운영체제 요약 정리

핵심 개념 및 시험 대비 학습 노트

목차

정리 범위와 원문 확인 사항 . . . 3

전체 학습 내용 개요 . . . 4

장별 요약 및 상세 정리 . . . 8

제1장. 서론 . . . 8

제2장. 운영체제 구조 . . . 20

제3장. 프로세스 . . . 21

제4장. 스레드와 병행성 . . . 37

제5장. CPU 스케줄링 . . . 49

제6장. 동기화 도구들 . . . 65

제7장. 동기화 예제 . . . 74

제8장. 교착 상태 . . . 83

제9장. 메인 메모리 . . . 93

제10장. 가상 메모리 . . . 102

제11장. 대용량 저장장치 구조 . . . 116

제12장. 입출력 시스템 . . . 134

제13장. 파일 시스템 인터페이스 . . . 147

제14장. 파일 시스템 구현 . . . 162

제15장. 파일 시스템 내부구조 . . . 175

제7부 도입. 보안과 보호 . . . 186

비교·공식·절차 모음 . . . 187

주요 개념 복습 . . . 203

개념 색인 . . . 206

최종 학습 점검표 . . . 209

정리 범위와 원문 확인 사항

제공된 615쪽 PDF의 모든 페이지를 대상으로 목차·본문·표·그림·수식·알고리즘·각주와 장말 자료를 검토한 학습 자료다. 실제 본문이 있는 제1장과 제3~15장, 제1·3·4·5·6·7부 도입을 정리한다. 원본 책 전체가 제공된 것은 아니므로, 책 전체의 완전한 요약으로 표시하지 않는다.

제공된 PDF의 전체 쪽수: 615

확인 가능한 범위: PDF 1~5쪽은 원본 목차, 6~615쪽은 제공된 도입 및 본문이다. 인쇄 1~679쪽 중 실제 제공된 610쪽에 해당한다. 실제 장별 범위는 아래 표와 각 장의 출처를 따른다.

원문 확인 범위와 제한

- 제2장 운영체제 구조 본문은 제공되지 않았다. 목차에서 제목·절 이름·시작 위치만 확인할 수 있어 제2장에는 미제공 안내만 남겼다.
- 제16~21장, 부록과 원본 찾아보기는 목차에만 있고 실제 본문이 없다. 이들 장의 학습 내용을 새로 작성하지 않았다.
- 인쇄 2, 60~116, 174, 280, 282, 318, 348, 382, 384, 490, 578, 616, 654쪽은 제공 파일에서 확인되지 않는다. 장 경계의 누락도 빈 페이지라고 추정하지 않았다.
- 마지막 PDF 615쪽은 제7부 보안과 보호의 도입이다. 그 도입에서 확인되는 정의만 정리하며, 뒤의 보안·보호 장 내용을 읽은 것으로 표시하지 않는다.
- 원문은 이미지 스캔이다. OCR은 독서 보조로 사용하고, 수록한 코드·수식·수치 예제와 중요한 그림은 원문 이미지로 대조하거나 원문 이미지를 그대로 보존했다. 의심되는 표기는 원문 확인 필요, 확인할 수 없는 내용은 원문에서 확인되지 않음으로 표시한다.
- 장말 연습문제는 점검하는 개념을 확인하는 데 사용했다. 원문에 없는 새 문제나 풀이·실행 결과를 만들지 않았으며, 외부 자료로 설명을 보충하지 않았다.

출처의 "책"은 원문에 인쇄된 쪽수, "원문 PDF"는 제공 파일의 첫 쪽을 1로 세는 위치다. 이 자료의 목차·쪽번호·색인은 생성된 PDF의 실제 쪽수를 사용한다. 원본 목차는 로마자 xix~xxiii로 인쇄돼 있어 본문 인쇄 쪽수와 구별한다.

전체 학습 내용 개요

제1부. 개관

출처: 책 1쪽 / 원문 PDF 6쪽

운영체제는 사용자와 하드웨어 사이의 중개자이며, 프로그램을 편리하고 효율적으로 실행할 환경을 제공한다. 하드웨어를 관리하면서 시스템의 정상 동작을 지키고, 설계 목표와 각 구성 부분의 입력·출력·동작을 명확히 해야 한다.

제3부. 프로세스 동기화

출처: 책 281쪽 / 원문 PDF 226쪽

병행 또는 병렬로 실행되는 스레드가 데이터를 공유할 때 접근을 제어하지 않으면 경쟁 조건으로 값이 손상될 수 있다. 동기화 도구는 공유 데이터 접근을 제어하지만, 잘못 사용하면 교착 상태나 성능 저하를 일으킬 수 있다.

제4부. 메모리 관리

출처: 책 383쪽 / 원문 PDF 324쪽

실행 중인 프로그램과 접근하는 데이터의 적어도 일부는 메인 메모리에 있어야 한다. 여러 프로세스를 메모리에 유지하는 관리 기법의 효과는 상황에 따라 다르며, 선택은 하드웨어 설계와 지원의 영향을 받는다.

제5부. 저장장치 관리

출처: 책 489쪽 / 원문 PDF 429쪽

대용량 저장장치는 파일과 데이터를 영구 저장한다. 장치는 전송 단위·접근 방식·동기성·공유 여부·읽기와 쓰기·속도가 다양하다. 입출력 서브시스템은 이를 제어하면서 단순한 인터페이스를 제공하고, 병목을 줄이도록 입출력을 최적화한다.

제6부. 파일 시스템

출처: 책 577쪽 / 원문 PDF 516쪽

파일은 작성자가 정의한 관련 정보의 모음이다. 파일 시스템은 파일을 물리적 저장장치에 매핑하고 사용자·프로그램의 접근과 조작 방법을 제공한다. 느릴 수 있는 물리 장치의 효율적인 접근과 공유·원격 접근 요구를 함께 고려한다.

제공된 장별 학습 범위

제1장. 서론

출처: 책 3-59쪽 / 원문 PDF 7-63쪽

- 운영체제를 하드웨어 자원 관리자이자 프로그램 실행 환경으로 이해한다. 컴퓨터 구성·인터럽트·저장장치·실행 모드에서 출발해 자원 관리, 보호와 보안, 가상화, 분산 시스템, 자료구조와 계산 환경으로 범위를 넓힌다.

제3장. 프로세스

출처: 책 117-173쪽 / 원문 PDF 64-120쪽

- 프로세스의 메모리·상태·PCB와 CPU 스케줄링을 연결한다. 생성·종료를 이해한 뒤 공유 메모리와 메시지 전달을 비교하고 실제 IPC 및 클라이언트·서버 통신으로 확장한다.

제4장. 스레드와 병행성

출처: 책 175-218쪽 / 원문 PDF 121-164쪽

- 프로세스 자원을 공유하는 스레드와 코어별 병렬 실행을 이해한다. 사용자·커널 스레드의 매핑, 스레드 API와 암묵적 스레딩을 살펴본 뒤 신호·취소·스레드 로컬 저장·커널 협력 문제를 정리한다.

제5장. CPU 스케줄링

출처: 책 219-279쪽 / 원문 PDF 165-225쪽

- CPU를 어떤 준비 프로세스·스레드에 언제 할당할지 정하는 정책과 평가 기준을 연결한다. 기본 알고리즘을 수치 예제로 비교하고 다중 처리기·실시간 조건 및 운영체제 구현 사례로 확장한다.

제6장. 동기화 도구들

출처: 책 283-317쪽 / 원문 PDF 227-261쪽

- 공유 데이터에 병행하게 접근할 때 생기는 경쟁 조건과 임계구역 문제를 정의하고, 하드웨어 명령에서 mutex 락·세마포·모니터까지 동기화 수단을 살펴본다.
- 상호 배제를 제공하는 것과 실행의 진행을 보장하는 것은 함께 검토해야 한다. 교착 상태·우선순위 역전 및 경합 정도에 따른 성능 차이도 이 장의 범위이다.

제7장. 동기화 예제

출처: 책 319-347쪽 / 원문 PDF 262-290쪽

- 유한 버퍼·readers-writers·식사하는 철학자 문제에 동기화 도구를 적용하고, Windows·Linux 커널 및 POSIX·Java의 구현과 API를 비교한다.
- 트랜잭션 메모리·OpenMP·함수형 언어도 원문이 제시하는 동기화 대안으로 정리한다.

제8장. 교착 상태

출처: 책 349-381쪽 / 원문 PDF 291-323쪽

- 한정된 자원을 점유하고 서로 기다리는 교착 상태를 시스템 모델과 자원 할당 그래프로 표현한다.
- 예방·회피·탐지·회복의 차이, 안전 상태와 은행원 알고리즘의 조건을 원문의 그래프·행렬 예와 함께 정리한다.

제9장. 메인 메모리

출처: 책 385-426쪽 / 원문 PDF 325-366쪽

- CPU가 만드는 논리 주소와 실제 메모리의 물리 주소를 연결하는 방법을 메모리 보호·주소 바인딩·MMU의 관점에서 정리한다.
- 연속 할당과 단편화에서 출발하여 페이징·TLB·페이지 테이블 구조·스와핑으로 확장하고, 원문이 제시한 Intel·ARM 주소 변환 사례를 다룬다.

제10장. 가상 메모리

출처: 책 427-488쪽 / 원문 PDF 367-428쪽

- 프로세스 전체를 메모리에 두지 않고 실행하는 가상 메모리의 이점과 요구 페이징·쓰기 시 복사·페이지 교체를 다룬다.
- 교체 알고리즘의 기준과 프레임 할당을 함께 비교하고 스래싱·지역성·작업 집합·압축·커널 할당·입출력 잠금까지 성능과 조건을 연결한다.

제11장. 대용량 저장장치 구조

출처: 책 491-534쪽 / 원문 PDF 430-473쪽

- HDD와 비휘발성 메모리(NVM)의 물리적 차이가 주소 매핑, 스케줄링, 오류 처리에 미치는 영향을 살펴본다. 저장장치 초기화·부팅·스왑 공간·연결 방식과 RAID의 신뢰성 및 성능을 연결해 정리한다.

제12장. 입출력 시스템

출처: 책 535-576쪽 / 원문 PDF 474-515쪽

- 포트·버스·컨트롤러의 하드웨어 동작을 응용 인터페이스와 커널 서비스에 연결한다. 폴링·인터럽트·DMA, 봉쇄형·비봉쇄형·비동기식 요청, 버퍼·캐시·스풀의 역할을 구별하고 요청 전체의 처리 경로를 따라간다.

제13장. 파일 시스템 인터페이스

출처: 책 579-615쪽 / 원문 PDF 517-553쪽

- 파일이라는 논리적 저장 단위와 속성·연산·접근 방법을 정리한다. 디렉터리의 이름 구조가 공유·탐색·삭제에 미치는 영향을 비교하고 파일 보호와 메모리 사상 파일의 동작을 살펴본다.

제14장. 파일 시스템 구현

출처: 책 617-653쪽 / 원문 PDF 554-590쪽

- 파일의 논리적 인터페이스를 실제 저장 블록으로 구현하는 자료구조와 알고리즘을 살펴본다. 디렉터리·블록 할당·가용 공간·캐시의 절충을 정리하고 크래시 복구와 WAFL의 스냅숏을 연결한다.

제15장. 파일 시스템 내부구조

출처: 책 655-678쪽 / 원문 PDF 591-614쪽

- 여러 파일 시스템을 하나의 이름 공간에 연결하는 마운트와 VFS를 정리한다. 다중 사용자 공유·원격 고장·일관성 의미를 구분하고 원문의 NFS 버전 3 중심 설명으로 마운트·원격 연산·경로 변환·캐싱을 따라간다.

제7부 도입. 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

- 제공된 마지막 도입 페이지에서 보안과 보호의 역할을 구별한다.

장	원문 범위
제1장. 서론	출처: 책 3-59쪽 / 원문 PDF 7-63쪽
제2장. 운영체제 구조	본문 미제공 / 출처: 원문 PDF 1쪽

장	원문 범위
제3장. 프로세스	출처: 책 117-173쪽 / 원문 PDF 64-120쪽
제4장. 스레드와 병행성	출처: 책 175-218쪽 / 원문 PDF 121-164쪽
제5장. CPU 스케줄링	출처: 책 219-279쪽 / 원문 PDF 165-225쪽
제6장. 동기화 도구들	출처: 책 283-317쪽 / 원문 PDF 227-261쪽
제7장. 동기화 예제	출처: 책 319-347쪽 / 원문 PDF 262-290쪽
제8장. 교착 상태	출처: 책 349-381쪽 / 원문 PDF 291-323쪽
제9장. 메인 메모리	출처: 책 385-426쪽 / 원문 PDF 325-366쪽
제10장. 가상 메모리	출처: 책 427-488쪽 / 원문 PDF 367-428쪽
제11장. 대용량 저장장치 구조	출처: 책 491-534쪽 / 원문 PDF 430-473쪽
제12장. 입출력 시스템	출처: 책 535-576쪽 / 원문 PDF 474-515쪽
제13장. 파일 시스템 인터페이스	출처: 책 579-615쪽 / 원문 PDF 517-553쪽
제14장. 파일 시스템 구현	출처: 책 617-653쪽 / 원문 PDF 554-590쪽
제15장. 파일 시스템 내부구조	출처: 책 655-678쪽 / 원문 PDF 591-614쪽
제7부 도입. 보안과 보호	출처: 책 679쪽 / 원문 PDF 615쪽

장별 요약 및 상세 정리

제1장. 서론

출처: 책 3-59쪽 / 원문 PDF 7-63쪽

학습 주제

- 운영체제를 하드웨어 자원 관리자이자 프로그램 실행 환경으로 이해한다. 컴퓨터 구성·인터럽트·저장장치·실행 모드에서 출발해 자원 관리, 보호와 보안, 가상화, 분산 시스템, 자료구조와 계산 환경으로 범위를 넓힌다.

설명의 흐름

1. 컴퓨터 구성과 인터럽트 → 저장장치와 처리기 구조 → 운영체제의 제어·자원 관리 → 보호·가상화·분산 환경 → 커널 자료구조와 공개 소스 학습

1.1 운영체제가 할 일

출처: 책 4-6쪽 / 원문 PDF 8-10쪽

운영체제는 컴퓨터 하드웨어를 관리하고 응용 프로그램의 실행 기반을 제공하며 사용자와 하드웨어 사이를 중재하는 소프트웨어이다. 하드웨어는 CPU·메모리·입출력 장치로 기본 계산 자원을 제공한다. 응용 프로그램은 이 자원을 사용해 사용자의 문제를 해결하며, 운영체제는 여러 사용자와 프로그램 사이의 하드웨어 사용을 제어하고 조정한다.

관점	핵심 역할과 조건
사용자 관점	개인용 컴퓨터에서는 사용의 편리함이 주요 목표이다. 모바일 장치는 터치·음성 인터페이스를 제공한다. 내장형 컴퓨터는 사용자 개입 없이 동작하기도 한다.
시스템 관점	자원 할당자로서 CPU 시간·메모리·저장공간·입출력 장치의 상충하는 요청을 효율적이고 공정하게 처리한다. 제어 프로그램으로서 부적절한 사용을 방지한다.

운영체제에 포함되는 요소에는 보편적으로 합의된 경계가 없다. 원문은 항상 실행되는 프로그램인 **커널**, 관리에 쓰이는 시스템 프로그램, 개발자에게 추가 서비스를 제공하는 미들웨어를 구별한다. 커널과 관련된 프로그램이라도 반드시 커널의 일부인 것은 아니다. 모바일 운영체제에는 데이터베이스·멀티미디어·그래픽 지원 프레임워크도 포함된다.

1.2 컴퓨터 시스템의 구성

출처: 책 7쪽 / 원문 PDF 11쪽

범용 컴퓨터는 하나 이상의 CPU, 공유 메모리, 공통 버스로 연결된 장치 컨트롤러로 구성된다. 컨트롤러는 특정 유형의 장치를 담당하고 로컬 버퍼와 특수 목적 레지스터를 가진다. **장치 드라이버**는 컨트롤러의 특성을 알고 운영체제의 다른 부분에 일관된 인터페이스를 제공한다. CPU와 장치 컨트롤러가 병렬로 작동하므로 공유 메모리 접근은 동기화되어야 한다.

1.2.1 인터럽트

출처: 책 7-12쪽 / 원문 PDF 11-16쪽

인터럽트는 하드웨어가 CPU의 처리가 필요한 이벤트를 알리는 핵심 수단이다. 드라이버가 컨트롤러 레지스터에 값을 적재하면 컨트롤러가 작업을 시작하고, 완료 후 인터럽트로 알린다. CPU는 하던 실행을 중단하고 서비스 루틴으로 제어를 옮긴 뒤 중단된 연산을 재개한다.

인터럽트 처리의 기본 순서

1. CPU는 명령어 실행을 마칠 때 인터럽트 요청 라인을 검사한다.
2. 인터럽트 번호를 인터럽트 벡터의 인덱스로 사용해 해당 핸들러로 이동한다.
3. 핸들러는 변경될 상태를 저장하고 원인을 확인하여 필요한 처리를 수행한다.
4. 상태를 복원하고 인터럽트 복귀 명령으로 이전 실행을 재개한다.

인터럽트 벡터는 서비스 루틴 주소의 배열이다. 중간 검색 루틴을 거치지 않고 적절한 루틴을 호출하여 처리 시간을 줄인다. 장치 수가 벡터 항목 수보다 많으면 각 항목에 핸들러 리스트를 연결하는 **인터럽트 체인**을 사용하고 요청을 처리할 핸들러를 찾을 때까지 호출한다.

구분	원문의 설명
마스크 불가능 인터럽트	복구할 수 없는 메모리 오류 같은 이벤트를 위해 예약한다.
마스크 가능 인터럽트	중요한 명령 시퀀스 실행 전에 잠시 끌 수 있으며 장치의 서비스 요청에 사용된다.
우선순위 레벨	낮은 우선순위 처리를 연기하고 높은 우선순위 인터럽트가 낮은 우선순위 실행을 선점하게 한다.

인터럽트 서비스는 빈번하고 시간에 민감하므로 효율적이어야 한다. 복귀 주소와 변경되는 레지스터 상태를 보존하지 않으면 원래 실행을 올바르게 재개할 수 없다.

1.2.2 저장장치 구조

출처: 책 12-15쪽 / 원문 PDF 16-19쪽

실행할 프로그램은 먼저 **메인 메모리**에 적재되어야 한다. 일반적인 메인 메모리는 DRAM으로 구현되는 휘발성 저장장치이다. 전원을 켤 때 운영체제를 적재하는 부트스트랩 프로그램은 RAM에 영구 보관할 수 없으므로 EEPROM 등 비휘발성 펌웨어를 사용한다.

적재(load)는 메인 메모리의 바이트 또는 워드를 CPU 레지스터로 옮기고, 저장(store)은 반대로 옮긴다. 명령 실행은 명령 인출·해독·피연산자 인출·연산·결과 저장으로 진행될 수 있다. 메모리는 주소가 생성된 방법이나 그 내용이 명령인지 데이터인지를 구별하지 않고 주소의 연속으로 취급한다.

메인 메모리는 모든 프로그램과 데이터를 영구 저장하기에는 작고 전원 손실 시 내용을 잃는다. 보조저장장치는 이를 확장해 대량의 데이터를 영구 보존한다. HDD와 NVM은 프로그램이 메모리에 적재되기 전 저장소이며, 처리 중 데이터의 원천과 목적지가 되기도 한다.

구분	원문의 범위
휘발성 저장장치	전원 제거 시 내용을 잃는다. 안전하게 보존할 데이터는 비휘발성 장치에 기록해야 한다.

구분	원문의 범위
기계적 비휘발성 저장장치	HDD·광 디스크·자기 테이프 등. 일반적으로 전기적 장치보다 용량이 크고 바이트당 비용이 낮다.
전기적 비휘발성 저장장치(NVM)	플래시·FRAM·NRAM·SSD 등. 일반적으로 더 빠르지만 비싸고 용량이 작다.
3차 저장장치	백업·드물게 쓰는 데이터·장기 보관을 위한 느리고 큰 저장장치이다.

저장 계층은 속도·용량·비용·휘발성을 함께 고려한다. 작고 빠른 메모리는 CPU에 가깝게 두며, 접근 시간이나 전송 속도 차이가 큰 두 수준 사이에는 캐시를 둘 수 있다. 원문은 1바이트=8비트, 워드를 구조 본연의 데이터 단위로 설명한다. 저장장치의 바이트 단위와 네트워킹의 비트 단위를 구별한다.

그림 1.6 저장장치 계층 구조

그림·표 출처: 책 14쪽 / 원문 PDF 18쪽

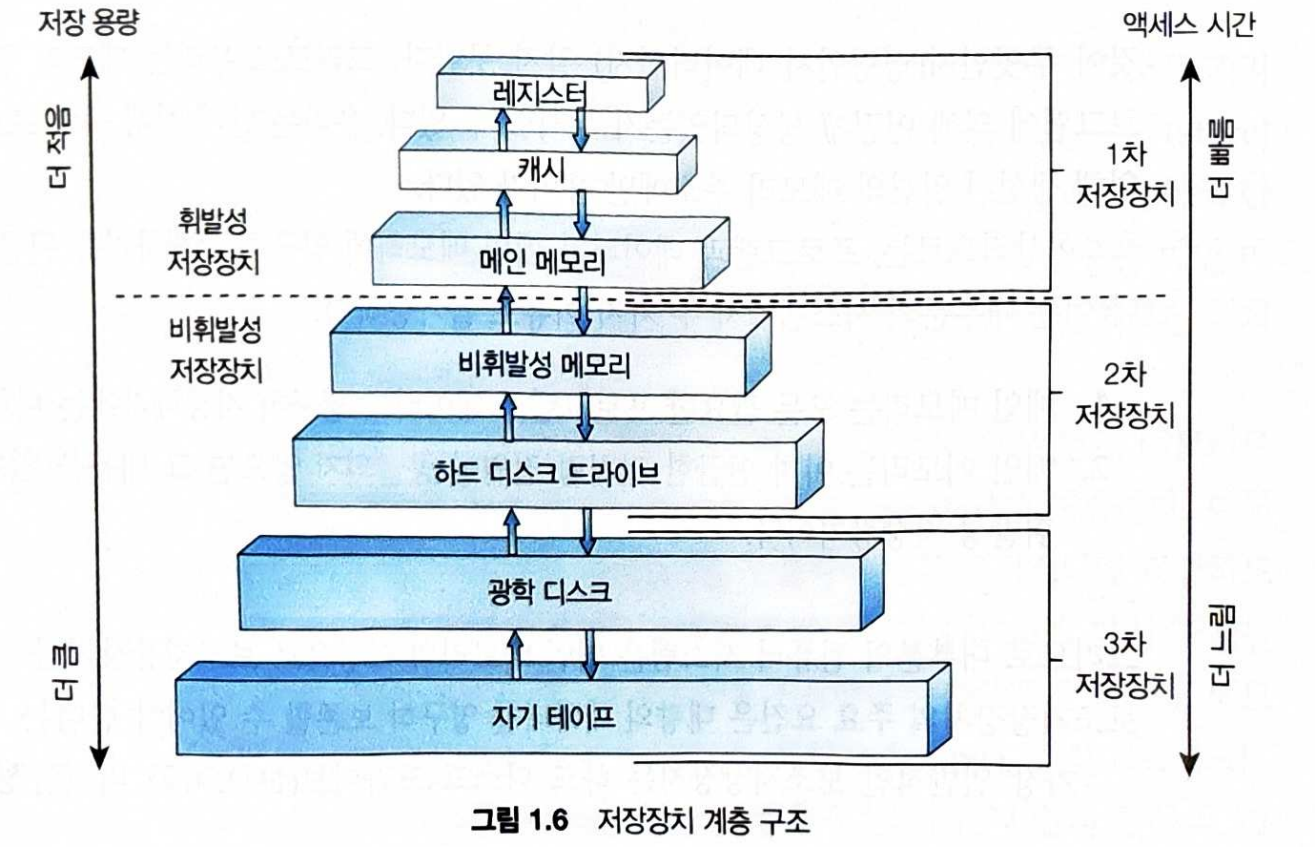


그림 1.6 저장장치 계층 구조

그림 1.6 저장장치 계층 구조

레지스터에서 자기 테이프로 내려가며 저장 용량은 커지고 접근 시간은 길어진다. 휘발성·비휘발성 및 1·2·3차 저장장치의 경계를 원문 그대로 보존하였다.

1.2.3 입출력 구조

출처: 책 15-16쪽 / 원문 PDF 19-20쪽

소량 데이터에는 인터럽트 구동 입출력이 적합하지만 대량 전송에서 바이트마다 인터럽트를 발생시키면 오버헤드가 커진다. **직접 메모리 접근(DMA)**은 버퍼·포인터·입출력 카운트를 설정한 후 컨트롤러가 CPU 개입 없이 메모리와 장치 버퍼 사이에서 블록 전체를 전송하게 한다. 블록 전송이 끝날 때 인터럽트를 발생시키므로 그동안 CPU는 다른 작업을 할 수 있다. 스위치 구조는 공유 버스 경쟁 없이 여러 구성요소의 동시 통신을 허용하여 DMA를 더 효과적으로 만들 수 있다.

1.3 컴퓨터 시스템 구조

출처: 책 16-22쪽 / 원문 PDF 20-26쪽

1.3.1 단일 처리기 시스템

출처: 책 16-17쪽 / 원문 PDF 20-21쪽

원문은 단일 처리 코어를 가진 범용 CPU가 하나인 경우를 단일 처리기 시스템으로 설명한다. 디스크·키보드·그래픽 컨트롤러의 전용 처리기는 제한된 명령어만 실행하고 사용자 프로세스를 실행하지 않는다. 그러한 전용 처리기가 있다고 해서 다중 처리기 시스템이 되는 것은 아니다. 코어는 명령을 실행하고 로컬 데이터용 레지스터를 갖는 기본 계산 단위이다.

1.3.2 다중 처리기 시스템

출처: 책 17-20쪽 / 원문 PDF 21-24쪽

다중 처리기 시스템은 여러 처리기가 버스와 메모리·주변 장치 등을 공유한다. 처리기를 늘리면 처리량이 증가하지만 협력 오버헤드와 공유 자원 경합 때문에 N개 처리기의 속도 향상이 반드시 N배가 되지는 않는다. **대칭형 다중 처리(SMP)**에서는 각 CPU가 운영체제 기능과 사용자 프로세스를 포함한 모든 작업을 수행한다.

각 CPU는 자체 레지스터와 로컬 캐시를 가지며 물리 메모리를 공유한다. 한 CPU가 유휴이고 다른 CPU가 과부하인 상태를 줄이려면 작업과 자료구조를 적절히 공유해야 한다. **다중 코어**는 한 칩 안에 여러 계산 코어를 두는 방식이다. 칩 내부 통신의 효율과 낮은 전력 소모가 이점이며 로컬 캐시와 공유 캐시를 결합할 수 있다. 운영체제에는 N개 코어가 N개 CPU처럼 보인다.

공유 버스 경합이 확장성의 병목이 되면 **NUMA** 구조에서 각 CPU 또는 CPU 그룹에 로컬 메모리를 둔다. 모든 CPU는 하나의 물리 주소 공간을 공유하지만 로컬 메모리보다 원격 메모리 접근 지연이 크다. 운영체제의 스케줄링과 메모리 관리가 이 단점을 줄여야 한다. 블레이드 서버는 각 보드가 독립적으로 부팅하고 자신의 운영체제를 실행한다는 점에서 전통적인 공유형 다중 처리기와 다르다.

1.3.3 클러스터형 시스템

출처: 책 20-22쪽 / 원문 PDF 24-26쪽

클러스터형 시스템은 둘 이상의 독립적인 노드를 네트워크로 연결한 약결합 시스템이다. 일반적으로 저장장치를 공유하고 LAN이나 고속 상호 연결망을 사용한다. 노드가 서로를 감시하다가 고장 노드의 저장장치 소유권을 넘겨받고 응용을 다시 실행함으로써 높은 가용성을 제공한다.

구성	동작
비대칭형 클러스터	다른 호스트가 응용을 실행하는 동안 긴급 대기 호스트는 감시만 하다가 고장 시 활성 서버가 된다.

구성	동작
대칭형 클러스터	여러 호스트가 응용을 실행하며 서로 감시한다. 모든 하드웨어를 이용하지만 여러 응용이 실행 가능해야 효율적이다.

남은 하드웨어 수준에 비례해 서비스를 계속하는 것이 우아한 성능 저하이며, 결함허용 시스템은 단일 구성요소 오류에도 계속 작동하도록 감지·진단·수정 기법을 둔다. 고성능 계산에 클러스터를 사용하려면 응용을 병렬화하여 각 노드가 일부 문제를 해결한 뒤 결과를 합쳐야 한다. 공유 데이터의 병렬 접근에는 충돌을 막는 접근 제어·잠금이 필요하며 원문은 분산 잠금 관리자(DLM)를 예로 든다. SAN은 여러 호스트가 여러 저장장치에 연결되도록 한다.

1.4 운영체제의 작동

출처: 책 23-24쪽 / 원문 PDF 27-28쪽

부팅 시 펌웨어의 **부트스트랩 프로그램**이 CPU 레지스터·컨트롤러·메모리를 초기화하고 커널을 찾아 메모리에 적재한다. 커널이 실행되면 서비스를 제공하고, 일부 서비스는 부팅 때 적재되어 계속 실행되는 시스템 데몬이 맡는다. 원문은 Linux의 systemd를 다른 데몬을 시작하는 시스템 프로그램으로 제시한다.

실행할 일이 없으면 시스템은 이벤트를 기다린다. 하드웨어 인터럽트 외에 **트랩(예외)**은 0으로 나누기·유효하지 않은 메모리 접근 등의 오류나 사용자 프로그램의 특정 요청으로 발생하는 소프트웨어 인터럽트이다. 사용자 요청은 시스템 콜을 통해 운영체제 서비스를 요구한다. 원문의 Hadoop 보충 설명은 분산 파일 시스템·YARN의 자원 및 작업 관리·MapReduce의 병렬 처리가 클러스터에서 결합됨을 보여 준다.

1.4.1 다중 프로그래밍과 다중 태스킹

출처: 책 24-26쪽 / 원문 PDF 28-30쪽

다중 프로그래밍은 여러 프로세스를 메모리에 유지하여 한 프로세스가 입출력 등을 기다릴 때 다른 프로세스를 실행한다. 실행 가능한 일이 있는 동안 CPU를 유휴로 두지 않아 이용률을 높이는 것이 목적이다. **다중 태스킹**은 이를 확장하여 프로세스 전환을 자주 수행함으로써 사용자에게 빠른 응답을 제공한다.

여러 프로세스를 동시에 유지하려면 메모리 관리가, 준비된 여러 프로세스 중 다음 대상을 고르려면 CPU 스케줄링이 필요하다. 가상 메모리는 일부만 메모리에 적재된 프로세스도 실행하게 하여 논리 메모리를 물리 메모리와 분리한다. 파일·저장장치 관리, 자원 보호, 통신·동기화와 교착 상태 처리도 여러 프로세스의 질서 있는 실행과 연결된다.

1.4.2 이중 모드와 다중 모드 운용

출처: 책 26-28쪽 / 원문 PDF 30-32쪽

운영체제와 사용자 코드가 자원을 공유하므로 하드웨어는 **사용자 모드**와 **커널 모드**를 구별한다. 원문의 모드 비트 예에서는 커널 모드가 0, 사용자 모드가 1이다. 부팅은 커널 모드에서 시작하고 사용자 프로그램으로 제어를 넘길 때 사용자 모드로 전환한다. 인터럽트·트랩·시스템 콜이 발생하면 하드웨어가 커널 모드로 바꾼다.

특권 명령은 커널 모드에서만 허용한다. 입출력 제어·타이머 관리·인터럽트 관리 등이 그 예이다. 사용자 모드에서 실행하려 하면 불법 명령으로 트랩을 발생시켜 운영체제로 제어를 넘긴다. 모드는 둘 이상으로 확장될 수 있으며 원문은 Intel 보호 링, ARMv8의 여러 모드, 가상 머신 관리자를 위한 모드를 소개한다.

시스템 콜의 제어 흐름

1. 사용자 프로세스가 운영체제의 작업 수행을 요청한다.

2. 트랩과 인터럽트 벡터를 통해 서비스 루틴으로 제어가 이동하고 커널 모드가 된다.

3. 커널이 호출 종류와 인수의 정확성·합법성을 검사하여 요청을 수행한다.

4. 사용자 모드로 바꾸고 시스템 콜 다음 명령으로 복귀한다.

프로그램이 불법 명령이나 자신의 영역 밖 메모리에 접근하면 하드웨어가 오류를 검출한다. 운영체제는 적절한 오류 메시지를 제공하고 비정상 종료시킬 수 있으며, 원문은 조사·수정을 위한 메모리 덤프도 설명한다.

그림 1.13 사용자 모드에서 커널 모드로의 전환

그림·표 출처: 책 26쪽 / 원문 PDF 30쪽

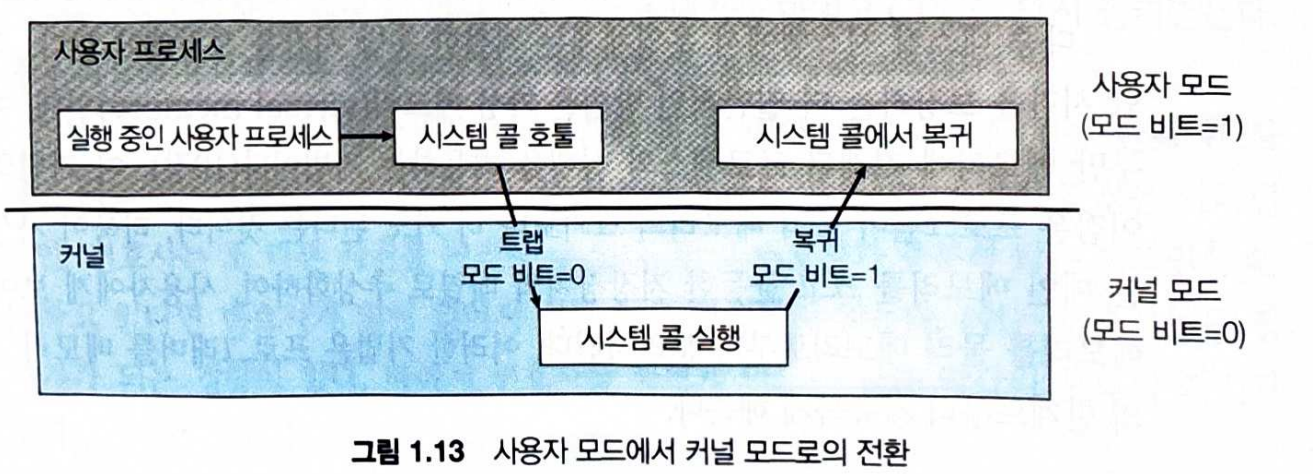


그림 1.13 사용자 모드에서 커널 모드로의 전환

그림 1.13 사용자 모드에서 커널 모드로의 전환

시스템 콜의 트랩에서 모드 비트가 0으로, 복귀에서 1로 바뀌는 원문 흐름이다.

1.4.3 타이머

출처: 책 28-29쪽 / 원문 PDF 32-33쪽

타이머는 사용자 프로그램의 무한 루프나 서비스 미호출 때문에 운영체제가 CPU 제어를 잃는 일을 방지한다. 지정한 시간 후 인터럽트를 발생시키며 가변 타이머는 고정률 클록과 계수기로 구현할 수 있다. 운영체제가 계수기를 설정하고 매 틱마다 감소시켜 0일 때 인터럽트를 발생시킨다. 제어를 사용자에게 넘기기 전에 설정되어야 하며 타이머 값 변경은 특권 명령이다. 원문 Linux 보충 설명에서 HZ는 초당 타이머 인터럽트 빈도, jiffies는 부팅 이후 인터럽트 누적 개수이다.

1.5 자원 관리

출처: 책 28-34쪽 / 원문 PDF 32-38쪽

1.5.1 프로세스 관리

출처: 책 29-30쪽 / 원문 PDF 33-34쪽

프로세스는 실행 중인 프로그램의 인스턴스이며 CPU 시간·메모리·파일·입출력 장치와 초기화 데이터를 필요로 한다. 프로그램은 디스크에 놓인 수동적 개체이지만 프로세스는 다음 명령을 가리키는 프로그램 카운터를 갖는 능동적 개체이다. 동일 프로그램을 실행해도 별도의 프로세스가 될 수 있다. 단일 스레드에는 한 실행 순서가, 다중 스레드에는 각 스레드의 프로그램 카운터가 있다.

- 사용자·시스템 프로세스를 생성하고 제거한다.
- 프로세스와 스레드에 CPU를 스케줄하고 중지·재개한다.

- 프로세스 동기화와 통신을 위한 기법을 제공한다.
- 종료 시 재사용 가능한 자원을 회수한다.

1.5.2 메모리 관리

출처: 책 30-31쪽 / 원문 PDF 34-35쪽

메인 메모리는 CPU와 입출력 장치가 공유하는 바이트 배열이며 각 바이트에는 주소가 있다. 프로그램을 실행하려면 메모리에 적재하고 절대 주소로 매핑해야 한다. 여러 프로그램을 메모리에 유지하면 CPU 이용률과 응답성을 높일 수 있지만 메모리 관리가 필요하다. 운영체제는 사용 중인 영역과 소유 프로세스를 추적하고, 공간을 할당·회수하며, 적재하거나 제거할 프로세스 또는 그 일부를 결정한다. 관리 기법의 효율은 상황에 따라 달라지고 필요한 하드웨어 지원의 영향을 받는다.

1.5.3 파일 시스템 관리

출처: 책 31-32쪽 / 원문 PDF 35-36쪽

운영체제는 물리 저장 매체의 차이를 감추고 **파일**이라는 논리 저장 단위를 정의한다. 파일은 생성자가 정의한 관련 정보의 집합이며 프로그램과 데이터를 표현할 수 있다. 운영체제는 파일을 물리 매체로 매핑하고 디렉터리로 조직하며 사용자별 접근을 통제한다. 파일·디렉터리 생성과 제거, 조작 프리미티브, 보조저장장치 매핑, 안정적인 매체로의 백업이 책무이다.

1.5.4 대용량 저장장치 관리

출처: 책 32쪽 / 원문 PDF 36쪽

HDD·NVM 같은 보조저장장치는 적재 전 프로그램과 데이터의 주요 온라인 저장소이다. 운영체제는 마운팅·언마운팅, 가용 공간 관리, 저장장소 할당, 디스크 스케줄링, 분할과 보호를 맡는다. 매우 빈번하게 사용되므로 장치와 알고리즘의 효율이 전체 시스템 속도에 영향을 줄 수 있다. 백업·장기 보존용 3차 저장장치는 덜 빈번하게 쓰이더라도 장치 할당·반환과 데이터 이주 등의 관리가 필요하다.

1.5.5 캐시 관리

출처: 책 33-34쪽 / 원문 PDF 37-38쪽

캐싱은 정보를 더 빠른 저장장치에 일시 복사하는 원리이다. 먼저 캐시를 검사하여 있으면 직접 쓰고, 없으면 원래 저장장치에서 가져와 사용하면서 캐시에 둔다. 캐시는 제한된 크기이므로 크기와 교체 정책이 중요하다. CPU 레지스터는 메인 메모리의 고속 캐시처럼 볼 수 있으며 레지스터 할당은 프로그래머·컴파일러가 정한다. 하드웨어만으로 관리되는 캐시는 이 책의 운영체제 제어 대상과 구별한다.

한 데이터의 복사본이 디스크·메인 메모리·캐시·레지스터에 동시에 존재하면 값의 갱신 시점이 서로 달라진다. 다중 태스킹에서는 각 프로세스가 최신 값을 보도록 해야 한다. 다중 처리기에서는 여러 로컬 캐시의 복사본을 일치시키는 **캐시 일관성** 문제가 생기며 통상 하드웨어에서 처리한다. 분산 환경의 파일 복제본 갱신은 더욱 복잡하다.

1.5.6 입출력 시스템 관리

출처: 책 34-35쪽 / 원문 PDF 38-39쪽

입출력 서브시스템은 장치의 구체적인 특성을 사용자와 운영체제의 나머지 부분에서 숨긴다. 버퍼링·캐싱·스풀링을 포함하는 메모리 관리 부분, 일반 장치 드라이버 인터페이스, 특정 하드웨어 드라이버로 구성된다. 해당 장치의 특성을 직접 아는 것은 그 장치 드라이버이다.

1.6 보안과 보호

출처: 책 35-36쪽 / 원문 PDF 39-40쪽

개념	정의와 역할
보호(protection)	프로그램·프로세스·사용자의 자원 접근을 제어한다. 허가된 사용과 허가되지 않은 사용을 구분하고 제어 명세와 강제 시행 방법을 제공한다.
보안(security)	시스템 외부와 내부의 공격을 방어한다. 보호 기능이 작동해도 인증 정보가 도난되면 자료가 복사·삭제될 수 있다.

운영체제는 사용자 이름과 유일한 사용자 식별자를 연결하고 로그인 인증에서 이를 결정한다. 식별자는 사용자의 프로세스·스레드에 연관된다. 그룹 식별자는 여러 사용자에게 공통 접근 권한을 부여할 때 사용한다. 제한된 작업에 권한 상승이 필요할 수 있으며 원문은 UNIX setuid를 통해 파일 소유자의 유효 사용자 식별자로 실행하는 경우를 제시한다. 보호와 보안은 사용자를 구별하는 기반에 의존하지만 동일한 개념은 아니다.

1.7 가상화

출처: 책 36-38쪽 / 원문 PDF 40-42쪽

가상화는 한 컴퓨터의 하드웨어를 여러 실행 환경으로 추상화하여 각 환경이 독립된 컴퓨터에서 실행되는 듯하게 만드는 기술이다. VMM은 게스트 운영체제를 실행하고 자원 사용을 관리하며 게스트 사이를 보호한다. 원문은 호스트 운영체제 위의 응용 형태 VMM과 호스트 자체 역할을 하는 VMM을 모두 소개한다.

원문의 구별에서 에뮬레이션은 소스와 대상 CPU가 다를 때 하드웨어를 소프트웨어로 모의하고 명령을 번역하므로 비용이 크다. 가상화에서는 동일 CPU용으로 컴파일된 운영체제를 다른 운영체제 환경에서 실행한다. 활용 사례는 다른 운영체제의 응용 사용, 여러 플랫폼용 개발·시험·디버깅, 데이터 센터의 자원 관리이다.

1.8 분산 시스템

출처: 책 38-39쪽 / 원문 PDF 42-43쪽

분산 시스템은 물리적으로 떨어진 이기종 컴퓨터가 네트워크로 연결된 집합이다. 자원 공유는 계산 속도·기능·데이터 가용성·신뢰성을 높일 수 있다. 운영체제는 원격 접근을 파일 접근처럼 일반화하거나 네트워크 기능을 명시적으로 호출하게 한다. 네트워크는 프로토콜·거리·매체에 따라 다르며 원문은 LAN·WAN·MAN·PAN과 TCP/IP를 소개한다.

네트워크 운영체제의 컴퓨터는 다른 컴퓨터와 파일·메시지를 공유하지만 서로 자율적으로 동작한다. 분산 운영체제는 컴퓨터들이 밀접하게 통신하여 하나의 운영체제가 전체 네트워크를 제어하는 것처럼 보이게 하므로 자율성이 더 낮다.

1.9 커널 자료구조

출처: 책 39-42쪽 / 원문 PDF 43-46쪽

1.9.1 리스트·스택·큐

출처: 책 39-41쪽 / 원문 PDF 43-45쪽

배열은 각 원소에 직접 접근한다. 리스트는 데이터를 순서로 나타내며 연결 리스트는 포인터로 항목을 연결한다. 단일 연결 리스트는 후속 항목, 이중 연결 리스트는 앞뒤 항목을 가리키고 원형 연결 리스트의 마지막은 첫 항목을 가리킨다. 가변 수 항목의 삽입·삭제는 쉽지만 길이 n인 리스트의 특정 항목을 찾는 최악 성능은 O(n)이다.

구조	순서·연산	원문의 사용
스택	후입선출(LIFO), push·pop	함수 인수·지역 변수·복귀 주소를 보관하고 복귀 시 제거한다.
큐	선입선출(FIFO)	인쇄 작업이나 CPU 실행 대기 태스크를 순서대로 조직한다.

1.9.2 트리

출처: 책 41쪽 / 원문 PDF 45쪽

트리는 부모·자식 관계로 계층을 나타낸다. 일반 트리는 임의 수의 자식, 이진 트리는 최대 두 자식을 가진다. 이진 탐색 트리는 값의 순서 관계를 추가하지만 형태에 따라 최악의 탐색이 O(n)이 될 수 있다. 균형 이진 탐색 트리는 깊이를 제한하여 최악의 탐색 O(lg n)을 보장한다. 원문은 Linux CPU 스케줄링의 red-black 트리를 예로 든다.

1.9.3 해시 함수와 맵

출처: 책 41-42쪽 / 원문 PDF 45-46쪽

해시 함수는 데이터에 산술 연산을 수행해 테이블 인덱스로 쓸 수 있는 수를 반환한다. 구현에 따라 인출 성능이 O(1)만큼 좋아질 수 있다. 서로 다른 입력이 같은 인덱스로 매핑되는 **해시 충돌**은 각 항목에 연결 리스트를 두어 처리할 수 있지만 충돌이 늘면 효율이 떨어진다. 해시 맵은 키와 값을 연결하며 키에 해시 함수를 적용해 값을 찾는다.

1.9.4 비트맵

출처: 책 42-43쪽 / 원문 PDF 46-47쪽

비트맵은 n개 자원의 상태를 n개 비트로 나타낸다. 원문의 예에서는 0이 사용 가능, 1이 사용 불가능을 뜻하지만 반대로 정할 수도 있다. i번째 비트는 i번째 자원에 대응한다. 한 자원에 8비트 부울 값을 쓰는 것보다 공간을 절약하므로 디스크 블록처럼 많은 자원의 가용성을 표시할 때 유용하다.

1.10 계산 환경

출처: 책 43-49쪽 / 원문 PDF 47-53쪽

1.10.1 전통적 계산

출처: 책 43-44쪽 / 원문 PDF 47-48쪽

웹 포털·무선 연결·모바일 접근이 사무실과 가정의 전통적인 계산 환경 경계를 넓힌다. thin client는 웹 기반 계산을 이해하는 터미널로 보안과 유지보수 측면에서 워크스테이션을 대체하기도 한다. 배치 시스템은 미리 정해진 입력을 대량 처리하고 대화형 시스템은 사용자 입력을 기다린다. 시분할의 타이머·스케줄링 기법은 현대 워크스테이션과 서버에서도 프로세스에 계산 시간을 빈번하게 제공하는 데 사용된다.

1.10.2 모바일 컴퓨팅

출처: 책 44-45쪽 / 원문 PDF 48-49쪽

모바일 컴퓨팅은 가볍고 휴대 가능한 스마트폰·태블릿 환경이다. GPS·가속도계·자이로스코프를 활용하는 응용과 증강현실을 지원한다. 무선망이나 휴대 전화 데이터망으로 온라인 서비스에 접근한다. 원문은 PC에 비해 메모리·처리 능력이 제한되고 전력 소모가 중요한 설계 조건임을 설명하며 iOS와 Android를 예로 든다. 이 설명과 수치는 원문 집필 시점의 환경이다.

1.10.3 클라이언트·서버 컴퓨팅

출처: 책 45-46쪽 / 원문 PDF 49-50쪽

클라이언트가 생성한 요청을 서버가 충족시키는 분산 구조이다. 계산 서버는 작업 요청 인터페이스를 제공하고 수행 결과를 돌려준다. 파일 서버는 파일 생성·갱신·읽기·제거 인터페이스를 제공하며 웹 서버가 브라우저에 파일을 전달하는 경우가 예이다.

1.10.4 피어 간 계산

출처: 책 46-47쪽 / 원문 PDF 50-51쪽

피어 간(P2P) 시스템에서는 클라이언트와 서버가 고정적으로 구별되지 않고 각 노드가 요청·제공 상황에 따라 두 역할을 맡는다. 서비스가 여러 노드로 분산되어 전통적인 서버 병목을 줄일 수 있다. 서비스 발견에는 중앙 검색 서비스에 등록하는 방법과 요청을 방송하고 제공자가 응답하는 방법이 있다. 원문은 Napster·Gnutella·Skype로 이 구성을 설명한다.

1.10.5 클라우드 컴퓨팅

출처: 책 47-49쪽 / 원문 PDF 51-53쪽

클라우드 컴퓨팅은 계산·저장·응용을 네트워크 서비스로 제공한다. 가상화를 기반으로 이용할 수 있으며 사용자는 사용한 자원에 따라 비용을 지불할 수 있다. 공중·사유·혼합형이라는 운영 범위의 구별과 SaaS·PaaS·IaaS라는 제공 자원의 구별은 서로 독립적인 배타 분류가 아니다.

유형	원문 의미
공중·사유·혼합형	누구나 비용을 지불하고 사용하는 서비스 / 한 회사용 / 두 방식을 포함하는 방식
SaaS	인터넷으로 사용하는 응용
PaaS	응용에 맞게 준비된 소프트웨어 스택

유형	원문 의미
IaaS	인터넷으로 사용하는 서버·저장장치

클라우드 하부구조에는 전통적 운영체제, 가상 머신을 관리하는 VMM, VMM과 클라우드 자원·인터페이스를 관리하는 도구가 결합될 수 있다. 원문 그림 1.24는 IaaS 공중 클라우드의 고객 인터페이스·관리 서비스·가상 머신·저장장치·부하 분배 관계를 보여 준다.

1.10.6 실시간 내장형 시스템

출처: 책 48-49쪽 / 원문 PDF 52-53쪽

내장형 시스템은 특정 작업을 수행하며 사용자 인터페이스가 거의 없을 수 있다. 범용 운영체제와 전용 응용, 제한된 기능의 내장형 운영체제, 운영체제 없이 ASIC만 사용하는 형태가 모두 존재한다. **실시간 시스템**은 데이터 흐름이나 처리에 엄격한 시간 제약이 있어 정해진 제한 시간 안에 올바른 결과를 내야 정확하게 동작한 것으로 간주된다. 단순히 빠른 응답이 바람직한 일반 시스템과 이 조건을 구별한다.

1.11 무료 및 공개 소스 운영체제

출처: 책 50쪽 / 원문 PDF 54쪽

무료·공개 소스 운영체제는 소스 코드 형태로 제공되어 분석·수정·컴파일·실행을 통한 학습을 가능하게 한다. 원문의 **자유 소프트웨어**는 소스 공개뿐 아니라 사용·재배포·수정을 허용하는 라이선스를 요구한다. 따라서 자유 소프트웨어는 공개 소스이지만 모든 공개 소스가 같은 자유를 보장하는 것은 아니다. 반면 비공개 소스는 컴파일된 이진 코드만 제공할 수 있다. 이진 코드에서 역공학으로 원래 소스와 주석을 완전히 복원하는 것은 어렵다.

1.11.1 역사

출처: 책 51쪽 / 원문 PDF 55쪽

초기에는 사용자 집단과 제조업체가 프로그램을 소스 코드와 함께 교환·배포했지만 소프트웨어의 사용 대상을 제한하고 경쟁자로부터 아이디어를 보호하려는 목적에서 이진 파일만 배포하는 관행이 확산되었다. 원문은 이러한 변화가 자유·공개 소스 운동의 배경임을 설명한다.

1.11.2 무료 운영체제

출처: 책 51-52쪽 / 원문 PDF 55-56쪽

Richard Stallman의 GNU와 자유 소프트웨어 운동에서 자유는 가격보다 사용자의 권리를 뜻한다. 실행, 분석·수정, 수정 없이 배포·판매, 수정해서 배포·판매할 자유가 핵심이다. **카피레프트**는 재배포 후에도 이러한 자유를 유지하게 한다. 원문은 GPL에서 바이너리와 함께 소스를 배포하고 수정본을 포함한 사본에 동일 GPL을 적용한다고 설명한다.

1.11.3 GNU/Linux

출처: 책 52-53쪽 / 원문 PDF 56-57쪽

GNU가 개발한 컴파일러·도구와 Linus Torvalds의 Linux 커널이 결합하여 GNU/Linux를 이룬다. 원문은 커널만을 Linux, GNU 도구까지 포함한 전체를 GNU/Linux로 구별한다. 배포판은 기능·유틸리티·응용·하드웨어 지원·인터페이스·목적에서 차이가 난다. 라이브 CD·DVD는 설치 없이 부팅하여 사용할 수 있고 가상 머신 이미지는 기존 호스트에서 별도의 운영체제를 학습하게 한다. 원문에 수록된 다운로드 주소와 설치 지시는 학습 사례로만 검토하였다.

1.11.4 BSD UNIX

출처: 책 53-54쪽 / 원문 PDF 57-58쪽

BSD UNIX는 AT&T UNIX에서 출발하였고 원문은 4.4BSD-lite 및 FreeBSD·NetBSD·OpenBSD·DragonFlyBSD를 소개한다. 소스 트리와 버전 제어를 이용하면 파일 변경 이력과 동시 수정 충돌을 관리할 수 있다. macOS의 공개 소스 핵심인 Darwin은 BSD UNIX에 기반하고 커널 패키지 이름은 xnu이다.

1.11.5 Solaris

출처: 책 54쪽 / 원문 PDF 58쪽

원문은 Solaris의 BSD UNIX와 System V 기반 이력, OpenSolaris 공개, 그 기반에서 발전한 Illumos를 설명한다. 이 이력은 제공된 본문의 설명 범위이며 이후 프로젝트의 현황을 외부 자료로 보충하지 않는다.

1.11.6 학습 도구로서 공개 소스 시스템

출처: 책 54-55쪽 / 원문 PDF 58-59쪽

소스가 공개되면 동작 관찰이나 문헌만으로 추측하던 구현을 직접 검토할 수 있다. 프로그램을 수정·시험하고 다양한 목표·라이선스·구현을 비교할 수 있다. 가상화와 하드웨어 시뮬레이터는 전용 기계 없이 여러 운영체제나 역사적 시스템을 탐구하게 한다. 원문은 소스 공개의 학습 가치와 다양성을 강조한다.

1.12 요약과 원문 복습 범위

출처: 책 56-59쪽 / 원문 PDF 60-63쪽

원문 요약은 하드웨어 관리, 인터럽트, 저장 계층, 다중 프로그래밍·태스킹, 실행 모드, 프로세스·메모리·파일 관리, 보호·가상화·계산 환경·공개 소스를 연결한다. 장말 연습문제 1.1~1.11은 운영체제 목적, 실시간 제약, 구성 범위, 특권 명령, 캐시, 클라이언트·서버와 P2P를 점검한다. 추가 자료와 참고문헌은 존재와 주제를 검토했으며 그 외부 문헌의 내용이나 새 풀이를 포함하지 않는다.

핵심 정리

- 운영체제의 자원 할당과 제어 역할을 사용자 관점과 시스템 관점에 연결한다.
- 인터럽트는 상태 보존·핸들러 디스패치·복원으로 처리하며 DMA는 블록 전송의 CPU 개입을 줄인다.
- 다중 프로그래밍은 CPU 이용, 다중 태스킹은 빈번한 전환과 응답에 초점을 둔다.
- 커널 모드·특권 명령·타이머는 잘못된 프로그램으로부터 시스템 제어를 지킨다.
- 보호는 자원 접근 제어, 보안은 내부·외부 공격 방어이다.
- 자원 복사본이 늘면 최신 값과 일관성을 유지하는 문제가 생긴다.

혼동하기 쉬운 내용

다중 프로그래밍과 다중 태스킹

출처: 책 24-26쪽 / 원문 PDF 28-30쪽

항목	다중 프로그래밍	다중 태스킹
목적	실행할 일이 있으면 CPU를 계속 사용	프로세스를 빈번하게 전환하여 빠른 응답
관계	여러 프로그램을 메모리에 유지	다중 프로그래밍의 논리적 확장

장 마무리 복습

- 하드웨어 자원을 추상화·할당·보호하는 것이 프로그램 실행 환경의 토대이다.
- CPU 제어의 이동은 인터럽트·트랩·시스템 콜 및 실행 모드 전환과 연결된다.
- 단일 처리기·다중 처리기·클러스터는 범용 계산 단위와 결합 방식의 차이로 구별한다.
- 저장 계층은 속도·용량·비용·휘발성의 절충이며 캐시에는 크기·교체·일관성 문제가 있다.

제2장. 운영체제 구조

출처: 원문 PDF 1쪽

학습 주제

- 원본 목차에서 제2장의 제목과 절 구성을 확인했으나 본문은 제공되지 않았다.

2 본문 미제공 안내

출처: 원문 PDF 1쪽

제공 PDF는 인쇄 59쪽 다음에 117쪽으로 이어진다. 제2장 본문은 원문에서 확인되지 않음. 아래는 목차에서 확인한 절 이름이며, 해당 개념의 정의나 작동 원리를 요약한 것으로 표시하지 않는다.

목차의 절	절 이름
2.1	운영체제 서비스
2.2	사용자와 운영체제 인터페이스
2.3	시스템 콜
2.4	시스템 서비스
2.5	링커와 로더
2.6	응용 프로그램이 운영체제마다 다른 이유
2.7	운영체제 설계 및 구현
2.8	운영체제 구조
2.9	운영체제 빌딩과 부팅
2.10	운영체제 디버깅
2.11	요약

원문 확인 메모

- 목차는 인쇄 61~112쪽의 장·절을 안내하지만 실제 파일은 인쇄 59쪽 다음에 117쪽으로 넘어간다. 본문 내용을 목차 제목에서 추정하지 않았다.

제3장. 프로세스

출처: 책 117-173쪽 / 원문 PDF 64-120쪽

학습 주제

- 프로세스의 메모리·상태·PCB와 CPU 스케줄링을 연결한다. 생성·종료를 이해한 뒤 공유 메모리와 메시지 전달을 비교하고 실제 IPC 및 클라이언트·서버 통신으로 확장한다.

설명의 흐름

1. 실행 중인 프로그램과 PCB → 준비·대기 큐와 문맥 교환 → 생성·종료 → 공유 메모리·메시지 전달 → POSIX·Mach·Windows·파이프 → 소켓·RPC

3.1 프로세스 개념

출처: 책 118-122쪽 / 원문 PDF 65-69쪽

3.1.1 프로세스

출처: 책 118-120쪽 / 원문 PDF 65-67쪽

프로세스는 실행 중인 프로그램이다. 현재 활동은 프로그램 카운터와 프로세서 레지스터의 내용으로 표현된다. 프로그램은 명령어를 저장한 수동적 파일이지만 프로세스는 다음 명령과 자원의 집합을 가진 능동적 개체이다. 실행 파일이 메모리에 적재되면 프로세스가 된다. 같은 프로그램에 연관되어도 각 프로세스의 실행 순서와 데이터·힙·스택은 별개일 수 있다.

메모리 섹션	내용과 변화
텍스트	실행 코드. 실행 중 크기는 고정된다.
데이터	전역 변수. 실행 중 크기는 고정된다.
힙	실행 중 동적으로 할당되는 메모리. 할당·반환에 따라 변한다.
스택	함수 매개변수·복귀 주소·지역 변수 등 임시 데이터. 호출 시 활성화 레코드를 넣고 복귀 시 제거한다.

스택과 힙은 동적으로 커지더라도 서로 겹치지 않아야 한다. 원문의 C 메모리 배치 보충 설명은 전역 데이터를 초기화된 데이터와 bss로 구별하고 argc·argv 저장 영역도 보인다. size 명령의 text·data·bss와 그 합계를 나타내는 dec·hex 필드를 설명한다. JVM처럼 프로세스 자체가 다른 프로그램의 실행 환경이 될 수도 있다.

3.1.2 프로세스 상태

출처: 책 121쪽 / 원문 PDF 68쪽

상태	정의
새로운(new)	생성 중이다.
실행(running)	명령어를 실행하고 있다.
대기(waiting)	입출력 완료나 신호 등 이벤트를 기다린다.
준비(ready)	처리기 할당을 기다린다.
종료(terminated)	실행이 끝났다.

이름과 세부 구분은 운영체제마다 다를 수 있다. 한순간 한 처리기 코어에서 실행되는 프로세스는 하나이지만 준비·대기 상태에는 여러 프로세스가 있을 수 있다. 준비와 대기는 각각 CPU와 이벤트를 기다린다는 차이가 있다.

그림 3.2 프로세스 상태 다이어그램

그림·표 출처: 책 121쪽 / 원문 PDF 68쪽

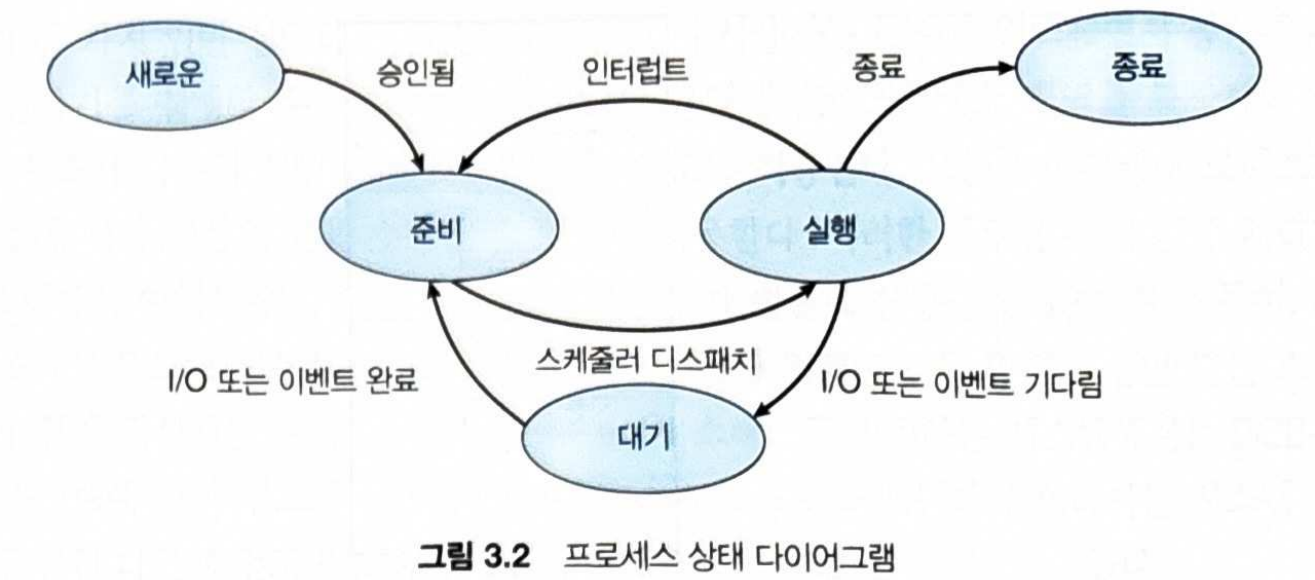


그림 3.2 프로세스 상태 다이어그램

승인·디스패치·인터럽트·입출력 대기와 완료·종료에 따른 상태 전이를 원문 그림으로 보존하였다.

3.1.3 프로세스 제어 블록

출처: 책 121-122쪽 / 원문 PDF 68-69쪽

프로세스 제어 블록(PCB)은 운영체제에서 각 프로세스를 표현하는 커널 자료구조이다. 실행을 시작하거나 다시 시작하는 데 필요한 상태와 회계 정보를 저장한다.

- 프로세스 상태와 다음 명령의 주소인 프로그램 카운터
- 인터럽트 후 복원을 위해 보존할 CPU 레지스터와 상태 코드
- 우선순위·스케줄링 큐 포인터·매개변수

- 기준·한계 레지스터, 페이지·세그먼트 테이블 등 메모리 관리 정보
- CPU 시간·경과 시간·제한·계정·프로세스 식별 정보
- 할당된 입출력 장치와 열린 파일 목록

3.1.4 스레드

출처: 책 122-123쪽 / 원문 PDF 69-70쪽

단일 실행 스레드를 가진 프로세스는 한 번에 하나의 실행 흐름을 수행한다. 여러 스레드를 허용하면 입력 처리와 철자 검사처럼 한 프로세스의 여러 일을 함께 수행할 수 있다. 다중 처리기에서는 스레드가 병렬 실행될 수 있다. 스레드 지원을 위해 PCB에 스레드별 정보를 포함하는 등 시스템 전체의 수정이 필요하며 상세 내용은 제4장에 이어진다.

3.2 프로세스 스케줄링

출처: 책 123쪽 / 원문 PDF 70쪽

프로세스 스케줄러는 실행 가능한 프로세스 가운데 CPU 코어에 할당할 대상을 선택한다. 다중 프로그래밍은 CPU 이용을 최대화하고 시분할은 빈번한 교체로 사용자 상호 작용을 지원한다. 메모리에 있는 프로세스 수는 다중 프로그래밍 정도이다. 입출력에 더 많은 시간을 쓰는 I/O 바운드와 계산에 더 많은 시간을 쓰는 CPU 바운드의 일반적 동작을 함께 고려해야 한다.

3.2.1 스케줄링 큐

출처: 책 123-126쪽 / 원문 PDF 70-73쪽

준비 큐는 CPU에서 실행되기를 기다리는 프로세스를, **대기 큐**는 입출력 완료 등 특정 이벤트를 기다리는 프로세스를 담는다. 준비 큐는 PCB를 연결한 리스트로 구현할 수 있다. 새 프로세스는 준비 큐에서 디스패치를 기다린다.

실행 중 프로세스의 이동

1. 입출력을 요청하면 해당 대기 큐에서 완료를 기다린다.
2. 자식을 생성한 뒤 종료를 기다리면 자식 종료 대기 큐로 간다.
3. 타임 슬라이스 만료나 인터럽트로 코어에서 제거되면 준비 큐로 돌아갈 수 있다.
4. 이벤트 완료 후 대기에서 준비로 돌아와 다시 스케줄된다. 종료하면 큐에서 제거되고 PCB와 자원이 반환된다.

Linux 보충 설명에서 task_struct는 프로세스 상태·스케줄링·부모와 자식·열린 파일·주소 공간을 표현한다. 활성 프로세스는 이중 연결 리스트로 조직되고 current가 현재 실행 프로세스를 가리킨다.

3.2.2 CPU 스케줄링

출처: 책 126쪽 / 원문 PDF 73쪽

CPU 스케줄러는 준비 큐에서 하나를 골라 CPU 코어를 할당한다. I/O 바운드는 짧게 실행한 뒤 대기하고 CPU 바운드는 긴 계산을 요구하므로 스케줄러는 자주 실행된다. **스와핑**은 메모리 부족 등의 상황에서 프로세스를 디스크로 내보내 다중 프로그래밍 정도를 낮추고 나중에 상태를 복원해 중단 지점부터 재개하는 기법이다.

3.2.3 문맥 교환

출처: 책 126-128쪽 / 원문 PDF 73-75쪽

문맥은 PCB에 표현되는 CPU 레지스터 값·프로세스 상태·메모리 관리 정보 등이다. 인터럽트 후 재개하려면 현재 상태를 저장하고 복원해야 한다. 다른 프로세스로 CPU를 넘기는 **문맥 교환**은 이전 프로세스의 문맥을 PCB에 저장하고 다음 프로세스의 문맥을 복구한다.

교환 중에는 유용한 사용자 작업을 하지 못하므로 순수한 오버헤드이다. 메모리 속도, 저장할 레지스터 수, 특수 명령, 레지스터 집합 지원 및 메모리 관리 기법의 복잡도에 따라 시간이 달라진다. 여러 레지스터 집합이 있으면 집합 포인터 변경으로 교환할 수 있으나 활성 프로세스가 더 많으면 메모리 복사가 다시 필요하다. 모바일 다중 태스킹 보충 설명은 전경·백그라운드와 Android의 사용자 인터페이스 없는 서비스를 구별한다.

3.3 프로세스에 대한 연산

출처: 책 128-136쪽 / 원문 PDF 75-83쪽

3.3.1 프로세스 생성

출처: 책 129-134쪽 / 원문 PDF 76-81쪽

프로세스를 생성하는 쪽은 **부모**, 새로 생성되는 쪽은 **자식**이다. 반복 생성으로 프로세스 트리를 이룬다. 운영체제는 유일한 정수 pid로 프로세스를 식별한다. 원문의 Linux 사례에서는 systemd가 pid 1인 루트 부모이며 ps·pstree로 프로세스 및 계층을 살펴본다.

자식은 운영체제로부터 직접 자원을 받거나 부모 자원의 일부만 사용하도록 제한될 수 있다. 부모가 자원을 나누거나 공유하고 초기화 데이터를 넘길 수도 있다. 부모는 자식과 병행 실행하거나 일부·전체 자식 종료를 기다린다. 주소 공간은 부모의 복사본일 수도, 새 프로그램으로 적재될 수도 있다.

UNIX **fork()**는 부모 주소 공간의 복사본인 새 프로세스를 만들며 두 프로세스는 호출 다음부터 실행한다. 부모에는 자식 pid가, 자식에는 0이 반환된다. **exec()**는 현재 주소 공간을 새 프로그램으로 덮어쓰며 오류가 없으면 돌아오지 않는다. 부모의 **wait()**는 자식 종료까지 기다린다.

그림 3.8 fork()로 별도 프로세스 생성

코드 출처: 책 132쪽 / 원문 PDF 79쪽

원문 코드·기호

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* 새 프로세스를 생성한다(fork) */
    pid = fork();

    if (pid < 0) { /* 오류가 발생했음 */
        fprintf(stderr,"Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* 자식 프로세스 */
        execlp("/bin/ls","ls",NULL);
    }
    else { /* 부모 프로세스 */
        /* 부모가 자식이 완료되기를 기다릴 것임 */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```

코드의 의미

부모는 자식 종료를 기다리고 자식은 주소 공간을 /bin/ls로 교체한다.

동작 과정

1. fork 반환값으로 실패·자식·부모를 구분한다.

2. 자식은 execlp(), 부모는 wait() 이후 printf()를 실행한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 원문에 없는 헤더를 추가하거나 코드의 선언·문장을 수정하지 않았다.

Windows **CreateProcess()**는 생성과 함께 지정된 프로그램을 자식 주소 공간에 적재한다. STARTUPINFO는 새 프로세스의 특성을, PROCESS_INFORMATION은 프로세스·스레드 핸들과 식별자를 제공한다. 원문 mspaint.exe 예제는 구조체 초기화 후 생성하고 WaitForSingleObject()로 종료를 기다린 뒤 핸들을 닫는다. fork()처럼 부모 주소 공간을 먼저 복사하는 방식과 구별한다.

3.3.2 프로세스 종료

출처: 책 134-136쪽 / 원문 PDF 81-83쪽

프로세스는 마지막 문장을 끝내고 **exit()**로 삭제를 요청하며 부모에게 종료 상태를 반환할 수 있다. 운영체제는 메모리·열린 파일·입출력 버퍼를 회수한다. 부모는 자원 초과·불필요한 작업·자신의 종료 등으로 자식을 종료시킬 수도 있다. 일부 시스템은 부모 종료 시 모든 후손을 종료하는 **연쇄식 종료**를 시행한다.

구분	발생 조건과 처리
좀비	종료했지만 부모가 wait()하지 않아 종료 상태를 담은 프로세스 테이블 항목이 남아 있다. 부모가 상태를 수집하면 pid와 항목도 반환된다.
고아	부모가 wait() 대신 먼저 종료한 자식이다. 전통적 UNIX는 init가 새 부모가 되어 종료 상태를 수집한다. 원문 Linux에서는 systemd 외 프로세스의 상속도 허용한다.

Android는 자원이 부족하면 중요도가 낮은 프로세스부터 종료한다. 원문의 중요도 순서는 전경 → 가시적 → 서비스 → 백그라운드 → 빈 프로세스이다. 여러 조건을 만족하면 가능한 높은 등급을 지정한다. 수명주기 지침은 종료 전에 상태를 보존하고 사용자가 돌아왔을 때 재개하도록 한다.

3.4 프로세스 간 통신

출처: 책 137-139쪽 / 원문 PDF 84-86쪽

다른 프로세스에 영향을 주거나 받는 **협력 프로세스**에는 정보 공유·계산 가속화·모듈성의 이점이 있다. 계산을 나누어 실제 속도를 높이려면 여러 처리 코어가 필요하다. 협력을 위한 **IPC**에는 공유 메모리와 메시지 전달이라는 두 기본 모델이 있다.

기준	공유 메모리	메시지 전달
정보 교환	공유 영역을 읽고 쓴다.	메시지를 보내고 받는다.
운영체제 관여	공유 영역 설정 후 일반 메모리 접근으로 수행한다.	통상 시스템 콜을 이용해 커널 개입 비용이 든다.
원문의 특징	설정 후 빠른 통신이 가능하지만 접근 조정은 프로세스가 책임진다.	적은 데이터 교환에 유용하고 분산 시스템에서 구현하기 쉽다.

Chrome 사례는 브라우저·렌더러·플러그인 프로세스를 분리하여 한 사이트의 고장이 다른 사이트 전체로 퍼지는 영향을 줄이고 렌더러의 디스크·네트워크 접근을 샌드박스로 제한하는 구성을 보여 준다. 이 사례는 원문이 기술한 구조의 설명이다.

3.5 공유 메모리 시스템에서의 프로세스 간 통신

출처: 책 139-141쪽 / 원문 PDF 86-88쪽

통신 프로세스는 공유 세그먼트를 만들고 각자의 주소 공간에 추가한다. 데이터의 형식·위치는 프로세스가 정하며 동일 위치의 동시 쓰기를 방지할 책임도 프로세스에 있다. **생산자·소비자**에서는 생산자가 채우고 소비자가 소모할 공유 버퍼와 동기화가 필요하다.

무한 버퍼에서 생산자는 계속 생산할 수 있지만 소비자는 항목을 기다릴 수 있다. 유한 버퍼는 비었으면 소비자가, 가득 찼으면 생산자가 대기한다. 원문의 원형 배열에서 in은 다음 빈 위치, out은 다음 소비 위치이다. in == out이면 비었고 (in + 1) % BUFFER_SIZE == out이면 가득 찬 것으로 본다.

그림 3.12 공유 메모리 생산자

코드 출처: 책 141쪽 / 원문 PDF 88쪽

생산자 처리 부분 전체 발췌이며 공유 변수 선언은 제외하였다.

원문 코드·기호

```
item next_produced;

while (true) {
    /* produce an item in next_produced */

    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */

    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

코드의 의미

원형 버퍼가 가득 차지 않았을 때 다음 항목을 저장한다.

동작 과정

1. 가득 찬 조건 동안 빈 반복으로 기다린다.

2. in 위치에 쓰고 모듈로 연산으로 다음 위치를 정한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 공유 변수 선언은 인쇄 140쪽에 따로 있다.

- 최대 BUFFER_SIZE - 1개만 수용한다.

그림 3.13 공유 메모리 소비자

코드 출처: 책 141쪽 / 원문 PDF 88쪽

소비자 처리 부분 전체 발췌이며 공유 변수 선언은 제외하였다.

원문 코드·기호

```
item next_consumed;

while (true) {
    while (in == out)
        ; /* do nothing */

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next_consumed */
}
```

코드의 의미

버퍼가 비어 있지 않을 때 다음 항목을 읽고 소비 위치를 전진시킨다.

동작 과정

1. in == out이면 기다린다.

2. out 위치를 읽고 다음 위치로 이동한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 생산되지 않은 항목을 읽지 않도록 비어 있는 조건을 검사한다.

이 구현은 BUFFER_SIZE개의 배열에 최대 BUFFER_SIZE - 1개 항목만 담는다. 배열 전체를 사용하는 새 해결책은 원문 연습 대상으로 남겨져 있으므로 추가하지 않는다. 이 절의 예제만으로 병행 접근의 동기화 문제가 해결되는 것은 아니며 원문은 이를 제6·7장으로 연결한다.

3.6 메시지 전달 시스템에서의 프로세스 간 통신

출처: 책 141-142쪽 / 원문 PDF 88-89쪽

메시지 전달은 동일 주소 공간을 공유하지 않고 통신·동기화하게 하며 네트워크로 분리된 프로세스에 유용하다. 최소 연산은 send(message)와 receive(message)이다. 고정 길이는 시스템 구현이 단순하지만 프로그램 작성을 어렵게 하고, 가변 길이는 프로그램 작성은 편하지만 시스템 구현이 복잡하다. 통신 링크에는 명명·동기화·버퍼링의 설계 선택이 있다.

3.6.1 명명

출처: 책 142-145쪽 / 원문 PDF 89-92쪽

직접 통신에서는 송신자와 수신자가 상대 프로세스를 명시한다. 통신을 원하는 두 프로세스 사이에 링크가 자동 구성되고 각 쌍에 하나의 연결이 생긴다. 대칭 주소 지정은 둘 다 상대 이름을, 비대칭은 송신자만 수신자 이름을 지정한다. 이름을 코드에 직접 넣으면 이름 변경 시 모든 참조를 바꿔야 하므로 모듈성이 제한된다.

간접 통신은 고유 식별자를 가진 메일박스·포트로 송수신한다. 공유 메일박스가 있을 때만 연결되고 하나의 링크가 여러 프로세스와 관련되거나 한 쌍 사이에 여러 링크가 있을 수 있다. 여러 수신자가 같은 메일박스를 공유하면 링크당 두 프로세스로 제한하거나 한 번에 하나만 수신하게 하거나 시스템이 한 수신자를 선택하는 규칙이 필요하다.

프로세스 소유 메일박스는 소유자가 수신하고 종료하면 사라진다. 운영체제 소유 메일박스는 독립적으로 존재하며 생성·송수신·삭제 기법이 필요하다. 생성자의 소유권과 수신 권한은 적절한 시스템 콜로 이전할 수 있다.

3.6.2 동기화

출처: 책 145-146쪽 / 원문 PDF 92-93쪽

방식	동작
봉쇄형 보내기	수신 프로세스나 메일박스가 메시지를 받을 때까지 송신자가 기다린다.
비봉쇄형 보내기	보낸 후 자신의 실행을 계속한다.
봉쇄형 받기	메시지가 사용 가능할 때까지 기다린다.
비봉쇄형 받기	유효한 메시지 또는 널을 반환받는다.

보내기와 받기가 모두 봉쇄형이면 송신자와 수신자의 **랑데부**가 된다. 생산자는 메시지가 전달될 때까지, 소비자는 메시지가 도착할 때까지 기다리는 방식으로 생산자·소비자를 표현할 수 있다.

3.6.3 버퍼링

출처: 책 146쪽 / 원문 PDF 93쪽

큐 용량	송신자의 대기 조건
무용량	대기 메시지를 보관하지 못하므로 수신자가 받을 때까지 기다린다.
유한 용량	빈자리가 있으면 넣고 계속하지만 가득 차면 공간을 기다린다.
무한 용량	메시지를 계속 보관할 수 있어 송신자는 버퍼 때문에 봉쇄되지 않는다.

무용량은 버퍼 없는 시스템으로, 나머지는 자동 버퍼링으로 부른다. 직접·간접 통신이라는 명명 방식과 버퍼 용량이라는 조건을 서로 다른 설계 기준으로 구별한다.

3.7 IPC 시스템의 사례

출처: 책 146-160쪽 / 원문 PDF 93-107쪽

3.7.1 POSIX 공유 메모리

출처: 책 147-149쪽 / 원문 PDF 94-96쪽

원문 API의 순서

1. shm_open()으로 이름 있는 공유 메모리 객체를 만들거나 연다. 이름·열기 플래그·접근 허가권을 지정하고 파일 설명자를 받는다.
2. ftruncate()로 바이트 단위 크기를 설정한다.
3. mmap()으로 객체를 프로세스의 주소 공간에 매핑하고 반환 포인터로 읽거나 쓴다.
4. 소비자는 사용을 마친 뒤 shm_unlink()로 이름 있는 공유 메모리를 제거한다.

원문 생산자는 OS라는 4096바이트 객체에 Hello와 World! 문자열을 쓰며 포인터를 기록한 바이트 수만큼 이동한다. MAP_SHARED는 공유하는 프로세스가 변경 값을 접근하도록 지정한다. 아래 코드는 원본 그림으로 보존하였다. 별도 실행 출력은 원문에 실행 결과 미제시.

그림 3.16 POSIX 공유 메모리 생산자 코드

그림·표 출처: 책 148쪽 / 원문 PDF 95쪽

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

#include <sys/mman.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int fd;
/* pointer to shared memory obect */
char *ptr;

    /* create the shared memory object */
    fd = shm_open(name,O_CREAT | O_RDWR,0666);

    /* configure the size of the shared memory object */
    ftruncate(fd, SIZE);

    /* memory map the shared memory object */
    ptr = (char *)
      mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    /* write to the shared memory object */
    sprintf(ptr,"%s",message_0);
    ptr += strlen(message_0);
    sprintf(ptr,"%s",message_1);
    ptr += strlen(message_1);

    return 0;
}
```

그림 3.16 POSIX 공유 메모리 API를 설명하는 생산자 프로세스

그림 3.16 POSIX 공유 메모리 생산자 코드

원문 코드 전체를 이미지로 보존한다. 객체 생성→크기 설정→매핑→문자열 쓰기의 순서이다. 원문에 실행 결과 미제시.

그림 3.17 POSIX 공유 메모리 소비자 코드

그림·표 출처: 책 149쪽 / 원문 PDF 96쪽

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

#include <sys/mman.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* shared memory file descriptor */
int fd;
/* pointer to shared memory obect */
char *ptr;

    /* open the shared memory object */
    fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = (char *)
      mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    /* read from the shared memory object */
    printf("%s",(char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```

그림 3.17 POSIX 공유 메모리 API를 설명하는 소비자 프로세스

그림 3.17 POSIX 공유 메모리 소비자 코드

원문 코드 전체를 이미지로 보존한다. 객체 열기→매핑→읽기→이름 제거 순서이며 원문 플래그 조합을 수정하지 않았다. 원문에 실행 결과 미제시.

원문 확인 필요: 소비자 그림은 shm_open()에 O_RDONLY를 쓰면서 mmap() 보호 인수에는 PROT_READ | PROT_WRITE를 표시한다. 원본의 플래그를 임의로 고치지 않고 그림 그대로 남겼다.

3.7.2 Mach 메시지 전달

출처: 책 149-153쪽 / 원문 PDF 96-100쪽

Mach의 태스크 간 통신은 대부분 포트로 메시지를 주고받는다. 포트는 크기가 정해진 단방향 메일박스이며 여러 송신자와 하나의 수신자를 가질 수 있다. 응답은 별도 포트로 보낸다. 포트 권한은 태스크에 속해 같은 태스크의 스레드가 공유한다. 생성자는 수신 권한을 갖고 다른 태스크에 송신 권한을 부여할 수 있다. Task Self·Notify 포트와 부트스트랩 서버는 커널 통신·이벤트 알림·포트 발견을 지원한다.

mach_msg()의 메시지는 고정 헤더와 가변 본체로 구성된다. 헤더에는 크기·소스·대상 포트 등의 메타데이터가 있다. 단순 메시지는 해석되지 않는 데이터, 복잡한 메시지는 out-of-line 메모리 포인터나 포트 권한을 포함할 수 있다. 같은 송신자의 메시지는 FIFO로 넣지만 서로 다른 송신자 사이의 절대 순서는 보장하지 않는다.

큐가 가득 차면 송신자는 무한 대기·제한 시간 대기·즉시 복귀·일시 캐시 중에서 선택할 수 있다. 캐시 선택은 다른 요청을 계속 처리해야 하는 서버의 일회성 응답에 쓰이며 송신 스레드당 하나만 커널에 보관한다. 같은 시스템 내부에서는 가상 메모리 매핑을 이용해 실제 데이터 복사를 피할 수 있지만 다른 시스템 사이에는 이 기법이 적용되지 않는다.

3.7.3 Windows

출처: 책 153-154쪽 / 원문 PDF 100-101쪽

ALPC(고급 로컬 프로시저 호출)는 동일 기계의 프로세스 사이 통신에 최적화된 메시지 전달 설비이다. 서버가 공개 연결 포트를 만들면 클라이언트가 연결을 요청하고 서버는 한 쌍의 사적 통신 포트로 채널을 만든다. 콜백은 응답 대기 중에도 다른 요청을 처리하도록 한다.

데이터 범위	전달 방법
256바이트 이하	포트 메시지 큐를 중간 저장소로 사용하여 복사한다.
큰 메시지	채널에 연관된 공유 메모리인 섹션 객체와 그 포인터·크기를 보낸다.
섹션 객체보다 더 큼	서버가 클라이언트 주소 공간을 직접 읽고 쓰는 API를 사용할 수 있다.

ALPC는 Windows API의 직접 사용 가능한 부분이 아니다. 응용은 표준 RPC를 호출하고 같은 시스템의 RPC가 간접적으로 ALPC를 통해 처리된다.

3.7.4.1 일반 파이프

출처: 책 155-160쪽 / 원문 PDF 102-107쪽

파이프는 쓰기 종단의 생산자와 읽기 종단의 소비자를 연결한다. 일반 파이프는 단방향이므로 양방향 전송에는 서로 반대 방향의 파이프 두 개가 필요하다. UNIX의 pipe(fd)는 fd[0]을 읽기, fd[1]을 쓰기 종단으로 만들며 read()·write()로 접근한다. 일반적으로 부모가 만든 뒤 fork()로 자식이 열린 파이프를 상속한다.

원문의 Greetings 예제에서 부모는 쓰고 자식은 읽으며 양쪽은 쓰지 않는 종단을 닫는다. 쓰는 쪽이 닫았을 때 읽는 쪽이 EOF를 검출하도록 하는 데 이 절차가 중요하다. Windows의 익명 파이프도 부모·자식 사이의 단방향 통신이다. CreatePipe() 이후 상속 허용과 표준 입력·출력 재지정을 명시하며 ReadFile()·WriteFile()로 읽고 쓴다. 양쪽 운영체제의 일반 파이프는 동일 기계의 관련 프로세스 사이에서 사용한다.

파이프를 비교할 때 방향, 반이중·전이중, 프로세스 관계 요구, 네트워크 지원을 구별한다. 반이중은 한순간 한 방향, 전이중은 양방향 동시 전송이다.

3.7.4.2 지명 파이프

출처: 책 160-161쪽 / 원문 PDF 107-108쪽

구분	UNIX FIFO	Windows 지명 파이프
관계·수명	부모·자식일 필요가 없고 파일 시스템에서 명시적으로 제거할 때까지 존재	부모·자식일 필요가 없으며 여러 프로세스가 사용 가능
전송	반이중, 바이트 단위. 양방향에는 보통 두 FIFO 사용	전이중, 바이트·메시지 단위
범위	같은 기계	같거나 다른 기계

UNIX는 mkfifo()로 만들고 일반 파일처럼 open()·read()·write()·close()한다. 원문 셸 사례 ls | less와 dir | more는 앞 명령의 출력을 뒤 명령의 입력으로 전달하여 한 화면씩 표시하는 생산자·소비자 관계를 보여준다.

3.8 클라이언트 서버 환경에서 통신

출처: 책 161-168쪽 / 원문 PDF 108-115쪽

3.8.1 소켓

출처: 책 162-165쪽 / 원문 PDF 109-112쪽

소켓은 통신의 극점이며 네트워크 통신에는 각 프로세스에 하나씩 두 소켓이 필요하다. IP 주소와 포트 번호의 조합으로 구별하고 연결은 유일한 소켓 쌍으로 구성된다. 서버는 지정 포트에서 요청을 기다리고 클라이언트의 연결을 수락한다. 원문은 well-known 서비스 포트와 클라이언트에게 부여하는 포트를 구별한다.

Java의 Socket은 연결 기반 TCP, DatagramSocket은 비연결 UDP, MulticastSocket은 여러 수신자에게 보내는 소켓이다. Date 서버 예제는 포트 6013에서 accept()로 기다리고 연결되면 PrintWriter로 날짜·시간을 전송한 뒤 소켓을 닫는다. 클라이언트는 127.0.0.1:6013에 연결하여 스트림으로 읽고 닫는다. 127.0.0.1은 자기 기계를 가리키는 루프백 주소이다.

소켓은 효율적이고 널리 쓰이지만 구조화되지 않은 바이트 스트림만 제공하는 낮은 수준이다. 데이터의 구조와 해석은 클라이언트·서버의 책임이며, 더 높은 수준의 대안이 RPC이다. 원문의 날짜 출력에 구체적인 실제 실행 값은 원문에 실행 결과 미제시.

3.8.2 원격 프로시저 호출

출처: 책 164-168쪽 / 원문 PDF 111-115쪽

RPC는 네트워크의 다른 시스템에 있는 프로시저를 지역 프로시저처럼 호출하도록 추상화한다. 메시지는 원격 서비스 주소·함수 식별자·매개변수를 포함하고 수행 결과는 별도 메시지로 반환한다. 클라이언트와 서버의 **스텁**이 통신 세부사항을 숨긴다.

스텁을 통한 호출

1. 클라이언트가 원격 프로시저에 대응하는 스텁을 호출하고 인수를 건넨다.

2. 스텁이 서버 포트를 찾고 매개변수를 정돈(marshalling)하여 전송한다.

3. 서버 스텁이 메시지를 받아 풀어내고 실제 프로시저를 호출한다.

4. 필요한 반환값도 반대 방향으로 정돈·전달한다.

기계마다 정수의 바이트 순서가 다를 수 있다. XDR 같은 기종 중립 표현은 송신 시 변환하고 수신 시 해당 기종 표현으로 되돌려 차이를 다룬다. 원문은 big-endian과 little-endian 가운데 어느 쪽이 더 낫다는 문제가 아니라 구조상의 선택이라고 설명한다.

네트워크 오류는 요청 손실·중복 실행 문제를 만든다. 원문의 **최대 한 번** 방식은 타임스탬프와 처리 이력으로 중복 요청을 무시한다. **정확히 한 번** 설명은 여기에 수신·실행 ACK와 ACK 전까지의 재전송을 더한다. 서버 고장과 ACK 손실의 경우는 장말 연습문제의 검토 대상이므로 별도의 보장이나 풀이를 창작하지 않는다.

바인딩은 서버 포트를 찾는 과정이다. 고정 포트는 컴파일 시 정하지만 이후 변경이 어렵다. 동적 방식은 고정된 matchmaker에 RPC 이름을 보내 포트를 얻은 뒤 실제 요청을 보낸다. 초기에 오버헤드가 있지만 유연하다. 원문은 분산 파일 시스템의 read·write·rename·delete·status 요청도 활용 예로 든다.

그림 3.29 RPC의 실행

그림·표 출처: 책 168쪽 / 원문 PDF 115쪽

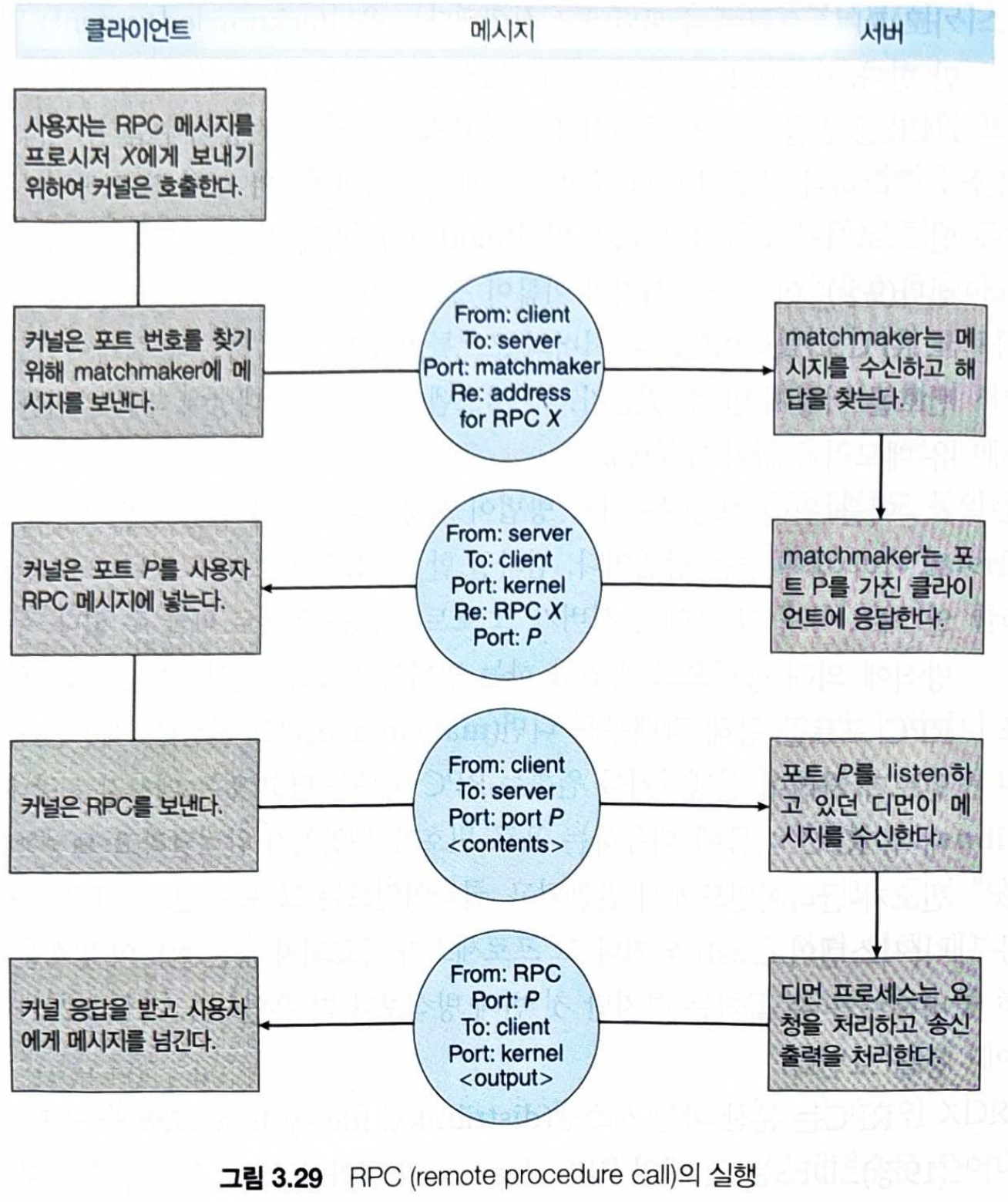


그림 3.29 RPC (remote procedure call)의 실행

그림 3.29 RPC의 실행

matchmaker에서 포트를 찾은 뒤 원격 서비스에 요청하고 결과를 반환받는 흐름이다.

3.8.2.1 Android RPC

출처: 책 167-169쪽 / 원문 PDF 114-116쪽

RPC는 같은 기계의 IPC에도 쓰이며 Android는 바인더 프레임워크로 이를 제공한다. 서비스는 사용자 인터페이스 없이 백그라운드 연산을 수행한다. bindService()가 서비스를 바인드하면 서버의 onBind()가 호출된다.

메시지 전달의 경우 onBind()는 Messenger를 반환한다. 단방향이므로 응답하려면 클라이언트의 Messenger를 Message의 replyTo에 포함한다. RPC의 경우 원격 메소드를 정의한 인터페이스를 반환하며 AIDL에서 Java 인터페이스와 스텁을 생성한다. 바인더가 매개변수 정돈·전달·서비스 호출·반환값 전송을 처리한다.

3.9 요약과 원문 복습 범위

출처: 책 169-173쪽 / 원문 PDF 116-120쪽

프로세스의 메모리·상태·PCB는 스케줄링과 문맥 교환의 기반이다. 생성·종료는 부모·자식 관계와 자원 회수로 이어지고 협력은 IPC를 요구한다. 장말 연습문제 3.1~3.7은 fork 결과·프로세스 수·문맥 교환·공유 상태·RPC 전달 의미를 점검한다. 문제의 코드도 검토하였으나 원문에 없는 실행 결과나 새 풀이를 작성하지 않았다. 추가 자료·참고문헌의 외부 내용은 사용하지 않았다.

핵심 정리

- 프로그램은 수동적 파일, 프로세스는 프로그램 카운터와 자원을 가진 실행 개체이다.
- 준비는 CPU를, 대기는 이벤트를 기다린다. PCB가 문맥 저장과 재개를 가능하게 한다.
- fork는 생성, exec는 주소 공간 교체, wait는 종료 대기·상태 수집이다.
- 좀비와 고아를 종료 상태 미수집과 부모 선행 종료로 구별한다.
- 공유 메모리의 데이터 형식·접근 동기화는 통신 프로세스의 책임이다.
- 메시지 명명·동기화·버퍼링은 독립된 설계 기준이다.
- 일반 파이프의 관계·범위와 지명 파이프의 운영체제별 차이를 구별한다.
- RPC 스텁은 정돈과 통신을 숨기지만 전달 오류와 바인딩 문제를 고려해야 한다.

혼동하기 쉬운 내용

IPC의 기본 모델

출처: 책 137-139쪽 / 원문 PDF 84-86쪽

항목	공유 메모리	메시지 전달
교환 단위	공유 영역의 데이터	송수신 메시지
커널 관여	영역 설정 후 일반 메모리 접근	통상 각 통신에 시스템 콜 사용
책임	자료 형식과 동시 접근 조정	이름·봉쇄·버퍼링 규칙 선택

좀비와 고아

출처: 책 135-136쪽 / 원문 PDF 82-83쪽

항목	좀비	고아
핵심 조건	종료한 자식의 상태를 부모가 아직 수집하지 않음	부모가 자식보다 먼저 종료
처리	부모의 wait가 항목을 회수	다른 부모가 인계받아 종료 상태를 수집

장 마무리 복습

- 프로세스 표현 → 큐 이동 → 문맥 교환의 연결을 따라 실행과 재개를 설명한다.
- 생성과 프로그램 교체는 다른 연산이며 자원 복사·공유와 부모 대기 조건을 구별한다.
- 공유 메모리의 접근 책임과 메시지 전달의 링크 설계 선택을 연결한다.
- 소켓의 바이트 스트림보다 높은 추상화를 RPC가 제공하며 스텁·정돈·바인딩이 이를 구현한다.

원문 확인 메모

- 소비자 공유 메모리 예제의 열기·매핑 플래그 조합은 원문 그대로 보존하고 원문 확인 필요로 표시했다.

제4장. 스레드와 병행성

출처: 책 175-218쪽 / 원문 PDF 121-164쪽

학습 주제

- 프로세스 자원을 공유하는 스레드와 코어별 병렬 실행을 이해한다. 사용자·커널 스레드의 매핑, 스레드 API와 암묵적 스레딩을 살펴본 뒤 신호·취소·스레드 로컬 저장·커널 협력 문제를 정리한다.

설명의 흐름

1. 스레드 구성과 이점 → 병행·병렬 및 다중 코어 과제 → 스레드 매핑 → 라이브러리 → 작업 중심의 암묵적 스레딩 → 다중 스레드의 주의점 → Windows·Linux 구현

4.1 개요

출처: 책 176-178쪽 / 원문 PDF 122-124쪽

스레드는 CPU 이용의 기본 단위이며 스레드 ID·프로그램 카운터·레지스터 집합·스택으로 구성된다. 같은 프로세스의 스레드는 코드·데이터·열린 파일·신호 같은 자원을 공유한다. 단일 스레드는 하나의 제어 흐름, 다중 스레드는 같은 프로세스 안의 여러 제어 흐름을 제공한다.

원문은 브라우저의 표시와 네트워크 검색, 문서 편집기의 입력·철자 검사, 이미지 축소판 생성 등을 예로 든다. 웹 서버는 요청을 듣는 스레드와 요청을 처리하는 스레드를 나눌 수 있다. 같은 작업을 위해 프로세스를 매번 새로 만드는 것보다 자원을 공유하는 스레드가 효율적일 수 있다. 커널도 장치·메모리 관리·인터럽트 처리를 담당하는 여러 스레드로 구성될 수 있다.

이점	의미와 조건
응답성	일부가 봉쇄되거나 긴 연산을 해도 다른 실행 흐름이 사용자에게 응답할 수 있다.

이점	의미와 조건
자원 공유	프로세스 간에는 명시적 IPC가 필요하지만 스레드는 같은 주소 공간과 자원을 공유한다.
경제성	프로세스보다 생성에 드는 시간·메모리가 적고 스레드 간 문맥 교환이 일반적으로 빠르다.
규모 적응성	다중 처리기에서 스레드를 다른 처리기에 병렬 배치할 수 있다. 단일 스레드는 처리기가 많아도 하나에서만 실행한다.

4.2 다중 코어 프로그래밍

출처: 책 178-182쪽 / 원문 PDF 124-128쪽

병행성은 여러 작업이 모두 진행되도록 지원하는 것이고 **병렬성**은 여러 작업을 같은 순간 수행하는 것이다. 단일 코어에서는 실행을 교대로 끼워 넣어 병행성을 제공할 수 있지만 병렬 실행은 하지 못한다. 다중 코어는 각 코어에 다른 스레드를 배치하여 실제 동시 실행을 가능하게 한다.

Amdahl의 법칙은 순차 부분과 병렬 부분이 있는 응용에 코어를 추가할 때 가능한 성능 이득을 나타낸다. 원문에서 S는 반드시 순차 실행되는 비율이고 N은 처리 코어 수이다.

원문 코드·기호

```
speedup ≤ 1 / (S + (1 − S) / N)
```

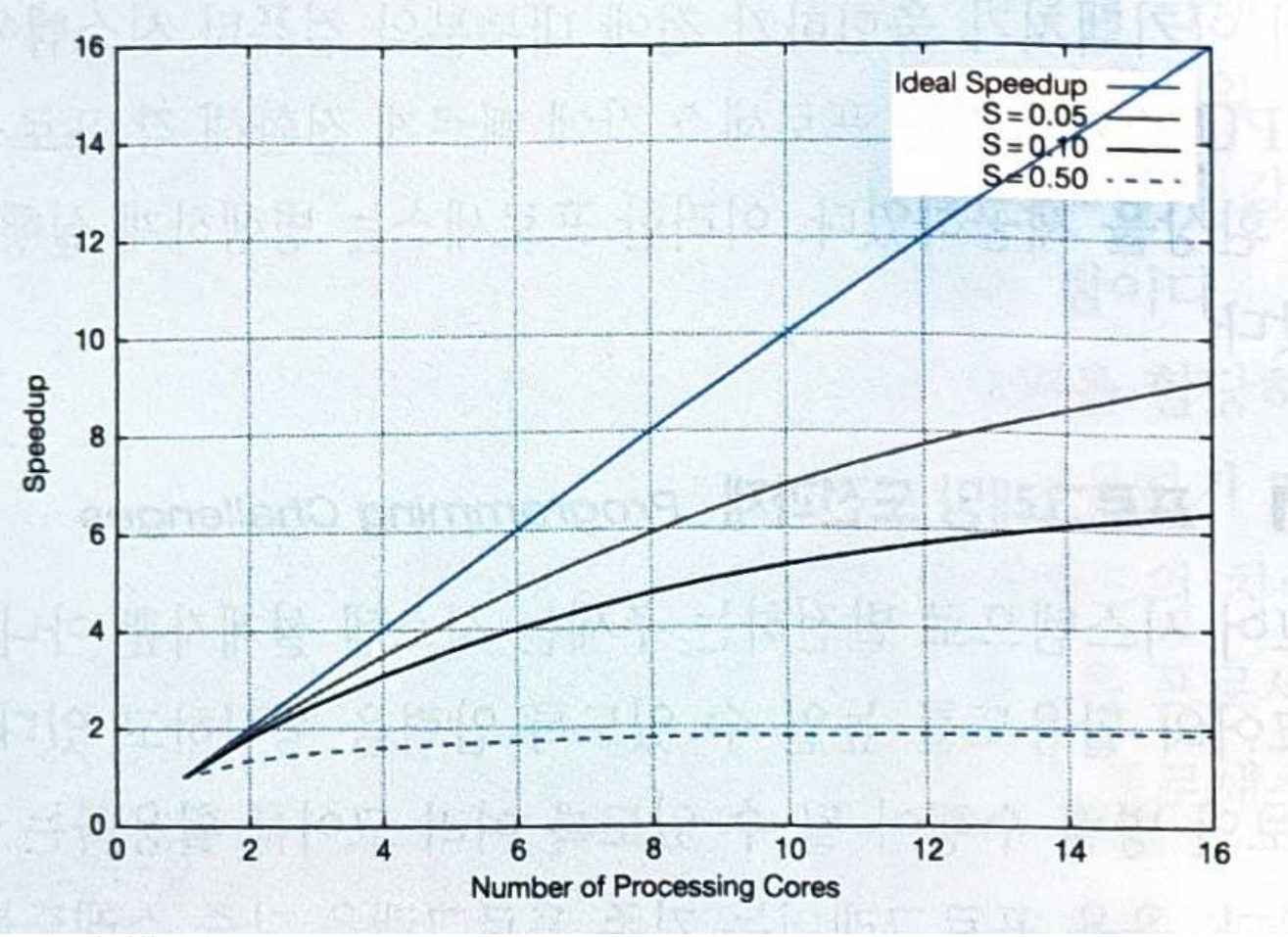
- S: 반드시 순차적으로 실행되어야 하는 부분

- N: 처리 코어 수

원문 예는 병렬 75%·순차 25%에서 2코어 약 1.6배, 4코어 2.28배를 제시한다. N이 무한히 커지면 1/S에 수렴하므로 순차 부분이 성능 향상을 제한한다.

Amdahl의 법칙 그래프

그림·표 출처: 책 180쪽 / 원문 PDF 126쪽



Amdahl의 법칙 그래프

코어 증가에 따른 속도 향상은 순차 부분 S의 크기에 제한됨을 보여 준다. 원문의 범례·축·곡선을 그대로 보존하였다.

원문 확인 필요: 같은 보충 설명은 순차 부분이 40%인 경우 최대 2.0배라고 기재한다. 제시된 1/S 설명과 수치가 일치하지 않으므로 이 문구를 정답 수치로 일반화하지 않고 원문 확인이 필요한 부분으로 남긴다. 임의로 수치를 고치지 않았다.

4.2.1 프로그래밍 도전과제

출처: 책 179-181쪽 / 원문 PDF 125-127쪽

- 태스크 인식: 독립적으로 병행 실행할 수 있는 부분을 찾는다.
- 균형: 나누어진 태스크가 전체 작업에 균등하게 기여하게 한다. 기여가 작은 작업에 별도 코어를 쓰는 것이 항상 유리하지 않다.
- 데이터 분리: 각 코어의 태스크가 처리할 데이터를 나눈다.
- 데이터 종속성: 한 태스크가 다른 태스크의 데이터에 의존하면 실행을 동기화한다.
- 시험·디버깅: 병렬 실행 경로가 다양하여 단일 스레드보다 검증이 어렵다.

4.2.2 병렬 실행의 유형

출처: 책 181-182쪽 / 원문 PDF 127-128쪽

유형	분배하는 것	원문 예
데이터 병렬	데이터의 부분집합. 각 코어는 같은 연산을 한다.	배열의 앞·뒤 부분을 다른 코어가 각각 합산한다.
태스크 병렬	각자 다른 연산을 수행하는 태스크	같은 배열에 대해 서로 다른 통계 연산을 수행한다.

두 방식은 상호 배타적이지 않으므로 한 응용이 혼합하여 사용할 수 있다. 데이터를 나누는지, 수행할 연산 자체를 나누는지를 구별한다.

4.3 다중 스레드 모델

출처: 책 182-185쪽 / 원문 PDF 128-131쪽

사용자 스레드는 커널 위에서 라이브러리가 관리하고 **커널 스레드**는 운영체제가 직접 관리한다. CPU에서 실행되려면 두 수준 사이의 매핑이 필요하다.

모델	매핑·이점	제약
다대일	여러 사용자 스레드 → 커널 스레드 하나. 사용자 공간 관리가 효율적이다.	하나의 봉쇄형 시스템 콜이 전체를 봉쇄한다. 여러 코어에서 병렬 실행할 수 없다.
일대일	사용자 스레드마다 커널 스레드 하나. 한 스레드 봉쇄 중 다른 스레드가 실행되고 병렬 실행 가능하다.	사용자 스레드 생성마다 커널 스레드가 필요하여 과도한 개수는 부담이 된다.
다대다	많은 사용자 스레드 → 같거나 더 적은 커널 스레드. 커널이 다른 실행 스레드를 스케줄할 수 있다.	융통성이 있지만 구현이 어렵다.
두 수준	다대다를 기본으로 하면서 특정 사용자 스레드를 특정 커널 스레드에 고정할 수 있다.	사용자·커널 수준의 협력이 필요하다.

원문은 Linux와 Windows의 일대일 모델 및 작업을 하부 스레드에 매핑하는 현대 병행 라이브러리를 대비한다. 커널 스레드 수를 제한하는 필요성과 병렬성·구현 복잡도를 함께 고려한다.

4.4 스레드 라이브러리

출처: 책 185-186쪽 / 원문 PDF 131-132쪽

스레드 라이브러리는 생성·관리를 위한 API를 제공한다. 사용자 공간만의 라이브러리는 지역 함수 호출로 구현되고, 커널 지원 라이브러리는 호출이 커널 시스템 콜로 이어진다. 원문은 Pthreads·Windows·Java를 설명한다. Pthreads는 사용자 또는 커널 수준 구현이 가능하며 Windows 라이브러리는 커널 수준이다. Java는 보통 호스트 운영체제의 라이브러리를 사용하지만 JVM 명세가 구체적 매핑을 정하지는 않는다.

비동기 스레딩은 부모가 자식을 만든 뒤 독립적으로 계속 실행한다. **동기 스레딩**은 자식들이 모두 종료하여 조인할 때까지 부모가 기다린 뒤 결과를 결합한다. 원문의 합산 예제는 동기 방식을 사용한다. Pthreads와 Windows의 전역 변수는 같은 프로세스 스레드가 공유하지만 Java는 전역 데이터 개념 대신 명시적으로 공유를 조정한다.

4.4.1 Pthreads

출처: 책 186-188쪽 / 원문 PDF 132-134쪽

Pthreads는 POSIX가 정한 스레드 생성·동기화 **동작의 명세**이며 그 자체가 하나의 구현은 아니다. 원문 합산 프로그램의 main과 runner는 sum을 공유한다. pthread_attr_init()으로 기본 속성을 설정하고 pthread_create()에 식별자·속성·시작 함수·인수를 전달한다. pthread_join()은 합산 스레드 종료를 기다리며 runner는 pthread_exit()로 종료한다. 여러 스레드의 조인은 반복문에서 각각 기다릴 수 있다.

그림 4.11 Pthreads 합산 프로그램

코드 출처: 책 187쪽 / 원문 PDF 133쪽

원문 코드·기호

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid,NULL);

    printf("sum = %d\n",sum);
}

/* The thread will execute in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

코드의 의미

main은 runner를 별도 스레드로 만들고 종료를 기다린 뒤 공유 sum을 출력한다.

동작 과정

1. 명령행 상한값을 runner의 인수로 전달한다.
2. runner가 1부터 upper까지 더하고 종료한다.

3. 부모는 pthread_join() 뒤 sum에 접근한다.

원문 실행 결과

원문에 실행 결과 미제시

4.4.2 Windows 스레드

출처: 책 188-190쪽 / 원문 PDF 134-136쪽

Windows의 합산 예제는 전역 Sum을 공유하고 Summation 함수에서 계산한다. CreateThread()에 보안 속성·스택 크기·시작 함수·인수·생성 플래그·식별자 수신 위치를 전달한다. 부모는 WaitForSingleObject()로 완료를 기다리고 핸들을 닫은 뒤 결과를 출력한다. 여러 스레드는 WaitForMultipleObjects()에 객체 수·핸들 배열·모든 객체 대기 여부·타임아웃을 지정하여 기다린다.

4.4.3 Java 스레드

출처: 책 190-192쪽 / 원문 PDF 136-138쪽

Java는 Thread를 상속하여 run()을 재정의하거나 Runnable 인터페이스를 구현한다. Runnable 객체를 전달해 Thread를 만들고 **start()**를 호출하면 새 스레드가 초기화되어 run()을 실행할 자격을 얻는다. 원문은 run()을 직접 호출하지 말 것을 명시한다. 부모는 join()으로 완료를 기다릴 수 있다. Runnable을 람다 표현식으로 작성하는 방법도 소개한다.

4.4.3.1 Java Executor 프레임워크

출처: 책 192-194쪽 / 원문 PDF 138-140쪽

Executor는 Runnable을 execute()에 전달하여 태스크 실행과 명시적인 Thread 생성·start 호출을 분리한다. 태스크를 생산하고 실행 스레드가 소비하는 모델이다. **Callable**은 결과를 반환할 수 있으며 submit()이 반환하는 **Future**의 get()으로 결과를 얻는다. execute()는 결과를 반환하지 않는다는 차이가 있다.

원문 합산 예제는 newSingleThreadExecutor()를 만들고 Callable 작업을 submit()한 뒤 Future.get()으로 결과를 기다린다. 이 구성은 스레드 생성과 결과를 분리하므로 반드시 스레드의 최종 종료까지가 아니라 결과가 사용 가능해질 때까지 기다리도록 한다.

4.5 암묵적 스레딩

출처: 책 194쪽 / 원문 PDF 140쪽

수백·수천 개 스레드를 직접 만들고 관리하는 부담을 줄이기 위해 생성·관리 책임을 컴파일러와 실행시간 라이브러리로 옮긴다. **암묵적 스레딩**에서 개발자는 병렬 가능한 작업을 식별하고 라이브러리가 이를 하부 스레드에 매핑한다. 정확성·자원 관리의 어려움 자체가 사라지는 것은 아니며 제6·8장의 문제와 연결된다.

4.5.1 스레드 풀

출처: 책 195-197쪽 / 원문 PDF 141-143쪽

스레드 풀 처리

1. 미리 만든 스레드들은 작업을 기다린다.
2. 요청이 오면 새 스레드 대신 작업을 풀에 제출한다.
3. 사용 가능한 스레드가 작업을 처리하고, 없으면 작업이 기다린다.
4. 작업을 마친 스레드는 풀로 돌아가 다음 작업을 기다린다.

기존 스레드 재사용은 생성 시간을 절약하고 동시 스레드 수를 제한하여 자원 고갈을 막는다. 작업과 실행 방법을 분리하면 지연 실행·주기 실행도 가능하다. 풀 크기는 CPU 수·메모리·최대 동시 요청 등에 따라 정하거나 활용도에 따라 동적으로 조절한다. 풀 제출 작업을 비동기 실행할 수 있는 경우에 잘 작동한다. 원문은 Windows QueueUserWorkItem()과 Android AIDL의 스레드 풀을 사례로 든다.

Java의 newSingleThreadExecutor()는 하나, newFixedThreadPool(size)는 지정된 수, newCachedThreadPool()은 재사용 가능한 무제한 풀을 만든다. shutdown() 이후 새 작업은 거부하고 이미 제출된 작업이 완료되면 종료한다.

4.5.2 Fork Join

출처: 책 197-201쪽 / 원문 PDF 143-147쪽

fork-join은 부모가 자식 작업을 나누고 완료를 기다린 뒤 결과를 결합하는 동기 모델이다. 암묵적 방식에서는 실제 스레드를 직접 만들지 않고 병렬 작업을 식별하며 라이브러리가 스레드 수와 배정을 정한다.

Java의 ForkJoinPool은 분할 정복 작업을 받는다. 충분히 작으면 직접 풀고, 그렇지 않으면 하위 문제를 fork한 뒤 join하여 결과를 합친다. ForkJoinTask의 하위 클래스인 RecursiveTask는 결과를 반환하고 RecursiveAction은 반환하지 않는다. 원문 SumTask 예제는 배열의 범위를 둘로 나누고 임계값 THRESHOLD=1000 아래에서 직접 합산한다. 언제 충분히 작다고 볼지는 실제 타이밍 시험이 필요하다.

작업 수가 스레드 수보다 훨씬 많을 수 있다. 각 작업 스레드는 자신의 큐를 유지하며 큐가 비면 **work stealing**으로 다른 큐의 작업을 가져와 부하를 분산한다.

4.5.3 OpenMP

출처: 책 201-203쪽 / 원문 PDF 147-149쪽

OpenMP는 C·C++·FORTRAN의 API와 컴파일러 디렉티브 집합으로 공유 메모리 병렬 프로그래밍을 돕는다. 개발자가 병렬 영역을 표시하면 실행시간 라이브러리가 스레드 실행을 관리한다. 원문은 parallel 영역과 parallel for로 반복을 분배하는 예를 제시한다. 스레드 수와 데이터의 공유·개별 사용 여부도 선택할 수 있다.

OpenMP 병렬 for

코드 출처: 책 202쪽 / 원문 PDF 148쪽

원문 병렬 반복문 전체 발췌이며 앞의 별도 parallel 영역 예제는 포함하지 않았다.

원문 코드·기호

```
#pragma omp parallel for
for (i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

코드의 의미

배열 a와 b의 원소 합을 c에 넣는 반복을 생성된 스레드들에 분배한다.

동작 과정

1. 디렉티브가 병렬화할 반복문을 지정한다.
2. 각 반복은 같은 인덱스의 a와 b를 더해 c에 저장한다.

원문 실행 결과

원문에 실행 결과 미제시

4.5.4 Grand Central Dispatch

출처: 책 203-204쪽 / 원문 PDF 149-150쪽

GCD는 macOS·iOS의 실행시간 라이브러리·API·언어 확장이다. 태스크를 디스패치 큐에 넣고 큐에서 꺼낼 때 관리 중인 풀의 가용 스레드에 배정한다. 직렬 큐는 FIFO로 꺼낸 하나가 끝나야 다음 작업을 실행한다. 병행 큐도 FIFO로 제거하지만 여러 태스크를 동시에 실행할 수 있다.

서비스 품질 클래스	원문의 역할
사용자 대화형	즉각적인 사용자 인터페이스·이벤트 반응. 작업량이 매우 작아야 한다.
사용자 시작	파일·URL 열기처럼 사용자가 시작하여 완료를 기다리는 비교적 긴 작업
유틸리티	데이터 가져오기처럼 오래 걸리지만 즉각적인 결과가 필요하지 않은 작업
백그라운드	색인·백업처럼 사용자에게 보이지 않고 시간에 민감하지 않은 작업

작업은 C 계열의 블록 또는 Swift closure로 나타낸다. GCD는 POSIX 스레드 풀의 크기를 응용 수요와 시스템 능력에 맞추어 적극적으로 늘리거나 줄인다.

4.5.5 Intel 스레드 빌딩 블록

출처: 책 204-206쪽 / 원문 PDF 150-152쪽

Intel TBB는 특별한 컴파일러나 언어 지원 없이 C++ 병렬 응용을 돕는 템플릿 라이브러리이다. 개발자는 태스크를 지정하고 스케줄러가 하부 스레드에 배정하며 부하 균형·캐시 인지를 제공한다. 병렬 루프·원자적 연산·상호 배제 잠금 및 병행 자료구조를 포함한다.

parallel_for는 반복 공간을 청크로 나누어 작업을 만들고 가용 스레드에 배정한다. 처리 코어 수에 맞춰 직접 배열을 분할하는 방식보다 하드웨어별 분할의 세부사항을 라이브러리에 맡길 수 있다. 원문은 apply(v[i])를 각 배열 원소에 적용하는 직렬 루프와 다음 템플릿 호출을 대비한다.

TBB parallel_for

코드 출처: 책 205쪽 / 원문 PDF 151쪽

원문 템플릿 호출 예제 전체 발췌이며 앞의 직렬 루프는 제외하였다.

원문 코드·기호

```
parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});
```

코드의 의미

반복 공간 0부터 n-1까지의 각 i에 대해 apply(v[i])를 실행한다.

동작 과정

1. 처음 두 인수가 반복 공간을 정한다.
2. 람다가 각 원소에 적용할 연산을 나타낸다.
3. 라이브러리가 청크와 작업·스레드 배정을 관리한다.

원문 실행 결과

원문에 실행 결과 미제시

4.6 스레드와 관련된 문제들

출처: 책 206-212쪽 / 원문 PDF 152-158쪽

4.6.1 fork() 및 exec() 시스템 콜

출처: 책 206쪽 / 원문 PDF 152쪽

다중 스레드에서 fork()가 모든 스레드를 복제할지 호출한 스레드만 복제할지 고려해야 한다. 원문은 두 버전을 제공하는 UNIX를 설명한다. 곧 exec()를 할 경우 전체 프로세스가 대체되므로 모든 스레드 복제는 불필요하고, 복사본을 그대로 실행할 경우 모든 스레드가 필요하다. 어느 스레드가 exec()를 호출하든 지정한 새 프로그램은 모든 스레드를 포함한 전체 프로세스를 대체한다.

4.6.2 신호 처리

출처: 책 206-208쪽 / 원문 PDF 152-154쪽

UNIX **신호**는 특정 이벤트에서 생성되고 프로세스에 전달되어 처리된다. 동기식 신호는 불법 메모리 접근·0으로 나누기처럼 실행 연산에서 생겨 그 연산을 한 스레드에 전달되어야 한다. 비동기 신호는 외부의 종료 요청·타이머 만료처럼 발생한다. 커널의 기본 처리기를 사용자 정의 처리기로 대체할 수 있고 신호에 따라 무시하거나 종료할 수 있다.

다중 스레드에서는 적용 스레드·모든 스레드·일부·전담 스레드 중 전달 대상을 정해야 한다. 비동기 신호는 허용된 스레드에 전달될 수 있지만 한 번만 처리해야 한다. kill()은 프로세스, pthread_kill()은 특정 스레드를 지정한다. Windows의 APC는 지정된 스레드에 이벤트 처리 함수를 호출하므로 UNIX의 프로세스 전체 신호 처리와 다르다.

4.6.3 스레드 취소

출처: 책 208-210쪽 / 원문 PDF 154-156쪽

방식	동작	주의
비동기 취소	목적 스레드를 즉시 종료	공유 자료 갱신 중단과 자원 회수 누락 문제가 생길 수 있다.
지연 취소	취소 요청을 표시하고 목적 스레드가 안전한 시점에 확인	질서 있게 정리할 기회를 준다.

pthread_cancel()은 취소 요청을 표시할 뿐이며 실제 동작은 목적 스레드의 설정에 달린다. 취소 비활성 상태에서는 요청이 보류될 수 있다. 기본 지연 취소는 취소 점에 도달할 때 발생한다. 봉쇄형 read() 등이 취소 점이며 pthread_testcancel()로 직접 검사할 수 있다. 요청이 있으면 이 함수는 복귀하지 않고 종료한다. 정리 핸들러는 종료 전에 자원을 해제하도록 한다.

원문은 Pthreads의 비동기 취소를 권장하지 않는다고 설명한다. Java는 interrupt()로 인터럽트 상태를 설정하고 스레드가 isInterrupted()로 검사하는 지연 취소와 유사한 정책을 사용한다. 요청을 보냈다는 사실과 실제 종료했다는 사실을 구별해야 한다.

4.6.4 스레드 로컬 저장장치

출처: 책 210-211쪽 / 원문 PDF 156-157쪽

TLS는 스레드마다 자신만 접근할 고유 데이터를 제공한다. 지역 변수는 한 함수 호출 동안만 보이지만 TLS는 여러 함수 호출에 걸쳐 보존된다. 정적 데이터와 비슷하지만 각 스레드마다 고유하다는 차이가 있다. 생성 과정을 직접 제어하지 못하는 풀에서도 스레드별 식별자 등을 저장할 수 있다. 원문은 Java ThreadLocal, Pthreads 키, C# ThreadStatic, gcc __thread를 소개한다.

4.6.5 스케줄러 액티베이션

출처: 책 211-212쪽 / 원문 PDF 157-158쪽

다대다·두 수준 모델에서는 사용자 라이브러리와 커널이 협력하여 필요한 커널 스레드 수를 조정해야 한다. **LWP(경량 프로세스)**는 사용자 라이브러리에 가상 처리기처럼 보이고 각 LWP는 커널 스레드 하나에 연결된다. 커널 스레드가 봉쇄되면 해당 LWP와 그 위의 사용자 스레드도 봉쇄된다.

봉쇄와 해제를 알리는 upcall

1. 커널이 스레드가 봉쇄됨과 식별자를 라이브러리에 알리고 새 가상 처리기를 제공한다.
2. upcall 처리기가 상태를 저장하고 다른 실행 가능 스레드를 스케줄한다.
3. 대기 이벤트가 끝나면 커널이 다시 알린다.
4. 처리기가 이전 스레드를 실행 가능으로 표시하고 가용 가상 처리기에서 실행하도록 한다.

입출력 중심 응용은 동시에 봉쇄되는 호출을 감당할 충분한 LWP가 필요할 수 있다. 원문 예에서 파일 읽기 5개에 LWP가 4개뿐이면 다섯 번째 요청은 한 LWP가 돌아올 때까지 기다린다.

4.7 운영체제 사례

출처: 책 213-214쪽 / 원문 PDF 159-160쪽

4.7.1 Windows 스레드

출처: 책 213-214쪽 / 원문 PDF 159-160쪽

Windows는 사용자 스레드 하나에 커널 스레드 하나를 대응시킨다. 스레드는 식별자·레지스터·프로그램 카운터·사용자 및 커널 스택·개별 저장 영역을 가진다. **ETHREAD**는 소속 프로세스·시작 루틴·KTHREAD 포인터, **KTHREAD**는 스케줄링·동기화·커널 스택·TEB 포인터를 갖는다. 둘은 커널 공간이다. **TEB**는 사용자 공간에서 식별자·사용자 스택·TLS 배열을 제공한다.

그림 4.21 Windows 스레드의 자료구조

그림·표 출처: 책 214쪽 / 원문 PDF 160쪽

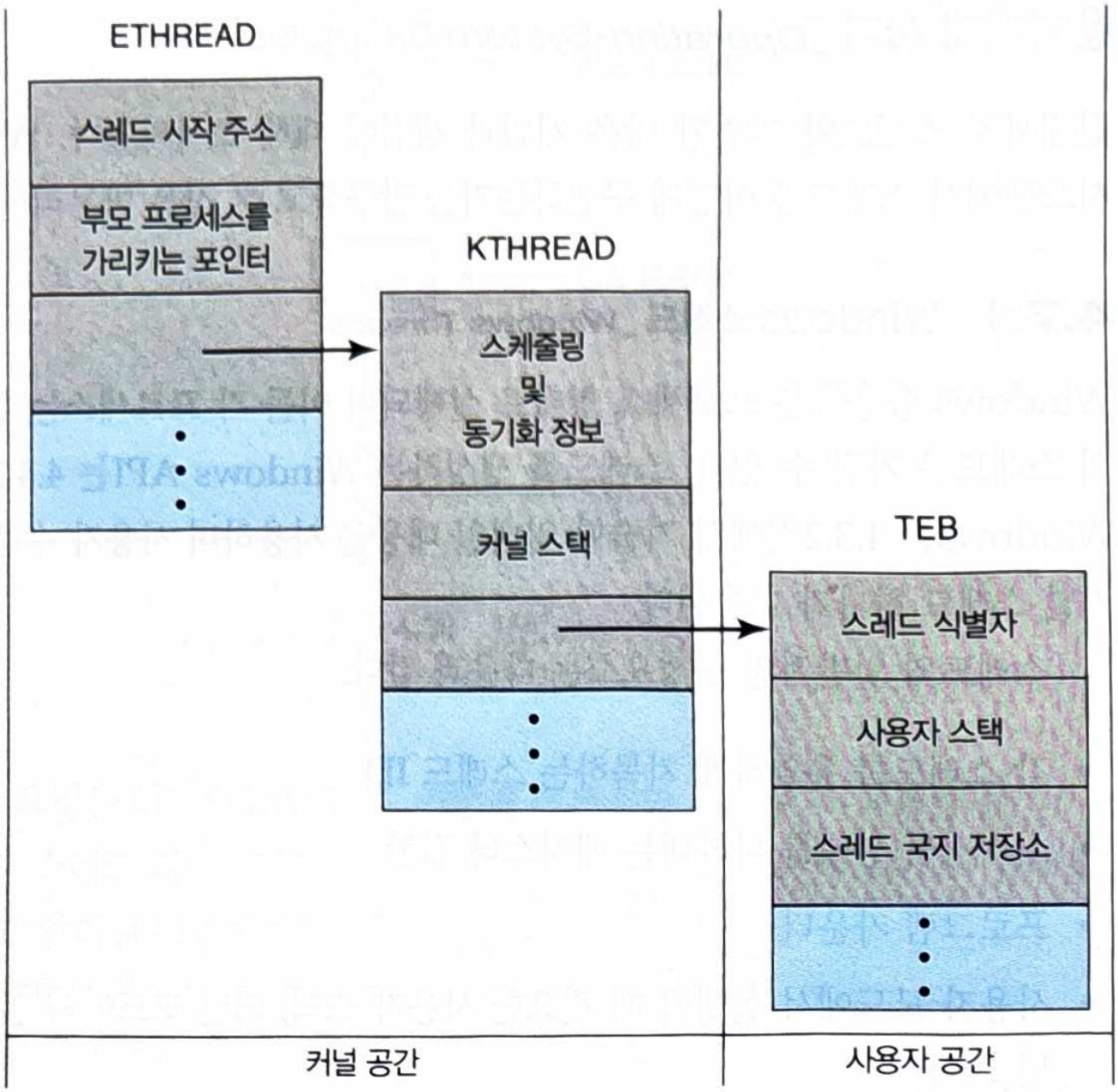


그림 4.21 Windows 스레드의 자료구조

그림 4.21 Windows 스레드의 자료구조

ETHREAD→KTHREAD→TEB의 포인터와 커널·사용자 공간 경계를 보여 준다.

4.7.2 Linux 스레드

출처: 책 214-215쪽 / 원문 PDF 160-161쪽

Linux는 프로세스와 스레드를 모두 **태스크**로 다룬다. clone()의 플래그가 부모·자식의 공유 범위를 결정한다. CLONE_FS는 파일 시스템 정보, CLONE_VM은 메모리 공간, CLONE_SIGHAND는 신호 처리기, CLONE_FILES는 열린 파일 집합을 공유한다. 여러 자료구조를 공유하면 스레드처럼, 공유하지 않으면 fork()에 가까운 프로세스처럼 동작한다.

task_struct가 데이터 자체가 아니라 관련 자료구조의 포인터를 가지므로 필요한 부분만 같은 구조를 가리키게 할 수 있다. 원문은 clone의 다른 플래그가 하나의 Linux 커널 아래 격리된 시스템인 컨테이너 생성에도 연결된다고 설명한다.

4.8 요약과 원문 복습 범위

출처: 책 215-218쪽 / 원문 PDF 161-164쪽

스레드의 공유 이점과 병렬화의 어려움은 함께 고려해야 한다. 매핑 모델과 스레딩 도구는 생성·관리 책임을 어디에 두는지 구별한다. 장말 연습문제 4.1~4.7은 성능 이점·Amdahl·병렬 유형·사용자와 커널 스레드·문맥 교환·생성 자원·LWP를 검토한다. 새 사례나 계산 풀이를 추가하지 않았고 추가 자료·참고문헌의 외부 내용은 사용하지 않았다.

핵심 정리

- 스레드는 레지스터·스택 등 실행 상태를 개별 보유하고 프로세스의 코드·데이터·자원을 공유한다.
- 병행은 진행, 병렬은 실제 동시 실행이며 순차 부분이 코어 추가의 이득을 제한한다.
- 매핑 모델의 봉쇄·병렬성·생성 비용을 함께 구별한다.
- 암묵적 스레딩은 개발자가 작업을 식별하고 라이브러리가 스레드 배정을 맡는 방식이다.
- 취소 요청과 실제 종료는 다르며 공유 상태와 자원의 안전한 정리가 필요하다.
- TLS는 함수 지역 변수와 다르며 스레드별로 고유한 데이터를 함수 호출에 걸쳐 제공한다.

혼동하기 쉬운 내용

데이터 병렬과 태스크 병렬

출처: 책 181-182쪽 / 원문 PDF 127-128쪽

기준	데이터 병렬	태스크 병렬
분할 대상	데이터 부분집합	서로 다른 작업
코어별 연산	동일	고유 연산
관계	두 방법을 혼합할 수 있다.	두 방법을 혼합할 수 있다.

비동기 취소와 지연 취소

출처: 책 208-210쪽 / 원문 PDF 154-156쪽

기준	비동기	지연
시점	즉시 종료	안전한 취소 점에서 요청을 확인
자원·공유 상태	갱신 중 중단·자원 미반환 위험	정리 기회를 제공

장 마무리 복습

- 응답성·자원 공유·경제성·규모 적응성을 원문의 사례와 연결한다.
- 독립 작업 분할만으로 충분하지 않으며 균형·데이터 분할·종속성·검증도 필요하다.
- 스레드 API의 생성·조인과 Executor·풀·fork-join의 작업 제출을 구별한다.

- 다중 스레드 fork·exec·신호·취소·TLS·LWP는 공유와 개별 실행 상태가 만나는 지점이다.

원문 확인 메모

- Amdahl 보충 설명의 순차 비율 40%와 최대 2.0배 문구는 제시 공식과 불일치하여 원문 확인 필요로 남겼다.

제5장. CPU 스케줄링

출처: 책 219-279쪽 / 원문 PDF 165-225쪽

학습 주제

- CPU를 어떤 준비 프로세스·스레드에 언제 할당할지 정하는 정책과 평가 기준을 연결한다. 기본 알고리즘을 수치 예제로 비교하고 다중 처리기·실시간 조건 및 운영체제 구현 사례로 확장한다.

설명의 흐름

1. 버스트·선점·디스패치 → 평가 기준 → 기본 스케줄링 → 스레드 경쟁 범위 → 다중 코어의 부하·선호도 → 실시간 기한 → 실제 운영체제 → 모델링과 평가

5.1 기본 개념

출처: 책 219-220쪽 / 원문 PDF 165-166쪽

다중 프로그래밍은 한 프로세스가 입출력 등을 기다리는 동안 다른 프로세스에 CPU를 할당하여 이용률을 높인다. 코어가 하나면 한순간 하나만 실행하고 다중 코어에서는 이 개념을 각 코어에 적용한다. 현대 운영체제에서 실제 스케줄 대상은 보통 커널 스레드이다. 원문은 일반적 논의에서는 프로세스 스케줄링이라는 표현을 쓰고 스레드에 특화된 경우 이를 구별한다.

5.1.1 CPU·I/O 버스트 사이클

출처: 책 220-222쪽 / 원문 PDF 166-168쪽

프로세스 실행은 **CPU 버스트**와 **I/O 버스트**가 교대로 나타나는 사이클이다. 마지막 CPU 버스트는 다음 입출력 대신 종료 요청으로 끝난다. 관측상 짧은 CPU 버스트는 많고 긴 버스트는 적다. I/O 중심 프로그램은 짧은 CPU 버스트가 많고 CPU 중심 프로그램은 긴 버스트를 가질 수 있으므로 이 분포가 알고리즘 선택에 중요하다.

5.1.2 CPU 스케줄러

출처: 책 220-221쪽 / 원문 PDF 166-167쪽

CPU 스케줄러는 준비된 메모리 내 프로세스 가운데 다음 실행 대상을 선택한다. 준비 큐는 반드시 FIFO가 아니며 우선순위 큐·트리·순서 없는 연결 리스트도 가능하다. 큐의 레코드는 일반적으로 PCB이다. 준비 상태라는 의미와 구현 자료구조를 혼동하지 않아야 한다.

5.1.3 선점 및 비선점 스케줄링

출처: 책 221-223쪽 / 원문 PDF 167-169쪽

결정이 가능한 상황	선택의 성격
실행 → 대기	현재 프로세스가 CPU를 내놓았으므로 다른 준비 프로세스를 선택해야 한다.
실행 → 준비	현재 프로세스의 계속 실행과 다른 프로세스 사이에서 선택할 수 있다.
대기 → 준비	새로 준비된 프로세스와 현재 실행 프로세스를 비교할 수 있다.
종료	실행할 다른 준비 프로세스를 선택해야 한다.

첫 번째와 네 번째 경우에만 교체하면 **비선점(협조적)** 스케줄링이다. 할당된 CPU는 종료하거나 대기로 전환할 때까지 유지한다. 다른 상황에서도 CPU를 뺏을 수 있으면 **선점**이다. 선점은 공유 자료 갱신 도중 다른 실행이 개입하여 경쟁 조건을 만들 수 있다.

비선점형 커널은 구조가 단순해지지만 실시간 응답에 불리하다. 선점형 커널은 공유 커널 자료구조를 mutex 등으로 보호해야 한다. 인터럽트의 영향을 받는 짧은 코드 부분에서 인터럽트를 잠시 끌 수 있지만 자주 사용하거나 긴 명령 구간에 적용하면 안 된다.

5.1.4 디스패처

출처: 책 223-224쪽 / 원문 PDF 169-170쪽

디스패처는 스케줄러가 선택한 프로세스에 실제 CPU 제어를 넘긴다. 문맥 교환, 사용자 모드 전환, 사용자 프로그램의 적절한 재개 위치로 이동하는 작업을 수행한다. 하나를 멈추고 다음 실행을 시작할 때까지의 시간이 **디스패치 지연**이므로 빠르게 수행되어야 한다.

원문 Linux 예제는 vmstat로 시스템의 문맥 교환을, /proc의 프로세스 상태 파일로 개별 횟수를 관찰한다. **자발적 교환**은 현재 사용할 수 없는 자원을 요청하여 CPU를 포기한 경우이고 **비자발적 교환**은 시간 할당량 만료·높은 우선순위 선점 등으로 빼앗긴 경우이다.

5.2 스케줄링 기준

출처: 책 225-226쪽 / 원문 PDF 171-172쪽

기준	정의	일반 목표
CPU 이용률	CPU가 바쁘게 일하는 비율	최대화
처리량	단위 시간에 완료한 프로세스 수	최대화
총처리 시간	제출부터 완료까지. 준비 대기·CPU 실행·입출력 시간을 포함	최소화
대기 시간	준비 큐에서 보낸 시간의 합	최소화
응답 시간	요청 제출부터 첫 응답이 시작될 때까지	최소화

첫 응답까지와 응답을 모두 출력하는 데 걸리는 시간은 다르다. 평균뿐 아니라 최댓값·최솟값이나 응답 시간의 편차가 중요한 환경도 있다. 원문의 기본 수치 예제는 단일 코어, 프로세스당 하나의 CPU 버스트,

밀리초 단위 및 평균대기 시간이라는 단순화 조건을 사용한다. 이 조건을 실제 모든 실행의 성능 보장으로 확대하지 않는다.

5.3 스케줄링 알고리즘

출처: 책 226-238쪽 / 원문 PDF 172-184쪽

5.3.1 선입 선처리 스케줄링

출처: 책 226-228쪽 / 원문 PDF 172-174쪽

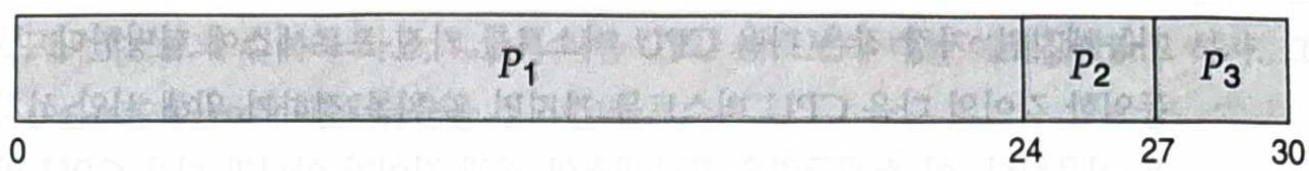
FCFS는 CPU를 먼저 요청한 순서로 할당한다. 준비 큐의 꼬리에 PCB를 넣고 머리에서 선택하는 FIFO로 구현할 수 있어 단순하다. 비선점이므로 현재 버스트를 마치거나 대기할 때까지 CPU를 점유한다. 긴 작업 뒤에 짧은 작업이 몰리면 평균대기 시간이 길어진다.

원문의 동시 도착 예	P1	P2	P3
CPU 버스트(ms)	24	3	3
P1→P2→P3일 때 대기	0	24	27
P2→P3→P1일 때 대기	6	0	3

첫 순서는 평균 (0 + 24 + 27)/3 = 17ms, 두 번째는 (6 + 0 + 3)/3 = 3ms이다. 같은 버스트 집합도 도착 순서에 따라 크게 달라진다. 아래는 원문 첫 순서의 간트차트이다.

FCFS의 원문 간트차트

그림·표 출처: 책 227쪽 / 원문 PDF 173쪽



FCFS의 원문 간트차트

원문 버스트 P1=24, P2=3, P3=3의 첫 도착 순서이다. 0·24·27·30 경계를 보존한다.

호위 효과는 긴 CPU 중심 프로세스가 CPU를 보유하는 동안 여러 I/O 중심 프로세스가 기다리는 현상이다. 이때 장치는 쉬고, CPU 중심 작업이 입출력으로 옮기면 짧은 작업들이 빨리 실행·재대기하여 이번에는 CPU가 쉬게 될 수 있다. 짧은 프로세스를 먼저 처리할 때보다 CPU·장치 이용률이 낮아지는 이유이다. 대화형 시스템에서 규칙적인 CPU 몫을 제공하기 어렵다는 한계도 있다.

5.3.2 최단 작업 우선 스케줄링

출처: 책 228-230쪽 / 원문 PDF 174-176쪽

SJF는 전체 작업 길이가 아니라 **다음 CPU 버스트**가 가장 짧은 프로세스를 고른다. 길이가 같으면 FCFS로 정한다. 주어진 작업 집합에서 짧은 버스트를 앞당기면 짧은 작업의 대기 감소가 긴 작업의 증가보다 커서 평균대기 시간을 최소화한다. 그러나 실제 다음 버스트를 미리 알 수 없으므로 예측이 필요하다.

프로세스	P1	P2	P3	P4
원문 버스트(ms)	6	8	7	3

프로세스	P1	P2	P3	P4
SJF 대기(ms)	3	16	9	0

동시 도착한 위 예의 실행 순서는 P4→P1→P3→P2이고 구간 경계는 0·3·9·16·24이다. 평균대기는 (3 + 16 + 9 + 0)/4 = 7ms이며 원문 FCFS 비교값은 10.25ms이다.

원문 코드·기호

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\tau_n, \quad 0 \le \alpha \le 1$$

- t_n: n번째 실제 CPU 버스트 길이
- τ_n: 앞서 축적한 예측값
- τ_{n+1}: 다음 CPU 버스트 예측
- α: 최근 실제값과 과거 예측에 줄 상대 가중치

α=0이면 과거 예측만, α=1이면 가장 최근 버스트만 반영한다. α=1/2이면 둘에 같은 무게를 준다. 원문 그림은 α=1/2, 초기 예측 10을 사용한다.

재귀적으로 대입하면 오래된 버스트에 대한 가중치는 (1-α)의 거듭제곱으로 감소한다. 원문의 전개식은 아래에 그대로 보존한다.

지수 평균의 원문 전개식

그림·표 출처: 책 230쪽 / 원문 PDF 176쪽

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \cdots + (1-\alpha)^j \alpha t_{n-j} + \cdots + (1-\alpha)^{n+1}\tau_0.$$

지수 평균의 원문 전개식

이전 예측을 재귀 대입하여 과거 버스트의 가중치가 감소함을 보이는 원문 수식을 보존한다.

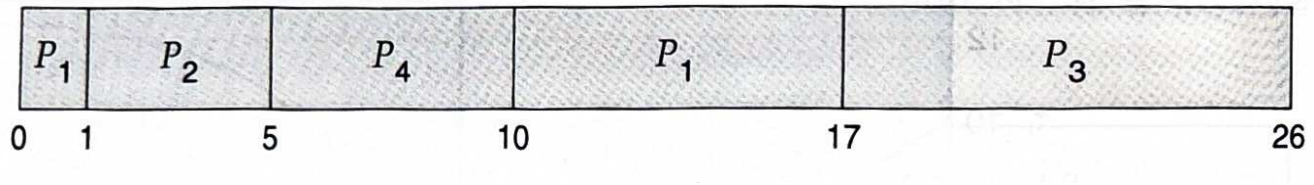
선점형 SJF는 새로 도착한 버스트와 현재 남은 시간을 비교하여 더 짧은 쪽을 실행하는 **최소 잔여 시간 우선(SRTF)**이다. 비선점형은 현재 버스트가 끝나도록 한다.

프로세스	P1	P2	P3	P4
도착(ms)	0	1	2	3
버스트(ms)	8	4	9	5

원문 예에서 P1이 시간 0에 시작하지만 시간 1에 남은 7보다 짧은 4의 P2가 도착하여 선점한다. 실행 순서는 P1(0~1)→P2(1~5)→P4(5~10)→P1(10~17)→P3(17~26)이다. 평균대기는 [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 6.5ms이고 비선점 비교값은 7.75ms이다.

선점형 SJF의 원문 간트차트

그림·표 출처: 책 230쪽 / 원문 PDF 176쪽



선점형 SJF의 원문 간트차트

도착과 남은 시간을 비교하여 P1→P2→P4→P1→P3로 전환하는 원문 차트이다.

5.3.3 라운드 로빈 스케줄링

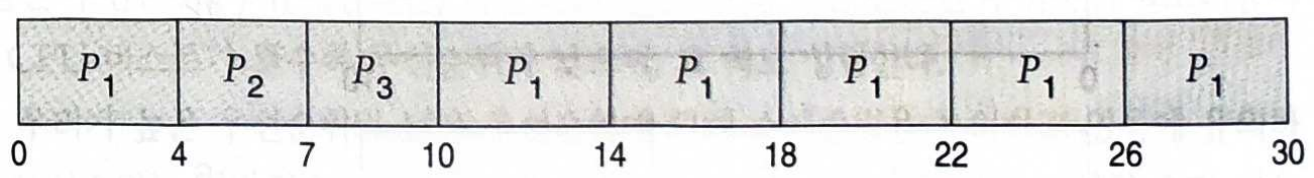
출처: 책 230-233쪽 / 원문 PDF 176-179쪽

RR은 FCFS에 시간 할당량 q와 선점을 추가한다. 준비 큐를 원형으로 돌며 각 프로세스에 한 번씩 q를 준다. q 이전에 버스트가 끝나면 자발적으로 넘기고, 길면 타이머 인터럽트 후 큐 꼬리로 돌아간다. 실행 가능한 프로세스가 하나가 아니면 연속 두 몫을 받지 않는다.

원문의 P1=24, P2=P3=3 및 q=4 예에서는 P1(0~4)→P2(4~7)→P3(7~10) 후 P1이 나머지를 수행한다. 대기는 6·4·7이고 평균은 17/3 = 5.66ms로 제시된다. 원문의 시간 경계를 보존한 차트는 아래와 같다.

RR의 원문 간트차트(q=4ms)

그림·표 출처: 책 231쪽 / 원문 PDF 177쪽



RR의 원문 간트차트(q=4ms)

P1=24, P2=P3=3의 순환 실행이며 각 시간 경계를 원문 그대로 보존하였다.

원문 코드·기호

```
다음 몫까지의 대기 상한: (n − 1)q
```

- n: 준비 큐의 프로세스 수

- q: 각 시간 할당량

각 프로세스가 최대 q 덩어리로 CPU의 1/n을 나누어 받는 원문 조건이다. 예를 들어 n=5, q=20ms이면 매 100ms마다 최대 20ms를 받는다.

q가 매우 크면 FCFS와 같아지고 너무 작으면 문맥 교환 오버헤드가 커진다. 따라서 q는 문맥 교환 시간보다 충분히 커야 하지만 과도하게 커서도 안 된다. q 증가가 평균 총처리 시간을 반드시 개선하는 것은 아니다. 많은 버스트가 한 몫 안에 끝나면 개선될 수 있다. 원문은 CPU 버스트의 약 80%가 q보다 짧아야 한다는 경험 기준을 제시한다.

5.3.4 우선순위 스케줄링

출처: 책 233-235쪽 / 원문 PDF 179-181쪽

우선순위 스케줄링은 가장 높은 우선순위 프로세스에 CPU를 준다. 같은 순위는 FCFS 또는 RR로 처리할 수 있다. 예상 다음 버스트의 역을 우선순위로 쓰는 SJF는 이 방식의 특수한 경우이다. 숫자가 크거나 작을 때 어느 쪽이 높은 우선순위인지는 시스템마다 다르며 원문의 기본 예는 낮은 숫자가 높은 순위를 뜻한다.

내부 우선순위는 시간 제한·메모리 요구·열린 파일 수·I/O 대 CPU 비율처럼 측정 가능한 값에서, 외부 우선순위는 중요성·비용 등의 외부 기준에서 정한다. 선점형은 더 높은 순위 도착 시 현재 실행을 뺏고 비선점형은 다음 선택 때 반영한다.

주요 한계는 낮은 순위가 끝없이 기다리는 **기아(무한 봉쇄)**이다. **노화(aging)**는 오래 대기한 프로세스의 우선순위를 점차 높여 이를 완화한다. 같은 순위에 RR을 쓰면 우선순위 집단 안에서도 몫을 나눈다. 원문은 최고 순위가 먼저 끝나고 다음 동일 순위 쌍이 q=2로 교대하는 예로 두 정책의 결합을 설명한다.

5.3.5 다단계 큐 스케줄링

출처: 책 235-237쪽 / 원문 PDF 181-183쪽

다단계 큐는 순위나 프로세스 유형별로 준비 큐를 나눈다. 각 큐는 자체 알고리즘을 사용할 수 있다. 원문은 대화형 전경 큐에 RR, 배치 배경 큐에 FCFS를 쓰는 경우를 든다. 전형적으로 프로세스는 진입할 때 정해진 큐에 남는다.

큐 내부뿐 아니라 큐 사이에도 정책이 필요하다. 고정 우선순위를 쓰면 상위 큐가 비어야 하위 큐를 실행하고 상위 작업 도착 시 하위를 선점한다. 다른 방식은 큐별로 CPU 비율을 나누는 것으로 원문은 전경 80%·배경 20%를 예로 든다. 높은 큐의 절대 우선권과 각 큐의 시간 몫은 다른 선택이다.

5.3.6 다단계 피드백 큐 스케줄링

출처: 책 237-239쪽 / 원문 PDF 183-185쪽

다단계 피드백 큐는 프로세스가 큐 사이를 이동할 수 있다. CPU를 많이 쓰면 낮은 큐로 내려가고 I/O 중심·대화형처럼 짧은 버스트는 높은 큐에 머문다. 오래 대기한 하위 큐 작업을 올리는 노화로 기아를 줄일 수 있다.

원문의 3단계 예

1. 새 프로세스는 큐 0에서 8ms를 받는다.
2. 끝나지 않으면 큐 1의 꼬리로 내려가고 큐 0이 비었을 때 16ms를 받는다.
3. 또 끝나지 않으면 큐 2로 간다. 큐 0·1이 비었을 때 FCFS로 실행한다.
4. 상위 큐에 새 프로세스가 오면 하위 큐의 실행을 선점한다.

그림 5.9 다단계 피드백 큐

그림·표 출처: 책 238쪽 / 원문 PDF 184쪽

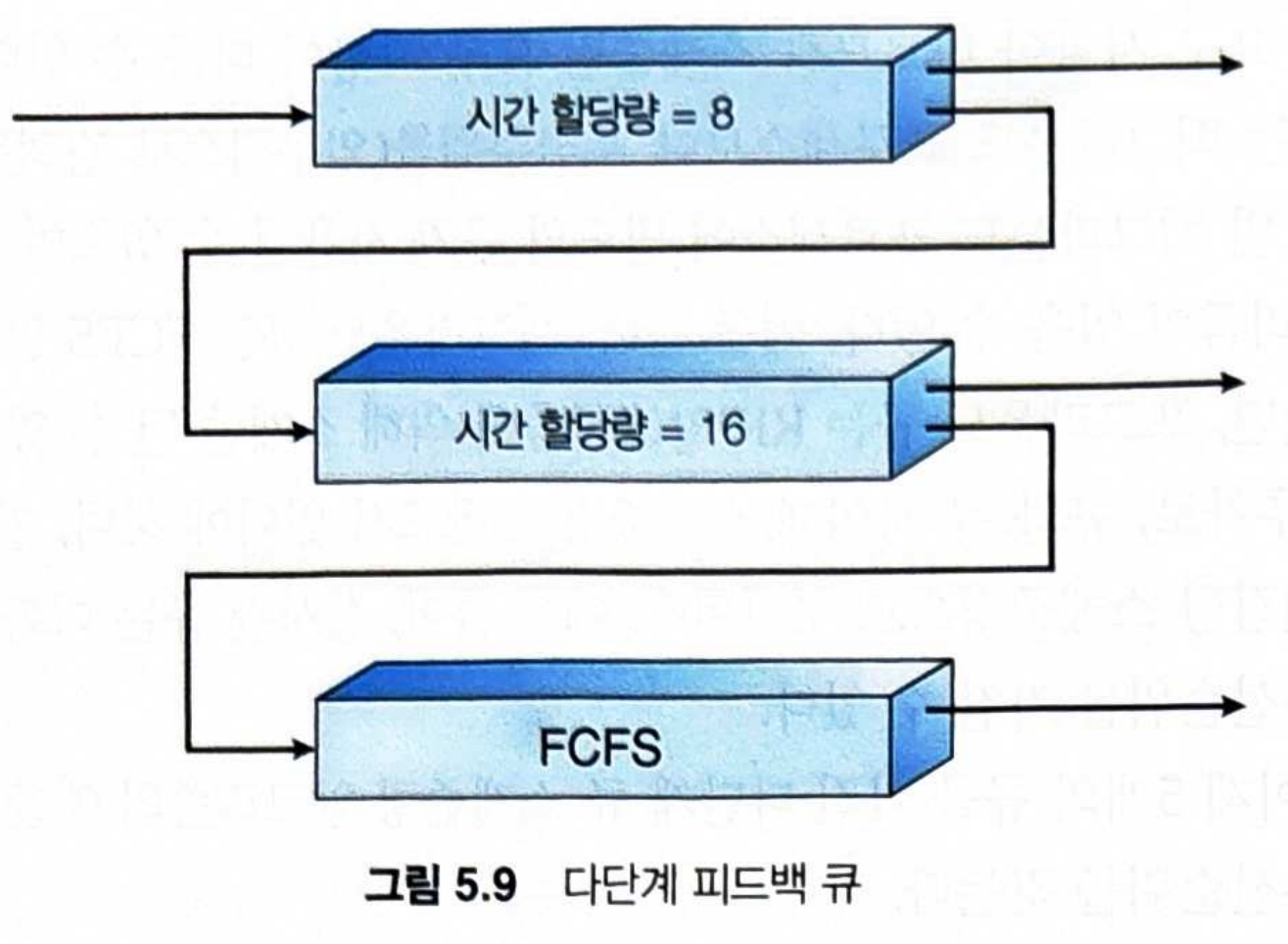


그림 5.9 다단계 피드백 큐

8ms→16ms→FCFS라는 원문 큐 이동과 실행 정책이다.

정의해야 할 매개변수는 큐 수, 각 큐 알고리즘, 승격 시기, 강등 시기, 새로 서비스가 필요할 때 들어갈 큐이다. 특정 시스템에 맞게 매우 유연하지만 이 매개변수의 적절한 선택이 필요하여 복잡하다.

5.4 스레드 스케줄링

출처: 책 239-241쪽 / 원문 PDF 185-187쪽

5.4.1 경쟁 범위

출처: 책 239-240쪽 / 원문 PDF 185-186쪽

범위	경쟁 대상과 결정 주체
PCS(프로세스 경쟁 범위)	같은 프로세스의 사용자 스레드들이 가용 LWP를 놓고 경쟁한다. 스레드 라이브러리가 결정한다.
SCS(시스템 경쟁 범위)	시스템 전체의 커널 스레드가 물리 CPU 코어를 놓고 경쟁한다. 커널이 결정한다.

사용자 스레드가 LWP에 스케줄되었다고 해서 곧바로 물리 CPU에서 실행되는 것은 아니다. 해당 커널 스레드도 CPU에 배정되어야 한다. 일대일 Windows·Linux는 SCS를 사용한다. PCS는 통상 프로그래머가 정한 우선순위를 따르고 더 높은 순위로 선점할 수 있지만 동일 순위의 타임 슬라이스를 보장하지는 않는다.

5.4.2 Pthread 스케줄링

출처: 책 240-241쪽 / 원문 PDF 186-187쪽

PTHREAD_SCOPE_PROCESS는 PCS, PTHREAD_SCOPE_SYSTEM은 SCS를 선택한다. 다대다에서 SYSTEM을 쓰면 사용자 스레드에 LWP를 만들고 연결하여 일대일에 가까운 방식이 된다. pthread_attr_getscope()는 현재 범위를 읽고 pthread_attr_setscope()는 속성에 범위를 지정한다. 오류 시 0이 아닌 값을 반환한다. 원문은 Linux·macOS에서 SYSTEM만 허용한다고 설명한다.

원문 그림 5.10은 기본 속성을 얻고 현재 범위를 출력한 뒤 SYSTEM을 설정하여 다섯 스레드를 생성·조인한다. 원문 확인 필요: 인쇄 240쪽 설명 중 PROCESS로 지정한다는 문장과 실제 그림의 SYSTEM 설정이 일치하지 않는다. 이 노트는 실제 코드의 SYSTEM을 기준으로 목적을 설명하고 불일치를 명시한다.

그림 5.10의 경쟁 범위 설정 부분

코드 출처: 책 241쪽 / 원문 PDF 187쪽

그림 5.10 중 경쟁 범위 설정의 두 줄만 발췌하였다. 속성 초기화·현재 범위 확인·스레드 생성·조인은 제외하였다.

원문 코드·기호

```
/* set the scheduling algorithm to PCS or SCS */
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
```

코드의 의미

스레드를 생성하기 전에 attr의 경쟁 범위를 시스템 경쟁 범위(SCS)로 지정한다.

동작 과정

1. 기존 attr에 SYSTEM을 설정한다.
2. 원문 뒤의 pthread_create()가 이 속성을 사용한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 인쇄 240쪽 PROCESS 문구와 다르므로 원문 확인 필요를 명시하였다.

5.5 다중 처리기 스케줄링

출처: 책 242-249쪽 / 원문 PDF 188-195쪽

5.5.1 다중 처리기 스케줄링의 접근 방법

출처: 책 242-243쪽 / 원문 PDF 188-189쪽

여러 CPU는 병렬 실행과 부하 공유를 가능하게 하지만 스케줄링을 복잡하게 한다. **비대칭 다중 처리**는 한 마스터가 스케줄링·입출력 등 시스템 활동을 맡아 자료 공유를 단순화하지만 마스터가 병목이 될 수 있다. **SMP**는 각 처리기가 스스로 스케줄한다.

준비 큐	장점	쟁점
공통 큐	유휴 처리기가 곧바로 작업을 선택할 수 있다.	같은 스레드 중복 선택·누락을 막는 동기화가 필요하며 락 경합이 병목이 될 수 있다.
처리기별 큐	공유 큐 접근 경합을 줄이고 캐시 선호도를 활용한다.	큐마다 부하가 달라질 수 있어 균등화가 필요하다.

5.5.2 다중 코어 프로세서

출처: 책 243-246쪽 / 원문 PDF 189-192쪽

한 칩의 여러 코어는 운영체제에 개별 논리 CPU처럼 보인다. 코어는 메모리 데이터가 준비되기를 기다리는 **메모리 스톨**로 계산 시간을 잃을 수 있다. **하드웨어 다중 스레딩**은 한 코어에 여러 하드웨어 실행 상태를 두고 한 스레드가 기다릴 때 다른 스레드로 전환한다. 각 하드웨어 스레드는 명령 포인터·레지스터 등 상태를 가지므로 운영체제에는 논리 CPU로 보인다.

거친 다중 스레딩은 긴 지연 이벤트에서 전환하여 파이프라인 정리·재충전 비용이 크다. 세밀한 다중 스레딩은 명령 주기 같은 짧은 경계에서 전환하도록 회로를 제공하여 전환 비용을 낮춘다. 물리 코어의 캐시·파이프라인 자원은 하드웨어 스레드가 공유한다.

원문의 두 단계 스케줄링

1. 운영체제가 논리 CPU인 하드웨어 스레드에 실행할 소프트웨어 스레드를 배정한다.
2. 코어가 실제 실행할 하드웨어 스레드를 선택한다.

두 결정은 무관하지 않다. 소프트웨어 스레드를 같은 코어의 하드웨어 스레드 둘에 넣으면 자원을 나누므로 서로 다른 코어에 배정하는 것보다 느릴 수 있다. 운영체제가 자원 공유 구조를 알면 더 적절한 배정을 할 수 있다.

5.5.3 부하 균등화

출처: 책 247쪽 / 원문 PDF 193쪽

부하 균등화는 어떤 처리기는 쉬고 다른 처리기에는 대기 스레드가 쌓이는 불균형을 줄인다. 주로 처리기별 큐를 사용하는 경우 필요하다. push 이주는 주기적으로 부하를 검사하여 바쁜 처리기에서 덜 바쁜 곳으로 옮기고, pull 이주는 유휴 처리기가 바쁜 쪽에서 작업을 가져온다. 둘은 함께 사용할 수 있다.

같은 스레드 수가 항상 같은 부하를 뜻하지 않는다. 우선순위나 다른 목표도 고려할 수 있으며 단순한 균등 기준은 스케줄링 목표와 충돌할 수 있다.

5.5.4 처리기 선호도 (원문 번호 5.5.2)

출처: 책 247-249쪽 / 원문 PDF 193-195쪽

원문 확인 필요: 이 항목의 소제목 번호는 인쇄 247쪽에서 5.5.2로 표시되어 앞의 다중 코어 절 번호와 중복된다. 본문의 다른 참조는 5.5.4를 사용하므로 정리 순서를 구별하고 실제 인쇄 번호를 제목에 병기하였다.

같은 처리기에서 계속 실행하면 최근 데이터가 남은 **warm cache**를 활용한다. 다른 곳으로 이주하면 캐시를 무효화·재충전하는 비용이 든다. 동일 처리기를 선호하되 보장하지 않는 것이 약한 선호도, 실행 가능한 CPU 집합을 지정하는 것이 강한 선호도이다.

NUMA에서는 로컬 메모리가 원격 메모리보다 빠르므로 CPU 스케줄러와 메모리 배치가 협력해 실행 CPU에 가까운 메모리를 써야 한다. 부하 균등화를 위해 이주하면 캐시·NUMA의 이점을 잃을 수 있다. 따라서 균등화와 메모리 접근 지연 최소화는 상충하는 목표이며 함께 조정해야 한다.

5.5.5 이기종 다중 처리

출처: 책 249쪽 / 원문 PDF 195쪽

HMP는 같은 명령어 집합을 실행하되 속도·전력 관리 특성이 다른 코어를 결합한다. 작업이 모든 코어에서 실행 가능하므로 특정 마스터만 시스템 작업을 하는 비대칭 다중 처리와 다르다. 원문의 ARM big.LITTLE에서 짧고 높은 성능을 요구하는 대화형 작업은 big, 긴 배경 작업은 효율적인 LITTLE에 배정할 수 있다. 전력 절약 모드에서는 big을 끄는 방법도 설명한다.

5.6 실시간 CPU 스케줄링

출처: 책 250쪽 / 원문 PDF 196쪽

구분	보장 범위
연성 실시간	중요 실시간 프로세스가 일반 프로세스보다 우선권을 가진다. 특정 시점 실행은 보장하지 않는다.
경성 실시간	태스크가 마감시간 안에 서비스를 받아야 한다. 기한 뒤 결과는 서비스를 받지 못한 것과 같은 문제가 된다.

5.6.1 지연시간 최소화

출처: 책 250-252쪽 / 원문 PDF 196-198쪽

이벤트 지연은 이벤트 발생부터 대응 서비스까지이다. 인터럽트 지연은 인터럽트 발생부터 서비스 루틴 시작까지로, 현재 명령 완료·종류 식별·상태 저장 등이 포함된다. 커널의 인터럽트 불능 시간이 영향을 주므로 매우 짧게 제한해야 한다. 경성 실시간에서는 단순히 짧을 뿐 아니라 정해진 한계보다 작아야 한다.

디스패치 지연은 현재 프로세스를 멈추고 다음 실행을 시작하는 시간이다. 선점형 커널이 이를 줄이는 데 효과적이다. 충돌 단계에는 커널 실행의 선점과 높은 순위가 필요한 자원을 낮은 순위가 해제하기까지의 대기가 포함되고, 이후 높은 순위를 CPU에 스케줄한다.

5.6.2 우선순위 기반 스케줄링

출처: 책 252-253쪽 / 원문 PDF 198-199쪽

실시간은 필요 시 빠르게 응답하도록 선점 우선순위 스케줄링을 요구한다. 하지만 높은 순위만 주는 것으로는 연성 기능에 그치며 경성은 기한을 지킬 추가 기법이 필요하다. 원문은 주기 태스크의 수행 시간 t, 마감시간 d, 주기 p를 정의한다.

원문 코드·기호

```
0 ≤ t ≤ d ≤ p,  실행 빈도 = 1/p
```

- t: 주기마다 필요한 고정 수행 시간
- d: 서비스를 완료해야 하는 마감시간
- p: 반복 주기

원문 주기 태스크 모델의 관계이다. 승인 제어는 기한 내 완료 가능한 요청을 허용하고 그렇지 않으면 거절한다.

5.6.3 Rate-Monotonic 스케줄링

출처: 책 253-255쪽 / 원문 PDF 199-201쪽

Rate-monotonic(RM)은 주기가 짧은 태스크에 높은 **정적 우선순위**를 주는 선점 정책이다. 주기 태스크는 각 버스트에서 고정된 수행 시간을 요구한다. 정적 우선순위 알고리즘 가운데 원문이 제시한 의미에서 최적이지만 모든 CPU 부하를 항상 기한 안에 처리하지는 못한다.

원문 예	P1	P2
주기 p	50	100
수행 t	20	35
이용률 t/p	0.40	0.35

총 75% 이용률이라도 P2를 먼저 실행하면 P1이 시간 55에 끝나 기한 50을 놓친다. RM에서는 P1이 0~20, P2가 20~50 실행한 뒤 다음 P1이 선점하여 50~70 실행하고 P2가 70~75에 끝나 두 기한을 만족한다.

원문 코드·기호

```
N(2^(1/N) - 1)
```

- N: 스케줄할 주기 프로세스 수

원문이 제시한 최악의 경우 보장 가능한 CPU 이용률 경계이다. 한 프로세스는 100%, 두 프로세스는 약 83%, 수가 커지면 약 69%로 내려간다. 경계 초과는 모든 경우 실패한다는 뜻이 아니라 보장할 수 없다는 뜻이다.

두 번째 예 p1=50·t1=25, p2=80·t2=35에서는 총 이용률이 원문 반올림으로 약 0.94이다. P1의 고정 우선권으로 P2가 시간 80의 기한을 놓친다. 원문 차트로 그 실패 시점을 보존하였다.

그림 5.23 RM이 기한을 놓치는 예

그림·표 출처: 책 255쪽 / 원문 PDF 201쪽

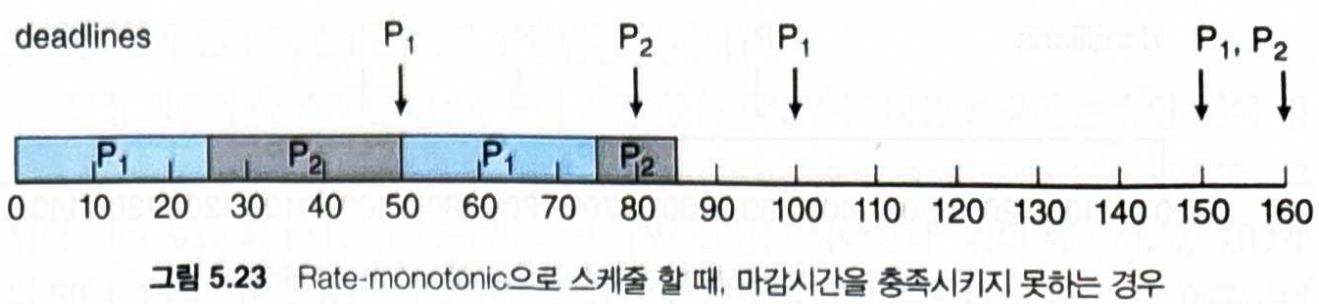


그림 5.23 Rate-monotonic으로 스케줄 할 때, 마감시간을 충족시키지 못하는 경우

그림 5.23 RM이 기한을 놓치는 예

주기 50·80, 수행 25·35에서 고정 우선순위 때문에 P2가 기한 80을 지키지 못한다.

5.6.4 Earliest-Deadline-First 스케줄링

출처: 책 255-256쪽 / 원문 PDF 201-202쪽

EDF는 마감시간이 빠를수록 높은 **동적 우선순위**를 준다. 프로세스가 실행 가능해질 때 기한을 알려야 하며 새로운 기한에 맞춰 순위를 재조정한다. RM과 달리 주기적일 필요도 고정 CPU 할당량일 필요도 없다.

RM이 실패한 p1=50·t1=25, p2=80·t2=35 예에서 EDF는 P1을 0~25에 실행한 뒤 P2를 실행한다. 시간 50에도 P2의 기한 80이 새 P1의 100보다 빠르므로 선점하지 않고 60까지 완료한다. P1은 60~85에 실행하여 두 번째 기한을 지킨다. 이후 시간 100에는 P1의 150이 P2의 160보다 빨라 선점한다.

그림 5.24 EDF 스케줄링

그림·표 출처: 책 256쪽 / 원문 PDF 202쪽

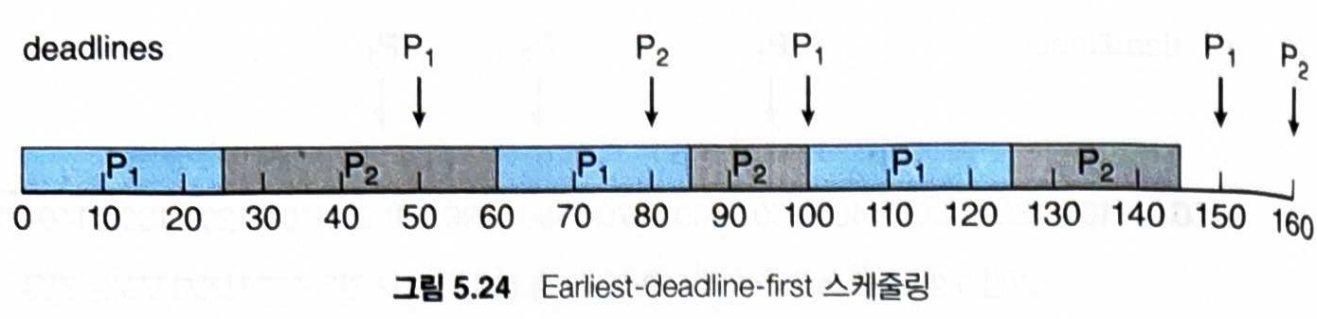


그림 5.24 EDF 스케줄링

같은 부하에서 절대 마감시간의 순서로 우선순위를 조정하는 원문 차트이다.

원문은 EDF를 이론적으로 최적이며 가능한 부하에서 CPU를 최대한 활용할 수 있는 방식으로 설명한다. 실제는 문맥 교환·인터럽트 비용 때문에 100%의 유효 이용이 불가능하다는 한계를 함께 둔다. 기한을 지킬 수 없는 임의의 부하까지 보장한다는 뜻으로 확대하지 않는다.

5.6.5 일정 비율의 몫 스케줄링

출처: 책 256-257쪽 / 원문 PDF 202-203쪽

원문 코드·기호

```
할당받는 CPU 시간 비율 = N/T
```

- T: 전체 시간 몫

- N: 응용에 할당한 몫

원문 예는 T=100에서 A=50·B=15·C=20을 주어 각각 50%·15%·20%를 보장한다. 승인 제어는 남은 몫 안의 요청만 허용한다.

위에서 이미 85몫을 배정했으므로 새 D가 30몫을 요구하면 거절한다. 몫을 보장하려면 자원이 충분할 때만 새 응용을 받아들이는 승인 제어가 함께 필요하다.

5.6.6 POSIX 실시간 스케줄링

출처: 책 257-258쪽 / 원문 PDF 203-204쪽

정책	동작
SCHED_FIFO	가장 높은 우선순위의 FIFO 스레드가 종료·봉쇄할 때까지 실행한다.
SCHED_RR	같은 순위에 시간 할당량을 제공하는 점을 제외하면 FIFO와 유사하다.
SCHED_OTHER	구체적 구현은 정의하지 않아 시스템별로 다르다.

POSIX.1b의 pthread_attr_getschedpolicy()와 pthread_attr_setschedpolicy()는 스레드 속성의 정책을 읽거나 설정한다. 오류 시 0이 아닌 값을 반환한다. 원문 그림 5.25는 현재 정책을 확인한 뒤 FIFO로 설정하고 다섯 스레드를 생성·조인한다. PCS·SCS라는 경쟁 범위와 FIFO·RR이라는 실행 정책을 서로 다른 속성으로 구별한다.

5.7 운영체제 사례들

출처: 책 259-267쪽 / 원문 PDF 205-213쪽

5.7.1 사례: Linux 스케줄링

출처: 책 259-262쪽 / 원문 PDF 205-208쪽

원문은 전통적 UNIX 변형, O(1) 스케줄러, 커널 2.6.23부터의 **CFS**로 이어지는 역사를 설명한다. 다음 내용은 제공된 책이 다루는 구현 범위이다. 스케줄링 클래스마다 순위와 알고리즘을 두고 가장 높은 클래스의 태스크를 선택한다. 기본 CFS와 실시간 클래스를 구별한다.

CFS는 고정 길이의 순위별 시간 할당량 대신 nice 값에 따른 CPU 비율을 제공한다. nice는 -20~19이고 작은 값이 높은 순위이며 기본은 0이다. **목적 지연시간**은 모든 실행 가능 태스크가 한 번씩 실행할 시간 간격이고 태스크 수가 많아지면 늘어날 수 있다.

vruntime은 우선순위를 반영한 가상 실행 시간이다. nice=0이면 실제 시간과 같고 낮은 우선순위는 더 빨리, 높은 우선순위는 더 느리게 증가한다. 다음 실행 대상은 가장 작은 vruntime이다. 같은 nice에서 I/O 중심은 짧게 쓰고 대기하므로 CPU 중심보다 vruntime이 작아져 깨어날 때 우선할 수 있다.

실행 가능 태스크는 vruntime을 키로 하는 적·백 트리에 들어가며 봉쇄되면 빠진다. 가장 왼쪽이 최소값이고 그 위치를 rb_leftmost에 캐시하여 다음 선택을 효율화한다. 원문은 실시간의 FIFO·RR을 일반 태스크보다 높은 전역 순위에 배치한다.

부하 균등화는 스레드 수만이 아니라 우선순위·평균 CPU 이용을 결합해 부하를 정의한다. 캐시·NUMA 불이익을 줄이기 위해 자원을 공유하는 코어를 **스케줄링 도메인**으로 계층화하고 가까운 도메인부터 균형을 맞춘다. 다른 NUMA 노드로의 먼 이주는 심각한 불균형에서 선택하며 전체가 바쁠 때는 로컬 접근 이점을 보존하려 한다.

5.7.2 사례: Windows 스케줄링

출처: 책 263-266쪽 / 원문 PDF 209-212쪽

Windows는 **우선순위 기반 선점**으로 높은 스레드를 우선 실행한다. 디스패처는 32단계 순위를 사용한다. 가변 클래스는 1~15, 실시간은 16~31이며 0은 메모리 관리용이다. 각 순위 큐를 높은 곳부터 조사하고 준비 스레드가 없으면 idle 스레드를 실행한다.

프로세스의 IDLE·BELOW_NORMAL·NORMAL·ABOVE_NORMAL·HIGH·REALTIME 클래스와 스레드의 상대 순위를 결합해 실제 순위를 정한다. 가변 스레드는 몫 만료 시 기본 순위 아래로는 내려가지 않게 순위를 낮추고, I/O 대기 해제 시 종류에 따라 높인다. 대화형 응답과 계산 중심 작업의 CPU 독점을 함께 고려하는 방식이다. 전경 NORMAL 프로세스에는 더 긴 시간 몫을 주는 정책도 설명한다.

UMS는 커널과 독립적인 사용자 모드 생성·관리를 지원한다. 앞선 fiber는 같은 TEB를 공유하는 제약이 있었지만 UMS는 각 실행 문맥을 따로 보관한다. 원문은 직접 사용자 스케줄러를 만들기보다 Concurrency Runtime 같은 상위 도구가 이를 활용하도록 설계됨을 설명한다.

다중 처리기에서는 같은 코어의 하드웨어 스레드를 SMT 집합으로 묶고 최근 실행·선호도를 고려한다. 각 스레드의 이상적인 처리기를 분산하여 부하를 여러 물리 코어에 나누며 프로세스마다 시작 시드 값을 달리하여 첫 코어 쏠림을 피한다.

5.7.3 사례: Solaris 스케줄링

출처: 책 266-269쪽 / 원문 PDF 212-215쪽

Solaris는 TS(시분할)·IA(대화형)·RT(실시간)·SYS(시스템)·FSS(공평 공유)·FP(고정 우선순위)라는 여섯 클래스를 전역 순위로 매핑한다. 클래스별 알고리즘과 우선순위가 다르며 가장 높은 전역 순위 스레드를 선택한다.

기본 TS는 다단계 피드백으로 순위를 동적으로 바꾸고 높은 순위에 짧은 몫, 낮은 순위에 긴 몫을 준다. 몫을 다 쓰면 순위를 낮추고 수면에서 복귀하면 높여 대화형 응답과 CPU 중심 처리량을 조정한다. IA는 같은 정책을 쓰되 창 응용의 순위를 높인다. RT는 일반 클래스보다 우선하며 SYS는 커널 스레드용이므로 커널 모드에 들어간 사용자 프로세스와 동일하지 않다.

FP는 순위를 동적으로 바꾸지 않는다. FSS는 프로젝트라는 프로세스 집합에 CPU 공유 권리를 부여한다. 인터럽트 서비스용 스레드는 일반 클래스 밖의 가장 높은 범위를 사용한다. 선택된 스레드는 봉쇄·몫 만료·더 높은 순위 선점까지 실행한다. 원문은 Solaris 9부터 일대일 스레드 모델로 바뀐 것도 함께 설명한다.

5.8 알고리즘의 평가

출처: 책 268-269쪽 / 원문 PDF 214-215쪽

모든 환경에 최선인 알고리즘은 없다. 먼저 이용률·처리량·응답·총처리 시간 가운데 상대 중요성과 제약을 정해야 한다. 원문은 최대 응답 300ms를 만족하면서 이용률을 높이는 식의 목표를 예로 든다. 그런 뒤 동일 기준으로 후보 알고리즘을 평가한다.

5.8.1 결정론적 모델링

출처: 책 269-271쪽 / 원문 PDF 215-217쪽

결정론적 모델링은 미리 정한 정확한 작업 부하에 각 알고리즘을 적용한다. 단순하고 빠르며 정확한 비교값을 주지만 그 입력에만 적용된다. 반복되는 동일 프로그램의 요구량을 정확히 측정할 수 있을 때 선택에도 쓸 수 있다.

원문의 동시 도착 입력	P1	P2	P3	P4	P5
버스트(ms)	10	29	3	7	12

이 입력의 원문 결과는 FCFS 평균 28ms, 비선점 SJF 평균 13ms, q=10인 RR 평균 23ms이다. 모든 프로세스와 버스트를 시간 0에 안다는 특정 조건에서 비교한 것으로, 이후의 알 수 없는 작업 부하에 그대로 일반화하지 않는다.

5.8.2 큐잉 모델

출처: 책 271-272쪽 / 원문 PDF 217-218쪽

작업이 날마다 달라지면 고정 입력보다 도착·CPU·I/O 버스트의 분포를 측정하거나 추정한다. CPU와 각 I/O 장치를 큐를 가진 서버로 표현하고 도착률·서비스율로 이용률·평균 길이·대기를 계산하는 것이 **큐잉 네트워크 분석**이다.

원문 코드·기호

```
n = λ × W
```

- n: 서비스 중인 프로세스를 제외한 평균 큐 길이
- λ: 큐에 도착하는 평균률
- W: 큐의 평균대기 시간

Little의 공식은 원문의 안정 상태 조건에서 도착 수와 이탈 수가 같음을 이용한다. 두 변수를 알면 나머지를 구한다. 원문 예 λ=7개/초, n=14개에서 W=2초이다.

분석 가능한 분포·알고리즘이 제한적이고 실제와 다른 단순 분포 및 독립 가정이 필요할 수 있다. 따라서 계산값은 실제 시스템의 근사이며 정확성에 한계가 있다.

5.8.3 모의실험

출처: 책 272-273쪽 / 원문 PDF 218-219쪽

모의실험은 주요 구성요소를 자료구조로 만들고 모형 시간을 증가시키면서 프로세스·장치·스케줄러의 활동을 반영해 통계를 수집한다. 수학적·경험적 분포의 난수로 입력을 만들 수 있지만 빈도만으로는 실제 이벤트의 순서 관계를 보존하지 못한다.

추적 파일은 실제 실행 이벤트의 순서를 기록하여 같은 입력으로 알고리즘들을 비교하게 한다. 상세할수록 정확해질 수 있지만 실행 시간·저장공간·설계와 디버깅 비용이 커진다. 원문 그림은 하나의 실제 추적을 FCFS·SJF·RR 모의실험에 각각 넣고 성능 통계를 비교한다.

5.8.4 구현

출처: 책 273-274쪽 / 원문 PDF 219-220쪽

실제 운영체제에 알고리즘을 구현해 실행하면 실제 환경을 평가할 수 있지만 코드·자료구조 변경과 시험 비용이 크다. 회귀 시험으로 새 버그와 예전 버그 재발을 살펴야 한다. 환경 자체도 프로그램 변화와 사용자의 정책 적응에 따라 변하므로 같은 작업을 반복하는 도구·스크립트와 지속적인 측정이 필요하다.

관리자가 매개변수를 조정하거나 API로 우선순위를 바꾸는 유연성이 도움이 되지만 특정 응용에 맞춘 최적화가 일반 환경에서도 개선을 주는 것은 아니다. 선택 기준·입력 대표성·실제 오버헤드를 함께 검토해야 한다.

5.9 요약과 원문 복습 범위

출처: 책 274-279쪽 / 원문 PDF 220-225쪽

원문 요약은 알고리즘 선택 규칙·단일 및 다중 코어·실시간·Linux/Windows/Solaris와 평가를 연결한다. 연습문제 5.1~5.10은 선점, 간트차트와 대기·총처리 시간, 큐 매개변수, I/O 중심 우대, PCS·SCS 및 우선순위 재계산을 점검한다. 문제의 입력·공식도 검토하였으나 새로운 계산 결과나 풀이를 만들지 않았다. 추가 자료와 참고문헌은 외부 지식을 도입하는 근거로 사용하지 않았다.

핵심 정리

- 스케줄러의 선택과 디스패처의 실제 제어 이전을 구별한다.
- 대기 시간은 준비 큐의 시간이며 총처리·첫 응답 시간과 다르다.
- SJF는 다음 버스트를 기준으로 하고 실제 구현에는 예측이 필요하다.
- RR의 q는 응답과 문맥 교환 비용의 절충이며 큰 q는 FCFS로 가까워진다.
- 우선순위의 숫자 방향은 시스템별로 다르며 기아에는 노화가 필요할 수 있다.
- 다단계 큐의 고정 소속과 피드백 큐의 이동을 구별한다.
- 다중 코어의 균등화는 캐시·NUMA 선호도와 충돌한다.
- 경성 실시간은 단순 우선권을 넘어 마감시간 보장이 필요하다.
- RM은 주기 기반 정적, EDF는 기한 기반 동적 순위이다.
- 알고리즘 평가값은 입력·가정·오버헤드의 한계와 함께 읽는다.

혼동하기 쉬운 내용

기본 CPU 스케줄링 선택 규칙

출처: 책 226-239쪽 / 원문 PDF 172-185쪽

알고리즘	선택 기준	주의
FCFS	도착·요청 순	비선점·호위 효과
SJF/SRTF	짧은 다음 버스트/남은 시간	예측이 필요함
RR	같은 q를 순환 배정	q와 문맥 교환의 절충
우선순위	가장 높은 순위	숫자 방향 확인·기아와 노화
다단계 큐	상위 큐 우선 또는 큐별 몫	소속이 일반적으로 고정
피드백 큐	실행 특성·대기에 따라 이동	승격·강등 등 매개변수 선택 필요

실시간 알고리즘의 순위

출처: 책 253-256쪽 / 원문 PDF 199-202쪽

기준	RM	EDF
우선순위	주기 짧을수록 높음·정적	기한 빠를수록 높음·동적
대상 조건	고정 수행 시간의 주기 태스크	주기·고정 수행 시간을 반드시 요구하지 않음
필요 정보	주기와 수행 시간	실행 가능 시 마감시간 통지

평가 방법의 범위

출처: 책 269-274쪽 / 원문 PDF 215-220쪽

방법	정보와 장점	한계
결정론적	정확한 고정 입력·빠르고 명확	해당 입력에 국한
큐잉	확률 분포로 평균 계산	분포·독립 가정의 근사
모의실험	모형과 추적 파일로 실제 순서 반영	시간·저장·모형 개발 비용
구현	실제 시스템 환경에서 실행	코드 변경·회귀 시험·환경 변화 비용

장 마무리 복습

- 버스트 특성 → 선택 기준 → 실제 디스패치 비용을 연결한다.
- 선점은 응답에 유리하지만 공유 자료 보호가 필요하다.
- 큐 구조·경쟁 범위·논리 CPU·물리 코어는 서로 다른 층의 결정이다.
- 부하·선호도·에너지·기한 등 목표가 달라지면 필요한 정책도 달라진다.
- 단일 수치 예제의 비교와 실제 시스템의 확률·시간 비용을 구별한다.

원문 확인 메모

- Pthread 경쟁 범위 본문의 PROCESS 문구와 실제 그림의 SYSTEM 설정 불일치는 원문 확인 필요로 명시했다.
- 운영체제 사례는 제공된 책이 설명한 버전·구현의 범위를 정리하였으며 이후 변경을 보충하지 않았다.
- 처리기 선호도 소제목은 5.5.2로 중복 인쇄되지만 본문 참조는 5.5.4를 사용한다. 실제 번호를 제목에 병기하고 원문 확인 필요로 표시했다.

제6장. 동기화 도구들

출처: 책 283-317쪽 / 원문 PDF 227-261쪽

학습 주제

- 공유 데이터에 병행하게 접근할 때 생기는 경쟁 조건과 임계구역 문제를 정의하고, 하드웨어 명령에서 mutex 락·세마포·모니터까지 동기화 수단을 살펴본다.

- 상호 배제를 제공하는 것과 실행의 진행을 보장하는 것은 함께 검토해야 한다. 교착 상태·우선순위 역전 및 경합 정도에 따른 성능 차이도 이 장의 범위이다.

설명의 흐름

1. 경쟁 조건 → 임계구역의 세 요구 조건 → Peterson의 해결안과 명령 재정렬의 한계
2. 메모리 장벽·원자적 명령·원자적 변수 → mutex·세마포 → 모니터와 조건 변수
3. 라이브니스 실패 → 동기화 도구의 경합별 평가

6.1 배경

출처: 책 283-285쪽 / 원문 PDF 227-229쪽

협력적 프로세스는 시스템 안의 다른 프로세스 실행에 영향을 주거나 영향을 받는다. 논리 주소 공간을 직접 공유하거나 공유 메모리·메시지 전달로 데이터를 공유할 수 있다. 병행 실행에서는 어느 명령 지점에서도 인터럽트와 실행 전환이 일어날 수 있고, 병렬 실행에서는 여러 코어의 명령 흐름이 동시에 진행된다. 공유 데이터에 대한 접근을 질서 있게 조정하지 않으면 데이터의 일관성이 손상될 수 있다.

원문의 유한 버퍼 예는 채워진 버퍼 수를 나타내는 정수 count를 추가한다. 생산자는 항목을 넣고 count를 증가시키며 소비자는 항목을 꺼내고 감소시킨다. 각 코드는 따로 실행하면 올바르지만 count++와 count--는 각각 읽기·레지스터 계산·저장으로 나뉠 수 있다. 초기 count가 5일 때 두 연산을 병행하면 4, 5, 6이 될 수 있지만 올바른 값은 5이다.

원문에서 count가 4가 되는 교차 실행

1. 생산자가 count=5를 자신의 레지스터로 읽고 6으로 증가시킨다.
2. 소비자가 아직 5인 count를 다른 레지스터로 읽고 4로 감소시킨다.
3. 생산자가 6을 count에 저장한 뒤 소비자가 4를 저장한다. 실제 채워진 버퍼는 5개지만 count는 4이다.

경쟁 상황(race condition)은 여러 프로세스가 같은 자료를 병행하게 접근·조작하고 최종 결과가 접근의 특정 순서에 의존하는 상황이다. 원문 예에서는 한순간에 하나의 프로세스만 count를 조작하도록 보장해야 하며, 이를 위해 프로세스 동기화가 필요하다.

6.2 임계구역 문제

출처: 책 286-288쪽 / 원문 PDF 230-232쪽

임계구역(critical section)은 적어도 하나의 다른 프로세스와 공유하는 데이터를 접근·갱신하는 코드 영역이다. 한 프로세스가 그 영역을 수행하는 동안 다른 프로세스는 자신의 해당 임계구역에서 실행할 수 없어야 한다. 임계구역 문제는 협력적으로 데이터를 공유하는 프로세스의 활동을 동기화할 프로토콜을 설계하는 문제이다.

프로세스의 일반 구조

1. 진입 구역(entry section)에서 임계구역 진입 허가를 요청한다.
2. 임계구역에서 공유 데이터를 조작한다.
3. 퇴출 구역(exit section)을 수행하고 나머지 구역(remainder section)을 실행한다.

요구 조건	정확한 의미
상호 배제	한 프로세스가 임계구역을 실행하면 다른 프로세스는 그들의 임계구역을 실행할 수 없다.

요구 조건	정확한 의미
진행	임계구역이 비어 있고 진입 희망자가 있다면 나머지 구역에서 실행 중이지 않은 프로세스들만 다음 진입자 결정에 참여하며, 결정을 무한정 연기할 수 없다.
한정된 대기	한 프로세스의 진입 요청 이후 허가될 때까지 다른 프로세스의 임계구역 진입 횟수에 한계가 있어야 한다.

각 프로세스가 0이 아닌 속도로 실행된다고 가정하지만 프로세스 사이의 상대 속도는 가정하지 않는다. 커널의 열린 파일 리스트, 다음 사용 가능한 PID, 메모리 할당·프로세스 리스트·인터럽트 자료구조도 경쟁 조건에 노출된다. 단일 코어에서 공유 변수를 수정하는 동안 인터럽트를 금지할 수 있지만, 다중 처리기에 이를 적용하면 모든 처리기로 전달하는 비용과 효율 저하, 클록 갱신의 영향을 고려해야 한다.

커널	선점과 특성
비선점형	커널 모드에서 실행 중인 프로세스는 커널을 나가거나 봉쇄되거나 자발적으로 CPU를 양보할 때까지 계속 실행한다. 원문은 한순간 커널 실행자가 하나라는 관점에서 커널 자료구조의 경쟁 조건을 설명한다.
선점형	커널 모드에서도 선점을 허용하므로 공유 자료구조를 신중히 보호해야 한다. SMP에서는 여러 처리기의 프로세스가 커널 모드에 있을 수 있다. 더 민첩한 응답과 실시간 프로세스의 선점에 유리하다.

6.3 Peterson의 해결안

출처: 책 288-291쪽 / 원문 PDF 232-235쪽

Peterson의 해결안은 임계구역과 나머지 구역을 번갈아 실행하는 **두 프로세스**에 한정된다. turn은 진입 순번, flag 배열은 각 프로세스가 진입할 준비가 되었는지를 나타낸다. 두 프로세스가 함께 들어가려 하면 turn에 대한 마지막 배정이 먼저 들어갈 프로세스를 결정한다.

Peterson의 해결안: 프로세스 P_i의 구조

코드 출처: 책 289쪽 / 원문 PDF 233쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ;

    /* critical section */

    flag[i] = false;

    /*remainder section */
}
```

코드의 의미

두 프로세스가 진입 의사와 순번을 공유하여 임계구역 진입을 조정한다.

동작 과정

1. flag[i]를 true로 만들고 turn을 상대 j로 지정한다.
2. 상대가 진입 의사를 갖고 상대 차례이면 대기한다.
3. 임계구역을 마치면 flag[i]를 false로 바꾼다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 공유 선언은 int turn;과 boolean flag[2];이다.
- 이 예는 현대 컴퓨터에서 명령 재정렬 때문에 올바른 실행을 보장할 수 없다는 원문의 제한과 함께 읽어야 한다.

원문의 증명은 두 프로세스가 동시에 임계구역에 있다면 flag가 모두 true여야 하지만 turn은 동시에 두 값을 가질 수 없다는 점으로 상호 배제를 보인다. 진입을 막는 것은 상대가 진입 의사가 있고 상대 차례인 경우뿐이다. 상대가 나가면 flag를 false로 바꾸고, 다시 들어가려 해도 turn을 넘겨주므로 기다리던 프로세스가 무한정 양보하지 않는다.

적용 한계: 현대 컴파일러와 프로세서는 데이터 종속성이 없는 읽기·쓰기를 재정렬할 수 있다. 원문은 x와 flag를 공유하는 두 스레드에서 x=100보다 flag=true가 먼저 보이면 기대한 100 대신 0을 출력할 수 있음을 보인다. Peterson의 진입 구역 첫 두 배정 순서가 바뀌면 두 스레드가 동시에 임계구역에 있을 수 있다. 따라서 논리적 증명만으로 현대 구조의 실행까지 보장한다고 해석하면 안 된다.

6.4 동기화를 위한 하드웨어 지원

출처: 책 291-297쪽 / 원문 PDF 235-241쪽

원문은 메모리 장벽, 원자적 하드웨어 명령, 원자적 변수를 제시한다. 이들은 직접 동기화에 쓰이거나 더 높은 수준 도구를 만드는 기본 구성요소가 된다.

6.4.1 메모리 장벽

출처: 책 292-293쪽 / 원문 PDF 236-237쪽

메모리 모델은 컴퓨터 구조가 프로그램의 메모리 접근에 보장하는 사항을 뜻한다. 원문은 한 처리기의 변경 결과가 다른 모든 처리기에 즉시 보이는 강한 순서와 즉시 보이지 않는 약한 순서를 구별한다. 구조마다 모델이 다르므로 공유 메모리 다중 처리기의 가시성을 임의로 가정할 수 없다.

메모리 장벽(memory barrier, memory fence) 명령이 실행되면 뒤의 적재·저장 전에 앞의 적재·저장이 완료되도록 한다. 저장된 변경이 다른 처리기에 보이도록 보장하여 재정렬의 문제를 다룬다. 원문의 x/flag 예에서는 읽는 쪽에서 flag 적재 뒤 x 적재 전에, 쓰는 쪽에서는 x 배정 뒤 flag 배정 전에 장벽을 둔다. 매우 낮은 수준의 연산이며 원문은 주로 커널 개발자의 특수 동기화 코드에 사용된다고 설명한다.

6.4.2 하드웨어 명령어

출처: 책 293-296쪽 / 원문 PDF 237-240쪽

원자적 명령은 검사·변경 또는 교환을 인터럽트되지 않는 하나의 단위로 수행한다. test_and_set()은 원래 값을 반환하면서 대상을 true로 설정한다. false로 초기화한 lock을 이에 사용하면 먼저 도착한 실행자만 진입하고 나갈 때 false로 바꿀 수 있다.

원자적 compare_and_swap()의 정의

코드 출처: 책 294쪽 / 원문 PDF 238쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;

    if (*value == expected)
        *value = new_value;

    return temp;
}
```

코드의 의미

현재 값이 expected와 같을 때에만 new_value를 저장하며, 성공 여부와 관계없이 원래 값을 반환한다.

동작 과정

1. 원래 *value를 temp에 보관한다.
2. 기댓값과 현재 값이 같으면 값을 바꾼다.
3. temp를 반환한다. 원문은 전체 명령을 원자적 단위로 가정한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 동시에 실행된 CAS 명령들은 임의의 순서로 순차 실행된다.

CAS 상호 배제 예에서 lock은 0으로 초기화되고 compare_and_swap(&lock, 0, 1)을 반복한다. 첫 성공자는 원래 값 0을 받아 임계구역에 들어가고 퇴출 시 lock=0으로 바꾼다. 그러나 단순 반복은 상호 배제를 만족해도 **한정된 대기를 만족하지 않는다**. 원문의 별도 알고리즘은 waiting 배열을 false, lock을 0으로 초기화하고 퇴출 시 대기자를 순환 탐색한다. 다음 대기자에게 진입을 넘겨 최대 n-1번만 양보하도록 한다.

원문의 Intel x86 설명은 cmpxchg가 CAS를 구현하고 대상 피연산자가 갱신되는 동안 lock 접두어로 버스를 잠가 원자적 실행을 강제한다고 기술한다. 여기서는 이 설명을 다른 구조의 세부 동작으로 확장하지 않는다.

6.4.3 원자적 변수

출처: 책 296-297쪽 / 원문 PDF 240-241쪽

원자적 변수는 정수·부울 같은 기본 자료형에 원자적 연산을 제공한다. CAS는 흔히 직접 사용되기보다 이런 도구의 기초가 된다. 원문의 증가 함수는 현재 값을 읽고 그 값이 유지되어 있을 때만 1 증가시키는 CAS를 시도하며 충돌하면 반복한다.

한 변수의 갱신이 원자적이라는 사실만으로 전체 알고리즘의 경쟁 조건이 사라지지는 않는다. 유한 버퍼에서 count가 원자적이어도 두 소비자가 count>0을 기다리다가 생산자가 항목 하나만 추가했을 때 함께 대기를 빠져나올 수 있다. 원문은 원자적 변수를 주로 카운터·시퀀스 생성기 같은 공유 데이터 한 개의 갱신에 적합한 도구로 제한하여 설명한다.

6.5 Mutex Locks

출처: 책 297-299쪽 / 원문 PDF 241-243쪽

Mutex 락은 임계구역 진입 전에 acquire()로 획득하고 나올 때 release()로 반환하는 도구이다. available이라는 불린 값이 사용 가능 여부를 나타낸다. 획득·반환 호출은 원자적이어야 하며 CAS로 구현할 수 있다.

원문의 단순 구현은 사용 불가능한 동안 반복 검사하는 **바쁜 대기**를 한다. 이 유형을 스핀락이라고 부른다. 하나의 CPU 코어를 여러 프로세스가 나누는 환경에서는 다른 프로세스가 쓸 CPU 주기를 낭비한다. 반면 대기하면서 문맥 교환을 하지 않는 이점이 있어 다중 코어에서 임계구역이 짧을 때 유리할 수 있다.

원문이 제시하는 짧은 기간의 기준은 락 유지 시간이 대기와 재개를 위한 **두 번의 문맥 교환 시간보다 짧은 경우**이다. 락을 얻으려다 봉쇄되면 경합, 즉시 얻을 수 있으면 비경합으로 본다. 많은 스레드가 경쟁하는 높은 경합은 응용의 전체 성능을 떨어뜨린다.

6.6 세마포

출처: 책 299-303쪽 / 원문 PDF 243-247쪽

세마포 S는 정수 변수이며 초기화를 제외하고 원자적인 wait()와 signal()만으로 접근한다. 원문의 고전적 wait는 S>0이 될 때까지 바쁘게 기다린 뒤 감소시키고 signal은 증가시킨다. 검사와 감소까지 인터럽트되지 않는 하나의 동작이어야 한다.

유형	값·용도
카운팅 세마포	유한 개 자원의 접근을 제어한다. 가용 자원 수로 초기화하고 획득 때 감소, 반환 때 증가시킨다.
이진 세마포	0과 1의 값만 가지며 mutex 락과 유사하게 상호 배제를 제공할 수 있다.

세마포는 실행 순서도 조정한다. 원문은 P_1의 S_1 뒤에 P_2의 S_2가 실행되어야 할 때 synch를 0으로 초기화하고, S_1 뒤에 signal(synch), S_2 앞에 wait(synch)를 둔다. 이 순서 제약은 하나의 임계구역 보호와 구별되는 용도이다.

바쁜 대기를 줄이는 구현에서는 세마포마다 정수 value와 프로세스 리스트 list를 둔다. wait에서 기다려야 하면 프로세스를 대기 상태로 바꾸고 CPU 스케줄러가 다른 프로세스를 선택한다. signal이 깨우면 준비 완료 상태·준비 큐로 돌아간다.

대기 큐를 사용하는 세마포 wait()

코드 출처: 책 301쪽 / 원문 PDF 245쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        sleep();
    }
}
```

코드의 의미

자원 수를 감소시키고 기다려야 하는 프로세스를 세마포 대기 리스트에 넣어 일시 중지한다.

동작 과정

1. value를 감소시킨다.
2. 음수이면 프로세스를 list에 넣고 sleep()으로 대기시킨다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 원문의 의사 코드이다. 일반 C 코드로 임의 수정하지 않았다.
- 세마포 연산 자체는 원자적이어야 한다.

대기 큐를 사용하는 세마포 signal()

코드 출처: 책 302쪽 / 원문 PDF 246쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

코드의 의미

세마포 값을 증가시키고 대기자가 있으면 하나를 꺼내 준비 상태가 되도록 깨운다.

동작 과정

1. value를 증가시킨다.
2. 값이 0 이하이면 list의 P를 제거하고 wakeup(P)를 수행한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 깨운 프로세스가 즉시 CPU에서 실행되는지는 스케줄링에 달려 있다.

이 구현에서는 음수 값의 절댓값이 대기 프로세스 수를 나타낸다. 이는 고전적 정의와 달리 감소 후 검사하기 때문이다. FIFO 큐로 대기자를 관리하면 한정된 대기를 보장하는 방법이 되지만 세마포의 정확한 사용 자체가 특정 큐잉 전략에 의존하는 것은 아니다. 단일 처리기는 연산 동안 인터럽트를 막을 수 있고 SMP는 CAS·스핀락 등의 기법이 필요하다. 바쁜 대기가 완전히 사라진 것은 아니며 짧은 세마포 연산의 임계구역에 국한된다.

6.7 모니터

출처: 책 303-310쪽 / 원문 PDF 247-254쪽

세마포는 호출 순서를 잘못 쓰면 발견하기 어려운 타이밍 오류를 낳는다. signal을 먼저 하고 wait를 나중에 하면 상호 배제가 깨질 수 있고, 퇴출 시 signal 대신 wait를 하면 영구 봉쇄될 수 있다. 어느 연산을 빠뜨려도 상호 배제 위반 또는 영구 대기로 이어진다. 이런 오류를 줄이기 위해 고급 언어 구조물인 모니터를 도입한다.

모니터는 내부에서 상호 배제가 보장되는 연산 집합을 포함한 추상 자료형이다. 지역 변수는 내부 함수만 접근하고 다른 프로세스가 직접 표현을 사용하지 못한다. 모니터 안에서는 항상 하나의 프로세스만 활성화된다. 프로그래머가 이 제약을 직접 코딩할 필요가 줄어든다.

조건 변수 x의 x.wait()는 호출자를 일시 중지하고 x.signal()은 기다리는 프로세스 하나를 재개한다. 기다리는 프로세스가 없으면 조건 변수 signal은 효과가 없다. 세마포의 signal은 항상 세마포 상태에 영향을 준다는 점과 다르다.

신호 후 정책	실행 관계
Signal and wait	신호를 보낸 P가 기다리고 재개된 Q가 모니터를 떠나거나 다른 조건을 기다린다.
Signal and continue	P가 계속하며 Q는 P가 모니터를 떠나거나 다른 조건을 기다릴 때까지 대기한다.
절충안	P가 signal 직후 모니터를 떠나 Q가 즉시 재개되도록 한다.

계속 실행 정책에서는 Q가 재개할 때 기다리던 논리 조건이 이미 참이 아닐 수 있다. 원문은 Java·C# 등이 모니터 개념을 편입하고 다른 언어에서도 유사한 동기화 지원을 제공한다고 설명한다.

세마포로 signal-and-wait 모니터를 구현할 때 mutex는 1, next는 0으로 초기화한다. next-count는 신호 후 대기하는 프로세스 수를 센다. 외부 프로시저는 mutex를 획득하고 본체를 수행한 뒤 next 대기자가 있으면 그쪽에 신호하고 아니면 mutex를 반환한다. 조건 x마다 x-sem과 x-count를 0으로 두어 조건 대기자와 신호 후 양보 관계를 연결한다.

수행 재개 순서가 FCFS만으로 충분하지 않으면 **x.wait(c)**의 정수 c를 우선순위 번호로 저장할 수 있다. x.signal()은 가장 작은 번호의 프로세스를 재개한다. 원문의 ResourceAllocator는 자원 사용을 희망하는 최대 시간이 가장 짧은 대기자에게 하나의 자원을 준다.

고급 도구라도 잘못된 사용 순서 자체를 보장하지 않는다. 허가 없이 자원 접근·반환, 얻은 자원 미반환, 반환 전에 재요청이 가능하다. 정확성을 위해 호출 순서를 지키는지와 모니터 밖에서 공유 자원을 직접 접근하지 않는지를 확인해야 한다. 대규모·동적 시스템에서 전수 검사는 비합리적일 수 있다는 한계도 원문에 명시된다.

6.8 라이브니스

출처: 책 310-312쪽 / 원문 PDF 254-256쪽

라이브니스(liveness)는 프로세스가 실행 수명주기 동안 진행하도록 시스템이 충족해야 할 속성이다. 무기한 대기는 진행·한정된 대기를 위반하는 라이브니스 실패이다. 바쁜 대기도 임의의 오랜 시간 지속되면 실패가 될 수 있다.

교착 상태는 집합 안의 모든 프로세스가 그 집합의 다른 프로세스만 일으킬 수 있는 사건을 기다리는 상황이다. 원문은 1로 초기화한 S, Q를 두 프로세스가 서로 반대 순서로 wait하여 각자 상대의 signal을 영원히 기다리는 예를 든다.

우선순위 역전은 높은 우선순위 H가 낮은 우선순위 L이 점유한 자원을 기다리는 동안 중간 우선순위 M이 L을 선점하여 H의 대기까지 길게 하는 문제이다. 원문은 세 단계 이상의 우선순위를 가진 시스템에서 설명한다. **우선순위 상속**은 해당 자원을 사용하는 L이 사용 완료까지 H의 우선순위를 물려받게 하고 이후 원래 값으로 돌려, M의 선점을 막는다. Mars Pathfinder의 반복 리셋 사례는 우선순위 상속을 활성화하여 해결한 원문 사례로 제시된다.

6.9 평가

출처: 책 312-314쪽 / 원문 PDF 256-258쪽

하드웨어 명령은 낮은 수준의 기반이며 CAS로 락 없는 알고리즘도 만들 수 있다. 이런 알고리즘은 오버헤드가 낮고 확장성 장점이 있지만 개발·테스트가 어렵다. CAS는 먼저 갱신을 시도하고 충돌하면 재시도하는 낙관적 접근, 상호 배제 락은 다른 갱신자가 있다고 가정하고 먼저 획득하는 비관적 접근으로 대비된다.

경합	원문의 일반적 성능 지침
없음	양쪽 모두 빠르지만 CAS 보호가 기존 동기화보다 다소 빠를 수 있다.
적당함	CAS는 대개 성공하며 실패해도 재시도가 적어 기존 동기화보다 빠를 수 있다.
심함	반복 충돌이 커지므로 기존 동기화가 기본적으로 CAS보다 빠르다.

한 개 공유 변수 갱신에는 원자적 정수가 가볍다. 임계구역 접근 보호에는 mutex가 이진 세마포보다 간단하고 오버헤드가 적어 선호되지만, 유한 개 자원 제어에는 카운팅 세마포가 적합하다. 여러 reader를 동시에 허용하는 reader-writer 락은 병행성을 높일 수 있다. 모니터·조건 변수는 사용이 간단하지만 구현에 따라 오버헤드와 높은 경합에서의 확장성 한계가 있다.

6.10 요약

출처: 책 315-317쪽 / 원문 PDF 259-261쪽

경쟁 조건은 공유 접근의 순서 의존성에서 발생한다. 임계구역 해결은 상호 배제·진행·한정된 대기를 함께 검토해야 한다. 하드웨어의 가시성과 원자성이 소프트웨어 도구의 토대이며, 세마포·모니터도 올바른 순서와 라이브니스 점검이 필요하다. 원문 연습 문제는 인터럽트 금지와 클록, 바쁜 대기, 단일·다중 처리기에서의 스핀락, 세마포 연산의 원자성 및 공유 은행 계좌의 경쟁 조건을 점검한다. 새 답안은 만들지 않았다.

핵심 정리

- 경쟁 조건의 핵심은 공유 데이터와 접근 순서에 따른 결과 변화이다.

- 임계구역 해법은 상호 배제·진행·한정된 대기 세 조건을 만족해야 한다.
- 명령 재정렬·메모리 가시성과 원자적 갱신은 구별한다.
- 조건 변수 signal은 대기자가 없으면 효과가 없지만 세마포 signal은 상태를 바꾼다.
- 상호 배제만 확인하지 말고 교착 상태·우선순위 역전·경합 성능도 점검한다.

장 마무리 복습

- 공유 데이터 → 경쟁 조건 → 임계구역 프로토콜의 관계를 연결한다.
- 하드웨어 지원 → 원자적 변수·락 → 세마포·모니터로 추상화 수준이 높아진다.
- 바쁜 대기와 봉쇄 대기의 비용은 문맥 교환과 경합 정도에 따라 다르다.

제7장. 동기화 예제

출처: 책 319-347쪽 / 원문 PDF 262-290쪽

학습 주제

- 유한 버퍼·readers-writers·식사하는 철학자 문제에 동기화 도구를 적용하고, Windows·Linux 커널 및 POSIX·Java의 구현과 API를 비교한다.
- 트랜잭션 메모리·OpenMP·함수형 언어도 원문이 제시하는 동기화 대안으로 정리한다.

설명의 흐름

1. 고전적 문제의 공유 상태와 동기화 조건 → 커널의 동기화 도구
2. POSIX의 락·세마포·조건 변수 → Java 모니터와 명시적 락
3. 전통적인 락 사용의 한계 → 트랜잭션·컴파일러 지시·언어 방식

7.1 고전적인 동기화 문제들

출처: 책 319-327쪽 / 원문 PDF 262-270쪽

원문은 새 동기화 방법의 능력을 확인하는 대표 문제로 세 가지 문제를 제시한다. 세마포로 일반 구조를 설명하되 실제 구현에서는 이진 세마포 대신 mutex 락을 사용할 수 있다고 구별한다.

7.1.1 유한 버퍼 문제

출처: 책 320-321쪽 / 원문 PDF 263-264쪽

n개의 버퍼는 각각 한 항목을 저장한다. **mutex=1**은 버퍼 풀의 배타적 접근, **empty=n**은 빈 버퍼 수, **full=0**은 채워진 버퍼 수를 나타낸다. 자원 수를 기다리는 세마포와 실제 버퍼 조작을 보호하는 세마포의 역할을 분리하여 읽는다.

유한 버퍼 생산자 구조

코드 출처: 책 320쪽 / 원문 PDF 263쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
while (true) {
    . . .
    /* produce an item in next_produced */
    . . .
    wait(empty);
    wait(mutex);
    . . .
    /* add next_produced to the buffer */
    . . .
    signal(mutex);
    signal(full);
}
```

코드의 의미

빈 버퍼를 확보하고 버퍼 풀을 배타적으로 수정한 뒤 채워진 버퍼 수를 알린다.

동작 과정

1. empty를 기다린 후 mutex를 획득한다.
2. 항목을 넣고 mutex를 반환한다.
3. full에 신호한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 원문 그림 안의 생략 표기를 그대로 유지한 구조용 의사 코드이다.

유한 버퍼 소비자 구조

코드 출처: 책 321쪽 / 원문 PDF 264쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
while (true) {
    wait(full);
    wait(mutex);
    . . .
    /* remove an item from buffer to next_consumed */
    . . .
    signal(mutex);
    signal(empty);
    . . .
    /* consume the item in next_consumed */
    . . .
}
```

코드의 의미

채워진 버퍼를 기다려 항목을 꺼낸 뒤 빈 버퍼가 생겼음을 알린다.

동작 과정

1. full을 기다린 후 mutex를 획득한다.
2. 항목을 제거하고 mutex를 반환한다.
3. empty에 신호하고 가져온 항목을 소비한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 생산자는 소비자를 위해 채워진 버퍼를, 소비자는 생산자를 위해 빈 버퍼를 만든다는 대칭성이 핵심이다.
- 원문 자체의 생략 표기를 보존하였다.

7.1.2 Readers-Writers 문제

출처: 책 321-323쪽 / 원문 PDF 264-266쪽

여러 reader가 동시에 읽는 것은 허용하지만 writer는 읽는 스레드나 다른 writer와 동시에 공유 데이터베이스에 접근하면 안 된다. 쓰는 동안 배타적 접근을 보장하는 문제가 **readers-writers 문제**이다.

변형	우선 규칙	기아 가능성
첫 번째 문제	writer가 아직 접근 허가를 얻지 못했다면, writer가 기다린다는 이유만으로 새 reader를 기다리게 하지 않는다.	writer
두 번째 문제	writer가 준비되어 기다리면 새 reader의 읽기 시작을 막아 가능한 빨리 쓰게 한다.	reader

원문은 첫 번째 문제의 세마포 해법을 제시한다. read_count는 읽는 프로세스 수, mutex는 그 수의 갱신 보호, rw_mutex는 writer의 배타적 접근과 reader 집단의 접근을 조정한다. 첫 reader가 rw_mutex를 기다리고 마지막 reader가 반환하며, 중간 reader들은 rw_mutex를 다시 기다리지 않는다. writer가 반환할 때 여러 reader 또는 writer 한 개 중 어느 쪽이 진행할지는 스케줄러에 달린다.

Reader-writer 락은 획득할 때 읽기·쓰기 모드를 지정한다. 읽기 모드는 여러 프로세스가 동시에 얻을 수 있지만 쓰기는 하나만 얻는다. 읽기 전용과 쓰기를 식별하기 쉽고 reader가 writer보다 많은 응용에 유용하다. 설정 오버헤드는 mutex·세마포보다 크지만 여러 reader의 병행성으로 이를 상쇄할 수 있다.

7.1.3 식사하는 철학자들 문제

출처: 책 323-327쪽 / 원문 PDF 266-270쪽

원형 테이블의 철학자 5명은 생각과 식사를 반복하고 각각 왼쪽·오른쪽 젓가락 두 개가 있어야 식사할 수 있다. 이는 **교착 상태와 기아 없이 여러 스레드에 여러 자원을 할당하는 문제**를 표현한다. 각 젓가락을 초기값 1의 세마포로 두고 모두 왼쪽부터 기다리게 하면 이웃의 동시 식사는 막지만 모두 왼쪽만 얻은 채 오른쪽을 기다리는 교착 상태가 가능하다.

- 동시에 테이블에 앉는 철학자를 최대 4명으로 제한할 수 있다.
- 임계구역 안에서 두 젓가락을 모두 집을 수 있을 때에만 집도록 할 수 있다.
- 홀수는 왼쪽부터, 짝수는 오른쪽부터 집게 하는 비대칭 순서를 사용할 수 있다.

모니터 해법은 각 철학자를 THINKING·HUNGRY·EATING 상태로 구별하고 양옆 이웃이 EATING이 아닐 때만 먹게 한다. 각자 self 조건 변수로 기다리며 pickup → 식사 → putdown 순서를 지킨다. 이 해법은 이웃의 동시 식사와 교착 상태를 막지만 **기아 가능성까지 없애지는 않는다**.

식사하는 철학자 문제의 모니터 해법

그림·표 출처: 책 326쪽 / 원문 PDF 269쪽

```
monitor DiningPhilosophers
{
   enum {THINKING, HUNGRY, EATING} state[5];
   condition self[5];

   void pickup(int i) {
      state[i] = HUNGRY;
      test(i);
      if (state[i] != EATING)
         self[i].wait();
   }

   void putdown(int i) {
      state[i] = THINKING;
      test((i + 4) % 5);
      test((i + 1) % 5);
   }

   void test(int i) {
      if ((state[(i + 4) % 5] != EATING) &&
       (state[i] == HUNGRY) &&
       (state[(i + 1) % 5] != EATING)) {
          state[i] = EATING;
          self[i].signal();
      }
   }

   initialization_code() {
      for (int i = 0; i < 5; i++)
         state[i] = THINKING;
    }
}
```

그림 7.7 식사하는 철학자 문제에 대한 모니터 해결안

그림 7.7. 모니터 의사 코드 전체. 원문 자체의 표기를 보존하였다.

pickup은 HUNGRY로 바꾸고 양옆의 식사 상태를 검사하며 불가능하면 자기 조건 변수에서 기다린다. putdown은 THINKING으로 바꾼 뒤 양옆 대기자를 검사한다. 원문에 실행 결과 미제시.

7.2 커널 안에서의 동기화

출처: 책 327-330쪽 / 원문 PDF 270-273쪽

Windows는 단일 처리기에서 전역 정보를 접근할 때 관련 인터럽트 핸들러 실행을 잠시 막고, 다중 처리기에서는 짧은 코드에 스핀락을 사용한다. 스핀락 소유 중에는 선점되지 않도록 한다. 커널 밖 스레드 동기화에는 mutex·세마포·event·타이머 등의 **dispatcher 객체**를 제공한다.

Dispatcher 상태	객체와 스레드의 관계
Signaled	객체가 사용 가능하고 획득 시 봉쇄되지 않는다.
Nonsignaled	객체가 사용 불가능하다. 획득하려는 스레드는 대기 상태·객체 대기 큐로 간다.
다시 signaled가 됨	커널은 객체 종류에 따라 하나 또는 여러 대기 스레드를 준비 상태로 옮긴다. Mutex는 하나, event는 기다리는 모든 스레드가 대상일 수 있다.

Windows의 critical-section 객체는 사용자 모드 mutex이며 커널 개입 없이 획득·반환할 수 있다. 다중 처리기에서는 먼저 회전하고 회전이 길면 커널 mutex를 할당하여 CPU를 양보한다. 경쟁이 있을 때만 커널 mutex 비용을 부담하는 방식이다.

원문이 설명하는 Linux 커널은 선점 가능하며 원자적 정수 atomic_t, mutex, 스핀락·세마포 및 reader-writer 형태를 제공한다. 단순 정수 갱신에는 원자적 연산이 효율적이지만 여러 변수가 관련된 경쟁에는 더 정교한 도구가 필요하다. mutex_lock()이 락을 얻지 못하면 수면하고 소유자가 반환할 때 깨어난다.

환경	짧은 구역 보호
단일 처리기	커널 선점을 불가능하게 하고 끝나면 다시 가능하게 한다.
다중 처리기	스핀락을 획득하고 끝나면 방출한다.

원문의 Linux 스핀락·mutex는 재귀적이지 않다. 같은 스레드가 해제 없이 다시 획득하려 하면 봉쇄된다. 각 스레드의 preempt_count로 소유 락 수를 추적하며 0보다 크면 커널 선점이 안전하지 않다. 락이 오래 유지되면 스핀락보다 세마포·mutex를 사용하는 것이 적합하다고 설명한다.

7.3 POSIX 동기화

출처: 책 330-334쪽 / 원문 PDF 273-277쪽

POSIX API는 특정 커널의 내부 기능과 달리 사용자 수준 프로그래머가 쓰는 인터페이스이다. pthread_mutex_t를 pthread_mutex_init()으로 초기화하고, 두 번째 인자로 NULL을 주면 기본 속성을 쓴다. pthread_mutex_lock()과 pthread_mutex_unlock()으로 임계구역 전후를 보호하며 사용 중인 락을 요청한 스레드는 반환까지 봉쇄된다. 원문은 성공 시 0, 오류 시 0이 아닌 코드가 반환된다고 설명한다.

세마포는 Pthreads 자체의 일부가 아닌 POSIX SEM 확장에 속하지만 많은 구현에서 함께 제공된다. **기명 세마포**는 sem_open()으로 생성·열고 관련 없는 프로세스도 이름을 이용해 같은 세마포를 공유한다. 원문의 "SEM", O_CREAT, 0666, 1 예는 없을 때 생성, 다른 프로세스의 읽기·쓰기 권한, 초기값 1을 뜻한다.

구분	생성·공유
기명	이름으로 공통 세마포에 접근한다. sem_open()은 기존 세마포의 디스크립터도 반환한다.
무명	sem_init()에 세마포 포인터·공유 플래그·초기값을 전달한다. 플래그 0은 같은 프로세스의 스레드 사이 공유, 0이 아니면 공유 메모리 영역에 두어 프로세스 사이 공유할 수 있다.

두 유형 모두 sem_wait()로 획득하고 sem_post()로 반환한다. Pthread 조건 변수는 pthread_cond_t와 pthread_cond_init()으로 선언·초기화하고 mutex와 연결한다. C에는 언어 수준 모니터가 없으므로 별도 mutex가 조건 데이터의 무결성을 지킨다.

Pthread 조건 변수로 a == b를 기다림

코드 출처: 책 333쪽 / 원문 PDF 276쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
pthread_mutex_lock(&mutex);
while (a != b)
    pthread_cond_wait(&cond_var, &mutex);

pthread_mutex_unlock(&mutex);
```

코드의 의미

조건 데이터 확인을 mutex로 보호하면서 조건이 참이 아니면 조건 변수에서 기다린다.

동작 과정

1. mutex를 먼저 획득하고 a와 b를 비교한다.
2. wait가 mutex를 해제하여 다른 스레드가 데이터를 갱신할 수 있게 한다.
3. 신호 후 락을 다시 얻어 조건을 루프에서 재확인한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 원문은 프로그램 오류 방지를 위해 신호 후 조건을 다시 확인하는 루프가 중요하다고 명시한다.

공유 데이터를 갱신하는 쪽은 mutex를 얻고 값을 바꾼 다음 pthread_cond_signal()로 대기자 하나에 신호하고 mutex를 해제한다. **signal 호출 자체는 mutex를 해제하지 않는다.** 뒤의 pthread_mutex_unlock()으로 락이 풀리면 신호받은 스레드가 락 소유권을 얻고 wait 호출에서 실행을 재개한다.

7.4 Java에서의 동기화

출처: 책 334-342쪽 / 원문 PDF 277-285쪽

Java 객체마다 락이 연결된다. **synchronized 메소드**는 해당 객체 락을 얻은 스레드만 진입하고 종료하면 반환한다. 다른 스레드는 객체의 **진입 집합(entry set)**에서 락이 풀리기를 기다린다. JVM 명세는 선택 순서를 강제하지 않는다고 원문은 설명한다. 공유 데이터를 조작하는 부분만 synchronized 블록으로 감싸면 전체 메소드보다 **락 범위(lock scope)**를 줄일 수 있다.

객체에는 별도로 **대기 집합(wait set)**도 있다. 락을 가진 스레드가 조건을 만족하지 못하면 wait()를 호출하여 락을 해제하고 봉쇄 상태가 되어 대기 집합으로 간다. notify()는 대기자 하나를 선택하여 진입 집합으로 옮기고 실행 가능 상태로 바꾼다. 이 스레드는 락을 다시 얻은 후에야 wait에서 돌아와 조건을 확인한다. 대기자가 없으면 notify는 무시된다.

원문의 가득 찬 유한 버퍼 처리

1. 생산자가 insert()에서 버퍼가 가득 찼음을 확인하고 wait()하여 락을 놓는다.
2. 소비자가 remove()에 진입해 항목을 제거하고 notify()한다. 이 순간에도 소비자가 락을 소유한다.
3. 생산자는 대기 집합에서 진입 집합으로 이동한다.
4. 소비자가 메소드를 종료하여 락을 반환한다. 생산자가 다시 획득하여 루프에서 빈 공간을 재확인하고 삽입한다.

Java 객체의 진입 집합과 대기 집합

그림·표 출처: 책 337쪽 / 원문 PDF 280쪽

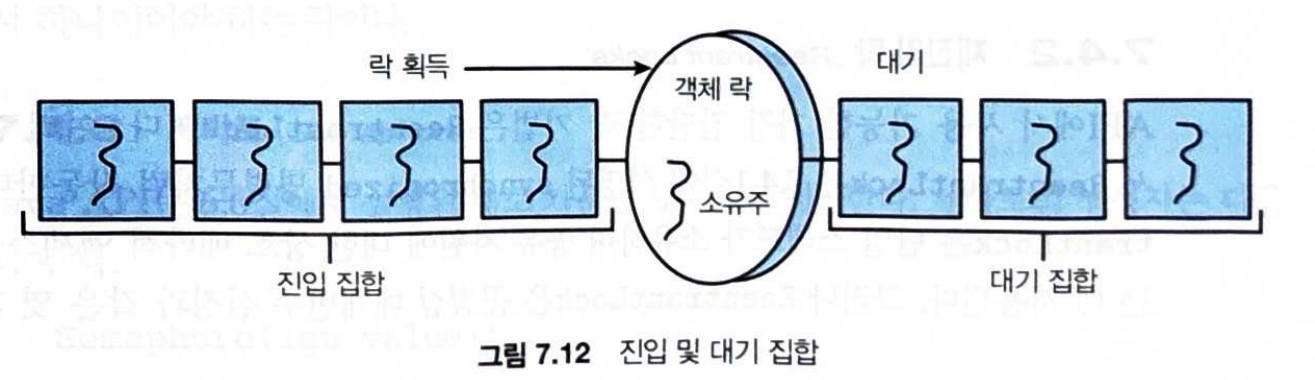


그림 7.12 진입 및 대기 집합

그림 7.12. 원문 관계도.

객체 락의 소유자는 하나이며 진입 대기와 조건 대기를 구별한다. notify가 즉시 락을 넘기는 것으로 해석하지 않는다.

ReentrantLock은 한 스레드가 소유하며 공정성 설정 등 추가 기능을 제공한다. 호출자가 이미 소유한 락도 다시 획득할 수 있어 재진입이라고 부른다. 사용할 수 없는 락은 unlock()으로 배정 가능해질 때까지 기다린다. 읽기만 하는 스레드의 병행성을 위해 ReentrantReadWriteLock도 제공된다.

Java ReentrantLock의 획득·반환

코드 출처: 책 339쪽 / 원문 PDF 282쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
Lock key = new ReentrantLock();

key.lock();
try {
    /* critical section */
}
finally {
    key.unlock();
}
```

코드의 의미

명시적 락으로 공유 자원에 상호 배타적으로 접근하며 finally에서 반환한다.

동작 과정

1. lock()으로 획득한 뒤 try 본체에서 임계구역을 수행한다.
2. 정상 완료 또는 try 안 예외 발생 때 finally에서 해제한다.

원문 실행 결과

원문에 실행 결과 미제시

원문 주의 사항

- 원문은 lock()을 try 밖에 둔다. 획득 실패 예외가 생겼는데 finally에서 해제하면 다른 예외로 원래 실패가 가려질 수 있기 때문이다.

Java의 카운팅 세마포는 Semaphore(int value)로 초기화하며 원문은 음수 초기값도 허용한다고 설명한다. acquire()는 인터럽트되면 InterruptedException을 던질 수 있다. 원문 예는 release()를 finally에 둔다. 조건 변수는 ReentrantLock의 newCondition()에서 얻으며 await()·signal()을 사용한다.

언어 수준 Java 모니터의 wait·notify는 객체의 무명 조건 변수 하나에 연결된다. 명시적 **Condition**은 기다리는 조건을 나누어 특정 조건의 대기자에게 알릴 수 있다. 원문의 5스레드 순번 예는 각 조건 변수 배열에서 자기 차례를 기다리고 작업 후 다음 turn에 신호한다. await는 연결된 재진입 락을 해제하지만 signal은 조건에만 알리며 실제 해제는 unlock()이 수행한다.

7.5 대체 방안들

출처: 책 342-345쪽 / 원문 PDF 285-288쪽

트랜잭션 메모리의 메모리 트랜잭션은 메모리 읽기·쓰기 연산의 원자적인 순서이다. 모든 연산이 완수되면 commit하고 그렇지 않으면 수행한 변경을 취소하여 시작 전으로 rollback한다. 원문이 가정한 atomic { S } 구조에서는 개발자 대신 트랜잭션 시스템이 원자성을 보장한다. 락을 쓰지 않는 이 방법에서는 락에 의한 교착 상태가 없고 병행 가능한 읽기를 시스템이 식별할 수 있다고 설명한다.

구현	원문의 방식과 한계
STM	특수 하드웨어 없이 컴파일러가 검사 코드를 넣어 동시 실행·저수준 락이 필요한 지점을 관리한다.

구현	원문의 방식과 한계
HTM	캐시 계층과 일관성 프로토콜로 충돌을 관리한다. 코드 계측 오버헤드가 작지만 하드웨어 구조의 변경이 필요하다.

OpenMP는 컴파일러 지시와 API로 공유 메모리 병렬성을 제공한다. #pragma omp critical 뒤 코드 구역은 한순간 하나의 스레드만 실행한다. 여러 임계구역에 이름을 주면 같은 이름을 가진 구역에서 동시에 두 스레드가 활동할 수 없다. 표준 mutex보다 사용이 쉽지만 개발자가 경쟁 가능성을 찾아 보호해야 하며 여러 임계구역이 관여하면 교착 상태가 여전히 가능하다.

함수형 프로그래밍 언어는 명령형 언어의 변경 가능한 상태와 다른 방식을 따른다. 원문은 한 번 값이 배정된 변수는 바뀌지 않는다는 점 때문에 이 장의 공유 상태 경쟁·교착 문제를 피하는 방향으로 설명하고 Erlang·Scala를 언급한다. 이 비교는 원문이 기술하는 범위에 한정하며 다른 언어 특성을 덧붙이지 않는다.

7.6 요약

출처: 책 345-347쪽 / 원문 PDF 288-290쪽

세 고전 문제는 자원 수 제어·읽기와 쓰기의 병행성·여러 자원 동시 할당을 각각 드러낸다. 커널 내부 동기화와 사용자 수준 POSIX·Java API를 구별하고, 락 소유권과 조건 대기의 이동을 함께 본다. 원문 연습 문제는 운영체제가 여러 락을 제공하는 이유, 철학자 교착, dispatcher 상태, 원자적 정수 연산 등을 확인한다. 외부 API 설명이나 새 계산 결과를 추가하지 않았다.

핵심 정리

- 유한 버퍼에서 empty·full은 자원 수, mutex는 버퍼 조작의 상호 배제를 담당한다.
- Reader 우선과 writer 우선은 서로 다른 쪽의 기아 가능성을 가진다.
- 철학자 모니터의 교착 상태 방지와 기아 방지는 같은 보장이 아니다.
- 조건 대기와 신호는 락의 획득·해제와 구별하며 재개 후 조건을 확인한다.
- Java의 진입 집합·대기 집합과 POSIX 조건 변수의 mutex 관계를 구분한다.

장 마무리 복습

- 고전 문제의 요구 조건 → 세마포·모니터의 상태와 연산 순서를 연결한다.
- 커널의 짧은 보호에는 스핀·선점 제어, 오래 기다리는 상황에는 봉쇄 도구라는 원문 맥락을 복습한다.
- 트랜잭션·OpenMP·함수형 방식은 각각 원자성 책임·지시 기반 보호·상태 모델을 바꾸는 접근으로 읽는다.

제8장. 교착 상태

출처: 책 349-381쪽 / 원문 PDF 291-323쪽

학습 주제

- 한정된 자원을 점유하고 서로 기다리는 교착 상태를 시스템 모델과 자원 할당 그래프로 표현한다.
- 예방·회피·탐지·회복의 차이, 안전 상태와 은행원 알고리즘의 조건을 원문의 그래프·행렬 예와 함께 정리한다.

설명의 흐름

1. 요청·사용·방출 모델 → 다중 스레드 교착·라이브락
2. 네 필요조건과 자원 할당 그래프 → 예방과 안전 상태 기반 회피
3. 은행원 알고리즘 → 현재 상태 탐지 → 종료·선점으로 회복

8.1 시스템 모델

출처: 책 349-351쪽 / 원문 PDF 291-293쪽

자원은 유한하며 여러 유형으로 나뉜다. 같은 유형의 인스턴스들은 서로 동등하여 그중 어느 것을 할당해도 요청을 충족해야 한다. CPU·파일·입출력 장치뿐 아니라 mutex·세마포도 자원이다. 각 락은 보호하는 자료구조가 다르므로 일반적으로 고유한 자원 클래스로 취급한다.

정상 자원 사용 순서

1. 요청: 즉시 허용되지 않으면 자원을 얻을 때까지 기다린다.
2. 사용: 할당받은 자원을 이용하여 작업한다.
3. 방출: 사용을 마치면 반환한다.

운영체제는 자원의 가용·할당 상태와 소유 스레드를 기록하고 사용할 수 없는 자원 요청자를 대기 큐에 둘 수 있다. **교착 상태**는 집합 안의 모든 스레드가 그 집합의 다른 스레드만 일으킬 수 있는 사건을 기다리는 상황이다. 주로 자원 획득·방출을 다루지만 IPC 등 다른 사건도 원인이 될 수 있다.

8.2 다중 스레드 응용에서의 교착 상태

출처: 책 351-354쪽 / 원문 PDF 293-296쪽

원문의 두 Pthread 스레드는 first_mutex와 second_mutex를 서로 반대 순서로 획득한다. 한쪽이 first_mutex, 다른 쪽이 second_mutex를 소유한 후 상대 락을 요청하면 둘 다 기다린다. 그러나 한 스레드가 두 락을 얻고 모두 반환한 다음 다른 스레드가 시작하면 교착은 없다. **가능한 코드 구조와 실제 특정 실행에서의 발생을 구분해야 한다.** 특정 스케줄링에서만 드러나므로 검사도 어렵다.

라이브락(livelock)은 스레드가 봉쇄되지는 않지만 실패하는 행동을 계속 재시도하여 진행하지 못하는 장애이다. 원문은 trylock으로 두 번째 락을 얻지 못하면 자기 락을 반환하고 다시 시도하는 두 스레드를 예로 든다. 서로 동시에 반복하면 계속 실패할 수 있다. 원문은 일반적으로 재시도 시간을 무작위화하여 피하는 접근을 설명하고 Ethernet 충돌 후 재전송 대기와 연결한다.

장애	진행하지 못하는 이유
교착 상태	집합 내 다른 스레드만 발생시킬 수 있는 사건을 기다리며 봉쇄된다.
라이브락	실패한 행동을 계속 재시도하여 봉쇄 없이도 앞으로 나아가지 못한다.

8.3 교착 상태 특성

출처: 책 353-358쪽 / 원문 PDF 295-300쪽

필요조건	원문의 의미
상호 배제	최소 한 자원이 비공유 모드로 점유되어 한 번에 한 스레드만 사용한다.
점유하며 대기	최소 한 자원을 소유하면서 다른 스레드 소유 자원을 추가로 기다린다.
비선점	자원은 강제로 회수할 수 없고 소유자가 작업을 마친 뒤 자발적으로 반환한다.
순환 대기	대기 스레드들의 각자가 다음 스레드가 점유한 자원을 기다리는 고리가 있다.

교착 상태가 가능하려면 네 조건이 **동시에** 성립해야 한다. 순환 대기는 점유하며 대기를 암시하므로 완전히 독립적인 조건은 아니지만 예방 방법을 설명하기 위해 나누어 본다.

자원 할당 그래프는 스레드를 원, 자원 유형을 사각형, 각 인스턴스를 사각형 내부 점으로 표시한다. 스레드 → 자원은 요청 간선, 인스턴스 → 스레드는 할당 간선이다. 요청이 충족되면 요청 간선을 할당 간선으로 바꾸고 반환하면 할당 간선을 지운다.

그래프 조건	판단
사이클 없음	교착 상태가 없다.
사이클의 자원 유형이 각각 단일 인스턴스	사이클은 교착 상태의 필요충분조건이다.
여러 인스턴스를 가진 자원 유형 포함	사이클은 필요조건이지만 충분조건은 아니다. 다른 스레드의 반환으로 고리를 풀 수 있다.

교착 상태인 자원 할당 그래프

그림·표 출처: 책 357쪽 / 원문 PDF 299쪽

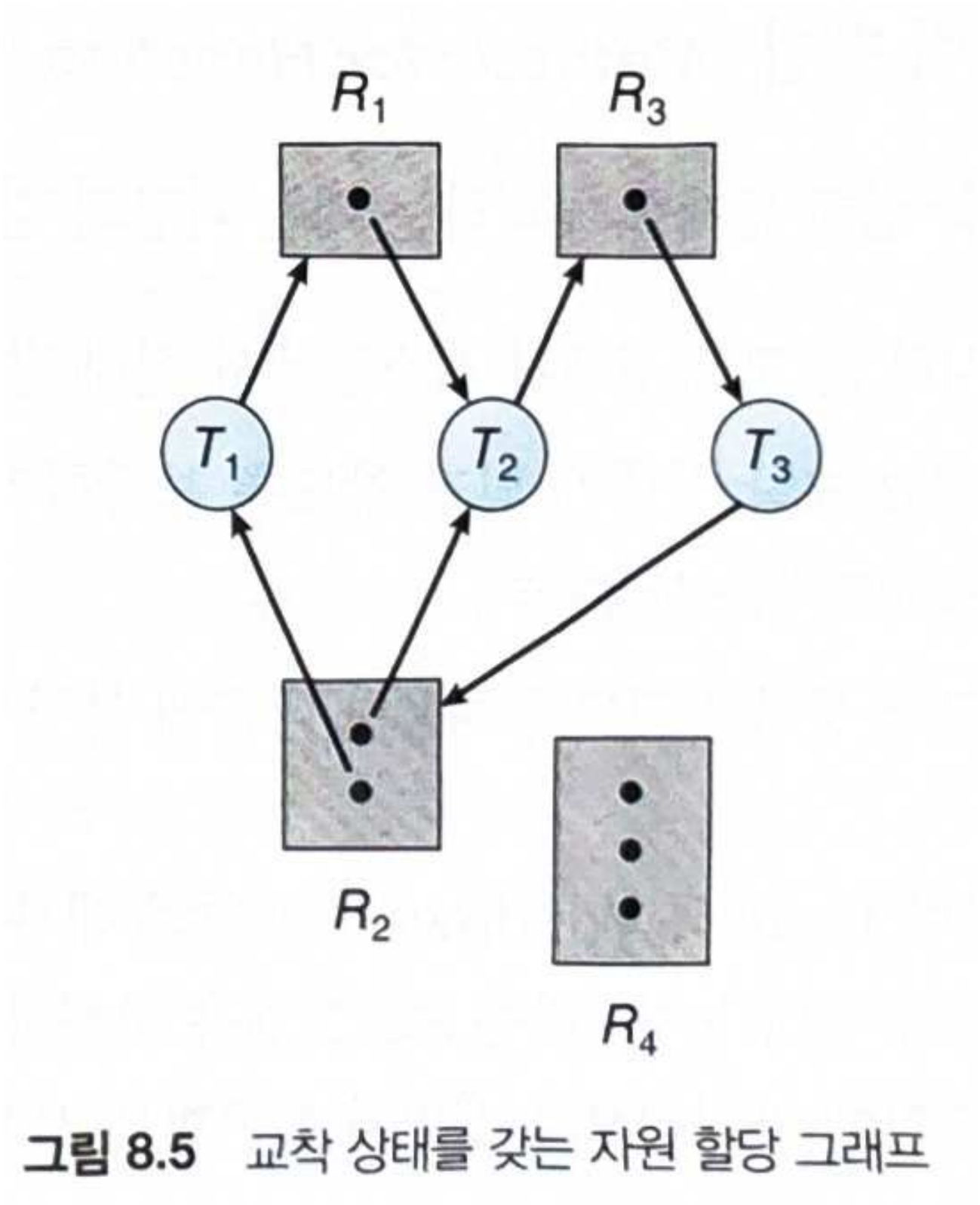


그림 8.5 교착 상태를 갖는 자원 할당 그래프

그림 8.5. 원문 그래프.

T_1·T_2·T_3가 서로 점유한 자원 반환을 기다린다. R_2의 인스턴스가 둘이어도 이 그림의 두 인스턴스는 기다리는 스레드에 점유되어 있다.

사이클이 있지만 교착 상태가 아닌 그래프

그림·표 출처: 책 358쪽 / 원문 PDF 300쪽

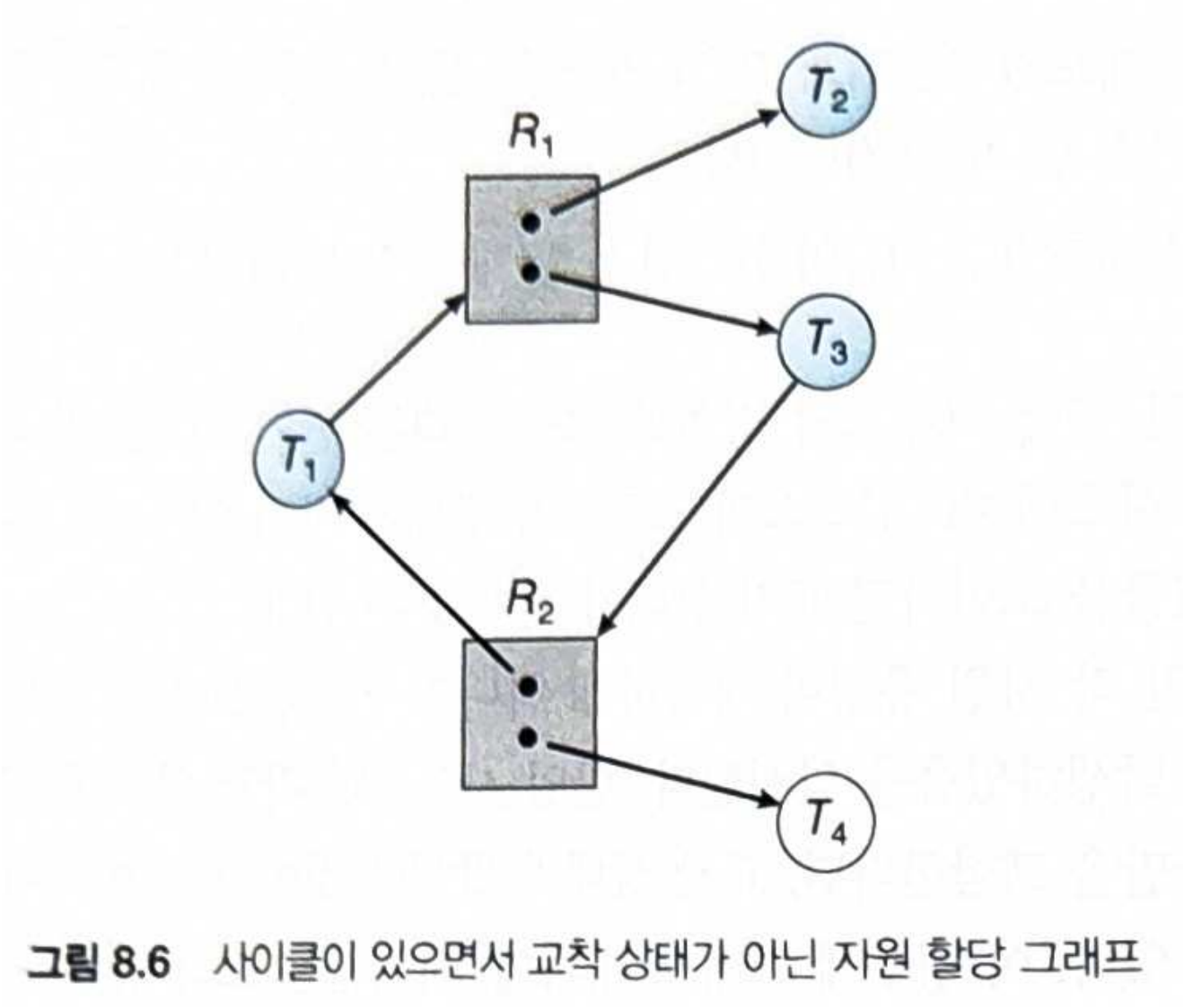


그림 8.6 사이클이 있으면서 교착 상태가 아닌 자원 할당 그래프

그림 8.6. 원문 그래프.

T_4가 R_2 인스턴스를 반환하면 T_3에 할당할 수 있어 사이클이 사라진다. 따라서 여러 인스턴스에서는 사이클만으로 교착 상태라고 단정할 수 없다.

8.4 교착 상태 처리 방법

출처: 책 358-360쪽 / 원문 PDF 300-302쪽

접근	기준과 비용
무시	교착이 발생하지 않는 것으로 취급한다. 원문은 드문 발생에 비해 지속적인 처리 비용이 클 수 있고 수동 복구를 사용할 수 있다고 설명한다.
예방·회피	절대 교착 상태가 되지 않게 제약하거나 현재 할당을 허가할지 판단한다.
탐지 후 회복	교착을 허용하되 상태 조사 알고리즘과 복구 알고리즘을 둔다.

예방은 네 필요조건 중 적어도 하나를 성립하지 않게 하도록 자원 요청 방법을 제한한다. **회피**는 스레드의 미래 최대 요구 등 추가 정보를 알고 현재 가용·할당 상태와 함께 요청 허가를 결정한다. 탐지·회복이

없으면 인식하지 못한 교착이 점차 자원을 묶어 시스템 성능을 낮출 수 있다. 자원 클래스별로 다른 기본 방법을 조합할 수도 있다.

8.5 교착 상태 예방

출처: 책 360-364쪽 / 원문 PDF 302-306쪽

깨뜨릴 조건	방법	한계
상호 배제	가능한 자원을 공유 가능하게 한다.	읽기 전용 파일과 달리 mutex 같은 자원은 본래 공유할 수 없어 일반적인 해결이 안 된다.
점유하며 대기	실행 전에 모두 요청하거나, 아무 자원도 없을 때만 새 요청을 허용한다.	오래 사용하지 않는 자원을 점유하여 이용률이 낮고 필요한 자원을 한꺼번에 얻지 못하는 기아가 가능하다.
비선점	새 요청을 즉시 못 들어주면 이미 가진 자원을 회수하고 나중에 함께 재획득한다. 또는 대기 중인 다른 스레드에서 선점한다.	상태 저장·복원이 쉬운 CPU 레지스터·트랜잭션 등에 적용되지만 mutex·세마포에는 일반적으로 적용되지 않는다.
순환 대기	모든 자원 유형에 전체 순서를 부여하고 오름차순 요청을 강제한다.	순서 설정만으로는 부족하며 프로그램이 지켜야 한다. 동적 락 획득에도 주의한다.

순환 대기 예방에서 함수 F는 각 자원 유형에 고유 자연수 번호를 준다. 이미 R_i를 가진 스레드는 더 큰 번호의 자원만 요청한다. 같은 유형의 인스턴스가 여러 개 필요하면 한 번의 요청으로 모두 받아야 한다. 대안은 새 자원 이상 순서의 기존 자원을 모두 먼저 반환하게 하는 것이다. 순환 고리가 있다면 순서가 계속 증가하여 결국 자기보다 작아져야 하는 모순이 생긴다는 것이 원문의 증명이다.

원문의 계좌 이체 예는 from·to 인자에 따라 동적으로 락을 얻으면 두 스레드가 반대 계좌 순서로 호출하여 교착할 수 있음을 보인다. Linux lockdep은 실행 중 락 획득·반환 패턴과 인터럽트 활성 상태를 검사하여 잠재적 교착을 보고한다. 원문은 개발·수정 검사 도구이며 속도 저하 때문에 실제 운영용으로 쓰지 않는다고 설명한다. 외부 도구 사용법은 덧붙이지 않았다.

8.6 교착 상태 회피

출처: 책 363-371쪽 / 원문 PDF 305-313쪽

예방의 요청 제한은 이용률·처리율을 낮출 수 있다. 회피는 각 스레드의 **최대 요구 수** 같은 추가 정보를 받아 요청 후의 할당 상태를 판단한다. 상태는 가용 자원 수·현재 할당 수·최대 요구 수로 표현한다. 현재 여유 자원이 충분하다는 사실만으로 요청을 허용하지 않는다.

8.6.1 안전 상태

출처: 책 365-366쪽 / 원문 PDF 307-308쪽

안전 상태는 모든 스레드가 최대 요구량을 요청해도 어떤 순서로 하나씩 끝낼 수 있는 **안전 순서**가 있는 상태이다. 순서의 각 스레드에 필요한 추가 자원은 현재 가용량과 앞서 끝난 스레드가 반납할 자원으로 충족해야 한다. 즉시 만족할 수 없다면 앞선 스레드의 종료까지 기다리게 한다.

안전 상태는 교착 상태가 아니다. 교착 상태는 불안전하지만 **불안전 상태가 반드시 교착 상태인 것은 아니다**. 이후 실제 요청에 따라 교착할 수 있다는 뜻이다. 원문의 12개 자원 예는 최대 요구 10·4·9, 현재 점유 5·2·2, 가용 3에서 $T_1 \rightarrow T_0 \rightarrow T_2$로 모두 끝낼 수 있다. 여기서 T_2에 하나 더 주면 남은 요청을 모두 처

리할 안전 순서가 없어 불안전해진다. 회피는 항상 안전 상태 안에 머무르게 하므로 가용 자원이 있어도 대기시킬 수 있다.

8.6.2 자원 할당 그래프 알고리즘

출처: 책 366-367쪽 / 원문 PDF 308-309쪽

각 자원 유형에 인스턴스가 하나씩인 경우 미래에 요청할 수 있음을 **예약 간선(claim edge)**이라는 점선으로 표시한다. 실행 전 모든 예약을 표시하며 실제 요청 시 요청 간선으로, 반환 시 할당 간선을 예약 간선으로 되돌린다. 모든 관련 간선이 예약인 경우에만 새 예약을 추가하도록 조건을 완화할 수 있다.

요청 간선을 할당 간선으로 바꾸어도 사이클이 생기지 않는 경우에만 허용한다. 생기면 불안전해지므로 자원이 현재 가용해도 기다리게 한다. 원문은 사이클 검사에 n^2 차수의 연산이 필요하다고 설명한다. 여러 인스턴스의 경우는 다음 은행원 알고리즘으로 다룬다.

8.6.3 은행원 알고리즘

출처: 책 368-371쪽 / 원문 PDF 310-313쪽

스레드는 시작할 때 자원 유형별 최대 개수를 미리 신고하며 총 보유량을 넘을 수 없다. 시스템은 요청을 들어준 뒤에도 안전한지 검사한다. 아래에서 n은 스레드 수, m은 자원 유형 수이며 벡터 비교는 모든 성분별 비교이다.

자료구조	크기와 의미
Available	m 벡터. 유형별 현재 가용 개수.
Max	n×m 행렬. 스레드별 유형별 최대 요구.
Allocation	n×m 행렬. 현재 할당된 개수.
Need	n×m 행렬. 앞으로 추가 요청할 수 있는 개수. Need[i][j] = Max[i][j] - Allocation[i][j].

은행원 안전성 알고리즘

그림·표 출처: 책 369쪽 / 원문 PDF 311쪽

8.6.3.1 안전성 알고리즘 _Safety Algorithm

시스템이 안전한지 아닌지를 알아낼 수 있는 알고리즘은 아래와 같다:

1. ***Work***와 ***Finish***는 크기가 m과 n인 벡터이다. ***Work*** = ***Available***로 초기 값을 준다. $i = 0, 1, ..., n - 1$에 대해서 ***Finish***[i] = ***false***를 초기 값으로 준다.
2. 아래 두 조건을 만족시키는 i 값을 찾는다.
   a. ***Finish***[i] == ***false***
   b. $Need_i \leq Work$
   그러한 i 값을 찾을 수 없다면 step 4로 간다.
3. $Work = Work + Allocation_i$
   Finish[i] = ***true***
   2번으로 간다.
4. 모든 i 값에 대해 ***Finish***[i] == ***true***이면 이 시스템은 안전 상태에 있다.

이 알고리즘으로 안전 여부를 알아내는 데에는 $m \times n^2$개의 연산이 필요하다.

8.6.3.1 원문 알고리즘.

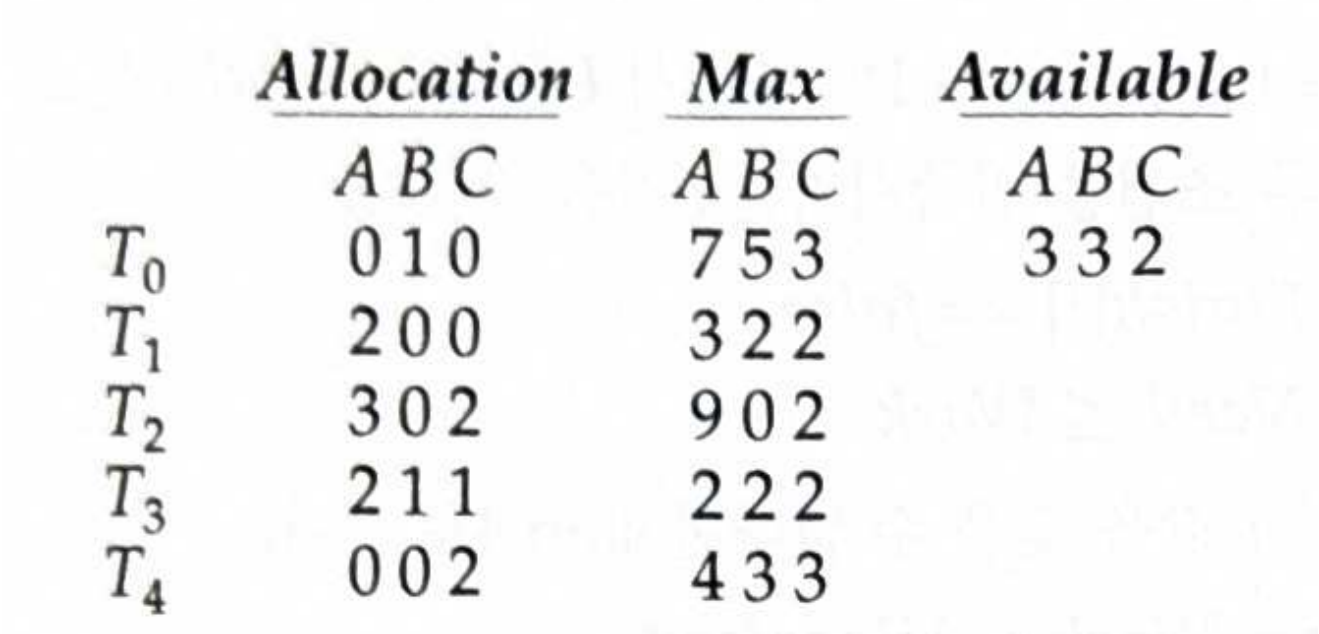
Work=Available, Finish=false에서 시작하여 $Need_i \leq Work$인 미완료 스레드를 찾아 끝낼 수 있다고 보고 $Allocation_i$를 Work에 돌려준다. 모두 Finish=true이면 안전하다. 원문의 부등호·기호를 보존하였다.

자원 요청 검사

1. $Request_i \leq Need_i$인지 검사한다. 아니면 오류로 처리한다.
2. $Request_i \leq Available$인지 검사한다. 아니면 당장 자원이 없어 대기한다.
3. 가상으로 할당해 Available을 줄이고 $Allocation_i$를 늘리며 $Need_i$를 줄인다.
4. 새 상태가 안전하면 확정하고, 불안전하면 이전 상태로 복원하고 기다리게 한다.

은행원 알고리즘의 원문 행렬 예

그림·표 출처: 책 370쪽 / 원문 PDF 312쪽



	Allocation	Max	Available
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

8.6.3.3의 최초 Allocation·Max·Available 표.

총 자원은 A=10, B=5, C=7이고 Available=(3,3,2)이다. 표의 값은 원문 그대로 보존하였다.

원문 최초 Need는 T_0=(7,4,3), T_1=(1,2,2), T_2=(6,0,0), T_3=(0,1,1), T_4=(4,3,1)이다. 안전 순서 $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2 \rightarrow T_0$가 있다. T_1의 (1,0,2) 요청을 가상 할당하면 Available=(2,3,0)이 되고 $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_0 \rightarrow T_2$가 안전하므로 즉시 허용한다. 이어 T_4의 (3,3,0)은 자원 부족, T_0의 (0,2,0)은 자원이 있어도 불안전하므로 허용하지 않는 것이 원문의 결과이다. 새 순서나 계산을 추가하지 않았다.

8.7 교착 상태 탐지

출처: 책 371-375쪽 / 원문 PDF 313-317쪽

예방·회피를 사용하지 않는 시스템은 교착 발생 여부를 검사하고 발생하면 회복할 방법이 필요하다. 실행 상태 정보 유지·탐지 알고리즘 실행·복구 손실이 모두 오버헤드이다.

각 자원 유형이 하나씩이면 자원 할당 그래프에서 자원 노드를 제거하여 **대기 그래프(wait-for graph)**를 만든다. $T_i \rightarrow T_j$는 T_i가 T_j 소유 자원의 반환을 기다림을 뜻한다. 이 경우 사이클이 있는 것과 교착이 동치이다. 원문은 그래프 사이클 탐지에 $O(n^2)$ 연산이 필요하다고 설명한다.

여러 인스턴스에서는 Available·Allocation과 현재 요청 행렬 **Request**를 사용한다. 회피의 Need가 미래 최대 추가 요구인 것과 달리 Request는 지금 기다리는 요청이다. Work=Available로 시작하고 Allocation이 0인 스레드는 Finish=true, 나머지는 false로 둔다. 미완료이면서 $Request_i \le Work$인 스레드를 찾아 $Allocation_i$를 돌려받는다고 가정한다. 더 찾지 못했을 때 Finish=false인 스레드들이 교착에 연루된다. 원문은 $m \times n^2$ 연산을 명시한다.

여러 인스턴스의 교착 상태 탐지 절차

그림·표 출처: 책 373쪽 / 원문 PDF 315쪽

1. ***Work***와 ***Finish***는 크기가 m과 n인 벡터이다. ***Work*** = ***Available***로 초기 값을 준다. $i = 0, 1, ..., n-1$에 대해서 $\boldsymbol{Allocation}_i \neq 0$이면 ***Finish***$[i]$ = ***false***이다. 그렇지 않으면 ***Finish***$[i]$ = ***true***이다.
2. 아래 두 조건을 만족시키는 i 값을 찾는다.
   a. ***Finish***$[i]$ == ***false***
   b. $\boldsymbol{Request}_i \leq \boldsymbol{Work}$
   그러한 i 값을 찾을 수 없다면 4번으로 간다.
3. $\boldsymbol{Work} = \boldsymbol{Work} + \boldsymbol{Allocation}_i$
   Finish$[i]$ = ***true***
   2번으로 간다.
4. 어떠한 i 값에 대해 $(0 \leq i < n)$ ***Finish***$[i]$ == ***false***이면 이 시스템은 교착 상태 상태에 빠져 있는 것이다. 그리고 T_i가 교착 상태 상태에 빠져있다.

이 탐지 알고리즘을 실행하는 데에는 $m \times n^2$개의 연산이 필요하다.

8.7.2 원문의 4단계 알고리즘.

현재 Request를 충족한 스레드가 더 요청하지 않고 끝낸다는 낙관적 가정으로 자원을 회수한다. 이 가정이 틀려 이후 교착하면 다음 탐지에서 나타난다는 설명도 원문에 있다.

원문의 예에서 A=7, B=2, C=6이고 Available=(0,0,0)이어도 $T_0 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1 \rightarrow T_4$로 끝낼 수 있어 교착이 아니다. T_2가 C 하나를 더 요청하면 T_0 자원을 회수해도 나머지 요청을 충족하지 못하여 T_1·T_2·T_3·T_4가 교착에 연루된다.

탐지 빈도는 교착 발생 빈도와 보통 연루되는 스레드 수에 달린다. 즉시 만족되지 않는 요청마다 검사하면 원인 시점을 찾기 쉽지만 비용이 크다. 일정 간격이나 CPU 이용률 하락 때만 검사하면 비용은 줄어도 원인 스레드를 특정하기 어려워진다. 원문의 BCC 도구·Java 스레드 덤프·데이터베이스 예는 모두 대기 관계를 관찰하여 사이클을 찾는 의미로 연결된다. 데이터베이스는 희생 트랜잭션을 중단·rollback하여 락을 풀고 나머지를 재개한다.

8.8 교착 상태로부터 회복

출처: 책 376-377쪽 / 원문 PDF 318-319쪽

탐지 후 운영자에게 알려 수동 처리하거나 시스템이 자동으로 회복할 수 있다. 자동 방법은 한 개 이상의 프로세스·스레드를 종료하여 순환 대기를 깨거나 점유 자원을 선점하는 것이다. 모든 교착 프로세스를 한꺼번에 중지하면 확실하지만 이미 수행한 계산을 잃는다. 한 개씩 중지하면 매번 재탐지하는 비용이 든다.

종료 대상은 우선순위, 지금까지와 남은 실행 시간, 점유한 자원의 종류·수, 앞으로 필요한 자원, 중지해야 할 프로세스 수 등을 고려한다. 파일 갱신 중 종료하면 파일이 부정확해질 수 있고 락을 풀어도 공유 데이터 무결성이 보장되는 것은 아니다.

선점 시 고려	원문 조건
희생자 선택	점유 자원·실행 시간 등의 비용을 최소화하는 순서를 선택한다.
후퇴(rollback)	자원이 없어진 프로세스는 정상 진행할 수 없어 안전한 상태로 되돌린다. 완전 재시작은 간단하지만 필요한 만큼만 후퇴하려면 더 많은 상태 정보를 유지해야 한다.
기아 방지	같은 프로세스가 계속 희생되지 않도록 한정된 횟수만 선택되게 해야 한다. 비용에 후퇴 횟수를 포함하는 것이 일반적인 해법이다.

8.9 요약

출처: 책 378-381쪽 / 원문 PDF 320-323쪽

교착 상태는 네 필요조건·요청과 할당 관계·현재와 미래 요구량을 연결하여 판단한다. 사이클의 의미는 자원 인스턴스 수에 따라 다르며 안전·불안전·교착을 구별한다. 원문 연습 문제는 안전 상태, 은행원 알고리즘, 봉쇄와 자원 순서, 탐지·기아 및 자원 선점 정책을 점검한다. 제공된 문제의 새 풀이를 작성하지 않았다.

핵심 정리

- 교착의 네 필요조건은 동시에 성립해야 한다.
- 자원 할당 그래프의 사이클은 단일 인스턴스에서는 필요충분, 여러 인스턴스에서는 필요조건이다.
- 불안전과 교착은 같지 않다. 회피는 요청 후 안전 순서를 보존하는지 본다.
- 은행원의 Need와 탐지의 현재 Request를 구분한다.
- 탐지만으로 끝나지 않으며 종료·자원 선점·후퇴·기아 대책을 함께 마련해야 한다.

장 마무리 복습

- 요청·사용·방출 → 대기 고리 → 네 조건의 제거 또는 상태 검사로 연결한다.
- 예방은 요청 규칙, 회피는 미래 최대 요구와 안전성, 탐지는 현재 요청·할당을 사용한다.
- 교착 복구의 선택은 잃는 계산·자료 무결성·반복 희생의 비용과 연결된다.

제9장. 메인 메모리

출처: 책 385-426쪽 / 원문 PDF 325-366쪽

학습 주제

- CPU가 만드는 논리 주소와 실제 메모리의 물리 주소를 연결하는 방법을 메모리 보호·주소 바인딩·MMU의 관점에서 정리한다.
- 연속 할당과 단편화에서 출발하여 페이징·TLB·페이지 테이블 구조·스와핑으로 확장하고, 원문이 제시한 Intel·ARM 주소 변환 사례를 다룬다.

설명의 흐름

1. 하드웨어와 주소 바인딩 → 논리·물리 주소 → 동적 적재·연결
2. 연속 할당의 배치와 단편화 → 페이지·프레임 변환 → TLB·보호·공유
3. 페이지 테이블 구조 → 스와핑 → IA-32·x86-64·ARMv8 사례

9.1 배경

출처: 책 385-393쪽 / 원문 PDF 325-333쪽

다중 프로그래밍으로 CPU 이용률·응답성을 높이려면 여러 프로세스를 메모리에 유지해야 한다. 메모리는 고유 주소를 가진 바이트들의 배열이며 CPU는 명령 인출·해독·피연산자 인출·실행·결과 저장을 수행한다. 메모리는 제시된 주소에 따라 읽고 쓸 뿐 그 주소가 생성된 방식이나 내용의 의미를 알지 못한다. 메모리 관리 방식은 하드웨어 설계와 긴밀하게 연결된다.

9.1.1 기본 하드웨어

출처: 책 386-388쪽 / 원문 PDF 326-328쪽

CPU가 직접 접근하는 범용 저장장치는 메인 메모리와 코어 레지스터이다. 명령·데이터가 디스크에 있으면 처리 전 메모리로 옮겨야 한다. 레지스터와 메인 메모리의 접근 속도 차이 때문에 CPU가 데이터 대기 중 stall될 수 있으며 캐시가 이를 완화한다.

동시에 운영체제와 다른 사용자 프로세스의 공간을 보호해야 한다. 각 프로세스에 독립된 합법 주소 영역을 정하고 사용자 모드의 모든 주소를 하드웨어가 검사한다. 원문의 **기준(base) 레지스터**는 가장 작은 합법 물리 주소, **상한(limit) 레지스터**는 영역 크기를 저장한다. 기준 이상이며 기준+상한보다 작은 범위인지 검사하는 그림 9.2의 관계가 보호 기준이다. 범위를 벗어나면 운영체제에 트랩한다.

두 레지스터는 커널 모드에서만 가능한 특권 명령으로 운영체제만 적재한다. 커널은 사용자 영역과 운영체제 영역을 모두 접근할 수 있어 프로그램 적재, 오류 덤프, 시스템 콜 인수 처리, 문맥 교환 등을 수행한다. 보호 검사를 소프트웨어 호출마다 수행하는 것으로 해석하지 않는다.

9.1.2 주소의 할당

출처: 책 388-390쪽 / 원문 PDF 328-330쪽

프로그램은 실행 파일에서 메모리로 적재되어 프로세스 문맥에 배치된다. 주소는 원시 프로그램의 심볼에서 컴파일러가 만드는 재배치 가능 주소, 링커·로더가 만드는 절대 주소로 바뀔 수 있다. 각 바인딩은 한 주소 공간을 다른 주소 공간에 사상한다.

시점	조건과 결과
컴파일 시간	적재 위치를 미리 알면 절대 코드를 만든다. 시작 위치를 변경하려면 다시 컴파일한다.
적재 시간	컴파일 때 위치를 모르면 재배치 가능 코드를 만든다. 실제 적재 시 바인딩하며 시작 위치가 바뀌면 다시 적재한다.
실행 시간	실행 중 다른 메모리 위치로 이동할 수 있도록 바인딩을 실행까지 미룬다. 특별한 하드웨어 지원이 필요하다.

9.1.3 논리 대 물리 주소 공간

출처: 책 390-391쪽 / 원문 PDF 330-331쪽

논리 주소는 CPU가 생성하는 주소이고 **물리 주소**는 메모리 주소 레지스터에 제시되어 메모리가 취급하는 주소이다. 컴파일·적재 시간 바인딩에서는 두 주소가 같지만 실행 시간 바인딩에서는 다르다. 이 책은 논리 주소와 가상 주소를 같은 뜻으로 사용한다.

메모리 관리 장치(MMU)는 실행 중 논리 주소를 물리 주소로 바꾼다. 단순한 동적 재배치에서는 재배치 레지스터 값을 모든 논리 주소에 더한다. 원문의 값 14000과 논리 주소 346은 물리 주소 14346으로 연결된다. 사용자 프로그램은 논리 주소를 생성·저장·비교할 수 있지만 실제 메모리 참조 때 하드웨어가 바인딩하므로 실제 물리 위치를 직접 접근하는 것으로 이해하면 안 된다.

9.1.4 동적 적재

출처: 책 391-392쪽 / 원문 PDF 331-332쪽

동적 적재(dynamic loading)는 루틴을 실제로 호출할 때까지 메모리에 올리지 않는다. 루틴은 디스크에 재배치 가능 형태로 대기하고 main이 실행되다가 다른 루틴을 부르면 이미 적재됐는지 조사한다. 없으면 연결 적재기가 가져오고 테이블을 갱신한 뒤 제어를 넘긴다. 오류 처리처럼 실행 빈도가 낮고 큰 루틴에 유리하다. 원문은 운영체제의 특별한 지원 없이 프로그램 설계로 가능하고 구현 라이브러리를 제공받을 수 있다고 설명한다.

9.1.5 동적 연결 및 공유 라이브러리

출처: 책 392-393쪽 / 원문 PDF 332-333쪽

동적 연결은 적재 자체를 미루는 동적 적재와 달리 라이브러리 연결을 실행 시점까지 미룬다. 정적 연결은 참조 라이브러리를 각 실행 이미지에 포함하지만 DLL은 실행할 때 찾아 주소를 조정한다. 여러 프로세스가 하나의 물리 사본을 공유할 수 있어 실행 이미지·메모리 공간을 줄인다.

라이브러리 갱신 뒤 새 버전을 쓰려면 정적 연결은 재연결이 필요하지만 동적 연결은 새 버전을 사용할 수 있다. 단, 여러 버전이 공존할 수 있으므로 프로그램·라이브러리에 버전 정보를 두어 원하는 버전을 구별한다. 동적 연결·공유에는 일반적으로 운영체제 지원이 필요하며 각자의 공간이 보호된 프로세스들이 같은 메모리를 이용할 수 있게 해야 한다.

9.2 연속 메모리 할당

출처: 책 393-397쪽 / 원문 PDF 333-337쪽

메모리는 운영체제 영역과 사용자 영역으로 나뉘며 연속 할당은 각 프로세스에 하나의 연속된 영역을 준다. 재배치 레지스터는 가장 작은 물리 주소, 상한 레지스터는 논리 주소 범위를 유지한다. MMU는 논리 주소가 범위 안인지 확인하고 재배치 값을 더한다. 디스패처는 문맥 교환 때 두 값을 적재한다.

가변 크기 파티션은 각각 프로세스 하나를 담는다. 운영체제는 사용 중·가용 영역을 기록한다. 프로세스 종료로 생긴 **hole**을 다시 할당하며 요청보다 크면 분할하고 반환 영역이 인접 hole과 붙으면 병합한다. 공간이 부족하면 거부하거나 대기 큐에 둘 수 있다.

전략	선택 기준·검색
최초 적합	충분히 큰 첫 공간을 고른다. 시작 또는 지난 검색 위치부터 찾고 발견하면 끝낸다.
최적 적합	요청을 만족하는 가장 작은 공간을 고른다. 크기순이 아니면 전체를 검색하며 작은 나머지를 남긴다.
최악 적합	가장 큰 공간을 고른다. 남은 공간도 다른 용도로 충분히 클 것을 기대한다. 크기순이 아니면 전체 검색이 필요하다.

원문 모의실험 설명에서는 최초·최적 적합이 시간·공간 효율에서 최악 적합보다 좋다. 최초와 최적 중 공간 효율이 항상 우세한 것은 없지만 최초가 일반적으로 빠르다. 이는 이름의 최적을 모든 조건의 최적 성능이라고 해석하지 않아야 하는 이유이다.

단편화	발생 위치·조건
외부 단편화	가용 공간 총합은 충분해도 작은 비연속 조각으로 나뉘어 요청을 만족하지 못한다.
내부 단편화	할당 단위 때문에 실제 요청보다 큰 영역을 주어 할당 내부의 일부를 쓰지 못한다.

원문은 최초 적합에서 N개 할당 블록에 대해 0.5N개 블록이 단편화로 손실될 수 있는 **50% 규칙**을 설명한다. 외부 단편화의 **압축(compaction)**은 사용 내용을 한쪽으로 모아 큰 가용 영역을 만든다. 실행 시간의 동적 재배치가 있어야 가능하며 정적 바인딩에서는 할 수 없고 이동 비용도 크다. 논리 공간을 비연속 물리 영역에 놓는 페이징은 다른 해결 방향이다.

9.3 페이징

출처: 책 397-408쪽 / 원문 PDF 337-348쪽

프레임은 물리 메모리의 고정 크기 블록, **페이지**는 논리 메모리의 같은 크기 블록이다. 페이지를 가용 프레임에 놓으면 프로세스 물리 공간이 연속일 필요가 없어 외부 단편화와 그 압축을 피한다. 논리 주소 공간 크기와 물리 메모리 용량을 분리하며 운영체제·하드웨어가 협력하여 구현한다.

9.3.1 기본 방법

출처: 책 397-402쪽 / 원문 PDF 337-342쪽

논리 주소는 페이지 번호 p와 페이지 오프셋 d로 나뉜다. 페이지 번호가 프로세스별 페이지 테이블의 인덱스이며 테이블에서 찾은 프레임 번호 f와 원래 오프셋으로 물리 주소를 만든다. 오프셋은 변하지 않는다.

MMU의 주소 변환

1. 논리 주소에서 p를 추출한다.
2. 페이지 테이블에서 p에 대응하는 f를 얻는다.
3. p를 f로 바꾸고 d는 유지하여 물리 주소를 만든다.

원문 코드·기호

```
논리 주소 공간 2^m, 페이지 크기 2^n 바이트 → 상위 m−n비트: p / 하위 n비트: d
```

- m: 논리 주소의 비트 수
- n: 페이지 안 오프셋의 비트 수

페이지 크기는 하드웨어가 정하며 2의 거듭제곱이므로 비트로 나누어 변환할 수 있다.

페이징 하드웨어의 페이지·프레임 변환

그림·표 출처: 책 398쪽 / 원문 PDF 338쪽

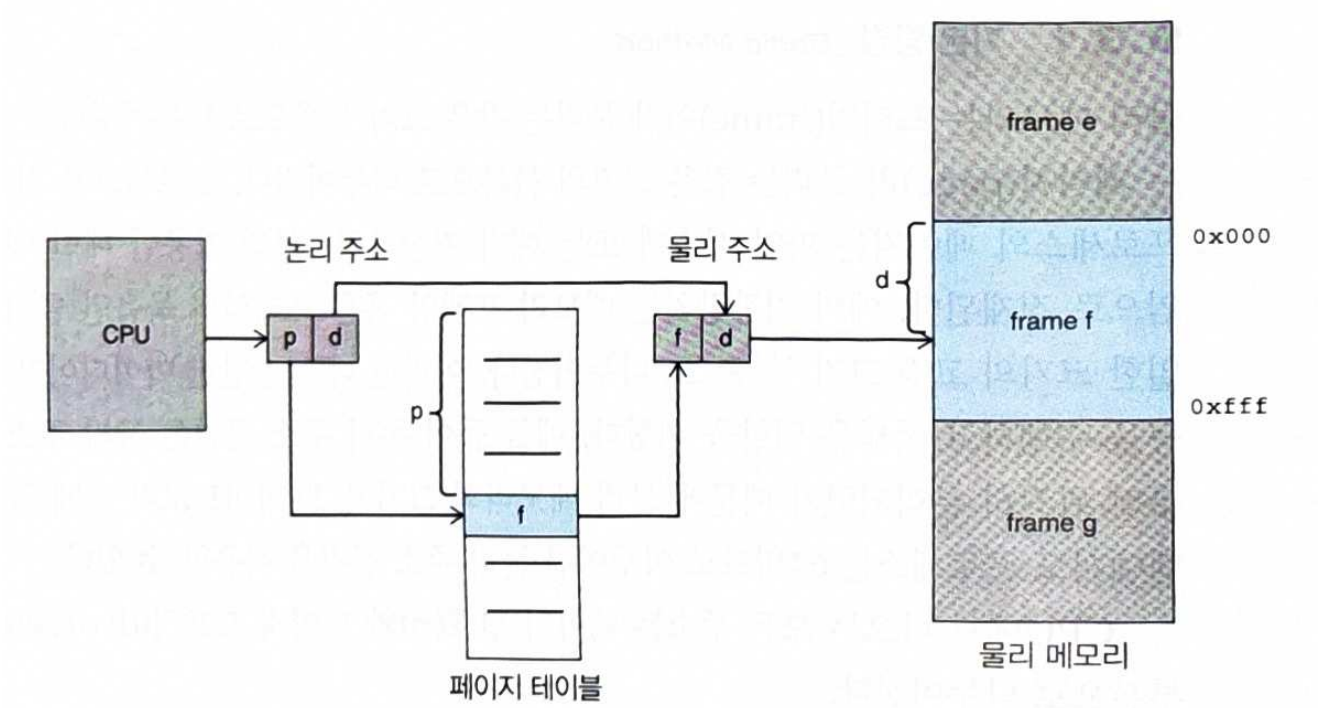


그림 9.8 페이징 하드웨어

그림 9.8. 원문 그림.

p로 프레임 f를 찾고 d를 그대로 유지하는 주소 변환을 보여준다.

원문의 4 B 페이지·32 B 물리 메모리 예에서 페이지 테이블은 페이지 0→프레임 5, 1→6, 2→1, 3→2이다. 논리 주소 0은 20, 3은 23, 4는 24, 13은 9로 사상된다. 이들은 원문에 명시된 결과이다. 프레임 단위로 할당하므로 외부 단편화는 없지만 마지막 페이지의 **내부 단편화**는 남는다.

원문 예에서 2,048 B 페이지와 72,766 B 요구는 마지막 페이지에 1,086 B를 사용하여 962 B를 낭비한다. 크기와 페이지 크기가 무관하면 평균적으로 반 페이지의 내부 단편화가 예상된다. 작은 페이지는 이를 줄이지만 페이지 테이블을 키우며 큰 페이지는 디스크 효율에 유리하다는 절충이 있다.

운영체제는 가용 프레임을 배정하고 페이지를 적재하여 테이블을 채운다. 프로그래머에게는 자기만의 연속 공간처럼 보이지만 실제 프로그램은 여러 프레임에 분산되고 다른 프로세스와 공존한다. 시스템의 **프레임 테이블**은 프레임별 가용 여부·소유 프로세스·페이지를 기록한다. 운영체제는 사용자 인수 주소를 변환하고 문맥 교환 때 테이블을 설정해야 하므로 페이징은 문맥 교환 비용도 늘린다.

9.3.2 하드웨어 지원과 TLB

출처: 책 402-406쪽 / 원문 PDF 342-346쪽

작은 페이지 테이블은 전용 레지스터로 구현할 수 있지만 각 문맥 교환에서 여러 값을 바꿔야 한다. 큰 테이블은 메인 메모리에 두고 **페이지 테이블 기준 레지스터(PTBR)**가 가리키게 하면 프로세스 변경 때 PTBR만 바꾸면 된다. 그러나 데이터를 얻기 위해 테이블과 실제 데이터에 두 번 접근하는 지연이 생긴다.

TLB(translation look-aside buffer)는 페이지 테이블 일부를 보관하는 매우 빠른 연관 메모리이다. 페이지 번호를 여러 키와 동시에 비교하여 프레임 번호를 얻는다. 맞으면 즉시 변환하고, 없으면 테이블을 읽

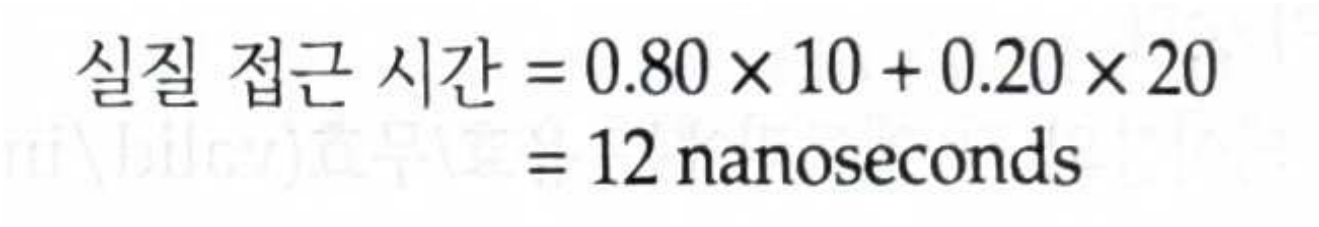
어 변환한 뒤 TLB에 추가한다. 가득 차면 LRU·라운드 로빈·무작위 등 정책으로 바꾸며 중요한 커널 항목을 고정할 수 있다.

각 TLB 항목의 **ASID(address-space identifier)**는 어느 프로세스의 주소 공간인지 식별한다. 현재 프로세스와 다르면 미스로 처리하여 다른 프로세스 정보가 잘못 사용되는 것을 막는다. ASID가 있으면 여러 프로세스 항목을 함께 보관하고, 없으면 새 테이블 선택 때 TLB를 flush해야 한다.

적중률(hit ratio)은 원하는 페이지 번호가 TLB에서 발견되는 비율이다. 원문의 계산식은 메모리 접근 10 ns, 테이블 미스의 두 번 접근 20 ns를 사용하여 80% 적중 때 12 ns, 99% 때 10.1 ns를 얻는다. 단일 테이블 접근을 가정한 예이며 여러 TLB 계층·여러 테이블 접근이면 추가 정보가 필요하다.

TLB 적중률80%의 원문 계산

그림·표 출처: 책 405쪽 / 원문 PDF 345쪽

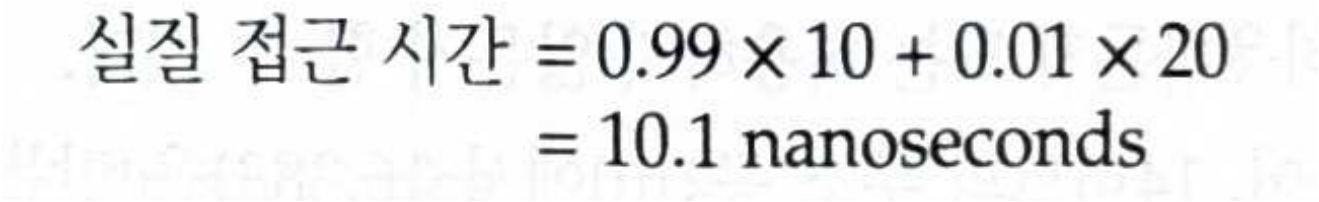


인쇄405쪽의 첫 계산식.

0.80×10+0.20×20=12 ns라는 원문 식을 보존하였다.

TLB 적중률99%의 원문 계산

그림·표 출처: 책 405쪽 / 원문 PDF 345쪽



인쇄405쪽의 두 번째 계산식.

0.99×10+0.01×20=10.1 ns라는 원문 식을 보존하였다.

원문 확인 필요: 같은 쪽의 앞 설명 문장에는 TLB 적중 시 "100 ns"가 기재되어 있으나 뒤의 식과 비교 설명은 "10 ns"를 사용한다. 이를 조용히 수정하지 않고 원문 내부 표기 불일치로 남긴다. 다단계 TLB의 일반 효과만 취하며 다른 수치를 새로 계산하지 않았다.

9.3.3 보호

출처: 책 406-407쪽 / 원문 PDF 346-347쪽

페이지 테이블의 **보호 비트**는 읽기·쓰기 또는 읽기 전용 같은 접근 권한을 지정한다. 주소 변환과 함께 검사하여 읽기 전용에 쓰면 보호 위반 트랩을 낸다. **유효/무효 비트**는 그 페이지가 프로세스의 합법 논리 주소 공간에 속하는지 나타내며 접근 허용 여부를 결정한다.

페이지 단위 보호는 마지막 페이지 안에서 실제 사용 범위보다 넓은 주소를 유효하게 둘 수 있다. 원문의 14비트 주소·2 KB 페이지 예에서 프로그램은 10,468까지 쓰지만 페이지 5 전체가 유효하여 12,287까지 유효로 취급된다. 이는 내부 단편화와 연결된다. 테이블 크기를 나타내는 **PTLR(page table length register)**로 유효 테이블 범위를 확인하는 방법도 제시된다.

9.3.4 공유 페이지

출처: 책 407-408쪽 / 원문 PDF 347-348쪽

공통 코드는 **재진입 코드**, 즉 실행 중 자체 수정이 없는 코드이면 여러 프로세스가 동시에 공유할 수 있다. 각 프로세스는 레지스터·데이터 저장영역을 따로 가지며 같은 물리 코드 사본을 각자의 테이블에 매핑한다. 원문의 libc=2 MB·40프로세스 예는 각각 복사할 때 80 MB 대신 공유 사본 2 MB가 필요하다고 비교한다.

읽기 전용 속성은 코드 정확성에만 맡기지 않고 운영체제가 강제해야 한다. 공유 라이브러리뿐 아니라 프로세스 간 공유 메모리 통신도 같은 프레임에 여러 테이블을 연결하는 방식으로 구현될 수 있다. 프로세스 간 공유가 각자의 데이터까지 모두 같은 것으로 만든다는 뜻은 아니다.

9.4 페이지 테이블의 구조

출처: 책 409-414쪽 / 원문 PDF 349-354쪽

주소 공간이 커지면 페이지 테이블 자체의 물리 메모리 비용이 커진다. 원문은 계층적·해시·역 페이지 테이블과 SPARC Solaris 사례로 테이블 저장·탐색의 절충을 설명한다.

9.4.1 계층적 페이징

출처: 책 409-411쪽 / 원문 PDF 349-351쪽

32비트 주소·4 KB 페이지·4 B 항목 예는 페이지 테이블만 프로세스당 4 MB를 요구한다. 테이블을 연속 공간에 두는 대신 **테이블 자체를 페이징**한다. 2단계에서는 20비트 페이지 번호를 바깥 인덱스 10비트와 안쪽 인덱스 10비트로, 오프셋은 12비트로 나눈다. 바깥에서 안쪽으로 변환하므로 forward-mapped라고도 부른다.

원문의 완전한 64비트 공간·4 KB 가정은 2단계로도 바깥 테이블이 지나치게 커지고 단계를 계속 늘리면 변환의 메모리 접근이 많아지는 문제를 보인다. 이는 다음 해시·역 테이블의 동기이다. 이 예의 전체 64비트 가정과 뒤에서 설명하는 실제 48비트 주소 사용 사례를 혼동하지 않는다.

9.4.2 해시 페이지 테이블

출처: 책 411-412쪽 / 원문 PDF 351-352쪽

큰 주소 공간에서 가상 페이지 번호를 해시한 위치에 연결 리스트를 둔다. 원소는 가상 페이지 번호·프레임 번호·다음 원소 포인터를 가진다. 해시 충돌 리스트를 따라 원하는 가상 페이지 번호와 일치하는 항목을 찾아 물리 프레임을 얻는다. **클러스터 페이지 테이블**은 한 항목이 한 페이지가 아니라 여러 페이지를 가리켜 넓게 퍼진 성긴 주소 공간에 유용하다.

9.4.3 역 페이지 테이블

출처: 책 412-413쪽 / 원문 PDF 352-353쪽

일반 테이블은 프로세스별 가상 페이지마다 항목을 가지지만 **역 페이지 테이블**은 시스템 전체에서 물리 프레임마다 하나의 항목을 둔다. 항목은 그 프레임의 페이지 주소와 소유 프로세스 ID를 기록한다. 원문 IBM RT 설명에서는 <process-id, page-number>를 검색하여 i번째 항목에서 찾으면 물리 주소는 <i, offset>이 된다.

물리 메모리 기준 항목만 유지하여 공간이 줄지만 가상 주소로 테이블 전체를 찾는 데 시간이 오래 걸릴 수 있다. 해시로 탐색을 줄이고 먼저 TLB를 확인하여 성능을 보완한다. 프레임당 항목 하나라 같은 물리

페이지를 여러 가상 주소로 동시에 나타내는 공유 메모리가 복잡해진다. 원문은 다른 프로세스가 참조하면 페이지 폴트와 매핑 변경을 거치는 방식의 한계를 설명한다.

9.4.4 Oracle SPARC Solaris

출처: 책 414쪽 / 원문 PDF 354쪽

원문의 변환 단계

1. 하드웨어가 TLB에서 변환 테이블 항목(TTE)을 찾는다.
2. 없으면 메모리의 변환 저장 버퍼(TSB)에서 최근 TTE를 찾고 TLB로 복사한다.
3. TSB에도 없으면 인터럽트하여 커널이 해시 테이블을 탐색한다.
4. 커널이 TTE를 만들어 TSB에 넣고 MMU에 제어를 돌려 변환을 끝낸다.

커널과 사용자 프로세스용 해시 테이블은 각 항목이 연속 가상 영역의 기준 주소와 범위를 나타내어 페이지마다 항목을 두는 비용을 줄인다. 하드웨어·운영체제가 함께 다단계 탐색을 줄이는 원문 사례로 읽는다.

9.5 스와핑

출처: 책 414-417쪽 / 원문 PDF 354-357쪽

스와핑은 프로세스나 일부를 메모리에서 백업 저장장치로 내보냈다가 실행을 계속하려고 가져오는 기법이다. 물리 메모리보다 프로세스 총 공간이 커도 동시에 유지할 수 있어 다중 프로그래밍 정도를 높인다. 전체 프로세스의 표준 스와핑은 이미지뿐 아니라 복원할 자료구조·다중 스레드별 정보·메타데이터도 보존해야 한다.

방식	원문 의미와 조건
표준 스와핑	전체 프로세스를 이동한다. 유휴 프로세스 메모리를 활성 프로세스에 줄 수 있지만 큰 이동 비용이 든다. 원문은 현대 시스템에서 드물고 Solaris의 극한 부족 상황을 예외로 든다.
페이징에서의 스와핑	프로세스의 일부 페이지만 page-out·page-in하여 전체 복사 비용을 줄인다.
모바일 시스템	원문은 일반적으로 스와핑을 피한다고 설명한다. 플래시 공간·쓰기 횟수·낮은 처리량 제약 때문이다.

원문의 iOS는 부족하면 응용이 메모리를 자발적으로 반환하게 하고 읽기 전용 코드는 제거 후 다시 읽을 수 있지만 수정한 데이터는 그렇게 제거하지 못한다. 충분히 반환하지 못하면 종료될 수 있다. Android는 비슷하게 프로세스를 종료하되 상태를 플래시에 저장하여 재시작을 돕는다. 이는 제공된 원문의 서술이며 현재 제품 동작으로 확대하지 않는다.

9.6 사례: Intel 32비트와 64비트 구조

출처: 책 417-421쪽 / 원문 PDF 357-361쪽

IA-32의 변환은 **논리 주소 → 세그먼테이션 장치 → 선형 주소 → 페이징 장치 → 물리 주소**이다. 두 장치가 MMU 역할을 나눈다. 세그먼트 디스크립터는 기준·상한 정보를 가지며 프로세스별 LDT와 공유 GDT를 구분한다. 논리 주소의 selector가 테이블·세그먼트·보호 정보를 선택하고 offset을 검사한 뒤 기준에 더해 32비트 선형 주소를 만든다.

IA-32의 4 KB 페이지는 2단계 페이징이다. 상위 10비트가 CR3이 가리키는 페이지 디렉터리, 다음 10비트가 페이지 테이블, 마지막 12비트가 오프셋이다. 페이지 디렉터리의 Page Size 플래그가 4 MB를 지정하

면 하위 테이블 대신 프레임을 직접 가리키고 하위 22비트를 오프셋으로 쓴다. 디스크의 페이지 테이블은 invalid 비트를 이용해 나중에 가져올 수 있다.

PAE(page address extension)는 원문에서 32비트 처리기의 물리 공간을 확장하는 3단계 테이블로 설명된다. 최상위 2비트가 페이지 디렉터리 포인터 테이블을 선택하고 항목을 64비트로 늘려 12비트 오프셋과 합한 36비트 물리 주소·64 GB를 지원한다. 사용하려면 운영체제 지원이 필요하며 원문은 32비트 Windows 데스크톱의 4 GB 제한도 별도 언급한다.

원문의 **x86-64**는 IA-32 명령 집합 확장에서 출발하며 이론적 64비트 전체와 실제 사용 비트 수를 구분한다. 제시된 구조는 48비트 가상 주소·4단계 테이블·4 KB/2 MB/1 GB 페이지와 52비트 물리 주소를 지원한다. 이 수치는 제공된 책의 설명으로 한정한다.

x86-64 선형 주소의 원문 필드

그림·표 출처: 책 421쪽 / 원문 PDF 361쪽

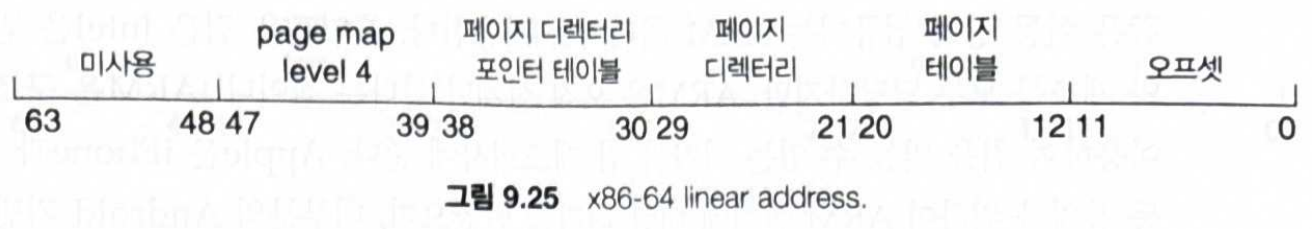


그림 9.25 x86-64 linear address.

그림 9.25. 원문 주소 구조.

47~39, 38~30, 29~21, 20~12의 계층 인덱스와 11~0 오프셋을 구분한다.

9.7 사례: ARM 구조

출처: 책 421-423쪽 / 원문 PDF 361-363쪽

원문은 64비트 **ARMv8**를 대상으로 설명하며 실제 사용 가상 주소는 48비트라고 구분한다. 4 KB·16 KB·64 KB의 변환 단위가 있고 페이지뿐 아니라 더 큰 연속 영역 region을 표현한다. 4 KB·16 KB는 최대 4단계, 64 KB는 최대 3단계 페이징을 사용할 수 있다.

변환 단위	페이지	Region
4 KB	4 KB	2 MB, 1 GB
16 KB	16 KB	32 MB
64 KB	64 KB	512 MB

4 KB 예에서는 TTBR이 레벨0 테이블을 가리킨다. 레벨1이 1 GB region을 직접 가리키면 하위 30비트, 레벨2가 2 MB region을 직접 가리키면 하위 21비트가 오프셋이다. 네 단계 모두 사용하면 마지막 12비트가 4 KB 페이지 안 오프셋이다. 구조의 다른 레벨·영역을 임의로 덧붙이지 않는다.

ARM TLB는 명령·데이터용 마이크로 TLB와 바깥 단일 메인 TLB로 나뉜다. 마이크로 TLB는 ASID를 지원한다. 먼저 마이크로, 다음 메인 TLB를 찾고 둘 다 미스이면 하드웨어가 테이블을 순회한다.

9.8 요약

출처: 책 423-426쪽 / 원문 PDF 363-366쪽

주소 생성·보호·사상은 하드웨어와 운영체제의 협력으로 이루어진다. 연속 할당의 외부 단편화를 페이징으로 피하지만 내부 단편화·테이블·변환 비용은 남는다. TLB와 여러 테이블 구조는 그 비용을 조절한다. 원문 연습 문제는 단편화 구분, 주소의 비트 수, 페이지·오프셋, 배치 전략, 일반·역 테이블 항목 수 등을 확인한다. 새 계산 문제나 답을 만들지 않았다.

핵심 정리

- 논리 주소는 CPU가 생성하며 MMU가 물리 주소로 변환한다.
- 바인딩 시점과 실행 중 재배치 가능 여부를 연결한다.
- 페이지 번호만 프레임 번호로 바뀌고 오프셋은 유지된다.
- PTBR·TLB·ASID·보호 비트·유효 비트의 역할을 구분한다.
- 페이지 테이블 구조는 저장 공간·탐색 비용·공유 방식의 절충이다.
- 제품·구조별 지원 수치는 원문의 시점과 범위로 읽는다.

장 마무리 복습

- 프로세스별 주소 공간 → 하드웨어 보호 → 바인딩·MMU 변환의 관계를 연결한다.
- 연속 공간의 단편화 → 비연속 프레임 할당 → 페이지 테이블 → TLB의 흐름으로 복습한다.
- 전체 주소 비트 수·실제 사용하는 비트·페이지 크기·단계 수를 서로 다른 기준으로 구분한다.

원문 확인 메모

- 인쇄405쪽(PDF345쪽) TLB 적중 시간의 본문 100 ns와 식의 10 ns가 일치하지 않아 원문 확인 필요로 표시하였다.

제10장. 가상 메모리

출처: 책 427-488쪽 / 원문 PDF 367-428쪽

학습 주제

- 프로세스 전체를 메모리에 두지 않고 실행하는 가상 메모리의 이점과 요구 페이징·쓰기 시 복사·페이지 교체를 다룬다.
- 교체 알고리즘의 기준과 프레임 할당을 함께 비교하고 스래싱·지역성·작업 집합·압축·커널 할당·입출력 잠금까지 성능과 조건을 연결한다.

설명의 흐름

1. 가상 주소 공간과 공유 → 요구 페이징의 폴트 처리·비용 → 쓰기 시 복사
2. 교체 알고리즘과 프레임 할당 → 지역성·작업 집합·스래싱 제어
3. 메모리 압축·커널 메모리 → 페이지 크기·TLB·프로그램 구조·I/O → 운영체제 사례

10.1 배경

출처: 책 427-431쪽 / 원문 PDF 367-371쪽

가상 메모리는 프로세스 전체가 메모리에 없어도 실행하게 하며 실제 물리 메모리와 프로그래머의 논리 메모리를 분리한다. 거의 실행되지 않는 오류 처리·사용되지 않는 배열 영역·드문 옵션까지 모두 올릴 필요가 없다. 작은 메모리로 큰 주소 공간을 제공하고 한 프로그램의 점유를 줄여 더 많은 프로그램을 수행하며 적재·스왑 I/O를 줄일 수 있다. 그러나 구현이 복잡하고 잘못 사용하면 성능이 크게 떨어진다.

프로세스의 가상 공간은 연속처럼 보이지만 물리 프레임은 흩어질 수 있다. 원문의 힙·스택 사이 **성긴 주소 공간(sparse address space)**은 공백도 주소 공간 일부이되 실제 확장 때만 물리 페이지를 요구한다. 공유 라이브러리와 공유 메모리는 서로 다른 프로세스의 가상 페이지를 같은 물리 페이지에 사상하고 fork 생성에서도 페이지를 공유할 수 있다.

10.2 요구 페이징

출처: 책 431-438쪽 / 원문 PDF 371-378쪽

요구 페이징(demand paging)은 페이지가 필요할 때만 메모리에 올린다. 접근하지 않는 페이지는 적재하지 않는다. 유효/무효 비트는 이 문맥에서 페이지가 메모리에 있는지 구별하며 무효는 논리적으로 잘못된 페이지이거나 유효하지만 보조저장장치에 있는 페이지를 뜻할 수 있다. 무효 페이지에 접근하면 **페이지 폴트 트랩**이 발생하므로 운영체제가 두 경우를 구별한다.

페이지 폴트 처리

1. 프로세스 내부 테이블을 검사하여 참조가 유효한지 확인한다.
2. 무효 참조면 프로세스를 중단하고, 유효하지만 메모리에 없으면 페이지를 가져온다.
3. 가용 프레임을 찾는다.
4. 보조저장장치에 그 프레임으로 페이지 읽기를 요청한다.
5. 읽기가 끝나면 페이지 테이블과 내부 테이블을 갱신한다.
6. 중단된 명령어를 다시 수행한다.

순수 요구 페이징은 시작할 때 한 페이지도 올리지 않고 첫 명령 인출의 폴트부터 시작할 수 있다. 한 명령이 명령·피연산자에 대해 여러 폴트를 낼 수 있지만 참조의 지역성 덕분에 특정 작은 부분을 집중적으로 사용하여 보통 만족할 성능을 낸다. 유효/무효 표시가 가능한 테이블과 메모리에 없는 페이지를 유지할 스왑 장치가 필요하다.

필수 조건은 폴트 후 명령을 정확히 재시작할 수 있다는 것이다. 레지스터·조건 코드·명령 카운터 등 상태를 보존하여 같은 위치에서 실행해야 한다. 여러 기억 장소를 바꾸고 원천·목적 영역이 겹칠 수 있는 이동 명령은 단순 재실행이 안 될 수 있다. 원문은 수정 전 양 끝을 먼저 접근하여 필요한 페이지를 확보하거나, 덮어쓸 값을 임시 레지스터에 보관하여 폴트 때 원상 복구하는 방법을 설명한다.

운영체제는 **가용 프레임 리스트**를 유지하며 페이지 폴트·스택·힙 확장에 프레임을 준다.
Zero-fill-on-demand는 할당 전 프레임을 0으로 채워 이전 내용을 지운다. 프레임의 이전 데이터를 남기는 것은 보안 취약점이 될 수 있다. 리스트가 0 또는 임계값 아래가 되면 다시 채울 방법이 필요하다.

원문 코드·기호

```
실질 접근 시간 = (1-p) × ma + p × 페이지 폴트 시간
```

- p: 페이지 폴트 확률, 0 ≤ p ≤ 1

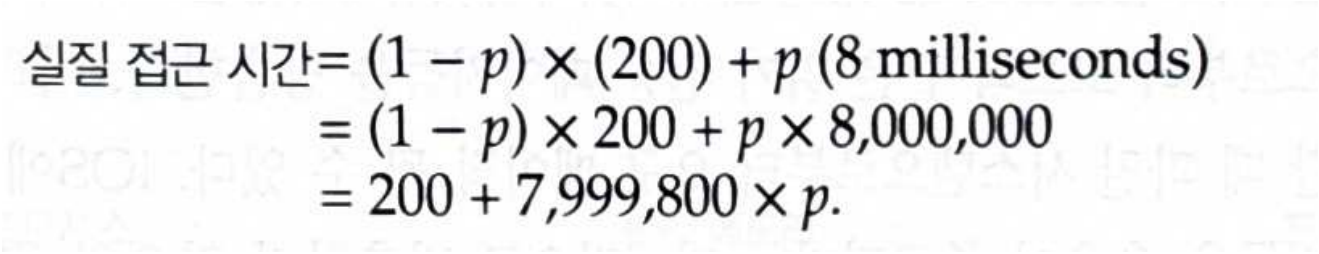
- ma: 메모리 접근 시간

폴트가 없을 때의 메모리 시간과 폴트가 있을 때의 처리 시간을 확률로 가중한다.

원문 상세 처리에는 트랩·상태 저장·유효성 및 디스크 위치 확인·읽기 대기와 전송·다른 프로세스 실행·I/O 완료 인터럽트·테이블 갱신·CPU 재배정·명령 재실행이 포함된다. 주요 요소는 인터럽트 처리, 페이지 읽기, 프로세스 재시작이다. 다른 프로세스를 실행하면 CPU 활용은 높지만 원래 처리 재개까지 추가 지연이 생긴다.

요구 페이징의 원문 성능 계산

그림·표 출처: 책 437쪽 / 원문 PDF 377쪽



인쇄437쪽의 200 ns·8 ms 가정 계산식.

실질 접근 시간=200+7,999,800×p(ns)이다. 원문의 단위 변환과 값을 이미지로 보존하였다.

위 예에서 1,000번에 한 번 폴트이면 8.2 μs로 약 40배 느려지고, 성능 저하를 10% 이내로 하려면 p<0.0000025라는 원문의 조건이 나온다. 폴트율을 낮추는 것이 매우 중요하다. 이는 HDD 기반 원문 가정의 결과이며 다른 장치에 새 수치를 대입하지 않는다.

스왑 공간은 파일 탐색·간접 할당을 거치지 않는 등 파일 시스템보다 I/O가 빠를 수 있다. 실행 이미지를 미리 복사하면 시작 비용이 들고, 먼저 실행 파일에서 요구 페이징한 뒤 교체할 때 스왑에 쓰는 절충이 있다. 수정되지 않는 실행 페이지는 버렸다가 파일에서 다시 읽을 수도 있다. 파일과 관련 없는 스택·힙의 **익명 메모리**는 별도 백업이 필요하다. 원문의 모바일 시스템은 읽기 전용 페이지를 방출·재적재하고 실행 중 생긴 페이지는 다른 정책으로 관리한다.

10.3 쓰기 시 복사

출처: 책 438-440쪽 / 원문 PDF 378-380쪽

쓰기 시 복사(copy-on-write, COW)는 fork 때 부모의 모든 페이지를 바로 복사하지 않고 부모·자식이 먼저 공유하게 한다. 수정 가능한 공유 페이지를 COW로 표시하고 둘 중 하나가 쓰면 복사본을 만들어 그 프로세스 주소 공간에 사상한다. 수정하지 않는 페이지는 계속 공유하며 실행 코드처럼 변경 불가능한 페이지는 COW 표시 없이 공유할 수 있다. 생성 직후 exec하는 자식을 위해 불필요한 복사를 줄이는 방법이다.

원문의 **vfork()**는 부모를 보류하고 자식이 부모 주소 공간을 그대로 사용한다. COW를 쓰지 않으므로 자식의 변경이 나중에 부모에게 보인다. 자식이 즉시 exec하는 용도이며 부모 페이지를 변경하지 않도록 주의해야 한다. 페이지 복사가 없어 효율적이라는 원문 범위로 설명한다.

10.4 페이지 교체

출처: 책 440-454쪽 / 원문 PDF 380-394쪽

프로세스 일부만 사용한다고 예상하여 과할당하면 실제로 더 많은 페이지가 필요해질 때 가용 프레임이 없어질 수 있다. 운영체제 메모리와 I/O 버퍼도 프레임을 필요로 한다. 논리적으로 사용자에게 투명해야 하는 페이징에서 단순 종료 대신 **페이지 교체**로 현재 프레임을 비워 요청 페이지를 올린다.

교체를 포함한 폴트 처리

1. 필요한 페이지의 보조저장장치 위치를 찾는다.
2. 가용 프레임이 없으면 교체 알고리즘으로 희생 프레임을 고른다.
3. 필요하면 희생 페이지를 기록하고 관련 테이블을 갱신한다.
4. 빈 프레임에 새 페이지를 읽고 테이블을 바꾼 뒤 명령을 재개한다.

희생 페이지 쓰기와 새 페이지 읽기가 모두 필요하면 저장장치 접근 두 번이 든다. **변경/더티 비트**는 메모리에서 쓴 적이 있는지 표시한다. 변경되지 않았고 백업 사본이 유지되면 다시 쓸 필요가 없다. 읽기 전용 코드도 버렸다가 다시 읽을 수 있다. 프레임 할당 알고리즘은 프로세스별 수를, 교체 알고리즘은 희생 페이지를 정한다.

교체 성능은 메모리 **참조열(reference string)**과 프레임 수를 정하고 폴트 횟수로 평가한다. 페이지 크기가 정해지면 전체 주소 대신 페이지 번호를 쓰고 연속한 같은 페이지 접근을 줄일 수 있다. 원문의 다음 비교는 처음 비어 있는 **3프레임**, 참조열 **7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1**을 사용한다.

10.4.2 FIFO 페이지 교체

출처: 책 444-446쪽 / 원문 PDF 384-386쪽

FIFO는 메모리에 가장 먼저 들어온 페이지를 교체한다. 적재 시간 또는 큐를 유지하여 머리에서 빼고 새 페이지를 끝에 넣는다. 이해·구현은 쉽지만 자주 쓰는 오래된 페이지도 내보낼 수 있다. 잘못된 교체는 이후 폴트를 늘릴 뿐 프로그램의 논리적 실행 자체를 틀리게 하지는 않는다.

FIFO의 원문 페이지 교체 추적

그림·표 출처: 책 445쪽 / 원문 PDF 385쪽

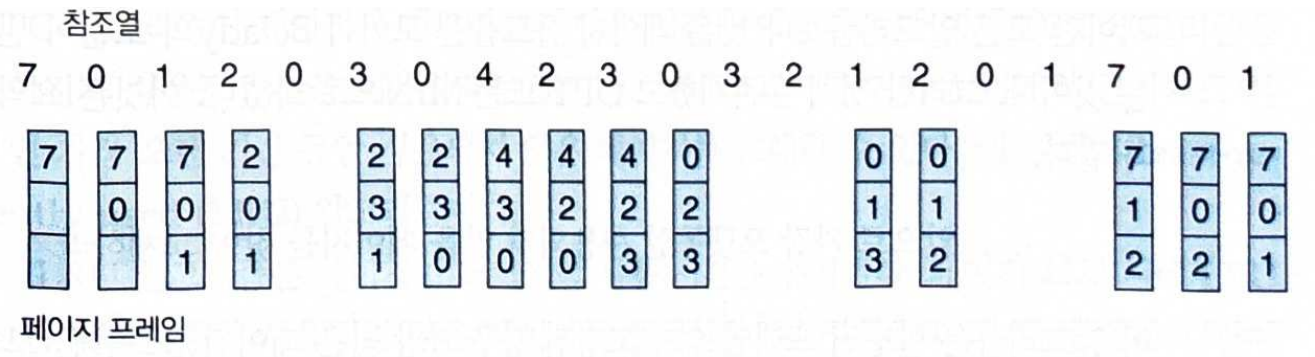


그림 10.12 FIFO 페이지 교체 알고리즘

그림 10.12. 3프레임 참조열 추적.

원문 결과는 페이지 폴트 15회이다. 표의 원래 페이지·프레임 배치를 보존하였다.

Belady의 모순은 프레임을 더 주었는데 폴트가 오히려 늘어나는 현상이다. 원문 참조열 1 2 3 4 1 2 5 1 2 3 4 5에서 FIFO는 3프레임에 9회, 4프레임에 10회이다. 프레임 증가가 모든 알고리즘에서 항상 폴트를 줄인다고 단정하면 안 된다.

10.4.3 최적 페이지 교체

출처: 책 446-447쪽 / 원문 PDF 386-387쪽

OPT/MIN은 앞으로 가장 오랫동안 사용되지 않을 페이지를 교체한다. 고정된 프레임 수에서 가장 낮은 폴트율을 보장하며 Belady의 모순이 없다. 그러나 미래 참조를 미리 알아야 하므로 실제 구현이 어렵고 비교 연구의 기준으로 사용한다.

최적 교체의 원문 추적

그림·표 출처: 책 447쪽 / 원문 PDF 387쪽

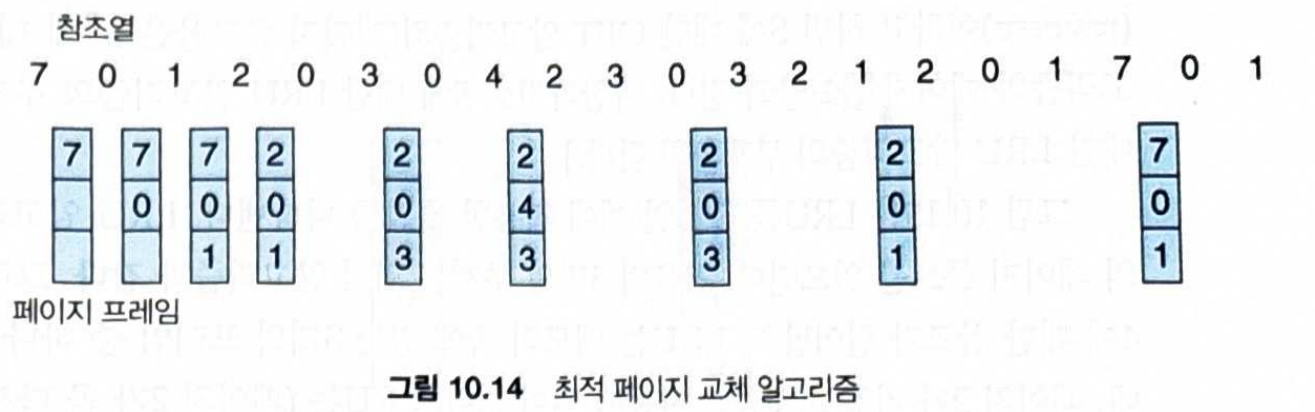


그림 10.14 최적 페이지 교체 알고리즘

그림 10.14. 같은 3프레임·참조열의 OPT.

원문 결과는 폴트 9회이다. 페이지2 요청 때 가장 나중에 다시 쓸 페이지7을 교체하고, 페이지3 요청 때 페이지1을 교체한다.

10.4.4 LRU 페이지 교체

출처: 책 447-449쪽 / 원문 PDF 387-389쪽

LRU(least-recently-used)는 가장 오랫동안 사용되지 않은 페이지를 교체한다. 가까운 과거를 가까운 미래의 근사로 본다. FIFO의 적재 시점, OPT의 미래 사용 시점, LRU의 마지막 사용 시점을 구별한다.

LRU의 원문 추적

그림·표 출처: 책 448쪽 / 원문 PDF 388쪽

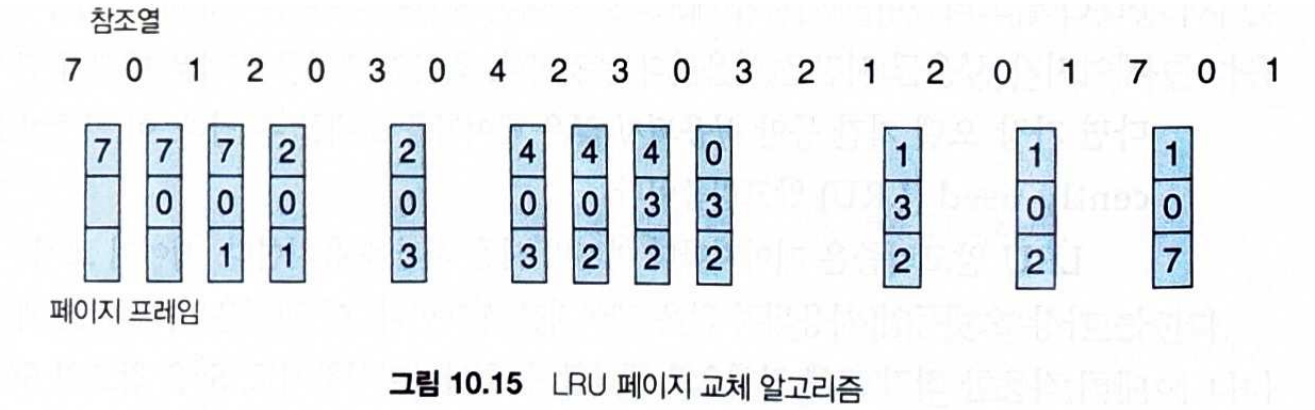


그림 10.15 LRU 페이지 교체 알고리즘

그림 10.15. 같은 3프레임·참조열의 LRU.

원문 결과는 폴트 12회이다. 페이지4 요청 때 과거에 가장 오래 안 쓴 페이지2를 선택하지만 곧 페이지2가 다시 필요해진다.

구현	유지 정보와 비용
계수기	페이지마다 마지막 사용 시간을 기록한다. 메모리 참조마다 갱신하고 희생자 선택 때 최소 시간을 탐색한다. 문맥 교환 관리·계수 overflow도 고려한다.
스택	참조된 페이지를 중간에서 제거해 꼭대기로 옮긴다. 맨 아래가 가장 오래 안 쓴 페이지이다. 연결 리스트 갱신 비용이 있지만 희생자 탐색은 필요 없다.

LRU·OPT는 **스택 알고리즘**이다. n프레임에 남는 페이지 집합이 n+1프레임 집합의 부분집합이므로 Belady의 모순이 없다. 두 정확한 LRU 구현은 매 참조의 갱신을 위해 하드웨어 지원이 필요하며 이를 소프트웨어 인터럽트로 수행하면 큰 지연이 든다. **원문 확인 필요:** 인쇄448쪽의 스택 설명 첫 문장은 "LFU"라고 표기하지만 해당 절·그림·후속 설명은 LRU를 다룬다. 표기를 조용히 수정한 인용으로 제시하지 않았다.

10.4.5 LRU 근사 페이지 교체

출처: 책 449-452쪽 / 원문 PDF 389-392쪽

정확한 사용 순서를 기록하는 하드웨어가 부족해도 **참조 비트**로 그동안 사용했는지는 알 수 있다. 운영체제가 0으로 두고 페이지를 읽거나 쓸 때 하드웨어가 1로 만든다. 이것은 정확한 순서는 아니지만 근사 교체의 기반이다.

부가적 참조 비트는 일정 간격마다 참조 비트를 8비트 기록의 맨 위에 넣고 나머지를 오른쪽으로 민다. 00000000은 최근 8구간 미사용, 11111111은 구간마다 최소 한 번 사용을 뜻한다. 정수로 작은 기록을 우선 교체하고 동점은 FIFO 등으로 처리한다. 비트 수와 간격은 조절할 수 있다.

2차 기회(클록) 알고리즘

1. FIFO 순서의 다음 희생 후보에서 참조 비트를 본다.
2. 0이면 교체한다.
3. 1이면 0으로 바꾸어 한 번 더 기회를 주고 다음 후보로 간다.
4. 순환 큐와 포인터로 반복한다. 모두 1이면 한 바퀴 돌며 0으로 만들고 다음 바퀴에서는 FIFO처럼 선택한다.

참조·변경 비트	개선된 2차 기회의 등급
(0,0)	최근 사용·변경이 없어 교체하기 가장 좋다.
(0,1)	최근 미사용이지만 변경되어 먼저 기록해야 한다.
(1,0)	최근 사용되어 다시 쓸 가능성이 있다.
(1,1)	최근 사용·변경되어 재사용 가능성과 기록 비용이 모두 있다.

개선된 알고리즘은 순환 탐색하며 가장 낮은 등급의 첫 페이지를 고른다. 목적은 최근 참조뿐 아니라 변경 페이지의 I/O 비용도 함께 고려하는 것이다. 여러 바퀴 탐색할 수 있다는 조건도 남긴다.

10.4.6 계수 기반 교체

출처: 책 452쪽 / 원문 PDF 392쪽

기법	원문 판단과 한계
LFU	참조 횟수가 가장 적은 페이지를 교체한다. 초기에 집중 사용했지만 나중에 안 쓰는 페이지가 남을 수 있어 주기적으로 계수를 오른쪽으로 이동하여 오래된 영향력을 줄일 수 있다.
MFU	횟수가 적은 페이지는 최근 들어왔고 앞으로 사용할 것이라는 판단에 근거한다.

원문은 두 방식 모두 구현 비용이 크고 OPT를 잘 근사하지 못해 일반적으로 널리 쓰이지 않는다고 설명한다. 특정 응용에서 유리할 가능성과 범용 기본 정책으로 선호되는지는 구별한다.

10.4.7 페이지 버퍼링

출처: 책 452-453쪽 / 원문 PDF 392-393쪽

교체 알고리즘과 함께 가용 프레임 풀을 유지하여 희생 페이지를 다 쓰기 전에 새 페이지부터 읽어 실행을 빨리 재개할 수 있다. 희생 페이지 기록이 끝나면 그 프레임을 풀에 넣는다. 변경 페이지 리스트를 여유 시간에 미리 기록하면 나중에 교체 때 쓰기 비용을 줄인다. 가용 풀 프레임의 원래 페이지를 기억하고 내용을 덮어쓰지 않았다면 재참조 때 디스크 I/O 없이 다시 줄 수 있어 잘못된 희생자 선택의 불이익을 줄인다.

10.4.8 응용과 페이지 교체

출처: 책 453-454쪽 / 원문 PDF 393-394쪽

자체 버퍼링·메모리 관리가 있는 데이터베이스는 범용 운영체제보다 자기 접근을 잘 알아 이중 버퍼링이 오히려 메모리를 두 번 사용할 수 있다. 연속 대량 읽기를 반복하는 데이터웨어하우스는 새 페이지보다 옛 페이지를 다시 쓸 수 있어 원문은 MFU가 LRU보다 효율적일 수 있는 예로 든다. **Raw I/O**는 단순 논리 블록 배열을 사용해 파일 시스템의 요구 페이징·잠금·선반입·이름 등의 서비스를 거치지 않는다. 이런 특수 응용과 달리 대부분은 정상 파일 시스템이 더 좋은 성능이라고 원문은 제한한다.

10.5 프레임의 할당

출처: 책 454-460쪽 / 원문 PDF 394-400쪽

가용 프레임을 프로세스에 얼마나 줄지는 교체 정책과 별개의 선택이다. 가용 프레임보다 많이 줄 수 없고 명령을 끝낼 수 있는 최소 수도 보장해야 한다. 명령 인출·간접 참조·피연산자 등 한 명령이 필요한 페이지가 동시에 있어야 재실행이 끝난다. 최소 수는 아키텍처가, 최대 수는 가용 물리 메모리가 정한다.

10.5.2 할당 알고리즘

출처: 책 455-456쪽 / 원문 PDF 395-396쪽

방법	원문 내용
균등 할당	운영체제 몫을 제외한 m프레임을 n프로세스에 m/n씩 준다. 원문 93프레임·5프로세스는 각각18, 나머지3은 풀이다.
비례 할당	프로세스 크기 s_i와 전체 크기 합 S에 비례하여 $a_i = s_i / S \times m$을 준다. 최소 수 이상 정수로 조정하고 합이 m을 넘지 않게 한다.
우선순위 반영	크기 대신 우선순위 또는 크기·우선순위 조합으로 비례시킬 수 있다.

원문 코드·기호

$$S = \Sigma\, s_i, \quad a_i = (s_i \,/\, S) \times m$$

- s_i: 프로세스 p_i의 크기
- m: 가용 프레임 전체 수
- a_i: p_i에 할당하는 근사 프레임 수

원문의 비례 할당식. 10페이지·127페이지에 62프레임을 나누는 예는 각각 약4·57프레임으로 제시된다.

균등·비례 모두 다중 프로그래밍 정도가 높아지면 각자 몫이 줄고 낮아지면 늘어난다. 단순 크기 기준은 우선순위를 똑같이 취급하므로 우선순위를 반영할지는 별도 정책이다.

10.5.3 전역 대 지역 할당

출처: 책 456-459쪽 / 원문 PDF 396-399쪽

구분	선택 범위·영향
전역 교체	다른 프로세스 프레임도 대상이다. 할당 수가 바뀌고 한 프로세스 성능이 다른 프로세스 행태에 영향을 받는다. 원문은 일반적으로 시스템 성능이 더 좋아 많이 쓰인다고 설명한다.
지역 교체	자기 프레임에서만 선택한다. 외부 영향이 적지만 다른 곳의 안 쓰는 프레임을 쓰지 못해 낭비할 수 있다.

주요(하드) 폴트는 페이지가 메모리에 없어 백업 저장장치에서 읽어야 한다. **사소한(소프트) 폴트**는 페이지가 이미 메모리에 있지만 그 프로세스 매핑이 없을 때 발생할 수 있다. 공유 라이브러리 매핑 또는 아직 덮어쓰지 않은 가용 풀 페이지 재할당은 테이블 갱신 등으로 해결하여 일반적으로 시간이 덜 든다.

전역 회수 전략은 가용 리스트가 0이 되기를 기다리지 않고 **최소 임계값** 아래에서 커널 리퍼(reaper)를 깨워 **최대 임계값**까지 회수한다. 충분해지면 멈추고 다시 부족하면 반복한다. 보통 LRU 근사를 사용하지만 심각한 부족에서는 더 공격적인 정책이나 원문의 Linux OOM 킬러로 프로세스를 종료할 수 있다. 원문은 메모리 백분율 기반 점수와 임계값 조절을 설명하며 현행 정책을 새로 보충하지 않는다.

10.5.4 비균등 메모리 접근

출처: 책 459-461쪽 / 원문 PDF 399-401쪽

NUMA에서는 CPU가 자기 로컬 메모리를 다른 CPU의 메모리보다 빨리 접근한다. 따라서 모든 프레임을 같게 보는 대신 실행 CPU에서 지연시간이 짧은 메모리를 할당해야 한다. 스케줄러가 직전 CPU와 가까운 위치에서 실행시키고 가상 메모리가 가까운 프레임을 주면 캐시 적중과 지연시간에 유리하다. 여러 스레드가 여러 보드에서 실행되면 더 복잡하다.

원문은 Linux의 스케줄링 도메인·NUMA 노드별 가용 리스트, Solaris의 CPU·메모리를 묶는 lgroup 계층을 예로 든다. Solaris는 그룹 안에서 충족하지 못하면 가까운 그룹에서 나머지를 찾아 전체 지연을 줄인다.

10.6 스래싱

출처: 책 461-467쪽 / 원문 PDF 401-407쪽

스래싱(thrashing)은 실제 실행보다 페이징에 더 많은 시간을 쓰는 과도한 페이징이다. 현재 활발히 쓰는 페이지를 담을 프레임이 부족하면 어떤 것을 교체해도 곧 다시 필요해져 폴트가 반복된다.

원문의 악순환

1. 프레임이 더 필요한 프로세스가 다른 프로세스의 페이지를 빼앗는다.
2. 그쪽도 폴트하여 페이징 장치 대기열이 늘고 준비 큐·CPU 이용률이 줄어든다.
3. 운영체제가 낮은 CPU 이용률을 보고 프로세스를 더 넣으면 프레임 경쟁이 심해진다.
4. 추가 폴트와 더 긴 대기가 다시 이용률을 낮추어 처리율이 급격히 떨어진다.

이 구간에서는 다중 프로그래밍 정도를 **낮춰야** 스래싱을 멈춘다. 지역 교체는 다른 프로세스의 프레임을 빼앗는 전파는 막지만 같은 페이징 장치 큐를 길게 하므로 영향을 완전히 없애지 못한다. 필요한 프레임 수를 확보하는 기준이 필요하다.

지역성(locality)은 함께 집중적으로 참조되는 페이지 집합이다. 프로세스는 함수·지역 변수·일부 전역 변수 등으로 형성된 한 지역에서 다른 지역으로 이동하며 지역들은 겹칠 수 있다. 현재 지역을 담으면 적재 후 안정되지만 작게 주면 계속 폴트한다. 원문은 이 모델을 캐싱의 기본 원리와도 연결한다.

10.6.2 작업 집합 모델

출처: 책 464-465쪽 / 원문 PDF 404-405쪽

작업 집합은 최근 Δ번 페이지 참조에 등장한 서로 다른 페이지들의 집합이며 지역성의 근사이다. 활발히 사용하면 포함되고 마지막 참조 후 Δ가 지나면 제외된다. Δ가 너무 작으면 지역을 빠뜨리고 너무 크면 여러 지역을 함께 담는다. 무한히 크면 실행 중 본 모든 페이지가 된다.

작업 집합의 원문 참조 창

그림·표 출처: 책 464쪽 / 원문 PDF 404쪽

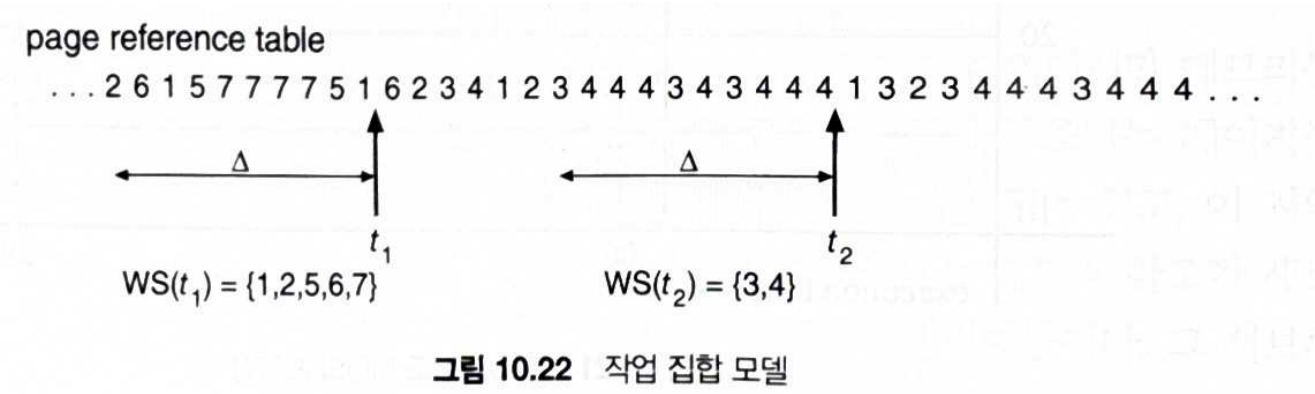


그림 10.22. Δ와 두 시점의 작업 집합.

원문 Δ=10에서 $WS(t_1)=\{1,2,5,6,7\}$, $WS(t_2)=\{3,4\}$이다. 참조열·창 위치를 새로 생성하지 않았다.

원문 코드·기호

$$D = \Sigma\ WSS_i$$

- WSS_i: 프로세스 i의 작업 집합 크기
- D: 전체 요구 프레임 수
- m: 시스템의 전체 메모리 프레임 수

D>m이면 지역성을 담을 프레임이 부족해 스래싱을 유발한다. 원문은 작업 집합에 맞게 할당하고 부족하면 한 프로세스를 연기하여 프레임을 나눈다.

매 참조마다 집합을 정확히 추적하기는 어렵다. 일정 간격 타이머와 참조 비트의 과거 기록으로 근사할 수 있다. 원문의 Δ=10,000·5,000참조 간격 예는 현재와 두 과거 비트를 검사한다. 더 많은 비트·더 잦은 인터럽트는 정확도를 높이지만 오버헤드도 늘린다.

10.6.3 페이지 폴트 빈도와 현재 관행

출처: 책 465-467쪽 / 원문 PDF 405-407쪽

PFF(page-fault frequency)는 폴트율의 상·하한으로 직접 제어한다. 높으면 프레임을 늘리고 낮으면 줄인다. 높아졌는데 줄 프레임이 없으면 프로세스를 스왑아웃하여 나눈다. 지역 전이 때 폴트율이 높아졌다 작업 집합 적재 후 낮아지는 원문 그래프의 맥락도 함께 본다. 원문이 제시하는 일반적인 실용 대책은 극한 상황을 제외하고 작업 집합을 함께 담을 충분한 물리 메모리를 갖추는 것이다.

10.7 메모리 압축

출처: 책 467-469쪽 / 원문 PDF 407-409쪽

메모리 압축은 수정 프레임을 스왑으로 내보내는 대신 여러 페이지를 압축하여 적은 프레임에 저장한다. 원문은 15·3·35의 세 프레임을 하나의 프레임7에 압축하여 원래 프레임들을 가용 리스트로 옮기고, 나중 참조 시 폴트로 압축을 풀어 복원하는 예를 든다. 압축을 유지할 프레임도 필요하지만 전체 사용량을 줄일 수 있다.

원문은 Android·iOS·Windows10·macOS의 지원을 설명하고 macOS는 먼저 압축한 뒤 부족하면 페이징한다고 기술한다. 압축률과 계산 시간 사이에 절충이 있고 여러 코어의 병렬 압축으로 개선할 수 있다. 제품명·압축 알고리즘 수치는 원문 범위를 넘어 갱신하지 않는다.

10.8 커널 메모리의 할당

출처: 책 469-473쪽 / 원문 PDF 409-413쪽

커널은 다양한 작은 자료구조를 할당하고 일부 하드웨어가 물리적으로 연속된 메모리를 요구한다. 사용자 프로세스의 비연속 프레임 배정과 달리 별도 풀을 사용하는 이유이다. 많은 운영체제가 커널 코드·데이터를 페이징하지 않으므로 단편화 낭비를 줄이는 것이 특히 중요하다.

기법	단위·동작	장점·한계
버디 시스템	연속 세그먼트를 2의 거듭제곱으로 나누고 요청을 가까운 위 크기로 올린다.	반환한 인접 버디를 쉽게 병합하지만 올림에 따른 내부 단편화가 있다.
슬랩 할당	자료구조별 캐시에 미리 만든 동일 종류 객체를 유지한다. 슬랩은 하나 이상 연속 페이지이다.	필요한 객체 크기로 할당·반환하여 원문은 단편화 낭비를 피하고 빈번한 요청을 빠르게 처리한다고 설명한다.

버디 시스템의 원문 분할

그림·표 출처: 책 470쪽 / 원문 PDF 410쪽

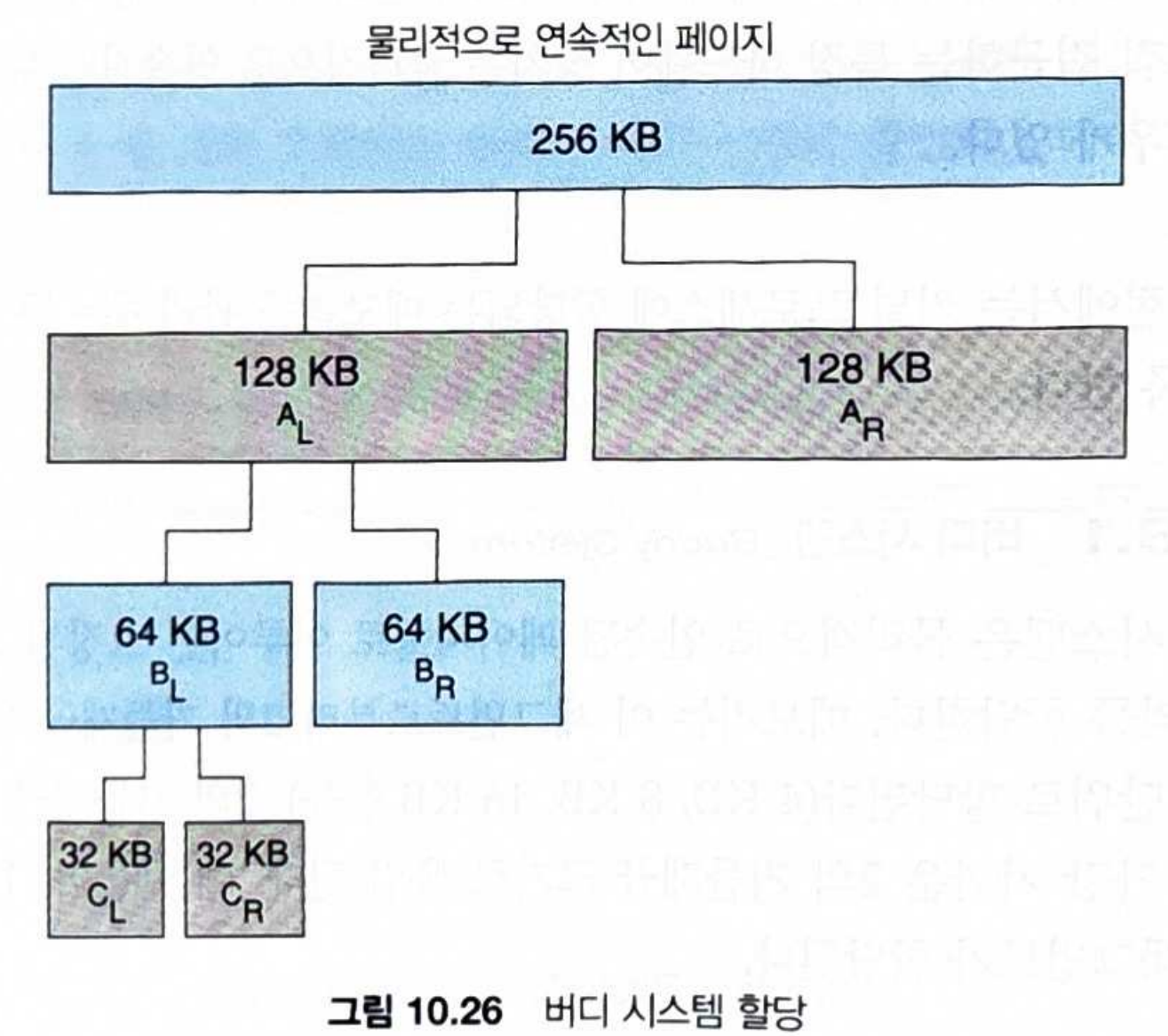


그림 10.26 버디 시스템 할당

그림 10.26. 256 KB에서 21 KB 요청을 위한 32 KB 버디까지 분할.

해제 후 C_l·C_r, B_l·B_r 등의 버디를 합쳐 원래 세그먼트로 돌아갈 수 있다.

슬랩의 상태는 full(모두 사용), empty(모두 가용), partial(혼합)이다. 먼저 partial의 free 객체, 없으면 empty, 그것도 없으면 새 연속 슬랩을 사용한다. 반환한 객체를 free로 표시하여 재사용하므로 매번 새 생성·해제 비용을 줄인다. 원문은 Solaris의 도입, Linux의 SLAB·제한 메모리용 SLOB·오버헤드를 줄이는 SLUB를 비교한다. SLOB는 크기별 세 목록에 최초 적합을 쓰며 SLUB는 메타데이터와 CPU별 큐 비용을 줄여 많은 처리기에서 개선한다고 설명한다.

10.9 기타 고려 사항

출처: 책 473-479쪽 / 원문 PDF 413-419쪽

페이지 교체·할당뿐 아니라 프리페이징, 페이지 크기, TLB reach, 테이블 정보, 프로그램 구조 및 I/O 고정이 실제 성능과 정확성에 영향을 준다.

10.9.1 프리페이징

출처: 책 473쪽 / 원문 PDF 413쪽

프리페이징은 시작·재시작의 연속 폴트를 줄이려고 필요한 일부·전체 페이지를 미리 가져온다. 이전 작업 집합을 기억해 재개 전에 올리는 방법이 있다. s개를 가져와 실제로 α만큼 쓰면 s×α개의 폴트를 줄이는 이익과 s×(1-α) 불필요 반입 비용을 비교한다. α가 0에 가까우면 불리하고 1에 가까우면 유리하다. 실행 코드의 필요 페이지는 예측이 어렵지만 순차 파일은 더 확실하다고 원문은 설명한다.

10.9.2 페이지 크기

출처: 책 473-475쪽 / 원문 PDF 413-415쪽

작은 페이지에 유리한 기준	큰 페이지에 유리한 기준
마지막 페이지 내부 단편화를 줄인다.	페이지 수와 테이블 크기를 줄인다.
필요한 지역을 더 정밀하게 가져와 메모리·불필요 I/O를 줄일 수 있다.	HDD의 탐색·회전 비용을 분산하고 한 번에 더 많이 전송한다.
정확한 필요 부분을 선택한다.	페이지 폴트·인터럽트·상태 저장·테이블 갱신 횟수를 줄인다.

한 크기가 모든 시스템에 최적이지 않다. 원문은 페이지가 커지면 같은 양의 데이터 전송을 한 번에 처리할 수 있지만 필요하지 않은 주변도 가져오며, 작으면 메모리는 알차지만 많은 폴트 오버헤드가 든다고 비교한다. 기존 시스템에서는 하드웨어 때문에 크기를 바꾸기 어렵다는 조건도 중요하다.

10.9.3 TLB Reach

출처: 책 475-476쪽 / 원문 PDF 415-416쪽

원문 코드·기호

```
TLB reach = TLB 항목 수 × 페이지 크기
```

- TLB reach: TLB로 직접 접근 가능한 메모리 공간의 크기

작업 집합을 담을 수 있으면 좋다. 항목 수 증가는 연관 메모리의 가격·전력 비용이 있고 페이지 확대는 내부 단편화를 늘릴 수 있다.

여러 페이지 크기·거대 페이지는 필요한 응용에서 reach를 높이는 대안이다. ARMv8의 contiguous 비트는 인접 블록을 하나의 TLB 항목으로 묶어 reach를 늘린다. 여러 크기를 다루는 소프트웨어 관리가 성능 비용을 더할 수 있지만 높은 적중률로 상쇄할 수 있다고 원문은 설명한다.

10.9.4 역 페이지 테이블

출처: 책 476쪽 / 원문 PDF 416쪽

역 테이블은 물리 프레임의 현재 가상 페이지 정보만 유지하므로 메모리에 없는 페이지의 디스크 위치를 모른다. 이를 위해 프로세스별 **확장 페이지 테이블**이 필요하다. 이 테이블은 매 메모리 참조가 아닌 폴트 때만 쓰므로 자체를 페이징할 수 있다. 폴트 처리 중 확장 테이블 때문에 다시 폴트가 날 수 있어 커널이 조심스럽게 처리해야 한다.

10.9.5 프로그램 구조

출처: 책 476-477쪽 / 원문 PDF 416-417쪽

자료·프로그램 구조는 지역성·작업 집합·폴트율에 영향을 준다. 다음은 원문의 행 중심 128×128 배열과 128워드 페이지 비교이다. 선언·변수·루프를 그대로 보존하며 제공되지 않은 프레임 수나 새 결과를 만들어 넣지 않았다.

원문의 배열 순회: 열을 먼저 진행

코드 출처: 책 477쪽 / 원문 PDF 417쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
int i, j;
int[128][128] data;

for (j = 0; j < 128; j++)
    for (i = 0; i < 128; i++)
        data[i][j] = 0;
```

코드의 의미

행 중심 저장·128워드 페이지에서 한 행 한 워드씩 번갈아 접근하는 원문 예이다.

동작 과정

1. 바깥 j, 안쪽 i 순서로 행을 계속 바꾼다.

원문 실행 결과

원문에 명시된 페이지 폴트 수: 16,384회. 화면 출력은 미제시.

원문 주의 사항

- 원문의 선언 문법을 수정하지 않고 그대로 보존하였다.
- 원문은 위 가정에서 폴트16,384회를 제시한다. 프레임 수 등 별도 조건을 새로 채우지 않았다.

원문의 배열 순회: 행을 먼저 진행

코드 출처: 책 477쪽 / 원문 PDF 417쪽

해당 원문 그림 또는 본문 코드에서 발췌하였다.

원문 코드·기호

```
int i, j;
int[128][128] data;

for (i = 0; i < 128; i++)
    for (j = 0; j < 128; j++)
        data[i][j] = 0;
```

코드의 의미

같은 행의 모든 워드를 처리하고 다음 행으로 넘어가 지역성을 높이는 원문 대비이다.

동작 과정

1. 바깥 i, 안쪽 j 순서로 한 페이지의 행을 먼저 처리한다.

원문 실행 결과

원문에 명시된 페이지 폴트 수: 128회. 화면 출력은 미제시.

원문 주의 사항

- 원문의 선언 문법을 일반 C 형태로 고치지 않았다.
- 원문에 주어진 가정·결과만 옮겼다.

원문은 한쪽 끝을 쓰는 스택의 지역성은 높고 참조를 분산하는 해시 테이블은 낮다고 설명한다. 단, 지역성만이 효율 기준은 아니며 탐색 속도·총 참조·페이지 접근 수도 본다. 컴파일러의 코드·데이터 분리와 재진입 코드, 로더의 페이지 경계 내 루틴 배치·자주 호출하는 루틴 묶기도 도움이 된다.

10.9.6 I/O 잠금과 페이지 잠금

출처: 책 478-479쪽 / 원문 PDF 418-419쪽

I/O 처리기가 사용자 버퍼의 물리 프레임에 전송하는 동안 다른 프로세스의 전역 교체로 그 프레임이 재할당되면 잘못된 메모리에 입출력할 수 있다. 원문은 두 방법을 제시한다. **시스템 버퍼에서만 I/O**한 뒤 사용자 버퍼로 옮기면 안전하지만 이중 이동 비용이 든다. 또는 **잠금 비트**로 I/O 페이지를 교체 대상에서 빼고 완료 시 해제한다.

커널의 일부·전체나 데이터베이스의 특수 버퍼도 고정할 수 있다. 악용하면 메모리 관리에 부담을 주므로 원문은 응용의 특권을 요구한다고 설명한다. 새로 읽었지만 아직 한 번도 못 쓴 낮은 우선순위 프로세스 페이지를 첫 재실행까지 지키는 용도도 있다. **잠금은 언젠가 풀려야 하며** 버그로 영구 유지되면 사용할 수 없는 프레임이 된다. Solaris의 잠금 힌트는 메모리 상황에 따라 무시할 수 있다.

10.10 운영체제의 예

출처: 책 480-483쪽 / 원문 PDF 420-423쪽

다음은 제공된 원문이 기술하는 Linux·Windows10·Solaris의 가상 메모리 사례이다. 현행 버전의 사실을 외부 자료로 보충하지 않는다. 공통으로 요구 페이징·COW와 LRU 근사 클록의 변형을 사용하지만 회수 구조와 임계값 정책은 다르다.

운영체제	원문이 제시하는 관리 방식
Linux	active_list는 사용 중인 페이지, inactive_list는 최근 안 써 회수할 수 있는 페이지를 유지한다. accessed 비트·리스트 위치를 갱신하고 inactive의 재참조는 active로 옮긴다. kswapd가 주기적으로 가용량을 확인하여 부족하면 회수한다.
Windows10	인접 페이지까지 읽는 클러스터 요구 페이징과 작업 집합 한계를 사용한다. 지역·전역 LRU 근사를 조합하고 부족하면 자동 작업 집합 트리밍을 수행한다.
Solaris	lotsfree 임계값 아래에서 pageout이 2차 기회와 유사한 두 바늘 스캔을 한다. 앞 바늘은 참조 비트를 지우고 뒤 바늘은 여전히0인 페이지를 회수한다.

Windows 원문의 데이터 클러스터는 3페이지, 나머지는 7페이지이며 작업 집합 초기 최소50·최대345를 제시한다. hard working-set limits가 없으면 여유 때 최대를 넘거나 부족 때 최소 아래로 갈 수 있다. 여유 메모리가 부족할 때 최대에 도달한 프로세스는 자기 집합의 지역 교체를 하고 전역 트리밍은 큰 유휴 프로세스를 먼저 줄인다.

Solaris는 scanrate·handspread로 두 바늘 간 검사 간격을 정한다. 가용량에 따라 slowscan에서 fastscan으로 늘리고 desfree가 부족하면 더 자주 수행한다. 계속 부족하면 프로세스 페이지를 회수하고 minfree도

못 채우면 새 요청마다 pageout을 수행한다. 공유 라이브러리 페이지를 건너뛰고 일반 파일 페이지와 프로세스 페이지를 구별하는 우선순위 페이징도 원문에 나온다.

10.11 요약

출처: 책 483-488쪽 / 원문 PDF 423-428쪽

가상 메모리의 유연성은 폴트·입출력·교체 비용 관리와 함께 이해해야 한다. 교체의 미래·과거·참조 정보, 프로세스별 프레임 수, 지역성과 작업 집합을 연결하고 커널·I/O의 별도 요구를 남긴다. 원문 연습 문제는 폴트 처리·비교 알고리즘·Belady 모순·요구 하드웨어·배열 순회·CPU와 디스크 이용률 등을 점검한다. 별도 풀이와 새 수치는 추가하지 않았다.

핵심 정리

- 요구 페이징의 무효 표시는 잘못된 참조와 단순 미적재를 모두 나타낼 수 있다.
- 폴트 후 중단 명령을 같은 상태에서 재시작할 수 있어야 한다.
- FIFO·OPT·LRU는 각각 적재 시점·미래 사용·마지막 사용을 기준으로 한다.
- 프레임 수와 교체 범위·지역성·작업 집합은 스래싱과 함께 검토한다.
- COW는 수정할 때 복사하고 vfork는 원문에서 부모 공간을 직접 공유한다.
- 페이지 크기·TLB reach·버디·슬랩·I/O 고정의 서로 다른 절충을 구분한다.

장 마무리 복습

- 가상 주소 추상화 → 필요한 페이지 적재 → 빈 프레임이 없으면 교체라는 흐름을 복습한다.
- 폴트율과 서비스 시간 → 실질 접근 시간 → 지역성·할당·작업 집합으로 연결한다.
- 스래싱 때 CPU 이용률 하락을 단순한 프로세스 부족으로 해석하면 악순환이 생긴다.
- 압축과 고정·커널 할당은 일반 사용자 페이지 교체와 다른 조건을 가진다.

원문 확인 메모

- 인쇄448쪽(PDF388쪽)의 LRU 스택 설명에 LFU라는 표기가 있어 원문 확인 필요로 남겼다.
- 인쇄477쪽(PDF417쪽)의 int[128][128] data; 선언은 원문의 학습 예제 표기를 그대로 보존하였다. 일반 C 문법으로 임의 수정하지 않았다.
- 477쪽 배열 예의 페이지 폴트 수는 원문 결과로 옮겼으며 원문에 명시되지 않은 추가 프레임 조건을 추정하지 않았다.

제11장. 대용량 저장장치 구조

출처: 책 491-534쪽 / 원문 PDF 430-473쪽

학습 주제

- HDD와 비휘발성 메모리(NVM)의 물리적 차이가 주소 매핑, 스케줄링, 오류 처리에 미치는 영향을 살펴본다. 저장장치 초기화·부팅·스왑 공간·연결 방식과 RAID의 신뢰성 및 성능을 연결해 정리한다.

설명의 흐름

1. HDD/NVM의 구조와 논리 블록 → 요청 스케줄링 → 오류 감지·수정 → 포매팅·부팅·손상 블록 → 스왑 → 연결 구조 → RAID와 객체 저장소

11.1 대용량 저장장치 구조의 개관

출처: 책 491-499쪽 / 원문 PDF 430-438쪽

출처: 책 491-499쪽 / 원문 PDF 430-438쪽

대용량 저장장치는 비휘발성 정보를 보관한다. 원문은 HDD와 NVM을 주요 보조저장장치로, 자기 테이프·광디스크·클라우드 저장장치 등을 느리고 큰 3차 저장장치의 예로 다룬다. 여러 종류를 함께 지칭할 때 NVS(nonvolatile storage) 또는 드라이브라고 한다. 물리 장치의 특성은 운영체제의 주소 매핑을 거쳐 논리 저장장치로 제공된다.

11.1.1 하드 디스크 드라이브

출처: 책 492-493쪽 / 원문 PDF 431-432쪽

출처: 책 492-493쪽 / 원문 PDF 431-432쪽

HDD는 자기 물질로 덮인 원형 플래터에 정보를 기록하고, 각 표면 위의 읽기-쓰기 헤드가 자기 패턴을 감지한다. 헤드들은 디스크 암에 붙어 함께 이동한다. 원형 **트랙**은 다시 **섹터**로 나뉘며, 같은 암 위치의 트랙 집합이 **실린더**이다. 섹터는 크기가 고정된 최소 전송 단위이다.

출처: 책 492-493쪽 / 원문 PDF 431-432쪽

전송률은 드라이브와 컴퓨터 사이의 데이터 흐름 속도이며, 위치 지정 시간(임의 접근 시간)은 원하는 실린더로 암을 옮기는 **탐색 시간**과 원하는 섹터가 헤드 아래까지 도는 **회전 지연시간**으로 구성된다. 컨트롤러의 DRAM 버퍼는 성능을 높인다. 공표된 전송률은 매체에서 비트를 읽는 비율일 수 있으므로 운영체제에 블록이 실제 전달되는 전송률과 구별해야 한다.

출처: 책 492-493쪽 / 원문 PDF 431-432쪽

헤드는 매우 얇은 공기나 가스층 위에서 움직인다. 표면에 닿는 **헤드 충돌**은 자기 표면을 손상시키며 일반적으로 수리할 수 없다. 드라이브 전체를 교체해야 할 수 있고, 다른 저장장치나 RAID로 보호하지 않은 데이터는 손실된다. 일부 섀시는 시스템 종료 없이 드라이브를 제거·교체할 수 있다.

11.1.2 비휘발성 메모리 장치와 NAND 컨트롤러

출처: 책 494-496쪽 / 원문 PDF 433-435쪽

출처: 책 494-496쪽 / 원문 PDF 433-435쪽

NVM 장치는 기계식이 아닌 전기식이다. 원문은 컨트롤러와 플래시 NAND 다이로 구성되는 장치에 초점을 맞춘다. 디스크와 유사한 외형이면 SSD라고 하며, USB 드라이브나 메모리 스틱 형태 또는 기판에 직접 실장하는 형태도 같은 원리로 취급한다. 움직이는 부분이 없어 HDD보다 안정적이고 탐색·회전 지연이 없으며 전력 소비도 적다. 반면 원문이 비교하는 시점에서는 메가바이트당 가격이 비싸고 대형 HDD보다 용량이 작다. 버스가 성능을 제한할 수 있어 시스템 버스에 직접 연결하기도 한다.

출처: 책 494-496쪽 / 원문 PDF 433-435쪽

NAND는 **페이지** 단위로 읽고 쓰지만 기존 데이터를 그대로 덮어쓸 수 없다. 덮어쓰려면 먼저 지워야 하며, 지우기는 여러 페이지로 된 **블록** 단위이다. 읽기가 가장 빠르고, 쓰기는 읽기보다 느리지만 지우기보다는 훨씬 빠르다. 여러 다이와 데이터 경로를 이용한 병렬 처리가 가능하다. 지우기마다 셀이 열화되므로 쓰기 마모가 수명을 제한한다. DWPD(Drive Writes Per Day)는 보증 기간 동안 하루에 드라이브 전체 용량만큼 데이터를 몇 번 쓸 수 있는지를 나타낸다.

출처: 책 494-496쪽 / 원문 PDF 433-435쪽

FTL·가비지 수집·과잉 공급의 관계

1. 블록이 다시 기록되면 새 페이지가 현재 데이터를 보유하고 이전 페이지의 데이터는 무효가 된다. 플래시 변환 계층(FTL)은 유효한 논리 블록을 실제 페이지로 매핑하고 물리 블록 상태를 추적한다.
2. 공간이 부족하면 유효하지 않은 페이지만 든 블록을 지운다. 유효·무효 페이지가 섞인 블록은 유효 데이터를 다른 위치로 복사한 뒤 지운다. 이것이 가비지 수집이다.
3. 이 복사를 위한 여유 공간과 쓰기 성능을 확보하려고 장치는 과잉 공급 영역을 따로 둔다. 지운 블록은 장치 상태에 따라 과잉 공급 공간이나 가용 풀로 돌아간다.
4. 삭제 횟수가 적은 블록에 데이터를 배치하여 후속 지우기를 분산한다. 이러한 마모 평준화는 일부 블록만 일찍 마모되어 전체 수명이 줄어드는 것을 막는다.

출처: 책 494-496쪽 / 원문 PDF 433-435쪽

컨트롤러는 데이터와 함께 오류 수정 코드를 저장하고 읽을 때 검사한다. 수정 가능한 오류가 반복되는 페이지는 불량으로 표시하여 쓰지 않는다. 그래도 장치 전체 고장이나 데이터 오염이 가능하므로 RAID로 보호할 수 있다. 이 알고리즘들은 대체로 컨트롤러 내부에서 구현되지만 그 동작에 따라 성능이 달라진다.

11.1.3 휘발성 메모리와 자기 테이프

출처: 책 496-498쪽 / 원문 PDF 435-437쪽

출처: 책 496-498쪽 / 원문 PDF 435-437쪽

RAM 드라이브는 시스템 DRAM의 일부를 장치 드라이버가 저장장치처럼 제공하는 것이다. raw 블록 장치로도 쓸 수 있지만 보통 파일 시스템을 만들어 표준 파일 연산으로 사용한다. 캐시·버퍼가 운영체제나 프로그래머에 의해 할당되는 것과 달리 사용자는 파일을 생성·읽기·쓰기·삭제하는 방식으로 임시 데이터를 보관한다. 매우 빠른 임시 파일·프로그램 간 데이터 공유·부팅 중 임시 루트 파일 시스템에 유용하다. 휘발성이므로 크래시·종료·전원 차단 뒤에는 내용이 지속되지 않는다.

출처: 책 496-498쪽 / 원문 PDF 435-437쪽

원문의 별도 설명에서 **자기 테이프**는 비휘발성과 큰 용량을 제공하지만 임의 위치를 찾는 데 오래 걸리므로 보조저장장치보다는 백업·잘 쓰지 않는 정보의 보관·시스템 간 전송에 적합하다. 위치를 찾은 뒤에는 HDD와 비슷한 전송 속도로 읽고 쓸 수 있다. 따라서 임의 위치 접근의 지연과 위치를 찾은 후의 순차 전송 속도를 구별해야 한다.

11.1.4 보조저장장치 연결 방법

출처: 책 498쪽 / 원문 PDF 437쪽

출처: 책 498쪽 / 원문 PDF 437쪽

보조저장장치는 시스템 버스나 I/O 버스에 연결된다. 원문은 ATA·SATA·eSATA·SAS·USB·FC를 예로 들고, NVM에는 PCI 버스에 직접 연결하여 처리량을 높이고 지연을 줄이는 NVMe를 설명한다. **호스트 컨트롤러(HBA)**는 컴퓨터 쪽에서 버스 전송을 맡고, 장치 내부 컨트롤러는 실제 하드웨어를 작동시킨다.

출처: 책 498쪽 / 원문 PDF 437쪽

명령과 데이터 전달

1. 컴퓨터는 보통 메모리 매핑 I/O 포트로 호스트 컨트롤러에 명령을 놓는다.
2. 호스트 컨트롤러는 장치 컨트롤러에 명령을 전달하고, 장치 컨트롤러는 드라이브를 작동시킨다.

3. 매체와 내장 캐시 사이에서 데이터가 이동하고, 캐시와 호스트 DRAM 사이에는 DMA로 데이터가 전송된다.

11.1.5 주소 매핑

출처: 책 498-499쪽 / 원문 PDF 437-438쪽

출처: 책 498-499쪽 / 원문 PDF 437-438쪽

운영체제는 저장장치를 **논리 블록의 1차원 배열**로 다룬다. 논리 블록은 최소 전송 단위이며 물리 섹터나 반도체 페이지로 매핑된다. 실린더·헤드·섹터 또는 칩·블록·페이지의 조합보다 LBA(논리 블록 주소)가 다루기 쉽다.

출처: 책 498-499쪽 / 원문 PDF 437-438쪽

현대 드라이브에서 LBA를 실제 위치로 단순 환산하기는 어렵다. 손상 섹터는 예비 섹터로 대체되고, 트랙당 섹터 수가 일정하지 않을 수 있으며, 제조업체가 내부 매핑을 관리하기 때문이다. 따라서 스케줄러는 보통 가까운 LBA가 가까운 물리 블록에 해당한다는 경향을 이용한다.

출처: 책 498-499쪽 / 원문 PDF 437-438쪽

구분	원문의 설명
CLV(고정 선형 속도)	트랙의 비트 밀도를 일정하게 유지한다. 바깥 트랙은 길어 더 많은 섹터를 담으며, 안쪽으로 갈수록 회전 속도를 높여 데이터 비율을 유지한다. CD-ROM·DVD-ROM의 예가 제시된다.
CAV(고정 각 속도)	회전 속도를 일정하게 유지하며 안쪽에서 바깥쪽으로 갈수록 비트 밀도를 줄인다. 원문은 HDD에서 사용하는 방법으로 설명한다.

출처: 책 498-499쪽 / 원문 PDF 437-438쪽

원문은 실린더를 구역으로 나누어 같은 구역 안에서는 트랙당 섹터 수를 일정하게 하고 구역 사이에서는 다르게 구성하는 방법도 설명한다. HDD와 NVM을 결합한 장치·볼륨 관리자는 일반 장치와 특성이 달라 캐싱과 스케줄링도 달라질 수 있다.

11.2 디스크 스케줄링

출처: 책 500쪽 / 원문 PDF 439쪽

출처: 책 500쪽 / 원문 PDF 439쪽

HDD 관리의 목표는 접근 시간을 줄이고 전송 대역폭을 높이는 것이다. 장치 대역폭은 첫 요청부터 마지막 전송 완료까지의 전체 시간으로 총 전송 바이트 수를 나눈 값이다. I/O 시스템 콜은 읽기·쓰기 여부, 열린 파일 핸들, 메모리 주소, 데이터 양을 전달한다. 장치나 컨트롤러가 바쁘면 요청은 큐에 대기하고, 드라이버는 큐를 재정렬할 수 있다.

출처: 책 500쪽 / 원문 PDF 439쪽

현대 드라이브는 실제 헤드·트랙 위치를 호스트에 완전히 공개하지 않는다. 따라서 디스크 스케줄링은 물리적 탐색 최소화뿐 아니라 공정성·적시성·순차 요청의 병합을 고려한다. 특정 요청 집합의 최적 순서를 계산하는 비용이 얻는 이익보다 클 수도 있다.

11.2.1 선입 선처리 스케줄링

출처: 책 501쪽 / 원문 PDF 440쪽

출처: 책 501쪽 / 원문 PDF 440쪽

FCFS는 요청이 도착한 순서대로 처리한다. 본질적으로 공평해 보이지만 빠른 서비스를 보장하지 않는다. 원문 큐는 98, 183, 37, 122, 14, 124, 65, 67이고 시작 헤드는 53이다. 이 순서 그대로 이동한 총 거리는 원문에 **640 실린더**로 제시된다. 122→14→124처럼 가까운 두 요청 사이를 크게 왕복하는 이동이 발생한다.

출처: 책 501쪽 / 원문 PDF 440쪽

그림 11.6 FCFS 디스크 스케줄링

그림·표 출처: 책 501쪽 / 원문 PDF 440쪽

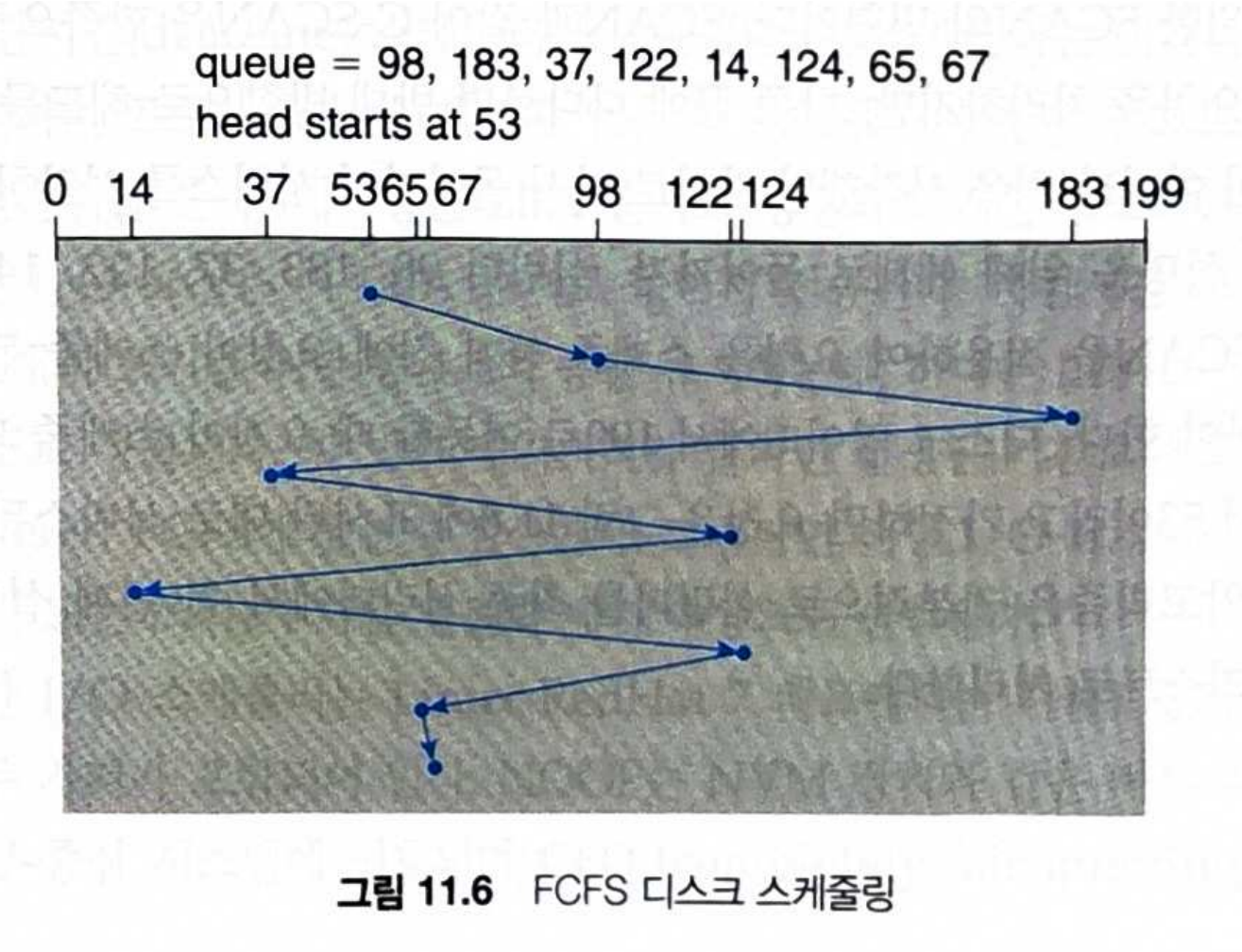


그림 11.6 FCFS 디스크 스케줄링

그림 11.6 FCFS 디스크 스케줄링 - 원문 그림 부분 발췌

동일한 요청 큐를 도착 순서로 처리할 때 헤드가 왕복하는 경로를 보인다. 총 이동거리 640은 원문에 제시된 결과이다.

11.2.2 SCAN 스케줄링

출처: 책 501-502쪽 / 원문 PDF 440-441쪽

출처: 책 501-502쪽 / 원문 PDF 440-441쪽

SCAN은 한 방향으로 이동하면서 경로상의 요청을 모두 처리한 뒤 반대로 이동한다. 원문은 한쪽 끝 또는 그 방향에 더 이상 대기 요청이 없는 경우 방향을 바꾼다고 설명하며, 엘리베이터 알고리즘이라고도 한다. 현재 위치뿐 아니라 진행 방향도 알아야 한다. 같은 큐에서 53에서 0 방향으로 출발하면 37·14를 처리하고 0에서 반전하여 65·67·98·122·124·183을 처리한다.

출처: 책 501-502쪽 / 원문 PDF 440-441쪽

헤드 앞에 새 요청이 생기면 진행하며 처리할 수 있지만 뒤에 생기면 돌아올 때까지 기다린다. 모든 실린더에 요청이 균일하게 도착한다고 가정하면 막 통과한 쪽 끝보다 오래 방문하지 않은 반대 끝에 대기 요청이 더 많다. 이 관찰이 C-SCAN의 출발점이다.

11.2.3 C-SCAN 스케줄링

출처: 책 502-503쪽 / 원문 PDF 441-442쪽

출처: 책 502-503쪽 / 원문 PDF 441-442쪽

C-SCAN은 대기 시간을 더 균등하게 하려는 SCAN의 변형이다. 한 방향으로 움직이며 요청을 처리하지만 끝에 도달하면 반대 방향의 요청을 처리하며 돌아오지 않고 시작 쪽으로 돌아간다. 실린더들을 마지막에서 처음으로 이어지는 순환 리스트처럼 취급한다. 원문 예는 시작 53에서 199 방향으로 이동하여 큰 실린더의 요청들을 처리하고 0으로 돌아간 뒤 14·37을 처리한다.

출처: 책 502-503쪽 / 원문 PDF 441-442쪽

그림 11.8 C-SCAN 디스크 스케줄링

그림·표 출처: 책 503쪽 / 원문 PDF 442쪽

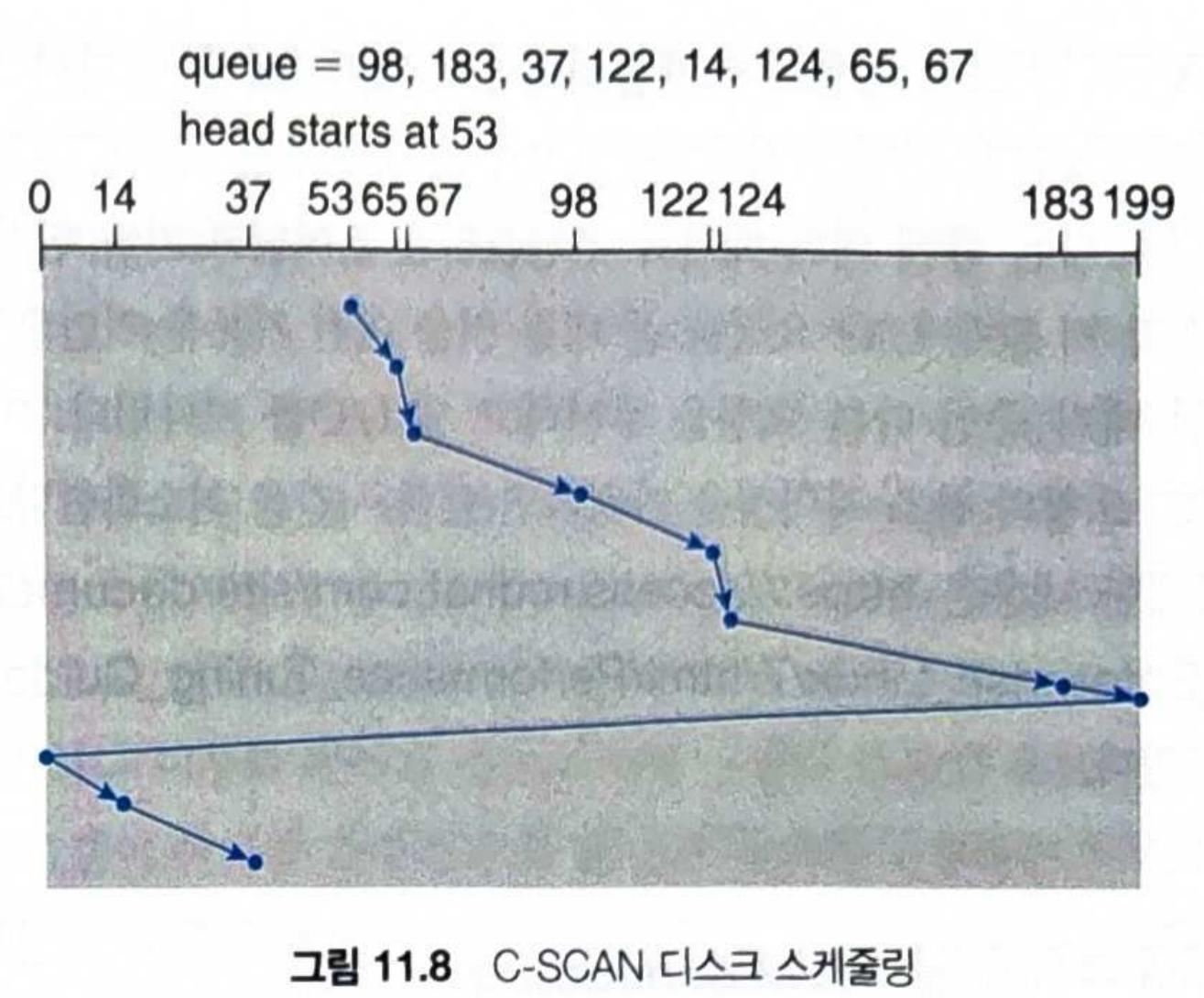


그림 11.8 C-SCAN 디스크 스케줄링 - 원문 그림 부분 발췌

큰 실린더 쪽 요청들을 처리한 뒤 반대 끝으로 돌아가는 경로를 표시한다. 돌아가는 구간과 요청을 처리하는 구간을 구별한다.

11.2.4 디스크 스케줄링 알고리즘의 선택

출처: 책 503-504쪽 / 원문 PDF 442-443쪽

출처: 책 503-504쪽 / 원문 PDF 442-443쪽

성능은 요청 수와 유형에 좌우된다. 큐에 요청이 하나뿐이면 모든 알고리즘이 사실상 FCFS처럼 동작한다. SCAN과 C-SCAN은 부하가 큰 시스템에서 기아 가능성을 줄이지만 완전히 없애지는 않는다.

출처: 책 503-504쪽 / 원문 PDF 442-443쪽

원문의 Linux **마감시간 스케줄러**는 읽기·쓰기마다 LBA 순서 큐와 FCFS 큐를 두어 총 네 큐를 유지한다. 읽을 때 프로세스가 봉쇄될 가능성이 더 높아 읽기에 우선순위를 준다. LBA 순서로 배치를 처리한 뒤 FCFS 큐에서 오래 기다린 요청을 검사하고, 그런 요청이 있으면 그 요청이 든 LBA 큐를 다음 배치로 선택한다. 원문의 기본 대기 기준은 500ms이다.

출처: 책 503-504쪽 / 원문 PDF 442-443쪽

원문에서 NOOP는 빠른 NVM을 쓰는 CPU 중심 시스템에 적합한 단순 스케줄러이고, CFQ는 실시간·최선 노력·유휴의 세 큐를 우선순위대로 관리한다. CFQ는 이전 기록으로 같은 프로세스의 후속 I/O를 예측하여 기다림으로써 참조 지역성과 탐색 감소를 이용하지만 배타적 우선순위로 기아가 생길 수 있다. 이 설명은 책에서 제시한 배포판·버전의 사례이며 현재 설정으로 확대하지 않는다.

11.3 NVM 스케줄링

출처: 책 504-505쪽 / 원문 PDF 443-444쪽

출처: 책 504-505쪽 / 원문 PDF 443-444쪽

NVM에는 움직이는 헤드가 없어 HDD의 이동 최소화 알고리즘의 이익이 작다. 보통 간단한 FCFS를 쓰며 Linux NOOP는 인접 요청을 병합한다. 읽기 서비스 시간은 일정하지만 플래시 쓰기 시간은 일정하지 않으므로 일부 SSD 스케줄러는 인접 쓰기만 병합하고 읽기는 FCFS로 처리한다.

출처: 책 504-505쪽 / 원문 PDF 443-444쪽

무작위 접근에서 HDD는 헤드 이동 때문에 제약을 받지만 NVM은 더 많은 IOPS를 처리할 수 있다. 순차 전송에서는 헤드 탐색이 적어 NVM의 상대적 이득이 작아진다. NVM 쓰기는 읽기보다 느리며, 장치가 얼마나 찼는지와 얼마나 마모되었는지에 따라 성능이 달라진다.

출처: 책 504-505쪽 / 원문 PDF 443-444쪽

쓰기 증폭이 생기는 과정

1. 응용의 쓰기를 위해 새 페이지를 기록해야 하지만 유효·무효 데이터가 섞인 블록에서 공간 회수가 필요하다.
2. 유효 페이지를 읽어 과잉 공급 공간으로 복사하고, 무효 데이터만 남은 블록을 지운다.
3. 이 블록을 다시 여유 공간에 포함한다. 응용의 한 번 쓰기에 추가 읽기와 추가 쓰기가 생긴다.

출처: 책 504-505쪽 / 원문 PDF 443-444쪽

이처럼 가비지 수집과 공간 관리를 위해 장치 자체가 생성하는 추가 I/O가 **쓰기 증폭**이다. 최악에는 매 쓰기마다 여러 I/O가 촉발된다. 파일 삭제 사실을 장치에 알려 불필요한 블록을 지우게 하는 방법은 수명과 성능 개선에 도움이 된다.

11.4 오류 감지 및 수정

출처: 책 505-506쪽 / 원문 PDF 444-445쪽

출처: 책 505-506쪽 / 원문 PDF 444-445쪽

오류 감지는 기록·전송·보관 중 데이터가 변했는지를 알아내어 전파를 막거나 경고하는 것이다. 바이트와 함께 패리티 비트를 저장하면 모든 단일 비트 오류를 감지하지만 이중 비트 오류는 놓칠 수 있다. 패리티는 XOR로 계산할 수 있고 추가 저장 공간이 필요하다. 체크섬은 계산한 값을 저장·비교하는 방식이며 CRC도 다중 비트 오류를 감지하는 방법으로 제시된다.

출처: 책 505-506쪽 / 원문 PDF 444-445쪽

ECC(오류 수정 코드)는 감지뿐 아니라 일부 오류를 보정할 만큼 정보를 추가로 저장한다. 디스크는 섹터별, 플래시는 페이지별 ECC를 사용한다. 기록할 때 값을 계산해 저장하고 읽을 때 다시 계산하여 비교한다. 손상 비트가 적어 수정할 수 있으면 소프트 오류이고, ECC로 수정할 수 없으면 하드 오류이다. 가능한 수정 범위는 알고리즘과 저장한 추가 정보의 양에 좌우된다. 컨트롤러가 통상적인 읽기·쓰기 때 이 처리를 자동 수행한다.

11.5 저장장치 관리

출처: 책 506-510쪽 / 원문 PDF 445-449쪽

11.5.1 드라이브 포매팅, 파티션, 볼륨

출처: 책 506-508쪽 / 원문 PDF 445-447쪽

출처: 책 506-508쪽 / 원문 PDF 445-447쪽

저수준(물리적) 포매팅은 섹터·페이지를 컨트롤러가 읽고 쓸 수 있는 구조로 초기화한다. 헤더·데이터 영역·트레일러가 마련되며 번호와 오류 탐지·수정 정보가 저장된다. NVM은 페이지 초기화와 FTL 생성이 필요하다. HDD는 대체로 공장에서 포맷되어 나온다. 섹터가 커지면 헤더·트레일러 비중이 줄지만 운영체제가 지원하는 섹터 크기는 제한될 수 있다.

출처: 책 506-508쪽 / 원문 PDF 445-447쪽

파일을 보유하기 위한 운영체제의 작업

1. 파티션: 블록·페이지 그룹을 나누어 별도 장치처럼 취급한다. 파티션 정보는 정해진 형식과 위치에 기록된다.
2. 볼륨 생성·관리: 파티션 하나를 바로 볼륨으로 쓰거나 여러 장치·파티션을 RAID 등으로 묶는다. 마운트 가능한 파일 시스템이나 파일 시스템을 담은 이미지 파일도 볼륨이라고 할 수 있다.
3. 논리적 포매팅: 가용·할당 공간의 맵과 빈 디렉터리 등 초기 파일 시스템 자료구조를 기록한다.

출처: 책 506-508쪽 / 원문 PDF 445-447쪽

마운트하면 파일 시스템을 사용할 수 있다. Windows는 C:·D:처럼 볼륨을 구분하고 Linux는 루트에서 시작하는 트리 내부에 다른 파일 시스템을 마운트한다. 파일 시스템은 블록을 클러스터로 묶어 순차 I/O를 늘리고 파일 데이터와 메타데이터를 가까이 배치하여 탐색을 줄인다.

출처: 책 506-508쪽 / 원문 PDF 445-447쪽

raw I/O는 파티션을 논리 블록의 배열로 직접 쓰며 버퍼 캐시·잠금·선반입·공간 할당·이름·디렉터리 같은 파일 시스템 서비스를 우회한다. 데이터베이스나 스왑처럼 전용 관리를 원하는 경우 유용하지만 대부분의 응용은 파일 시스템을 사용한다.

11.5.2 부트 블록

출처: 책 508-509쪽 / 원문 PDF 447-448쪽

출처: 책 508-509쪽 / 원문 PDF 447-448쪽

원문의 부팅 과정

1. 펌웨어의 작은 부트스트랩 로더가 CPU 레지스터·컨트롤러·메모리 등을 초기화한다.
2. 아직 전체 장치 드라이버가 없을 때에도 고정 위치의 부트 블록을 읽을 만큼의 기능으로 더 큰 부트스트랩 프로그램을 적재한다.
3. 부트스트랩 프로그램이 커널을 찾아 시작하고, 커널이 서브시스템과 서비스를 적재한다.

출처: 책 508-509쪽 / 원문 PDF 447-448쪽

원문의 Windows 예에서는 첫 논리 블록·페이지인 **MBR**에 부트 코드, 파티션 표, 부트 파티션 표시가 있다. 펌웨어가 MBR을 읽고 부트 파티션의 첫 섹터·페이지를 읽으면 그 부트 섹터가 커널로 안내한다. 부트 파티션이 있는 장치를 부트 디스크 또는 시스템 디스크라고 한다. 펌웨어는 갱신 가능하지만 악성 수정으로 감염될 수도 있다는 주의가 제시된다.

11.5.3 손상된 블록

출처: 책 509-511쪽 / 원문 PDF 448-450쪽

출처: 책 509-511쪽 / 원문 PDF 448-450쪽

단순한 방식은 포맷 중이나 검사 프로그램으로 손상 블록을 찾아 파일 시스템이 할당하지 않게 표시하는 것이다. 컨트롤러가 관리하는 방식에서는 손상 목록과 운영체제에 보이지 않는 예비 섹터를 유지하고 손상 섹터를 교체한다. 이것이 **섹터 예비/포워딩**이다. 논리 주소는 같지만 실제 위치가 달라져 스케줄링 최적화를 약화시킬 수 있어 같은 실린더의 예비 섹터를 우선 사용할 수 있다.

출처: 책 509-511쪽 / 원문 PDF 448-450쪽

섹터 밀어내기는 손상 위치부터 예비 섹터까지 여러 섹터를 옮겨 논리적 순서를 유지하는 방법이다. 원문은 논리 블록 17의 결함과 202 뒤의 예비 섹터를 이용해 202부터 차례로 뒤로 복사하는 예를 든다. 복구 가능한 소프트 오류는 사본을 만든 뒤 교체할 수 있지만 하드 오류의 데이터는 백업 등의 개입이 필요하다. NVM은 탐색 위치를 고려할 필요가 없어 예비 페이지·과잉 공급 영역과 불량 페이지 표로 관리한다.

11.6 스왑 공간 관리

출처: 책 511-513쪽 / 원문 PDF 450-452쪽

출처: 책 511-513쪽 / 원문 PDF 450-452쪽

가상 메모리는 보조저장장치를 메모리의 확장으로 쓰지만 훨씬 느리므로 스왑 설계가 성능에 크게 작용한다. 원문은 초기의 프로세스 전체 스와핑과 현대의 페이지 단위 스와핑을 구별한다. 스왑 공간의 크기는 메모리 관리 방식·물리/가상 메모리 크기에 좌우된다. 부족하면 프로세스 중단이나 시스템 크래시가 가능하고, 너무 크면 일반 파일 공간을 줄인다. 여러 장치의 스왑 영역으로 I/O 부하를 분산할 수도 있다.

출처: 책 511-513쪽 / 원문 PDF 450-452쪽

위치	장점·조건	제약
파일 시스템의 스왑 파일	일반 파일 루틴으로 생성·삭제·관리하기 편하다.	파일 시스템을 거친다.
전용 raw 파티션	파일·디렉터리를 저장하지 않고 속도에 초점을 맞춘 스왑 루틴으로 관리한다. 내용이 짧게 유지되고 부팅 때 초기화된다.	고정 크기를 늘리려면 재파티션하거나 다른 스왑 공간을 추가해야 한다.

출처: 책 511-513쪽 / 원문 PDF 450-452쪽

출처: 책 511-513쪽 / 원문 PDF 450-452쪽

11.6.3의 Solaris 예는 실행 파일의 코드 페이지를 교체할 때 스왑에 쓰지 않고 버린 뒤 필요하면 파일 시스템에서 다시 읽는다. 스왑은 파일로 뒷받침되지 않는 스택·힙·미초기화 데이터 같은 **익명 메모리**의 백업에 쓴다. 이후 Solaris는 페이지 생성 시점이 아니라 실제 퇴출 시점에 스왑 공간을 할당한다.

출처: 책 511-513쪽 / 원문 PDF 450-452쪽

출처: 책 511-513쪽 / 원문 PDF 450-452쪽

Linux도 익명 메모리에 스왑을 사용하며 파일이나 전용 파티션의 여러 영역을 허용한다. 원문 예의 각 영역은 연속된 4KB 페이지 슬롯으로 구성되고 **스왑 맵**의 정수 카운터가 슬롯 사용을 나타낸다. 0이면 가용, 양수이면 사용 중이며 값은 그 스왑 페이지를 공유하는 수이다. 원문 그림의 카운터 3은 세 프로세스가 같은 페이지를 쓰는 경우이다.

11.7 저장장치 연결

출처: 책 513-517쪽 / 원문 PDF 452-456쪽

출처: 책 513-517쪽 / 원문 PDF 452-456쪽

방법	접근 형태와 특징
호스트 연결	로컬 I/O 포트로 특정 장치의 논리 블록을 읽고 쓴다. USB·SATA·FC 등의 연결이 사용된다.
NAS	NFS·CIFS 등의 RPC로 네트워크상의 파일을 공유한다. 로컬 파일 시스템 같은 이름·접근을 제공하지만 데이터 네트워크 대역폭과 성능 비용이 있다.
iSCSI	IP 네트워크로 SCSI 명령과 논리 블록을 전달한다. 파일 시스템을 제공하는 NFS·CIFS와 달리 클라이언트가 raw 블록을 쓰거나 파일 시스템을 만든다.
클라우드 저장장치	WAN·인터넷을 거쳐 원격 데이터 센터의 저장장치에 접근하며 대체로 API를 사용한다. LAN용 NAS보다 지연·연결 장애의 가능성을 고려한다.
SAN	서버와 저장장치 유닛을 저장장치 프로토콜로 연결하는 사유 네트워크이다. 동적 할당·공유가 유연하고 일반 데이터 네트워크와의 경쟁을 줄인다.

출처: 책 513-517쪽 / 원문 PDF 452-456쪽

NAS의 NFS·CIFS는 파일 공유를 위해 잠금 기능을 제공한다. SAN 스위치는 호스트의 접근을 허용·금지하고 여러 호스트에 저장장치를 배정한다. SAN은 포트가 많아 비용이 크지만 근거리 연결이므로 NAS가 더 많은 호스트를 연결할 수도 있다. 저장장치 배열은 드라이브·컨트롤러·캐시·관리 소프트웨어를 갖춘 독립 장치이며 RAID·스냅숏·복제·압축·중복 제거·암호화 등을 구현할 수 있다. SSD만 쓰거나 HDD와 조합하여 용도에 맞게 배치·캐싱할 수도 있다.

11.8 RAID 구조

출처: 책 517-529쪽 / 원문 PDF 456-468쪽

11.8.1 중복으로 신뢰성 향상

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

RAID는 여러 드라이브를 이용하여 성능과 신뢰성을 높이는 구성이다. 원문은 I가 과거 inexpensive에서 independent로 해석되는 추세도 소개한다. 드라이브 수만 늘리고 데이터 사본은 하나만 유지하면 전체의 고장 가능성이 커진다. 중복 정보는 보통 불필요하지만 고장 시 분실 정보를 재구성하는 데 쓰인다.

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

원문 코드·기호

```
100,000 / 100 = 1,000시간
```

원문의 단일 드라이브 MTBF가 100,000시간인 100개 드라이브 집합의 고장 간 평균 시간 예이다.

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

미러링은 하나의 논리 드라이브를 두 물리 드라이브로 만들고 모든 쓰기를 양쪽에 수행한다. 하나가 고장 나면 나머지에서 읽지만, 복구 전에 두 번째도 고장 나면 데이터가 손실된다. 신뢰성은 드라이브 MTBF뿐 아니라 교체·복구의 평균 시간에도 좌우된다.

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

원문 코드·기호

```
100,000² / (2 × 10) = 500 × 10⁶시간
```

드라이브 고장이 독립적이고 단일 MTBF가 100,000시간, 평균 수리 시간이 10시간이라는 원문의 가정에서 얻는 평균 데이터 분실 시간이다. 원문은 약 57,000년으로 표현한다.

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

이 독립성은 현실에서 보장되지 않는다. 전력·재해·제조 결함은 두 드라이브에 함께 영향을 주고, 노후화·복구 중 부하도 실패 가능성을 바꾼다. 두 미러 블록을 동시에 쓰다가 전원이 끊기면 둘 다 불완전할 수 있다. 순서대로 쓰거나 보호된 NVRAM 쓰기 지연 캐시를 두어 완료할 수 있게 하는 대응이 제시된다.

11.8.2 병렬성을 이용한 성능 향상

출처: 책 519-520쪽 / 원문 PDF 458-459쪽

출처: 책 519-520쪽 / 원문 PDF 458-459쪽

미러의 두 드라이브로 읽기 요청을 분산하면 개별 읽기의 전송률 자체와 별도로 여러 읽기 처리 능력을 높일 수 있다. **스트라이핑**은 데이터를 드라이브에 나누어 저장한다. 비트 수준 예는 바이트의 각 비트를 여러 드라이브에 나누어 같은 시간에 더 많은 데이터를 읽는 것이다. 블록 수준에서는 블록을 여러 드라이브에 순환 배치하며 이것이 일반적으로 사용된다.

출처: 책 519-520쪽 / 원문 PDF 458-459쪽

원문 코드·기호

```
(i mod n) + 1
```

원문은 n개 드라이브에서 파일의 i번째 블록을 배치하는 드라이브 번호를 이 형태로 제시한다. 원문의 인덱스 표현을 보존한다.

출처: 책 519-520쪽 / 원문 PDF 458-459쪽

병렬성의 두 목적은 여러 작은 접근의 처리량을 부하 균등화로 높이는 것과 큰 접근의 응답 시간을 줄이는 것이다. 스트라이핑 자체는 중복이 아니므로 신뢰성을 높이지 않는다.

11.8.3 RAID 레벨과 구현

출처: 책 520-525쪽 / 원문 PDF 459-464쪽

출처: 책 520-525쪽 / 원문 PDF 459-464쪽

레벨	구성·보호	핵심 조건과 비용
0	중복 없는 블록 수준 스트라이핑	전송률은 높이지만 오류 복구용 중복 정보가 없다.
1	미러링	높은 신뢰성과 간단한 복구 대신 드라이브 사본이 필요하다.
4	블록 스트라이핑과 전용 패리티 드라이브	고장 섹터를 식별할 수 있을 때 나머지 데이터와 패리티로 복구한다. 작은 쓰기는 데이터와 패리티의 읽기-수정-쓰기가 필요하다.
5	데이터·패리티를 모든 드라이브에 분산	RAID 4의 전용 패리티 드라이브 집중을 줄인다. 같은 블록과 그 패리티는 동일 드라이브에 두지 않는다.
6	P+Q의 두 중복 정보	두 드라이브 고장을 허용한다. 같은 XOR 결과를 두 번 저장하는 것만으로는 추가 보호가 되지 않는다.
다차원 6	논리적 행·열 방향에 RAID 6 적용	여러 위치의 중복 정보를 이용한다. 그림은 단순화를 위해 전용 패리티 드라이브로 보이지만 실제 블록은 분산된다.
0+1	스트라이프 집합을 다시 미러링	단일 고장으로 한 스트라이프가 사용 불가능해질 수 있다.
1+0	미러 쌍들을 스트라이핑	한 드라이브 고장 뒤에도 해당 미러와 다른 쌍이 계속 사용될 수 있다.

출처: 책 520-525쪽 / 원문 PDF 459-464쪽

그림 11.15 RAID 레벨

그림·표 출처: 책 521쪽 / 원문 PDF 460쪽

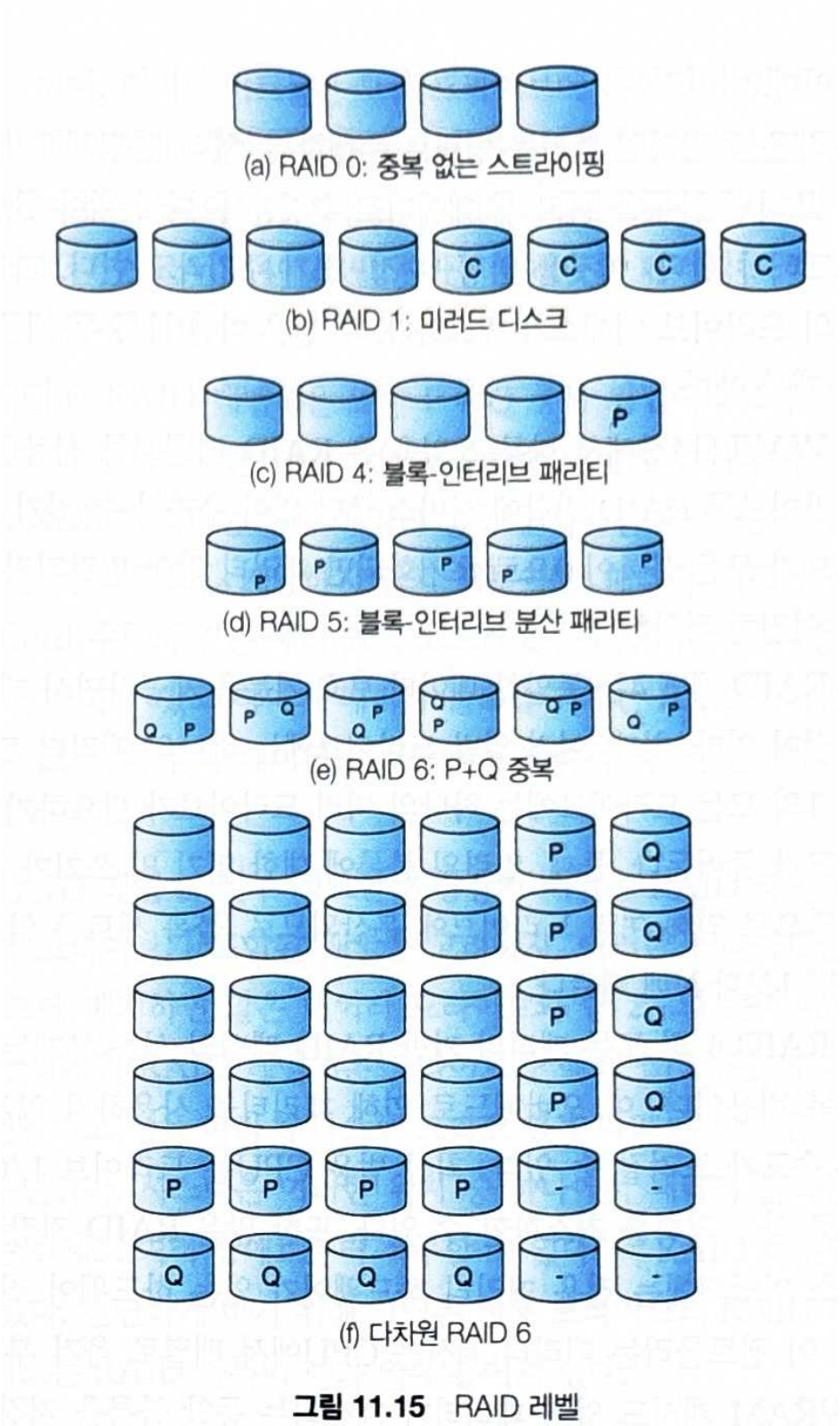


그림 11.15 RAID 레벨 - 원문 그림 부분 발췌

P는 오류 교정 정보, C는 두 번째 사본이다. 중복 없는 스트라이핑, 미러링, 전용·분산 패리티, 이중 패리티의 차이를 보존한다.

출처: 책 520-525쪽 / 원문 PDF 459-464쪽

RAID 4의 작은 쓰기는 이전 데이터·패리티 읽기 두 번과 새 데이터·패리티 쓰기 두 번, 총 네 드라이브 접근을 요구한다. XOR 계산·기록도 비용이다. 전용 패리티 하드웨어와 NVRAM 캐시는 CPU 부담을 줄이고 전체 스트라이프가 모이면 병렬 기록하여 읽기-수정-쓰기를 줄일 수 있다. WAFL은 0으로 초기화한 드라이브를 추가해도 패리티가 바뀌지 않는 RAID 4의 확장 특성을 이용한다.

출처: 책 520-525쪽 / 원문 PDF 459-464쪽

그림 11.16 RAID 0+1과 1+0의 단일 장애

그림·표 출처: 책 524쪽 / 원문 PDF 463쪽

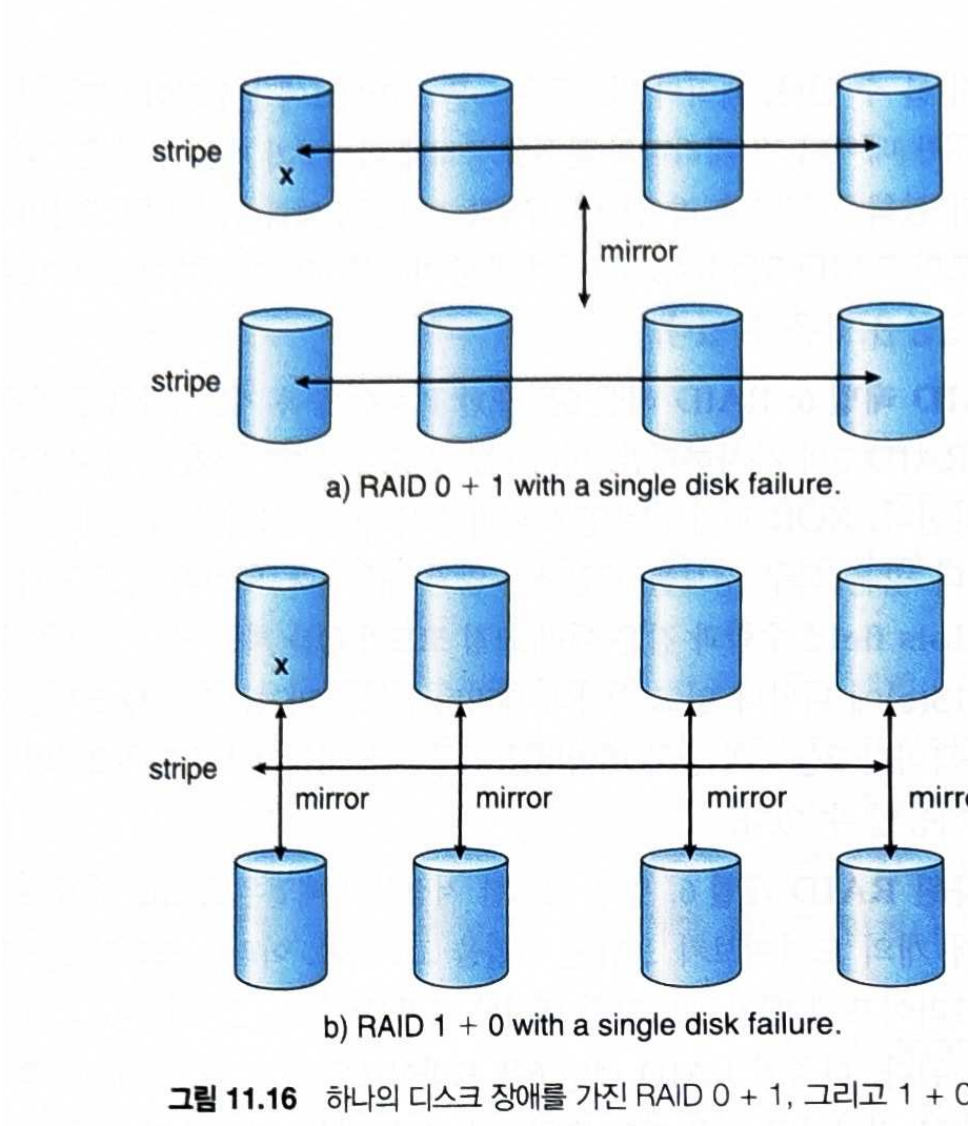


그림 11.16 하나의 디스크 장애를 가진 RAID 0 + 1, 그리고 1 + 0

그림 11.16 RAID 0+1과 1+0의 단일 장애 - 원문 그림 부분 발췌

스트라이핑 후 미러링하는 구조와 미러링 후 스트라이핑하는 구조에서 고장 영향 범위를 비교한다.

출처: 책 520-525쪽 / 원문 PDF 459-464쪽

RAID는 커널·볼륨 관리 소프트웨어, HBA, 저장장치 배열, SAN 내부 연결 계층에서 구현할 수 있다. 위치에 따라 비용·유연성·공유 범위가 다르다. 스냅숏은 갱신 전 파일 시스템 모습이고 복제는 다른 위치에 같은 내용을 자동 기록하는 것이다. 동기 복제는 로컬·원격 기록 뒤 완료하고, 비동기 복제는 묶어서 기록하므로 빠르고 거리에 덜 제한되지만 주 사이트 고장 시 미전달 데이터가 손실될 수 있다. **hot spare**는 데이터를 담지 않고 대기하다 고장 드라이브를 자동 대체하여 복구 지연을 줄인다.

11.8.4 RAID 레벨 선택

출처: 책 525-527쪽 / 원문 PDF 464-466쪽

출처: 책 525-527쪽 / 원문 PDF 464-466쪽

선택 기준에는 정상 성능뿐 아니라 고장 후 데이터를 계속 공급할 능력과 복구 시간·비용·공간 오버헤드가 포함된다. RAID 1은 복사로 복구하여 단순하고, 패리티 방식은 다른 드라이브에 접근해 재구성한다. 큰 RAID 5의 복구는 오래 걸릴 수 있다. 레벨 0은 데이터를 다시 적재할 수 있는 고성능 용도, 레벨 1은 빠른 복구, 0+1·1+0은 성능과 신뢰성이 함께 중요한 용도의 예로 제시된다.

출처: 책 525-527쪽 / 원문 PDF 464-466쪽

패리티 하나로 더 많은 데이터를 보호하면 공간 비용은 줄지만 복구 전 추가 고장에 노출될 수 있다. 드라이브 수를 늘리는 것도 전송률과 비용을 함께 바꾼다. 별도 사례인 InServ는 드라이브를 256MB chunklet으로 나누어 서로 다른 볼륨과 RAID 레벨에 쓸 수 있게 한다. 논리적으로 큰 공간을 먼저 제공하고 실제 쓰임에 따라 물리 저장장치를 할당하는 방식과 쓰기 시 복사 스냅숏도 설명된다.

11.8.5 확장

출처: 책 527쪽 / 원문 PDF 466쪽

출처: 책 527쪽 / 원문 PDF 466쪽

RAID의 중복·분산 개념은 테이프와 무선 데이터 전송에도 적용된다. 여러 테이프에 스트라이핑하면 처리량을 높이고 백업 시간을 줄일 수 있다. 데이터 조각과 패리티를 함께 전송하면 일부 조각을 수신하지 못해도 다른 조각에서 복구할 수 있다.

11.8.6 RAID의 문제점들

출처: 책 527-529쪽 / 원문 PDF 466-468쪽

출처: 책 527-529쪽 / 원문 PDF 466-468쪽

RAID는 물리 매체 오류를 보호하지만 모든 데이터 손상을 막지는 않는다. 잘못된 포인터, 미완료 쓰기(torn writes), 잘못된 쓰기, 컨트롤러 고장, RAID 소프트웨어 버그는 여전히 위험하다. 원문 ZFS는 데이터뿐 아니라 메타데이터도 체크섬으로 검사하며, 체크섬을 해당 블록 자체가 아니라 **그 블록을 가리키는 포인터와 함께** 둔다. 미러에 올바른 사본이 있으면 손상 블록을 갱신할 수 있다.

출처: 책 527-529쪽 / 원문 PDF 466-468쪽

그림 11.17 ZFS의 메타데이터·데이터 체크섬

그림·표 출처: 책 528쪽 / 원문 PDF 467쪽

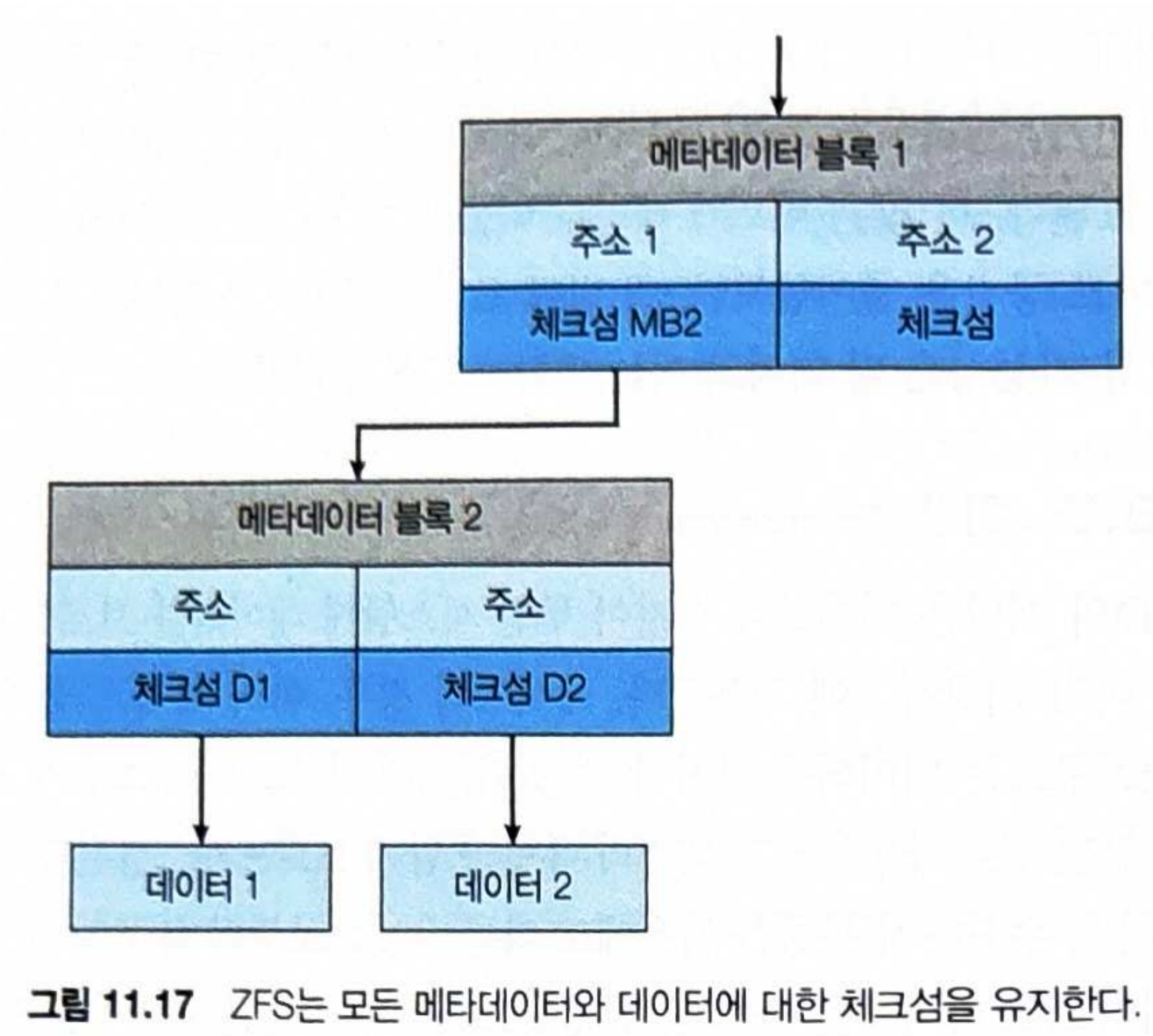


그림 11.17 ZFS는 모든 메타데이터와 데이터에 대한 체크섬을 유지한다.

그림 11.17 ZFS의 메타데이터·데이터 체크섬 - 원문 그림 부분 발췌

상위 메타데이터의 포인터와 체크섬이 하위 메타데이터·데이터를 함께 검증하는 관계이다.

출처: 책 527-529쪽 / 원문 PDF 466-468쪽

별도 볼륨·파일 시스템 크기는 수요가 변할 때 부적절해질 수 있고, 볼륨을 바꿔도 파일 시스템이 확장·축소를 지원하지 않을 수 있다. ZFS는 파일 시스템 관리와 볼륨 관리를 통합하고 RAID 드라이브를 **저장장치 풀**로 묶어 여러 파일 시스템에 필요에 따라 할당·반환한다. 쿼터는 파일 시스템 크기를 제한하고 예약은 지정 크기까지 확장을 보장한다. 이 변수들은 변경 가능하다.

11.8.7 객체 저장소

출처: 책 529-530쪽 / 원문 PDF 468-469쪽

출처: 책 529-530쪽 / 원문 PDF 468-469쪽

원문의 객체 접근 순서

1. 풀에 객체를 생성하고 객체 ID를 받는다.
2. 필요할 때 객체 ID로 접근한다.
3. 객체 ID로 삭제한다.

출처: 책 529-530쪽 / 원문 PDF 468-469쪽

객체 저장소는 일반 파일 시스템처럼 사용자가 디렉터리를 탐색하는 대신 프로그램이 ID로 접근하도록 설계된다. 관리 소프트웨어가 위치와 복제를 결정한다. 원문의 HDFS 예는 서로 다른 컴퓨터에 여러 사본을 두고 사본이 있는 컴퓨터에서 계산한 결과를 전송한다. 다른 컴퓨터의 데이터 접근에는 네트워크가 필요하여 고속 임의 접근보다는 대량 저장에 사용된다.

출처: 책 529-530쪽 / 원문 PDF 468-469쪽

컴퓨터와 디스크를 더해 풀을 늘리는 **수평 확장성**과 내용의 설명을 포함하는 자체 설명성이 특징이다. 원문은 콘텐츠를 기반으로 검색할 수 있어 콘텐츠 주소지정가능 저장소라고도 하며 형식이 정해지지 않은 구조화되지 않은 데이터를 저장한다고 설명한다. 이는 원문의 객체 저장소 설명 범위이다.

11.9 요약과 원문 연습문제의 점검 범위

출처: 책 530-534쪽 / 원문 PDF 469-473쪽

출처: 책 530-534쪽 / 원문 PDF 469-473쪽

원문 요약은 논리 블록·연결 방식·스케줄링·오류 처리·파티션/볼륨·스왑·RAID·객체 저장소를 다시 연결한다. 장말 연습문제(인쇄532~533쪽)는 스케줄링의 적용 조건, 회전 지연, 장치 간 I/O 균형, 코드 페이지와 스왑, 안정 저장장치, 전송 크기와 유효 전송률, RAID 읽기, HDD/NVM 선택을 점검한다. 원문에 제시되지 않은 풀이·계산 결과를 추가하지 않는다. 추가 자료와 참고문헌은 존재를 확인했으며 외부 내용을 요약에 보충하지 않았다.

핵심 정리

- HDD는 탐색·회전 지연이 중요하고 NVM은 삭제 단위·마모·가비지 수집·쓰기 증폭이 중요하다.
- FCFS의 공정성, SCAN의 양방향 처리, C-SCAN의 한 방향 처리와 귀환을 구분한다.
- 오류 감지와 수정, 논리 주소와 실제 위치, 파티션·볼륨·파일 시스템은 각각 다른 역할이다.
- 미러링은 중복을, 스트라이핑은 병렬성을 제공한다. RAID 레벨은 성능·공간 비용·복구 능력의 조합이다.
- RAID만으로 논리적 손상·소프트웨어 오류까지 모두 보호할 수는 없다.

장 마무리 복습

- 물리 구조 → 논리 블록 → 요청 큐 → 매핑·복구라는 연결을 먼저 떠올린다.
- 스왑 파일과 raw 파티션은 관리 편의와 성능·크기 변경의 조건이 다르다.
- NAS의 파일, iSCSI의 논리 블록, SAN의 저장장치 연결, 클라우드의 API 접근을 구별한다.
- ZFS는 포인터에 둔 체크섬과 공유 풀을 통해 무결성·크기 관리 문제를 다룬다.

제12장. 입출력 시스템

출처: 책 535-576쪽 / 원문 PDF 474-515쪽

학습 주제

- 포트·버스·컨트롤러의 하드웨어 동작을 응용 인터페이스와 커널 서비스에 연결한다. 폴링·인터럽트·DMA, 봉쇄형·비봉쇄형·비동기식 요청, 버퍼·캐시·스풀의 역할을 구별하고 요청 전체의 처리 경로를 따라간다.

설명의 흐름

1. 하드웨어와 핸드셰이킹 → 장치 드라이버의 추상화 → 응용 I/O 인터페이스 → 커널 서비스 → 이름 변환과 요청 생애 주기 → STREAMS → 성능과 구현 위치

12.1 개관

출처: 책 535-536쪽 / 원문 PDF 474-475쪽

출처: 책 535-536쪽 / 원문 PDF 474-475쪽

운영체제는 다양한 속도·기능의 입출력 장치를 관리·제어한다. 인터페이스 표준화는 새 장치의 결합을 쉽게 하지만 장치의 다양성도 계속 커진다. 포트·버스·컨트롤러라는 하드웨어 요소와 **장치 드라이버**라는 소프트웨어 모듈이 이 문제를 다룬다. 드라이버는 구체적인 장치 차이를 감추고 상위 커널 I/O 서브시스템에 일관된 인터페이스를 제공한다.

12.2 입출력 하드웨어

출처: 책 536-538쪽 / 원문 PDF 475-477쪽

출처: 책 536-538쪽 / 원문 PDF 475-477쪽

포트는 장치가 연결되는 접점이다. **버스**는 여러 장치가 공유하는 선과 메시지를 교환하는 프로토콜을 포함한다. 장치가 차례로 연결되어 포트로 이어지는 데이지 체인은 통상 버스처럼 동작한다. **컨트롤러**는 포트·버스·장치를 동작시키는 전자장치이며, 단순 칩부터 프로세서·마이크로코드·메모리를 갖춘 HBA까지 다양하다.

출처: 책 536-538쪽 / 원문 PDF 475-477쪽

원문의 PC 구조는 프로세서·메모리와 고속 장치를 PCI 계열 버스로 연결하고 느린 장치는 확장 버스에 연결한다. PCIe는 수신·송신 신호 쌍으로 구성된 레인을 사용하여 양방향 전송을 제공한다. 디스크 자체의 컨트롤러는 프로토콜뿐 아니라 불량 섹터 매핑·선반입·버퍼링·캐싱도 담당한다.

12.2.1 메모리 맵드 입출력

출처: 책 538-539쪽 / 원문 PDF 477-478쪽

출처: 책 538-539쪽 / 원문 PDF 477-478쪽

CPU는 컨트롤러 레지스터에 비트 패턴을 읽고 써서 I/O를 수행한다. 특수 I/O 명령으로 포트를 지정하는 방법과, 장치 제어 레지스터를 프로세서 주소 공간에 대응시키는 **메모리 맵드 I/O**를 구별한다. 후자는 보통의 데이터 전송 명령으로 레지스터를 접근한다. 그래픽 출력의 큰 매핑 영역처럼 많은 데이터를 다룰 때 유용하다.

출처: 책 538-539쪽 / 원문 PDF 477-478쪽

레지스터	호스트의 사용과 역할
입력(data-in)	입력 데이터를 읽는다.
출력(data-out)	출력할 데이터를 쓴다.
상태(status)	명령 완료·입력 준비·오류 여부를 읽는다.
제어(control)	명령을 내리거나 통신 방식·검사·속도 등의 모드를 지정한다.

출처: 책 538-539쪽 / 원문 PDF 477-478쪽

데이터 레지스터가 작더라도 FIFO 칩으로 버퍼를 확장하여 돌발적으로 많은 데이터가 들어오는 상황에 대응할 수 있다.

12.2.2 폴링

출처: 책 539-540쪽 / 원문 PDF 478-479쪽

출처: 책 539-540쪽 / 원문 PDF 478-479쪽

원문의 바이트 출력 핸드셰이킹

1. 호스트는 busy 비트가 소거될 때까지 검사한다.
2. 명령 레지스터의 쓰기 비트를 설정하고 data-out에 출력 바이트를 쓴다.
3. command-ready 비트를 설정한다.
4. 컨트롤러는 command-ready를 알아차리고 busy를 설정한다.
5. 컨트롤러는 쓰기 명령과 데이터를 읽어 장치로 출력한다.
6. 완료하면 command-ready·오류·busy 비트를 소거한다. 이 과정은 바이트마다 반복된다.

출처: 책 539-540쪽 / 원문 PDF 478-479쪽

첫 단계의 반복 검사가 **폴링(바쁜 대기)**이다. 기본 검사 자체는 짧지만 준비까지 오래 걸리면 CPU가 다른 유용한 일을 못 한다. 반대로 입력 장치를 너무 늦게 서비스하면 버퍼가 넘쳐 데이터를 잃을 수 있다. 상태가 바뀌었을 때 장치가 CPU에 알리는 인터럽트는 긴 반복 대기를 피하는 방법이다.

12.2.3 인터럽트

출처: 책 540-545쪽 / 원문 PDF 479-484쪽

출처: 책 540-545쪽 / 원문 PDF 479-484쪽

인터럽트 구동 I/O

1. 드라이버가 I/O를 시작하고 CPU는 다른 명령을 수행한다.
2. 컨트롤러가 입력 준비·출력 완료·오류를 알리는 인터럽트 요청을 보낸다.
3. CPU는 상태를 저장하고 인터럽트 핸들러로 제어를 옮긴다.
4. 핸들러가 원인을 조사하여 장치를 서비스하고 복귀 명령으로 이전 실행 상태를 회복한다.

출처: 책 540-545쪽 / 원문 PDF 479-484쪽

그림 12.3 인터럽트 구동식 입출력 사이클

그림·표 출처: 책 541쪽 / 원문 PDF 480쪽

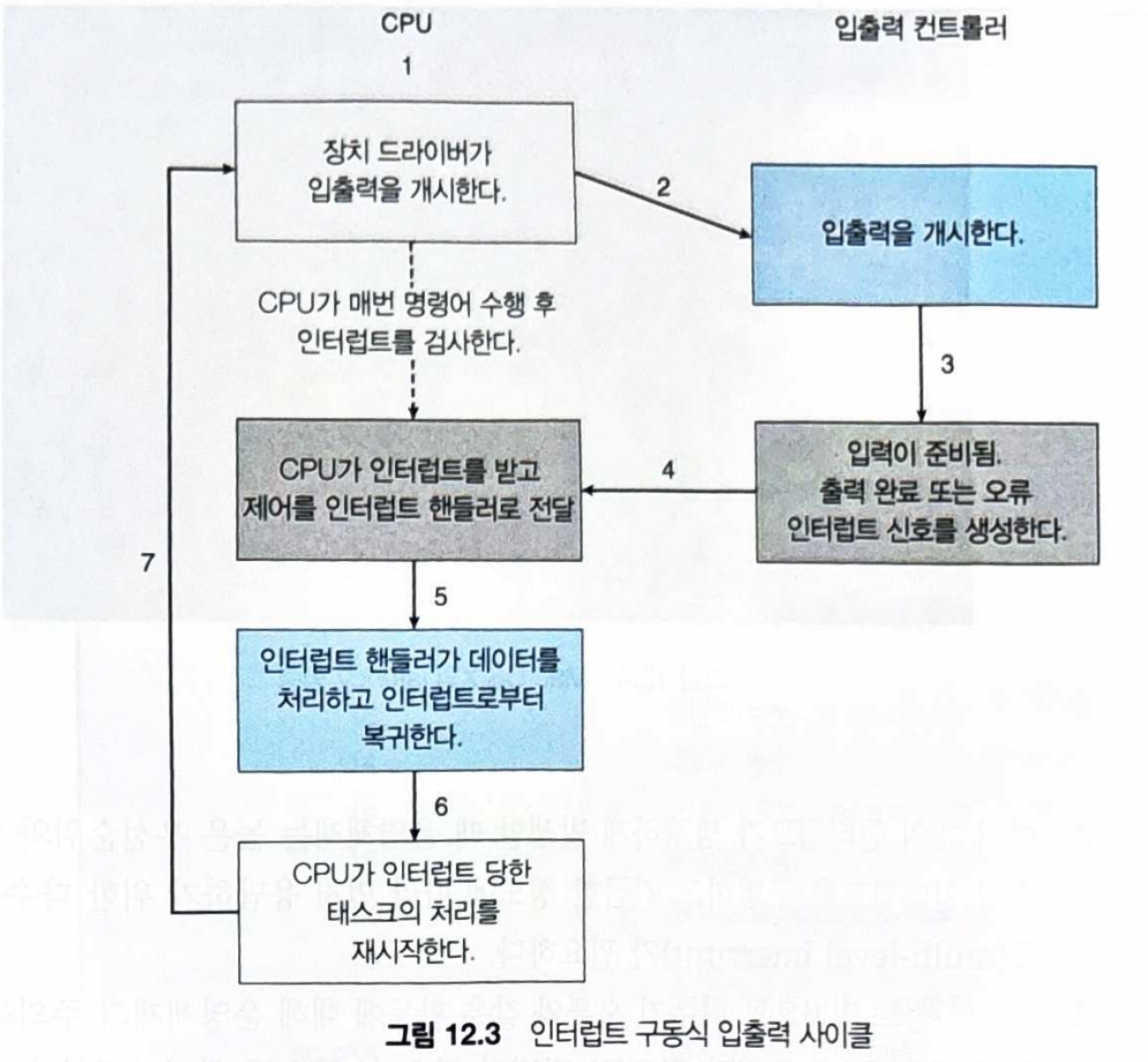


그림 12.3 인터럽트 구동식 입출력 사이클 - 원문 그림 부분 발췌

CPU 측 처리와 컨트롤러 측의 입출력·완료 신호 사이의 순서를 보여준다.

출처: 책 540-545쪽 / 원문 PDF 479-484쪽

현대 시스템은 인터럽트의 연기, 발생 장치 식별, 여러 우선순위, 예외·트랩 처리를 지원한다. **마스크 불가 인터럽트**는 회복 불가능한 메모리 오류 등의 긴급 상황에, **마스크 가능 인터럽트**는 필요하면 일시 중단할 수 있는 장치 요청에 사용된다. 인터럽트 벡터는 핸들러 주소 테이블을 선택하고, 여러 장치가 한 항목을 공유하면 사슬의 핸들러를 차례로 조사한다. 큰 테이블과 탐색 비용 사이의 절충이다.

출처: 책 540-545쪽 / 원문 PDF 479-484쪽

우선순위는 높은 인터럽트가 낮은 인터럽트 처리를 선점하게 한다. 시간·자원이 제한된 처리기를 1차와 2차로 나누어 1차는 상태 저장·문맥 교환·큐 삽입 등을, 별도로 스케줄되는 2차는 나머지 작업을 수행할 수 있다. 디스크 예에서 다음 I/O 시작은 급하지만 사용자 버퍼로 복사하는 작업은 덜 급하므로 우선순위를 달리한다. 커널 스레드와 우선순위로 이를 구현하면 다중 처리기에서 병렬 핸들링도 가능하다.

출처: 책 540-545쪽 / 원문 PDF 479-484쪽

페이지 폴트는 현재 프로세스를 대기시키고 페이지 입력을 스케줄한 뒤 다른 프로세스를 실행하게 한다. 입력 완료 인터럽트는 대기 프로세스를 준비 큐로 옮긴다. 시스템 콜의 트랩은 인자를 마련하고 사용자 상태를 저장한 뒤 커널 모드의 서비스를 실행하는 경로이다. 원문은 장치 인터럽트와 하드웨어 오류·시스템 콜을 모두 커널 루틴을 촉발하는 용도로 연결한다.

출처: 책 540-545쪽 / 원문 PDF 479-484쪽

인터럽트 처리에도 상태 저장·복귀 등의 비용이 있다. I/O 발생률이 낮을 때 인터럽트를 쓰다가 매우 높아져 폴링이 더 효율적이면 전환하는 혼합 방식도 있다. 따라서 인터럽트가 언제나 폴링보다 빠르다고 단정하지 않는다.

12.2.4 직접 메모리 접근(DMA)

출처: 책 545-547쪽 / 원문 PDF 484-486쪽

출처: 책 545-547쪽 / 원문 PDF 484-486쪽

CPU가 상태를 검사하며 바이트마다 직접 전송하는 **PIO**는 대용량 장치에 CPU 비용이 크다. **DMA**는 이 전송을 특수 컨트롤러에 맡긴다. 호스트는 출발·도착 포인터와 바이트 수를 담은 명령 블록을 메모리에 쓰고 그 주소를 DMA에 알려준다. 비연속 주소 목록을 사용하는 분산-수집 방식은 한 명령으로 여러 전송을 수행한다.

출처: 책 545-547쪽 / 원문 PDF 484-486쪽

DMA 전송

1. 장치 컨트롤러가 DMA-request로 전송을 요청한다.
2. DMA 컨트롤러가 메모리 버스를 확보하고 주소를 놓은 뒤 DMA-acknowledge를 보낸다.
3. 장치가 워드를 전송하고 요청 신호를 제거한다.
4. 전체 전송이 끝나면 DMA가 CPU에 인터럽트로 완료를 알린다.

출처: 책 545-547쪽 / 원문 PDF 484-486쪽

그림 12.6 DMA 전송의 단계

그림·표 출처: 책 546쪽 / 원문 PDF 485쪽

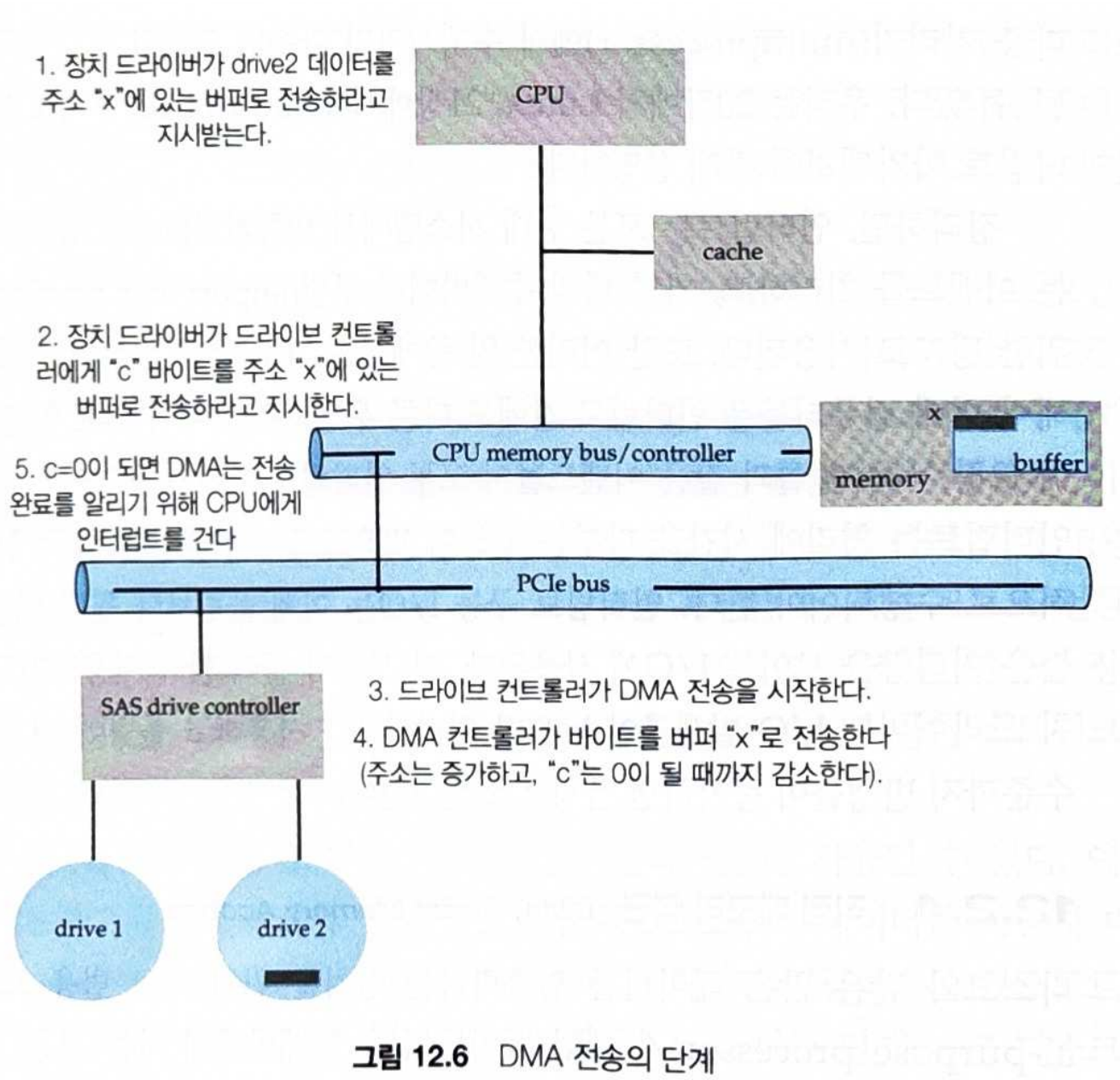


그림 12.6 DMA 전송의 단계

그림 12.6 DMA 전송의 단계 - 원문 그림 부분 발췌

원문의 주소 x, 전송량 c 및 완료 인터럽트의 관계를 보존한다. 주소는 증가하고 c는 0이 될 때까지 감소한다.

출처: 책 545-547쪽 / 원문 PDF 484-486쪽

DMA가 버스를 쓰는 동안 CPU는 캐시에는 접근할 수 있어도 주메모리 접근이 잠시 제한된다. 이 **사이클 스틸링**은 CPU를 늦추지만 전체 I/O 성능을 개선한다. 커널 버퍼를 거치면 사용자 공간으로 다시 복사하는 이중 버퍼링 비용이 있고, 직접 사용자 공간을 이용하면 전송 도중 수정에 주의해야 한다. DVMA는 가상 주소로 메모리 맵드 장치 사이의 전송을 지원한다. 일반 프로세스의 직접 I/O는 성능상 이익이 있어도 보안·안정성을 해칠 수 있어 범용 운영체제는 메모리와 장치를 보호한다.

12.2.5 입출력 하드웨어 요약

출처: 책 547-548쪽 / 원문 PDF 486-487쪽

출처: 책 547-548쪽 / 원문 PDF 486-487쪽

컨트롤러·포트·레지스터가 장치와 통신하는 기반이다. 호스트와 컨트롤러는 핸드셰이킹하고, 준비 상태는 폴링 또는 인터럽트로 확인하며, 큰 전송은 DMA로 위임한다. 이 하드웨어 차이를 상위의 일관된 인터페이스에 어떻게 수용하는지가 다음 절의 주제이다.

12.3 응용 입출력 인터페이스

출처: 책 548-551쪽 / 원문 PDF 487-490쪽

출처: 책 548-551쪽 / 원문 PDF 487-490쪽

추상화·캡슐화·계층화로 장치를 대표 범주와 표준 함수 집합으로 표현한다. 장치 드라이버가 이 함수를 구체적으로 구현하므로 상위 I/O 서브시스템은 하드웨어 차이에 덜 의존한다. 새 하드웨어는 기존 컨트롤러 인터페이스를 구현하거나 새 드라이버를 제공할 수 있다. 다만 드라이버 인터페이스는 운영체제마다 달라 각 운영체제용 드라이버가 필요하다.

출처: 책 548-551쪽 / 원문 PDF 487-490쪽

원문의 분류 기준	구분
전송 단위	문자 스트림 / 블록
접근 순서	순차 / 임의 접근
다른 사건과의 조율	동기식 / 비동기식 장치
사용자 수	공유 가능 / 전용
속도	지연·탐색 시간·전송률·연산 간 지연
입출력 방향	읽기·쓰기 / 읽기 전용 / 한 번 쓰기

출처: 책 548-551쪽 / 원문 PDF 487-490쪽

운영체제는 블록·문자 스트림·메모리 맵드 파일·네트워크 소켓·클록/타이머 등의 범주를 제공한다. 범주에 없는 고유 명령은 UNIX의 **ioctl()** 같은 호출로 전달할 수 있다. 원문의 세 인자는 장치 식별자, 명령을 나타내는 정수, 임의 자료구조를 가리키는 포인터이다. UNIX/Linux의 주(major) 번호는 드라이버·장치 유형을, 부(minor) 번호는 구체적 인스턴스를 선택한다.

12.3.1 블록 장치와 문자 장치

출처: 책 551-552쪽 / 원문 PDF 490-491쪽

출처: 책 551-552쪽 / 원문 PDF 490-491쪽

블록 인터페이스는 read·write·seek로 블록 장치를 제어하며 응용은 보통 파일 시스템을 거쳐 사용한다. 자체 버퍼·잠금을 쓰는 데이터베이스에는 운영체제의 중복 버퍼링·잠금이 부담이나 충돌이 될 수 있다. raw 접근은 장치 제어를 응용에 맡기고, **direct I/O**는 파일을 유지하되 버퍼링·잠금을 하지 않는 절충으로 소개된다.

출처: 책 551-552쪽 / 원문 PDF 490-491쪽

메모리 맵드 파일은 파일을 가상 주소에 대응시켜 메모리 읽기·쓰기로 접근한다. 해당 주소가 참조될 때 데이터가 실제로 메모리에 올라와 요구 페이징을 이용한다. 실행 파일 적재와 스왑 접근에도 이러한 인터페이스가 쓰인다. 문자 장치는 put·get처럼 한 문자씩 전송하고 상위 라이브러리가 줄 단위·편집·버퍼링을 제공할 수 있다. 키보드·마우스·모뎀이나 선형 바이트 스트림 출력 장치에 적합하다.

12.3.2 네트워크 장치

출처: 책 552쪽 / 원문 PDF 491쪽

출처: 책 552쪽 / 원문 PDF 491쪽

네트워크는 디스크의 read-write-seek와 다른 **소켓 인터페이스**를 사용한다. 소켓 생성, 원격 주소와 연결, 연결 완료 확인, 패킷 교환을 제공하여 하드웨어·프로토콜 내부를 감춘다. select()는 여러 소켓 가운데 수신할 데이터나 송신 여유가 있는 소켓을 알려 주어 응용의 바쁜 대기를 피한다. 원문은 FIFO·STREAMS·메시지 큐·소켓 등 다양한 IPC 방식의 존재도 설명한다.

12.3.3 클록과 타이머

출처: 책 553-554쪽 / 원문 PDF 492-493쪽

출처: 책 553-554쪽 / 원문 PDF 492-493쪽

클록·타이머는 현재 시각, 지난 시간, 지정 시각의 연산 실행을 제공한다. **프로그램 가능 인터벌 타이머**는 한 번 또는 주기적으로 인터럽트를 만들며 타임 슬라이스 종료, 더티 버퍼 기록, 네트워크 시간 제한 등에 쓰인다. 소프트웨어 가상 클록은 제한된 하드웨어 채널로 더 많은 요청을 지원한다.

출처: 책 553-554쪽 / 원문 PDF 492-493쪽

가상 타이머 관리

1. 요청을 마감시간순으로 정렬한다.
2. 하드웨어 타이머를 가장 이른 마감시간으로 설정한다.
3. 인터럽트가 발생하면 요청자에게 알리고 다음 마감시간으로 다시 설정한다.

출처: 책 553-554쪽 / 원문 PDF 492-493쪽

타이머 해상도에는 하드웨어뿐 아니라 가상 클록 유지 비용도 작용한다. 시스템 시각 오차는 NTP 같은 프로토콜로 보정할 수 있다는 것이 원문의 설명이다. 읽을 수 있는 고정밀 카운터는 정확한 간격 측정에 사용되지만 반드시 인터럽트 생성에도 쓰이는 것은 아니다.

12.3.4 봉쇄형과 비봉쇄형 입출력

출처: 책 554-556쪽 / 원문 PDF 493-495쪽

출처: 책 554-556쪽 / 원문 PDF 493-495쪽

방식	호출과 데이터·완료 관계
봉쇄형(blocking)	호출 스레드를 대기 큐로 옮기고 I/O가 끝나면 실행 가능한 상태로 돌린다. 코드 작성이 간단하다.
비봉쇄형(nonblocking)	오래 멈추지 않고 그 시점에 가능한 데이터 전송량을 반환한다. 전체 또는 일부 데이터일 수 있다.
비동기식(asynchronous)	요청을 제출한 뒤 즉시 돌아와 다른 일을 한다. 나중에 전체 작업 완료를 변수·시그널·소프트웨어 인터럽트·콜백 등으로 알린다.

출처: 책 554-556쪽 / 원문 PDF 493-495쪽

원문은 비봉쇄형 읽기가 지금 얻을 수 있는 데이터를 반환하는 반면 비동기식 읽기는 나중에 완전한 데이터를 제공하도록 요청한다는 차이를 강조한다. 여러 스레드를 사용하면 일부는 봉쇄형 호출로 기다리고 나머지는 연산할 수도 있다. select()에 대기시간 0을 주면 봉쇄 없이 상태만 검사하지만 실제 전송에는 뒤이은 읽기·쓰기가 필요하다.

출처: 책 554-556쪽 / 원문 PDF 493-495쪽

운영체제 내부에서도 쓰기나 네트워크 전송을 버퍼링한 뒤 먼저 응용에 복귀할 수 있다. 미기록 요청은 크래시 때 손실될 수 있어 기록 시간 제한과 명시적 플러시 방법을 둔다. 읽기는 아직 매체에 쓰지 않은 버퍼 내용과도 일관되어야 하며, 같은 파일을 여러 스레드가 접근하면 구현에 따라 잠금 프로토콜이 필요하다.

12.3.5 벡터형 입출력

출처: 책 556쪽 / 원문 PDF 495쪽

출처: 책 556쪽 / 원문 PDF 495쪽

벡터형 I/O는 한 시스템 콜로 여러 버퍼에 대한 작업을 수행하는 분산-수집 인터페이스이다. 원문의 UNIX readv 예처럼 여러 개별 호출과 중간 대형 버퍼 복사를 줄여 문맥 교환·시스템 콜 비용을 낮춘다. 일부 구현은 전체 작업의 원자성을 보장하여 같은 버퍼의 동시 I/O에 의한 오염을 막는다. 원자성은 모든 버전에 무조건 있다고 확대하지 않는다.

12.4 커널 입출력 서브시스템

출처: 책 556-565쪽 / 원문 PDF 495-504쪽

12.4.1 I/O 스케줄링

출처: 책 556-557쪽 / 원문 PDF 495-496쪽

출처: 책 556-557쪽 / 원문 PDF 495-496쪽

I/O 스케줄링은 요청 실행 순서를 정한다. 장치별 대기 큐를 재정렬하여 성능·공정성·평균 응답 시간을 개선한다. 원문 예는 헤드가 시작 부분에 있고 요청들이 끝·시작·중간 순서로 들어오면 2→3→1로 바꾸는 것이다. 긴급한 페이징 요청이 응용 I/O보다 우선할 수도 있다. 비동기 요청은 동시에 여러 개가 진행되므로 장치 상태 테이블의 종류·주소·상태와 해당 큐를 함께 추적한다.

12.4.2 버퍼링

출처: 책 557-559쪽 / 원문 PDF 496-498쪽

출처: 책 557-559쪽 / 원문 PDF 496-498쪽

버퍼는 두 장치 또는 장치와 프로그램 사이에서 전송 중인 데이터를 임시 저장하는 메모리이다. 첫째 목적은 생산자·소비자의 속도 차이를 완충하는 것이다. 이중 버퍼링에서는 한 버퍼를 장치에 기록하는 동안 다른 버퍼를 수신 데이터로 채워 두 동작의 타이밍을 분리한다. 둘째는 전송 크기 차이이다. 메시지를 작은 패킷으로 나누거나 수신 패킷을 조합할 때 버퍼를 쓴다.

출처: 책 557-559쪽 / 원문 PDF 496-498쪽

셋째는 **복제 시맨틱**이다. write() 이후 응용이 자기 버퍼를 수정해도 매체에는 호출 시점의 버전이 기록되어야 한다. 간단한 구현은 응용에 복귀하기 전에 커널 버퍼로 복사하고 그 사본을 기록하는 것이다. 복사 비용이 있지만 의미가 간단하다. 원문은 가상 메모리 매핑과 쓰기 시 복사 보호로 비슷한 효과를 더 효율적으로 얻을 수도 있다고 설명한다.

12.4.3 캐싱

출처: 책 559쪽 / 원문 PDF 498쪽

출처: 책 559쪽 / 원문 PDF 498쪽

캐시는 자주 사용할 자료의 사본을 빠른 저장 공간에 두는 것이다. 원문의 구분에서 버퍼는 전송 데이터의 유일한 보관 장소일 수 있고 캐시는 다른 곳의 원본에 대한 사본이다. 기능은 다르지만 한 메모리 영역이 두 역할을 겸할 수 있다. 파일 I/O는 먼저 버퍼 캐시를 검사하여 실제 I/O를 피하거나 미룰 수 있고, 지연된 쓰기를 모아 큰 단위로 스케줄할 수 있다.

12.4.4 스풀링 및 장치 예약

출처: 책 559-560쪽 / 원문 PDF 498-499쪽

출처: 책 559-560쪽 / 원문 PDF 498-499쪽

스풀은 프린터처럼 여러 작업을 뒤섞어 출력할 수 없는 장치의 출력 버퍼이다. 프로그램별 출력을 별도 보조저장장치 파일에 모으고 완성된 작업을 큐에 넣어 하나씩 출력한다. 디먼이나 커널 스레드가 관리하며 작업 열거·취소·중지 기능을 제공한다.

출처: 책 559-560쪽 / 원문 PDF 498-499쪽

전용 장치는 한 프로그램에 할당하고 명시적으로 반납하게 하거나 열린 핸들을 하나로 제한할 수도 있다. 프로세스 간 협조적 배타 접근을 지원하는 방식도 있다. 이러한 장치 예약에서 생길 수 있는 교착 상태는 원문의 해당 시스템 예에서 응용의 책임이다.

12.4.5 오류 처리

출처: 책 560-561쪽 / 원문 PDF 499-500쪽

출처: 책 560-561쪽 / 원문 PDF 499-500쪽

일시적 오류는 읽기·전송 재시도 등으로 복구할 수 있지만 핵심 장치의 영구 고장은 완전히 극복하기 어렵다. 시스템 콜은 성공·실패와 추가 오류 정보를 반환하며 UNIX errno가 예로 제시된다. SCSI의 sense key·sense code·qualifier·오류 로그는 더 세부적인 원인을 표현하지만 운영체제가 모든 하드웨어 정보를 응용에 전달하는 것은 아니다.

12.4.6 입출력 보호

출처: 책 561-562쪽 / 원문 PDF 500-501쪽

출처: 책 561-562쪽 / 원문 PDF 500-501쪽

입출력 명령을 특권 명령으로 두어 사용자가 직접 실행하지 못하게 하고, 시스템 콜을 받은 운영체제가 요청의 유효성을 검사한다. 메모리 맵드 I/O·포트 영역도 메모리 보호 대상이다. 다만 그래픽 성능처럼 직접 접근을 허용할 필요가 있으면 커널은 영역을 적절히 할당하고 잠금으로 중복 사용을 제어해야 한다.

12.4.7 커널 자료구조

출처: 책 562-563쪽 / 원문 PDF 501-502쪽

출처: 책 562-563쪽 / 원문 PDF 501-502쪽

커널은 열린 파일·네트워크 연결·문자 통신 등의 상태를 자료구조로 관리한다. UNIX의 파일 인터페이스는 일반 파일·raw 장치·프로세스 주소 공간을 모두 read()로 접근하게 하지만 실제 처리는 다르다. 열린 파일 레코드의 디스패치 테이블은 객체 유형에 맞는 읽기·쓰기·select·ioctl·close 함수를 가리킨다.

출처: 책 562-563쪽 / 원문 PDF 501-502쪽

원문은 독립적인 I/O 관리자와 메시지 전달 방식도 비교한다. 메시지 횟수가 늘어 비용이 생기지만 커널을 작게 하고 구조·설계의 유연성을 높일 수 있다. 이 비교는 원문이 설명하는 구현 사례의 범위에서 이해한다.

12.4.8 전원 관리

출처: 책 563-565쪽 / 원문 PDF 502-504쪽

출처: 책 563-565쪽 / 원문 PDF 502-504쪽

운영체제는 부하에 맞춰 프로세스를 이동시키거나 장치·CPU 코어를 유휴 상태로 만들어 전력·발열·냉각 비용을 줄인다. 일시 중단한 코어의 상태는 저장하고 재개 시 복원한다. 모바일 운영체제에서는 배터리 수명이 특히 중요하다.

출처: 책 563-565쪽 / 원문 PDF 502-504쪽

원문의 Android 기능	역할
전원 축소(power collapse)	개별 구성요소를 끄고 CPU를 깊은 슬립 상태로 둔다. 외부 자극·인터럽트에는 반응하여 빠르게 재개한다.
구성요소 수준 전원 관리	장치 트리의 부모·자식 관계와 드라이버의 사용 여부를 추적한다. 자식들이 모두 유휴이면 버스 등 상위 구성요소도 끌 수 있다.
wakelock	필요한 동안 응용이 획득하여 전원 축소 진입을 막고 작업이 끝나면 해제한다.

출처: 책 563-565쪽 / 원문 PDF 502-504쪽

전체 시스템은 부팅 때뿐 아니라 실행 중 장치 추가·제거와 상태·오류·전력 변화를 관리해야 한다. **ACPI**는 커널이 호출할 수 있는 하드웨어 관리 루틴을 제공한다. 원문의 정지 예에서는 커널→장치 드라이버→ACPI 루틴→장치의 순서로 지시한다.

12.4.9 커널 입출력 서브시스템 요약

출처: 책 565쪽 / 원문 PDF 504쪽

출처: 책 565쪽 / 원문 PDF 504쪽

커널 I/O 서브시스템은 이름·접근 제어·연산 허용 여부·파일 공간·장치 할당·버퍼/캐시/스풀·스케줄링·상태 감시·오류/복구·드라이버 초기화·전원 관리를 조정한다. 상위 계층은 드라이버가 제공하는 공통 인터페이스로 장치를 사용한다.

12.5 입출력 요구를 하드웨어 연산으로 변환

출처: 책 566-568쪽 / 원문 PDF 505-507쪽

출처: 책 566-568쪽 / 원문 PDF 505-507쪽

파일 이름은 파일 시스템의 디렉터리와 할당 정보로 연결된다. FAT는 이름을 할당 테이블 항목과 블록으로, UNIX는 이름을 inode와 할당 정보로 매핑한다. UNIX에서는 마운트 테이블의 가장 긴 일치 접두어가 장치를 정하고, 주 장치 번호는 드라이버를, 부 장치 번호는 장치 테이블 항목을 선택하며, 여기에서 컨트롤러 포트·메모리 주소를 얻는다. 여러 검색 테이블은 유연성과 동적 드라이버 적재를 제공하지만 잠금·오류 처리도 복잡해진다.

출처: 책 566-568쪽 / 원문 PDF 505-507쪽

원문의 봉쇄형 read 생애 주기

1. 열린 파일의 디스크립터로 read를 요청하고 커널이 인자를 검사한다.
2. 버퍼 캐시에 데이터가 있으면 바로 반환한다. 없으면 프로세스를 대기시키고 I/O를 스케줄한다.
3. 드라이버가 커널 버퍼를 확보하고 컨트롤러 레지스터에 명령을 기록한다.
4. 장치가 데이터를 전송한다. 원문의 가정에서는 DMA가 완료 인터럽트를 발생시킨다.
5. 벡터를 통해 핸들러가 드라이버에 알리고 드라이버는 완료 요청을 식별하여 I/O 서브시스템에 통지한다.
6. 커널이 사용자 버퍼로 데이터를 옮기고 프로세스를 준비 큐로 옮긴다.
7. 스케줄러가 CPU를 할당하면 프로세스가 시스템 콜에서 복귀한다.

출처: 책 566-568쪽 / 원문 PDF 505-507쪽

그림 12.14 입출력 요청의 생애 주기

그림·표 출처: 책 567쪽 / 원문 PDF 506쪽

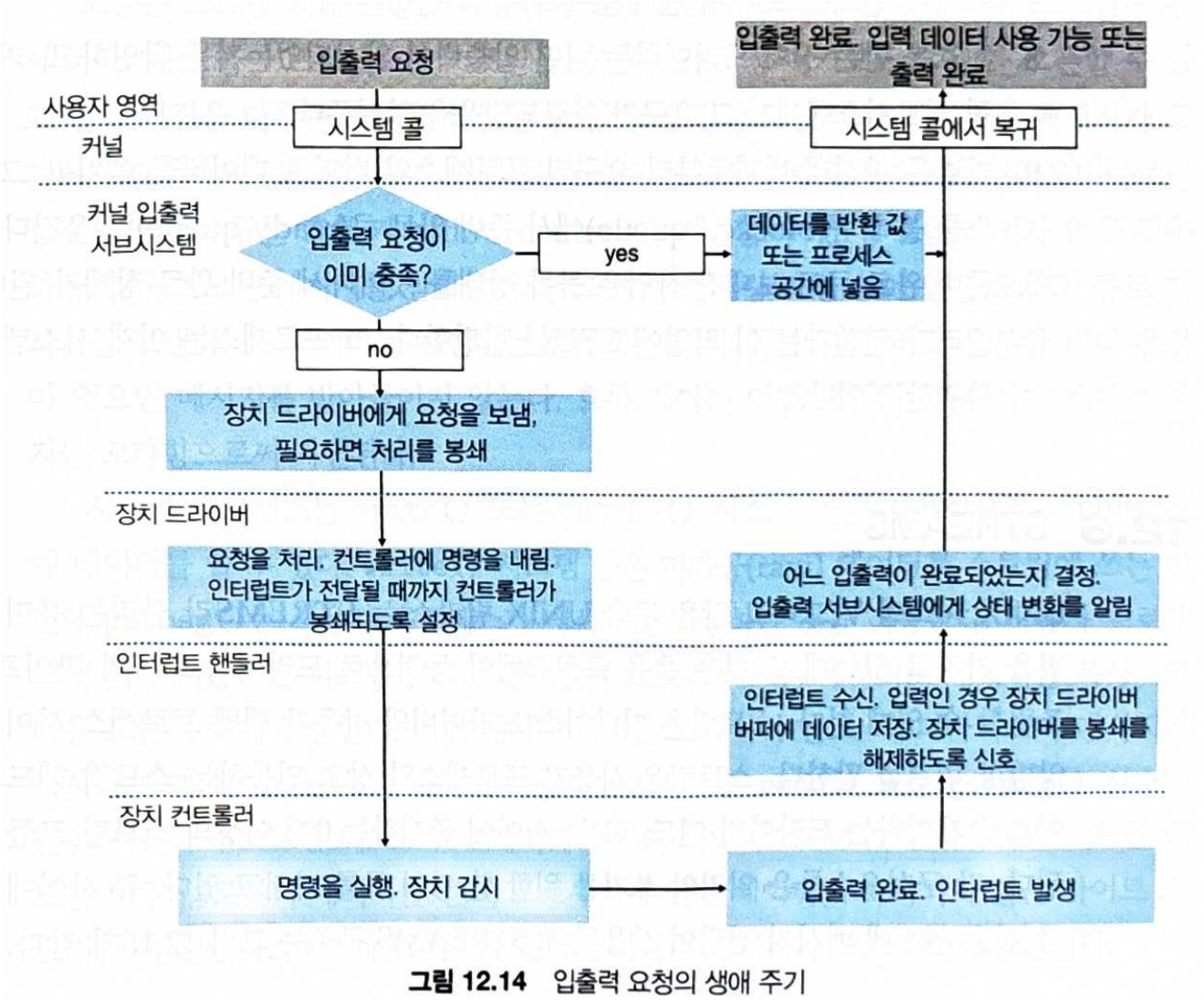


그림 12.14 입출력 요청의 생애 주기

그림 12.14 입출력 요청의 생애 주기 - 원문 그림 부분 발췌

사용자·커널·드라이버·인터럽트 핸들러·컨트롤러 사이의 요청과 완료 경로를 원문 그대로 보존한다.

12.6 STREAMS

출처: 책 568-570쪽 / 원문 PDF 507-509쪽

출처: 책 568-570쪽 / 원문 PDF 507-509쪽

STREAMS는 응용이 드라이버 코드의 파이프라인을 동적으로 조립하는 UNIX의 기법이다. 스트림은 사용자 프로세스와 장치 드라이버 사이의 완전 양방향 연결이며, 스트림 헤드·0개 이상의 모듈·드라이버 엔드로 구성된다. 각 요소에는 읽기·쓰기 큐가 있고 메시지 전달로 데이터를 교환한다. ioctl()로 모듈을 삽입할 수 있어 편집·네트워크 처리 등을 재사용한다.

출처: 책 568-570쪽 / 원문 PDF 507-509쪽

그림 12.15 STREAMS 구조

그림·표 출처: 책 569쪽 / 원문 PDF 508쪽

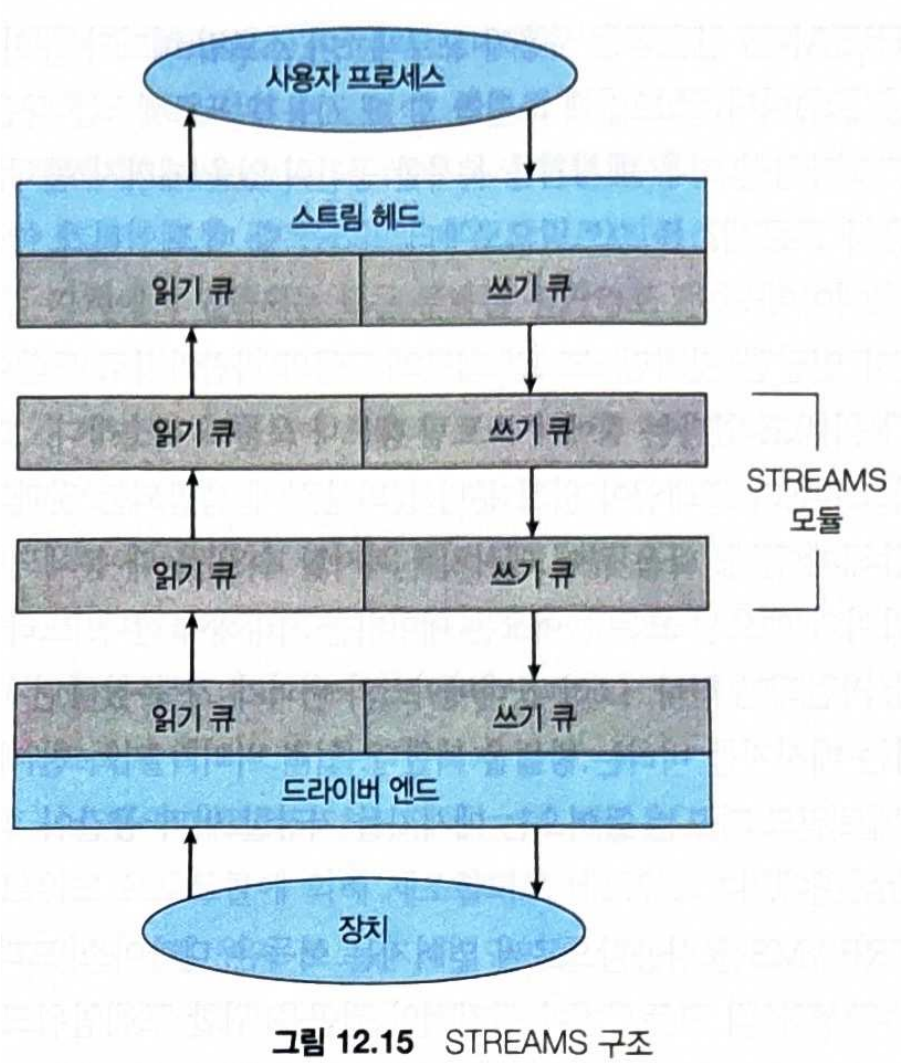


그림 12.15 STREAMS 구조

그림 12.15 STREAMS 구조 - 원문 그림 부분 발췌

헤드·모듈·드라이버 엔드의 읽기 큐와 쓰기 큐가 반대 방향으로 메시지를 전달하는 구조이다.

출처: 책 568-570쪽 / 원문 PDF 507-509쪽

흐름 제어를 사용하는 큐는 메시지를 버퍼링하고 여유가 없으면 새 메시지를 거절하여 인접 큐의 오버플로를 막는다. write/read는 바이트 스트림을, putmsg/getmsg는 메시지를 응용에 제공한다. 내부는 비동기적으로 동작하지만 프로세스가 헤드에 쓸 공간이나 읽을 데이터가 없으면 봉쇄된다. 드라이버 엔드는 외부 인터럽트에 응답해야 하므로 기다릴 수 없고 장치 버퍼가 차면 들어오는 메시지를 버릴 수 있다. 메시지 경계·제어 정보와 모듈 재사용이 원문의 장점이다.

12.7 성능

출처: 책 570-574쪽 / 원문 PDF 509-513쪽

출처: 책 570-574쪽 / 원문 PDF 509-513쪽

I/O 비용은 장치 시간에만 있지 않다. 드라이버 실행·봉쇄/해제·문맥 교환·인터럽트·캐시 영향·장치/메모리/커널/사용자 사이의 복사가 CPU와 메모리 버스에 부하를 준다. 원격 로그인에서 문자 하나를 전달하고 다시 반향하는 예는 작은 데이터도 여러 계층과 문맥 교환을 거칠 수 있음을 보인다. 프런트 엔드 처리기와 I/O 채널은 이 부담을 주 CPU에서 덜어 준다.

출처: 책 570-574쪽 / 원문 PDF 509-513쪽

- 문맥 교환·복사·인터럽트 빈도를 줄인다. 큰 전송, 지능형 컨트롤러, 바쁜 대기가 작은 폴링을 적절히 이용한다.
- DMA·채널로 CPU 부담을 낮추고 I/O와 연산을 중첩한다.
- 원시 연산을 장치 하드웨어에서 병렬 수행하고 CPU·메모리·버스·I/O 부하가 균형을 이루게 한다.

출처: 책 570-574쪽 / 원문 PDF 509-513쪽

구현 위치	원문의 장점	비용·제약
응용 코드	신축적이며 실험·디버깅이 쉽고 버그가 시스템 전체를 중단시킬 가능성이 작다.	문맥 교환 비용, 커널 내부 서비스 이용의 제약
커널	성능을 높이고 커널 서비스와 결합할 수 있다.	커널이 커지고 복잡해지며 오류가 크래시·데이터 손상으로 이어질 수 있다.
장치·컨트롤러 하드웨어	가장 높은 성능을 지향한다.	개발이 오래 걸리고 수정·개선·부하에 따른 정책 변경이 어렵다.

출처: 책 570-574쪽 / 원문 PDF 509-513쪽

원문의 저장장치 성능 그림은 용량과 지연시간이라는 서로 다른 축에서 캐시·DRAM·NVM·SSD·HDD와 네트워크 지연을 비교한다. 큰 용량이 낮은 지연을 뜻하지 않으며 네트워크를 통한 접근은 추가 지연을 가진다는 관계를 읽는다. 수치가 아닌 도표상의 관계 범위에서 정리한다.

12.8 요약과 원문 연습문제의 점검 범위

출처: 책 574-576쪽 / 원문 PDF 513-515쪽

출처: 책 574-576쪽 / 원문 PDF 513-515쪽

장말 문제는 기능을 컨트롤러에 둘 때의 장단점, 핸드셰이킹 비트, 폴링·인터럽트·혼합 전략, DMA의 병행성, CPU/버스/장치 속도의 균형, STREAMS의 엔드와 모듈을 점검한다. 새 프로토콜·풀이를 만들지 않는다. 추가 자료·참고문헌의 범위는 확인했으나 외부 설명을 가져오지 않았다.

핵심 정리

- 폴링은 상태를 반복 검사하고 인터럽트는 상태 변화를 통지한다. 발생률·대기 길이와 처리 비용에 따라 선택한다.
- DMA는 CPU의 전송 부담을 줄이지만 메모리 버스 사용·버퍼 보호·추가 복사 문제를 함께 고려한다.
- 비봉쇄형의 현재 가능한 결과와 비동기식의 나중 완료 통지를 구별한다.
- 버퍼링·캐싱·스풀링은 목적이 다르며 같은 영역이 버퍼와 캐시 역할을 겸할 수 있다.
- 이름→드라이버→컨트롤러와 요청→완료 인터럽트→프로세스 재개를 연결한다.

혼동하기 쉬운 내용

봉쇄형·비봉쇄형·비동기식 I/O

출처: 책 554-556쪽 / 원문 PDF 493-495쪽

방식	반환과 완료
봉쇄형	완료를 기다려 결과를 받는다.
비봉쇄형	지금 가능한 전송량을 반환한다.
비동기식	먼저 돌아온 후 전체 요청 완료를 별도로 알린다.

버퍼·캐시·스풀

출처: 책 557-560쪽 / 원문 PDF 496-499쪽

개념	목적
버퍼	전송 중 임시 보관, 속도·크기 차이와 복제 시맨틱 지원
캐시	다른 곳에 있는 데이터의 빠른 사본으로 재사용
스풀	섞어 처리할 수 없는 장치에 작업별 출력을 모아 순차 전달

장 마무리 복습

- 응용은 장치를 직접 제어하기보다 드라이버와 커널 서비스의 표준 인터페이스를 사용한다.
- 일관성·보호·전력 관리 역시 I/O 서브시스템의 역할이다.
- STREAMS는 양방향 큐와 메시지·흐름 제어로 모듈을 결합한다.
- 성능은 장치 속도뿐 아니라 문맥 교환·인터럽트·복사·계층 간 이동 비용에 좌우된다.

제13장. 파일 시스템 인터페이스

출처: 책 579-615쪽 / 원문 PDF 517-553쪽

학습 주제

- 파일이라는 논리적 저장 단위와 속성·연산·접근 방법을 정리한다. 디렉터리의 이름 구조가 공유·탐색·삭제에 미치는 영향을 비교하고 파일 보호와 메모리 사상 파일의 동작을 살펴본다.

설명의 흐름

1. 파일·속성·연산 → 유형·구조와 내부 단편화 → 순차/직접/색인 접근 → 디렉터리 구조 → 접근 보호 → 메모리 사상과 공유

13.1 파일 개념

출처: 책 579-580쪽 / 원문 PDF 517-518쪽

출처: 책 579-580쪽 / 원문 PDF 517-518쪽

파일은 보조저장장치에 기록된 관련 정보의 집합이며, 운영체제는 저장장치의 물리 특성을 추상화하여 일관된 논리적 저장 단위로 제공한다. 일반적으로 비휘발성 장치에 사상되므로 재부팅 뒤에도 내용이 존속한다. 사용자에게는 논리적 보조저장장치의 최소 할당 단위이다. 내용의 의미는 작성자·사용자가 정하며 비트·바이트·행·레코드의 연속이나 프로그램·데이터 등으로 해석한다.

출처: 책 579-580쪽 / 원문 PDF 517-518쪽

파일 시스템은 실제 파일의 집합과 모든 파일의 정보를 제공하는 디렉터리 구조로 이루어진다. 파일 인터페이스는 원래의 저장 용도를 넘어 proc 파일 시스템의 시스템 정보 접근에도 이용된다.

13.1.1 파일 속성

출처: 책 580-581쪽 / 원문 PDF 518-519쪽

출처: 책 580-581쪽 / 원문 PDF 518-519쪽

속성	원문의 역할
이름	사람이 읽는 기호형 이름
식별자	파일 시스템 안에서 파일을 고유하게 구별하는 숫자 등의 꼬리표
유형	여러 파일 유형을 지원할 때 필요한 정보
위치	장치와 그 장치 안의 위치를 나타내는 포인터
크기	현재 크기와 최대 허용 크기
보호	읽기·쓰기·실행 등의 접근 통제
타임스탬프·사용자 식별	생성·변경·사용 시각과 보호·보안·감시 정보

출처: 책 580-581쪽 / 원문 PDF 518-519쪽

출처: 책 580-581쪽 / 원문 PDF 518-519쪽

이름의 대소문자 구별은 시스템마다 다르다. 파일은 만든 프로세스와 독립적으로 존속하고 복사·전송될 수 있지만 공유·동기화 수단이 없다면 사본은 서로 독립적으로 바뀐다. 확장 속성에는 문자 인코딩·체크섬 같은 정보가 포함될 수 있다. 디렉터리 항목은 보통 이름과 식별자를 두고 식별자로 다른 속성을 찾는다. 디렉터리도 비휘발성으로 유지해야 하므로 저장장치에 보관하고 필요한 부분만 메모리에 반입한다.

13.1.2 파일 연산과 열린 파일 관리

출처: 책 582-586쪽 / 원문 PDF 520-524쪽

출처: 책 582-586쪽 / 원문 PDF 520-524쪽

연산	핵심 작업
생성	공간을 확보하고 디렉터리 항목을 만든다.
열기	이름·요청 모드·권한을 확인하고 후속 호출에 쓸 핸들을 반환한다.
쓰기·읽기	핸들로 데이터를 전달하며 현재 파일 위치를 갱신한다.
위치 재설정(seek)	현재 위치를 설정한다. 실제 I/O가 반드시 필요한 것은 아니다.
삭제	항목과 공간을 해제한다. 하드 링크가 있으면 마지막 링크가 없어질 때까지 내용은 남는다.
절단	다른 속성을 유지하며 길이를 0으로 하고 공간을 해제할 수 있다.

출처: 책 582-586쪽 / 원문 PDF 520-524쪽

매번 이름을 검색하지 않도록 **열린 파일 테이블**을 사용한다. 대부분의 시스템은 명시적 open을 요구하며 접근 모드를 권한과 비교한다. 닫으면 관련 테이블 항목과 락을 해제할 수 있다. 여러 프로세스가 동시에 여는 시스템은 **프로세스별 테이블**에 현재 위치·접근 권한 등을, **범 시스템 테이블**에 장치 위치·크기·접근 날짜 등 프로세스 독립 정보를 둔다. 프로세스별 항목이 범 시스템 항목을 가리킨다. 열린 계수는 마지막 사용자가 닫아 0이 될 때 항목을 회수하도록 한다.

출처: 책 582-586쪽 / 원문 PDF 520-524쪽

현재 위치 포인터는 각 프로세스의 사용 상태이므로 디스크상의 고유 속성과 분리한다. 열린 계수는 하드 링크 참조 계수와 구별하여 문맥에 맞게 읽는다. 전자는 열린 테이블의 수명에, 후자는 같은 파일에 대한 이름 참조와 파일 내용의 삭제에 관련된다.

출처: 책 582-586쪽 / 원문 PDF 520-524쪽

락의 구분	동작
공유 / 배타	공유 락은 여러 프로세스가 함께 획득할 수 있고, 배타 락은 한 프로세스만 가진다. 모든 운영체제가 둘 다 지원하지는 않는다.
강제적(mandatory)	운영체제가 잠긴 파일에 다른 프로세스가 접근하지 못하게 한다.
권고적(advisory)	응용이 협조하여 락을 획득·해제해야 하며 운영체제가 모든 접근을 차단하지는 않는다.

출처: 책 582-586쪽 / 원문 PDF 520-524쪽

락은 접근할 때만 유지하도록 하며 여러 락의 획득으로 교착 상태가 생기지 않게 해야 한다. 원문의 Java 예는 FileChannel을 얻고 처음 부분의 배타 락과 나머지 부분의 공유 락을 획득한 뒤 release로 해제한다. 아래 코드는 원문을 이미지로 보존한다. 원문에 실행 결과 미제시.

출처: 책 582-586쪽 / 원문 PDF 520-524쪽

그림 13.2 Java의 파일 락킹 예제

그림·표 출처: 책 586쪽 / 원문 PDF 524쪽

```
import java.io.*;
import java.nio.channels.*;

public class LockingExample {
  public static final boolean EXCLUSIVE = false;
  public static final boolean SHARED = true;

  public static void main(String args[]) throws IOException {
    FileLock sharedLock = null;
    FileLock exclusiveLock = null;

    try {
      RandomAccessFile raf = new RandomAccessFile("file.txt","rw");

      // get the channel for the file
      FileChannel ch = raf.getChannel();

      // this locks the first half of the file - exclusive
      exclusiveLock = ch.lock(0, raf.length()/2, EXCLUSIVE);

      /** Now modify the data . . . */

      // release the lock
      exclusiveLock.release();

      // this locks the second half of the file - shared
      sharedLock = ch.lock(raf.length()/2+1,raf.length(),SHARED);

      /** Now read the data . . . */

      // release the lock
      sharedLock.release();
    } catch (java.io.IOException ioe) {
      System.err.println(ioe);
    }
    finally {
      if (exclusiveLock != null)
            exclusiveLock.release();
      if (sharedLock != null)
            sharedLock.release();
    }
  }
}
```

그림 13.2 Java의 파일 락킹 예제

그림 13.2 Java의 파일 락킹 예제 - 원문 그림 부분 발췌

원문의 EXCLUSIVE=false, SHARED=true, lock 인자와 finally의 해제 처리를 보존한다. 데이터 수정·읽기의 본문은 원래부터 주석으로만 제시된다.

13.1.3 파일 유형

출처: 책 587-588쪽 / 원문 PDF 525-526쪽

출처: 책 587-588쪽 / 원문 PDF 525-526쪽

운영체제가 파일 유형을 인식하면 유형에 맞는 연산을 선택하거나 부적절한 연산을 막을 수 있다. 이름의 확장자, 작성 프로그램 속성, 시작 부분의 매직 넘버가 원문의 식별 방법이다. 확장자는 실행 가능·소스·목적·배치·문서·라이브러리·보관·멀티미디어 등의 용도를 암시한다. 그러나 모든 시스템에서 확장자가 강제되는 것은 아니며 응용의 힌트일 수도 있다.

출처: 책 587-588쪽 / 원문 PDF 525-526쪽

원문의 macOS 예는 작성 프로그램을 파일 속성으로 기록하여 데이터 파일을 열 때 해당 프로그램을 실행한다. UNIX는 일부 파일의 시작 부분 매직 넘버를 쓰지만 모든 파일이 이를 가지지는 않는다. 확장자와 작성 프로그램의 취급도 시스템마다 다르므로 특정 식별법을 보편 규칙으로 확대하지 않는다.

13.1.4 파일 구조

출처: 책 588-589쪽 / 원문 PDF 526-527쪽

출처: 책 588-589쪽 / 원문 PDF 526-527쪽

파일 유형은 내부 구조를 암시한다. 운영체제가 여러 구조를 직접 지원하면 편리한 연산을 제공할 수 있지만 코드가 커지고 복잡해지며 새로운 구조의 응용을 제약할 수 있다. 원문의 텍스트·실행 파일만 지원하는 가정에서는 암호화 데이터가 어느 유형에도 맞지 않는 문제가 생긴다. 반대로 UNIX처럼 바이트열로 다루면 유연하지만 해석과 운용은 응용의 책임이 된다. 다만 프로그램 적재·실행을 위해 실행 파일의 구조는 운영체제가 이해해야 한다.

13.1.5 파일의 내부 구조

출처: 책 589쪽 / 원문 PDF 527쪽

출처: 책 589쪽 / 원문 PDF 527쪽

논리 레코드 크기와 물리 블록 크기는 다를 수 있어 여러 레코드를 한 블록에 **패킹**하거나 다시 풀어야 한다. UNIX의 바이트 스트림은 파일 시작부터의 오프셋으로 바이트를 지정하고 파일 시스템이 물리 블록으로 묶는다. 이 변환은 응용이나 운영체제가 담당할 수 있다.

출처: 책 589쪽 / 원문 PDF 527쪽

공간은 블록 단위로 할당하므로 마지막 블록 일부가 남는 **내부 단편화**가 발생한다. 원문 예는 512바이트 블록에서 1,949바이트 파일에 네 블록(2,048바이트)을 할당하여 99바이트가 남는 경우이다. 블록을 크게 하면 이 낭비가 커질 수 있다.

13.2 접근 방법

출처: 책 590-592쪽 / 원문 PDF 528-530쪽

13.2.1 순차 접근

출처: 책 590쪽 / 원문 PDF 528쪽

출처: 책 590쪽 / 원문 PDF 528쪽

순차 접근은 저장된 레코드 순서로 파일을 처리한다. read-next는 다음 부분을 읽고 현재 위치를 증가시키며, 쓰기는 끝에 추가하고 새 끝으로 포인터를 옮긴다. 되감기나 정해진 레코드 수만큼의 이동을 제공할 수도 있다. 테이프 모델에 기반하지만 순차 장치와 임의 접근 장치 모두에 사용할 수 있고 편집기·컴파일러가 예로 제시된다.

13.2.2 직접 접근

출처: 책 590-591쪽 / 원문 PDF 528-529쪽

출처: 책 590-591쪽 / 원문 PDF 528-529쪽

직접(상대) 접근은 고정 길이 레코드·블록을 번호로 지정하여 읽기·쓰기를 임의 순서로 수행한다. read(n)·write(n) 또는 position-file(n) 후 순차 연산으로 표현할 수 있다. 사용자가 보는 상대 블록 번호는 실제 디스크 주소와 다르며 운영체제가 위치를 할당하고 다른 파일 영역의 접근을 막는다. 번호의 시작이 0인지 1인지는 시스템에 따라 다르다.

출처: 책 590-591쪽 / 원문 PDF 528-529쪽

원문 코드·기호

`L × N` 위치에서 `L`바이트

- L: 논리 레코드 길이

- N: 0부터 세는 레코드 번호

0번을 첫 레코드로 세고 논리 레코드의 길이가 L일 때 N번째 레코드의 요청 위치이다. 고정 길이라는 전제를 유지한다.

출처: 책 590-591쪽 / 원문 PDF 528-529쪽

원문의 순차 연산	직접 접근으로의 구현
reset	cp = 0;
read next	read cp; 이후 cp = cp+1
write next	write cp; 이후 cp = cp+1

출처: 책 590-591쪽 / 원문 PDF 528-529쪽

그림 13.5의 cp는 현재 위치이다. 직접 접근에 현재 위치 추적을 더하면 다음 레코드 순서로 처리할 수 있음을 보이는 원문의 표를 정리했다. 이는 별도의 새 알고리즘이 아니다.

출처: 책 590-591쪽 / 원문 PDF 528-529쪽

직접 접근으로 순차 접근을 구현하는 것은 간단하지만 순차 접근만으로 직접 접근을 흉내 내는 것은 비효율적이다. 원문의 항공편 번호로 예약 블록을 찾는 사례는 원하는 블록을 먼저 결정한 뒤 직접 읽는 의미를 보여준다.

13.2.3 기타 접근 방법

출처: 책 591-593쪽 / 원문 PDF 529-531쪽

출처: 책 591-593쪽 / 원문 PDF 529-531쪽

색인 접근은 먼저 레코드의 색인에서 포인터를 찾은 뒤 해당 블록을 직접 읽는다. 파일이 커 색인도 메모리에 못 들어가면 색인 파일에 대한 상위 색인을 둘 수 있다. 원문의 ISAM은 작은 마스터 색인→2차 색인 블록→실제 파일 블록의 순서로 찾고 블록 안에서 검색한다.

출처: 책 591-593쪽 / 원문 PDF 529-531쪽

원문의 UPC 가격 예는 10자리 코드와 6자리 가격의 16바이트 레코드, 1,024바이트 블록당 64레코드, 블록의 첫 UPC로 구성한 메모리 색인을 설명한다. 다만 120,000레코드를 2,000블록·색인 2,000항목으로 설명하는 수치 연결은 **원문 확인 필요**이다. 원문 수치를 다른 값으로 고쳐 계산하지 않고, 색인에서 블록을 정하여 적은 I/O로 탐색한다는 구조를 보존한다.

13.3 디렉터리 구조

출처: 책 593쪽 / 원문 PDF 531쪽

출처: 책 593쪽 / 원문 PDF 531쪽

디렉터리는 파일 이름을 파일 제어 블록에 대응시키는 심볼 테이블로 볼 수 있다. 필요한 연산은 이름·패턴 검색, 생성·삭제, 항목 나열, 이름 변경, 전체 순회이다. 이름이 커져 항목에 맞지 않거나 삭제로 빈 공간이 생기면 공간 관리도 필요하다. 순회는 검색뿐 아니라 백업·보관과 공간 회수에도 이용된다.

13.3.1 1단계 디렉터리

출처: 책 594쪽 / 원문 PDF 532쪽

출처: 책 594쪽 / 원문 PDF 532쪽

모든 파일을 한 디렉터리에 두므로 개념이 단순하지만 모든 이름이 고유해야 한다. 사용자나 파일 수가 늘면 이름 충돌과 기억·관리 문제가 커진다. 긴 이름을 허용해도 파일을 조직하는 문제 자체가 없어지지는 않는다.

13.3.2 2단계 디렉터리

출처: 책 594-596쪽 / 원문 PDF 532-534쪽

출처: 책 594-596쪽 / 원문 PDF 532-534쪽

마스터 파일 디렉터리(**MFD**)가 사용자별 파일 디렉터리(**UFD**)를 가리킨다. 각 UFD 안에서만 이름이 고유하면 되므로 다른 사용자는 같은 파일명을 쓸 수 있다. 생성·삭제는 자신의 UFD만 검색하고 UFD의 생성·삭제는 적절한 관리 권한이 필요하다. 사용자 간 격리는 되지만 다른 사용자의 파일을 공유하려면 접근 허용과 사용자명+파일명의 경로가 필요하다.

출처: 책 594-596쪽 / 원문 PDF 532-534쪽

시스템 프로그램을 모든 UFD에 복사하면 공간을 낭비한다. 대신 현재 UFD에서 찾지 못하면 시스템 파일 디렉터리 등 정해진 순서를 찾는 **탐색 경로**를 둔다. 사용자가 자신의 경로를 가질 수도 있다. 볼륨 문자·디렉터리·버전의 경로 구문은 시스템마다 다르다.

13.3.3 트리 구조 디렉터리

출처: 책 596-598쪽 / 원문 PDF 534-536쪽

출처: 책 596-598쪽 / 원문 PDF 534-536쪽

2단계를 임의 높이로 확장하면 사용자가 하위 디렉터리를 만드는 트리가 된다. 최상위 루트와 파일별 고유 경로를 갖는다. 디렉터리도 특별 취급되는 파일로 구현할 수 있으며 일반 파일과 구별하고 별도 생성·삭제 연산을 사용한다. 프로세스의 **현재 디렉터리**를 기준으로 파일을 찾고, 자식 프로세스는 보통 부모의 현재 디렉터리를 따른다.

출처: 책 596-598쪽 / 원문 PDF 534-536쪽

경로	기준
절대 경로	루트부터 목적 파일까지 명시
상대 경로	현재 디렉터리부터 목적 파일까지 명시

출처: 책 596-598쪽 / 원문 PDF 534-536쪽

원문 예에서 현재 디렉터리가 /spell/mail이면 prt/first와 /spell/mail/prt/first는 같은 파일을 가리킨다. 디렉터리 삭제는 비어 있을 때만 허용하는 정책과 하위 내용을 함께 재귀 삭제하는 정책이 있다. 후자는 편하지만 한 명령의 실수로 많은 내용을 잃을 수 있다. 트리에서는 경로와 접근 허용을 통해 다른 사용자 파일에도 접근할 수 있다.

13.3.4 비순환 그래프 디렉터리

출처: 책 599-601쪽 / 원문 PDF 537-539쪽

출처: 책 599-601쪽 / 원문 PDF 537-539쪽

비순환 그래프는 파일·하위 디렉터리를 여러 곳에서 공유하도록 트리를 일반화한다. 물리적으로 같은 파일을 공유하면 수정이 다른 참조에도 반영되지만 독립 사본 둘은 그렇지 않다. 링크는 다른 파일의 경로를 가진 간접 포인터로 구현할 수 있다. 동일 항목 내용을 여러 디렉터리에 복사하는 방식은 속성 변경을 일치시켜야 하는 문제가 있다.

출처: 책 599-601쪽 / 원문 PDF 537-539쪽

그림 13.10 비순환 그래프 디렉터리 구조

그림·표 출처: 책 599쪽 / 원문 PDF 537쪽

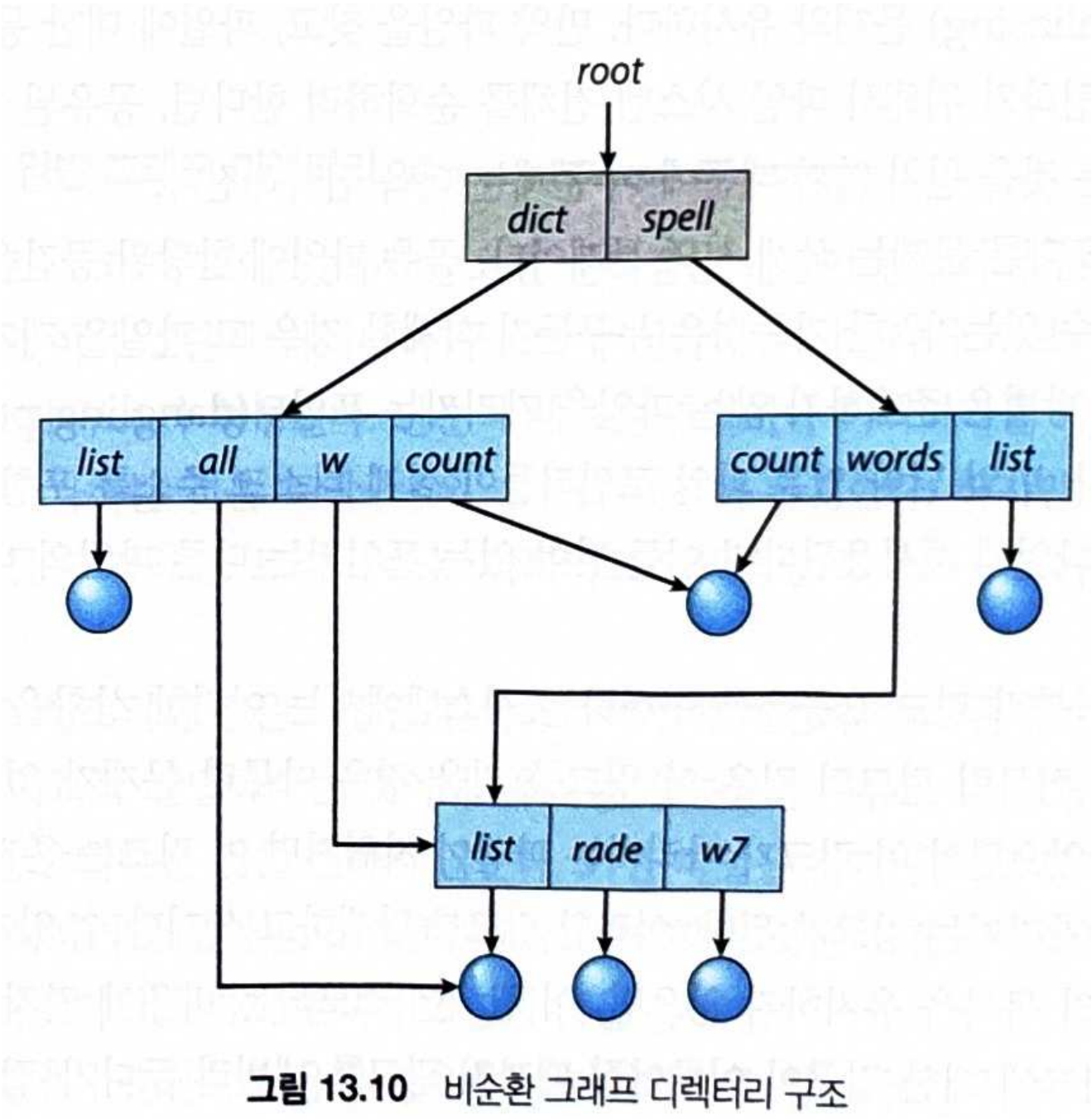


그림 13.10 비순환 그래프 디렉터리 구조

그림 13.10 비순환 그래프 디렉터리 구조 - 원문 그림 부분 발췌

여러 경로가 동일한 파일·서브디렉터리로 이어지는 공유 관계를 보여준다.

출처: 책 599-601쪽 / 원문 PDF 537-539쪽

여러 절대 경로가 한 파일을 가리키므로 탐색·통계·백업에서 같은 대상을 중복 처리하지 않아야 한다. 공유 파일을 한 참조만 지웠다고 제거하면 **dangling pointer**가 생긴다. 심볼릭 링크는 원래 파일이 삭제되면 없는 이름을 가리킬 수 있으며, 접근 시 해석 실패로 처리하거나 따로 관리할 수 있다. 같은 이름의 새 파일이 생기는 문제도 고려해야 한다.

출처: 책 599-601쪽 / 원문 PDF 537-539쪽

다른 방식은 모든 참조가 없어질 때까지 파일을 유지하는 것이다. 가변 길이 참조 목록 대신 **참조 계수**를 두어 링크 추가 때 증가, 삭제 때 감소시키고 0일 때 파일을 지울 수 있다. 원문 UNIX의 하드 링크는 inode에 이 계수를 유지한다. 디렉터리에 대한 다중 참조를 제한하는 것은 비순환성을 지키는 데 도움이 된다.

13.3.5 일반 그래프 디렉터리

출처: 책 601-603쪽 / 원문 PDF 539-541쪽

출처: 책 601-603쪽 / 원문 PDF 539-541쪽

기존 구조에 링크를 추가하면 순환이 생길 수 있다. **일반 그래프**는 공유의 유연성이 크지만 잘못된 탐색은 무한 반복할 수 있고 순환 내부의 참조 계수가 0이 되지 않아 도달 불가능한 공간을 회수하지 못할 수 있다.

출처: 책 601-603쪽 / 원문 PDF 539-541쪽

원문의 가비지 수집

1. 파일 시스템을 순회하면서 접근 가능한 모든 대상을 표시한다.

2. 두 번째 탐색에서 표시되지 않은 대상을 수집하여 가용 공간 목록에 넣는다.

출처: 책 601-603쪽 / 원문 PDF 539-541쪽

디스크 기반 전체 탐색은 비용이 커 드물게 쓰인다. 링크 추가 시 순환 검사도 비싸다. 디렉터리 순회에서 링크를 따라가지 않거나 탐색 수를 제한하는 등의 방안을 고려한다. 비순환 구조는 검색과 마지막 참조 판단이 상대적으로 간단하다는 점과 대조된다.

13.4 보호

출처: 책 603-607쪽 / 원문 PDF 541-545쪽

13.4.1 보호와 접근의 유형

출처: 책 603-604쪽 / 원문 PDF 541-542쪽

출처: 책 603-604쪽 / 원문 PDF 541-542쪽

신뢰성은 물리 손상·사고·오류에서 내용을 보존하는 문제이고 **보호**는 부적절한 접근을 통제하는 문제이다. 복사본·백업은 신뢰성에, 인증·암호화·방화벽과 파일별 접근 권한은 보호에 관계된다. 모든 접근 금지나 완전 개방 대신 필요한 연산을 통제한다.

출처: 책 603-604쪽 / 원문 PDF 541-542쪽

- 통제할 수 있는 기본 연산: 읽기, 쓰기, 실행, 끝에 추가, 삭제, 이름·속성 나열, 속성 변경.
- 이름 변경·편집·복사 같은 고급 작업은 기본 연산의 조합이므로 기본 연산에서 보호를 적용할 수 있다.
- 규모와 목적에 따라 보호 요구가 다르므로 모든 환경에 같은 정책을 무조건 적용하지 않는다.

13.4.2 접근 제어

출처: 책 604-607쪽 / 원문 PDF 542-545쪽

출처: 책 604-607쪽 / 원문 PDF 542-545쪽

ACL(접근 제어 리스트)은 파일·디렉터리에 누가 어떤 연산을 할 수 있는지 연결한다. 요청 사용자의 신원과 연산을 목록에서 검사한다. 세밀하지만 사용자를 일일이 열거하면 목록이 길어지고 디렉터리 공간 관리가 복잡해진다. 이를 줄이기 위해 소유자·그룹·기타 사용자로 분류하고 필요한 대상에 ACL을 더하는 결합 방식을 쓸 수 있다.

출처: 책 604-607쪽 / 원문 PDF 542-545쪽

UNIX 권한	원문의 의미
r	파일 읽기. 디렉터리에서는 내용 열람을 제어한다.
w	쓰기 접근을 제어한다.
x	파일 실행. 디렉터리에서는 해당 디렉터리로 이동하는 데 필요하다.
세 사용자 필드	소유자·그룹·기타에 각 rwx 3비트, 총 9비트

출처: 책 604-607쪽 / 원문 PDF 542-545쪽

원문의 Sara 사례는 책 전체를 공동 작업하는 그룹과 한 장만 접근시킬 방문자의 요구를 구분한다. 그룹 하나만으로 부족할 때 해당 장의 ACL에 방문자를 추가한다. 권한·ACL을 바꾸는 행위 자체도 엄격히 통제해야 한다. 두 허가가 충돌할 때의 우선순위도 정책으로 정해야 하며, 원문의 Solaris 사례는 더 구체적으로 부여된 ACL을 우선한다. 명시한 시스템의 규칙을 모든 시스템에 확대하지 않는다.

13.4.3 다른 보호 방법

출처: 책 607-608쪽 / 원문 PDF 545-546쪽

출처: 책 607-608쪽 / 원문 PDF 545-546쪽

파일마다 암호를 요구할 수도 있지만 암호가 너무 많으면 기억하기 어렵고 하나를 공통 사용하면 유출 시 전부 노출된다. 디렉터리별 암호나 파일·파티션 암호화가 대안으로 제시되며 암호 관리가 중요하다.

출처: 책 607-608쪽 / 원문 PDF 545-546쪽

디렉터리는 파일과 다른 연산을 가지므로 생성·삭제·목록 조회 등을 별도로 보호해야 한다. 이름이나 존재 자체가 중요한 정보일 수 있다. 경로로 파일에 접근하려면 파일뿐 아니라 그 경로의 디렉터리 접근도 허용되어야 한다. 여러 경로가 있으면 어떤 경로를 쓰는가에 따라 접근 권한이 다를 수 있다.

13.5 메모리 사상 파일

출처: 책 608-612쪽 / 원문 PDF 546-550쪽

13.5.1 메모리 사상 파일의 기본 기법

출처: 책 608-610쪽 / 원문 PDF 546-548쪽

출처: 책 608-610쪽 / 원문 PDF 546-548쪽

메모리 사상 파일은 프로세스 가상 주소의 일부를 파일 블록과 대응시킨다. 처음 참조할 때 요구 페이징으로 페이지 폴트가 발생하여 해당 내용을 메모리로 읽고, 이후에는 메모리 접근으로 처리한다. 매 접근마다 read/write 시스템 콜을 하는 부담을 줄인다. 사상했다고 파일 전체가 즉시 적재되는 것은 아니다.

출처: 책 608-610쪽 / 원문 PDF 546-548쪽

메모리에 쓴 내용이 보조저장장치에 즉시 동기 기록되는 것은 아니다. 기록·닫기·메모리 확보 시 처리는 원문이 설명한 시스템의 정책에 따른다. 어떤 시스템은 명시적 사상 호출을 요구하며 Solaris는 일반 파일 I/O도 커널 주소 공간의 매핑을 통해 처리하는 사례로 소개된다.

출처: 책 608-610쪽 / 원문 PDF 546-548쪽

원문의 기록 시점 설명에서는 일반적으로 파일을 닫을 때 메모리 이미지의 변경을 파일에 반영하고 가상 주소의 사상을 제거한다. 메모리 부족으로 공간을 확보할 때는 변경을 잃지 않도록 스왑 공간에 반영한다고 설명한다. 이 처리와 공유 프로세스가 같은 물리 페이지의 변경을 보는 시점은 별개의 관계이다.

출처: 책 608-610쪽 / 원문 PDF 546-548쪽

여러 프로세스의 페이지 테이블이 같은 물리 페이지를 가리키면 같은 파일의 변경을 공유할 수 있다. **쓰기 시 복사**를 사용하면 읽기 전용으로 공유하다 수정할 때 별도 페이지를 만든다. 공유 내용의 동시 수정이 필요하면 상호 배제 기법도 사용해야 한다. 같은 파일을 여러 주소 공간에 사상하는 방법은 공유 메모리 통신을 구현하는 데 쓰인다.

13.5.2 Windows API에서 공유 메모리

출처: 책 610-613쪽 / 원문 PDF 548-551쪽

출처: 책 610-613쪽 / 원문 PDF 548-551쪽

원문의 생산자·소비자 흐름

1. 생산자가 CreateFile()로 temp.txt를 열고 HANDLE을 얻는다.
2. CreateFileMapping()으로 SharedObject라는 이름의 공유 메모리 객체를 만든다.
3. MapViewOfFile()로 자신의 가상 주소 공간에 뷰를 만들고 반환 포인터로 메시지를 기록한다.
4. 소비자는 OpenFileMapping()으로 같은 이름의 객체를 열고 MapViewOfFile()로 뷰를 만든다.
5. 소비자가 메시지를 읽고, 각 프로세스는 UnmapViewOfFile()과 CloseHandle()로 뷰·핸들을 정리한다.

출처: 책 610-613쪽 / 원문 PDF 548-551쪽

뷰는 전체 파일 또는 오프셋·크기로 지정한 일부일 수 있다. 원문은 마지막 세 인자를 0으로 두어 전체 파일의 뷰를 지정한다. 실제 페이지는 요구 페이징으로 접근할 때 적재된다. 아래 생산자·소비자 코드는 원문 그대로 이미지로 보존한다. 본문에 메시지 읽기가 설명되어 있지만 별도 실행 출력 화면은 원문에 실행 결과 미제시.

출처: 책 610-613쪽 / 원문 PDF 548-551쪽

그림 13.15 공유 메모리에 기록하는 Windows 생산자

그림·표 출처: 책 611쪽 / 원문 PDF 549쪽

```
#include <windows.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
   HANDLE hFile, hMapFile;
   LPVOID lpMapAddress;

   hFile = CreateFile("temp.txt", /* file name */
      GENERIC_READ | GENERIC_WRITE, /* read/write access */
      0, /* no sharing of the file */
      NULL, /* default security */
      OPEN_ALWAYS, /* open new or existing file */
      FILE_ATTRIBUTE_NORMAL, /* routine file attributes */
      NULL); /* no file template */

   hMapFile = CreateFileMapping(hFile, /* file handle */
      NULL, /* default security */
      PAGE_READWRITE, /* read/write access to mapped pages */
      0, /* map entire file */
      0,
      TEXT("SharedObject")); /* named shared memory object */

   lpMapAddress = MapViewOfFile(hMapFile, /* mapped object handle */
      FILE_MAP_ALL_ACCESS, /* read/write access */
      0, /* mapped view of entire file */
      0,
      0);

   /* write to shared memory */
   sprintf(lpMapAddress,"Shared memory message");

   UnmapViewOfFile(lpMapAddress);
   CloseHandle(hFile);
   CloseHandle(hMapFile);
}
```

그림 13.15 Windows API를 사용하여 공유 메모리에 기록하는 생산자

그림 13.15 공유 메모리에 기록하는 Windows 생산자 - 원문 그림 부분 발췌

temp.txt와 SharedObject, 원문 API 인자·주석·정리 순서를 그대로 보존한 전체 코드이다.

출처: 책 610-613쪽 / 원문 PDF 548-551쪽

그림 13.16 공유 메모리를 읽는 Windows 소비자

그림·표 출처: 책 612쪽 / 원문 PDF 550쪽

```
#include <windows.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
   HANDLE hMapFile;
   LPVOID lpMapAddress;

   hMapFile = OpenFileMapping(FILE_MAP_ALL_ACCESS, /* R/W access */
      FALSE, /* no inheritance */
      TEXT("SharedObject")); /* name of mapped file object */

   lpMapAddress = MapViewOfFile(hMapFile, /* mapped object handle
      FILE_MAP_ALL_ACCESS, /* read/write access */
      0, /* mapped view of entire file */
      0,
      0);

   /* read from shared memory */
   printf("Read message %s", lpMapAddress);

   UnmapViewOfFile(lpMapAddress);
   CloseHandle(hMapFile);
}
```

그림 13.16 Windows API를 사용하여 공유 메모리를 읽는 소비자

그림 13.16 공유 메모리를 읽는 Windows 소비자 - 원문 그림 부분 발췌

동일한 SharedObject의 뷰를 만들고 포인터로 읽는 원문 전체 코드이다. 출력 문자열을 새로 실행하거나 결과를 추정하지 않는다.

13.6 요약과 원문 연습문제의 점검 범위

출처: 책 613-615쪽 / 원문 PDF 551-553쪽

출처: 책 613-615쪽 / 원문 PDF 551-553쪽

파일은 논리 레코드를 물리 저장장치에 대응시키는 추상 자료형이며 디렉터리는 이름과 조직을 제공한다. 구조가 1단계→2단계→트리→공유 그래프로 확장될수록 유연성과 함께 탐색·삭제·공간 회수의 복잡성이 증가한다. 파일 보호는 사용자뿐 아니라 연산의 종류와 경로·디렉터리를 함께 고려한다.

출처: 책 613-615쪽 / 원문 PDF 551-553쪽

장말 문제는 파일 보관 정책, 유형·구조 지원, 이름과 디렉터리 조직, open/close, 디렉터리 보호, 권한 표현과 제어 리스트를 점검한다. 본문에 없는 문제 풀이를 추가하지 않는다. 추가 자료의 다중 레벨 디렉터리·NFS 소개와 참고문헌 존재를 확인했고 외부 내용을 보충하지 않았다.

핵심 정리

- 파일의 이름·식별자·장치 위치, 파일 포인터·열린 계수·링크 참조 계수를 문맥별로 구별한다.
- 공유 파일은 독립 사본과 다르며 링크와 참조 계수는 삭제 시점을 바꾼다.
- 순차 접근, 상대 블록 번호에 의한 직접 접근, 색인에 의한 블록 선택을 구분한다.
- 일반 그래프에서 순환은 탐색과 참조 계수 기반 회수에 문제를 만든다.
- ACL·사용자 분류·디렉터리 권한과 메모리 사상의 공유·기록 조건을 함께 확인한다.

혼동하기 쉬운 내용

디렉터리 구조의 변화

출처: 책 594-603쪽 / 원문 PDF 532-541쪽

구조	해결·허용하는 것	새 제약
1단계	단순한 한 디렉터리	모든 파일명 고유
2단계	사용자별 같은 이름	공유와 경로 표현 필요
트리	사용자 하위 디렉터리·계층화	물리적 파일·하위 디렉터리 공유 제한
비순환 그래프	공유 허용	중복 순회·참조·삭제 관리
일반 그래프	순환까지 허용하는 유연성	탐색 종료·도달성·가비지 수집 문제

장 마무리 복습

- 파일 열기는 반복 이름 검색을 줄이고 열린 테이블의 사용 상태를 유지한다.
- 파일 구조를 운영체제가 많이 지원할수록 편의와 구현·확장 제약의 절충이 생긴다.
- 메모리 사상은 파일 접근을 가상 메모리 참조로 바꾸며 공유 물리 페이지와 쓰기 시 복사는 서로 다른 공유 의미를 가진다.

원문 확인 메모

- 인쇄592쪽 UPC 색인의 레코드 수·블록 수 연결은 원문 확인 필요로 표시하고 수치를 임의 수정하지 않았다.
- Java와 Windows 예제는 원문 코드의 정확한 기호·주석을 보존하기 위해 이미지로 수록했다. 원문 예제의 생략된 동작을 새 코드로 보충하지 않았다.

제14장. 파일 시스템 구현

출처: 책 617-653쪽 / 원문 PDF 554-590쪽

학습 주제

- 파일의 논리적 인터페이스를 실제 저장 블록으로 구현하는 자료구조와 알고리즘을 살펴본다. 디렉터리·블록 할당·가용 공간·캐시의 절충을 정리하고 크래시 복구와 WAFL의 스냅숏을 연결한다.

설명의 흐름

1. 계층별 역할 → 장치·메모리 자료구조와 open/close → 디렉터리 → 연속/연결/색인 할당 → 가용 공간 → 효율·캐시 → 일관성·로그·백업 → WAFL

14.1 파일 시스템 구조

출처: 책 618-620쪽 / 원문 PDF 555-557쪽

출처: 책 618-620쪽 / 원문 PDF 555-557쪽

파일 시스템의 두 설계 문제는 사용자에게 보이는 파일·속성·연산을 정의하는 일과 이를 실제 보조저장장치에 대응시키는 알고리즘·자료구조를 만드는 일이다. 원문은 재기록·직접 접근이 가능한 HDD와 덮어쓰기 특성이 다른 NVM을 구별하며, 효율을 위해 블록 단위로 I/O한다.

출처: 책 618-620쪽 / 원문 PDF 555-557쪽

층	역할
논리 파일 시스템	디렉터리·메타데이터·보호·FCB를 관리하고 이름을 하위 계층의 정보로 변환한다.
파일-구성 모듈	파일의 논리 블록과 실제 공간의 관계를 알고 가용 공간 관리자를 포함한다.
기본 파일 시스템	LBA 기반의 블록 읽기·쓰기, I/O 스케줄링, 버퍼·캐시를 관리한다.
I/O 제어	드라이버·인터럽트 핸들러가 일반 명령을 특정 하드웨어 명령으로 바꾸고 전송한다.

출처: 책 618-620쪽 / 원문 PDF 555-557쪽

메타데이터는 파일 내용 자체를 제외한 파일 시스템 구조이다. **FCB**는 소유·권한·내용의 위치 같은 파일 정보를 담고 UNIX에서는 inode가 그 역할을 한다. 계층화는 코드 중복을 줄여 여러 파일 시스템이 하위 코드를 공유하게 하지만 계층 통과 비용을 늘릴 수 있다. 파일 시스템마다 기능·성능·신뢰성·목적이 달라 여러 구현이 존재하고 FUSE는 사용자 수준 구현의 유연성을 제공하는 사례이다.

14.2 파일 시스템 구현

출처: 책 620-622쪽 / 원문 PDF 557-559쪽

14.2.1 파일 시스템 구현 개요

출처: 책 620-622쪽 / 원문 PDF 557-559쪽

출처: 책 620-622쪽 / 원문 PDF 557-559쪽

장치상의 구조	담는 정보
부트 제어 블록	해당 파티션에서 운영체제를 부팅하는 정보
볼륨 제어 블록	전체·가용 블록 수, 블록 크기, 가용 공간과 FCB의 수·포인터
디렉터리	이름과 파일 식별·제어 정보의 연결
파일별 FCB	고유 식별자와 자세한 파일 정보

출처: 책 620-622쪽 / 원문 PDF 557-559쪽

메모리에는 마운트된 파티션 정보, 최근 디렉터리 정보, FCB 사본을 담은 범 시스템 열린 파일 테이블, 이를 가리키는 프로세스별 열린 파일 테이블, 전송 중 블록 버퍼를 둔다. 마운트 때 적재하고 동작 중 갱신하며 해제 때 제거한다. 파일 생성은 새 FCB를 할당하고 디렉터리에 이름·FCB 연결을 추가한 뒤 기록하는 작업이다. 디렉터리를 특별 파일로 구현하는지 별개 객체로 보는지는 시스템에 따라 다르다.

14.2.2 사용법

출처: 책 622-623쪽 / 원문 PDF 559-560쪽

출처: 책 622-623쪽 / 원문 PDF 559-560쪽

open부터 close까지

1. open이 먼저 범 시스템 테이블을 확인한다. 이미 열려 있으면 그 항목을 가리키는 프로세스별 항목을 만든다.
2. 처음 열 때는 디렉터리에서 찾아 FCB를 범 시스템 테이블에 복사한다. 프로세스별 항목에는 현재 위치와 접근 모드 등을 둔다.
3. 프로세스는 반환된 파일 디스크립터(UNIX) 또는 핸들(Windows)로 후속 연산을 한다. 매번 파일 이름을 다시 검색하지 않는다.
4. close는 프로세스별 항목을 없애고 열린 계수를 줄인다. 마지막 사용자가 닫으면 갱신 정보를 디스크에 반영하고 범 시스템 항목을 회수한다.

출처: 책 622-623쪽 / 원문 PDF 559-560쪽

이름·디렉터리·FCB를 캐싱하면 실제 데이터 외의 파일 정보를 메모리에서 해결하여 저장장치 접근을 줄일 수 있다. 테이블은 파일별 공통 정보와 프로세스별 사용 상태를 구별하는 구조이다.

14.3 디렉터리 구현

출처: 책 623-624쪽 / 원문 PDF 560-561쪽

14.3.1 선형 리스트

출처: 책 624쪽 / 원문 PDF 561쪽

출처: 책 624쪽 / 원문 PDF 561쪽

파일 이름과 데이터 블록 포인터의 **선형 리스트**는 구현이 쉽지만 검색 시간이 길다. 생성은 같은 이름이 없는지 검색한 뒤 추가하고, 삭제는 공간을 돌려주고 항목을 비워 재사용한다. 사용 비트·특별 이름·잘못된 inode 번호 표시, 마지막 항목 복사, 연결 리스트 등으로 빈 항목을 관리할 수 있다.

출처: 책 624쪽 / 원문 PDF 561쪽

정렬하면 이진 검색과 정렬된 목록 출력이 쉬워지지만 삽입·삭제 때 정보 이동이 필요하다. 트리 구조나 최근 디렉터리 캐시가 보완할 수 있다. 디렉터리 검색은 빈번하므로 단순한 구현의 속도 비용이 중요하다.

14.3.2 해시 테이블

출처: 책 624-625쪽 / 원문 PDF 561-562쪽

출처: 책 624-625쪽 / 원문 PDF 561-562쪽

해시 테이블은 파일 이름을 값으로 바꾸어 항목 위치를 빠르게 찾는다. 충돌 처리가 필요하며 크기가 고정되면 확장 때 함수와 기존 항목을 재구성해야 할 수 있다. 원문의 64항목에서 65번째 항목을 추가하는 예는 더 큰 테이블과 재해싱의 필요를 보여준다. 체인 오버플로 방식은 각 해시 항목의 연결 리스트에 충돌 항목을 넣어 확장하지만 그 리스트를 탐색하는 비용이 있다.

14.4 할당 방법

출처: 책 625-633쪽 / 원문 PDF 562-570쪽

14.4.1 연속 할당

출처: 책 625-627쪽 / 원문 PDF 562-564쪽

출처: 책 625-627쪽 / 원문 PDF 562-564쪽

연속 할당은 파일이 연속한 블록을 차지하게 하고 디렉터리에 시작 블록과 길이를 기록한다. 시작이 b이고 길이가 n이면 b부터 b+n-1까지이며 i번째 블록은 b+i이다. 순차·직접 접근이 모두 쉽고 HDD의 헤드 이동을 줄일 수 있다. 이 이익은 가까운 논리 주소가 물리적으로도 가깝다는 전제를 가진다.

출처: 책 625-627쪽 / 원문 PDF 562-564쪽

가용 연속 영역 선택은 동적 공간 할당 문제이다. 최초 적합·최적 적합 등을 사용할 수 있지만 **외부 단편화**로 전체 빈 공간은 충분해도 요구 크기의 한 덩어리가 없을 수 있다. 다른 장치로 복사했다 돌아오거나 밀집화하면 개선되지만 시간·비용·정상 운영 중 성능 저하가 크다.

출처: 책 625-627쪽 / 원문 PDF 562-564쪽

파일 최종 크기도 예측하기 어렵다. 작게 예약하면 인접 공간이 없어 확장할 수 없고, 종료 후 다시 실행하거나 더 큰 공간에 전체를 복사해야 한다. 지나치게 선할당하면 오랫동안 사용하지 않는 공간이 낭비된다. **익스텐트**는 초기의 연속 영역이 부족할 때 연속 블록 묶음을 추가하여 위치·길이·다음 묶음 포인터를 기록하는 절충이다.

14.4.2 연결 할당과 FAT

출처: 책 627-630쪽 / 원문 PDF 564-567쪽

출처: 책 627-630쪽 / 원문 PDF 564-567쪽

연결 할당은 파일을 장치에 흩어질 수 있는 블록의 연결 리스트로 저장한다. 각 블록이 다음 블록을 가리키고 디렉터리는 시작·끝을 기억한다. 파일 크기를 미리 정하지 않고 가용 블록을 계속 붙일 수 있어 외부 단편화·밀집화 문제를 피한다. 반면 직접 접근은 앞의 링크를 따라가야 하므로 비효율적이다.

출처: 책 627-630쪽 / 원문 PDF 564-567쪽

포인터가 데이터 공간을 차지한다. 원문 예는 512바이트 블록의 4바이트 포인터로 508바이트만 데이터를 담는 경우이다. 여러 블록을 **클러스터**로 묶으면 포인터·할당 관리와 HDD 탐색을 줄이지만 내부 단편화와 작은 임의 요청의 전송량은 늘어난다. 링크 하나의 손상은 후속 데이터 접근을 잃게 할 수 있고 이중 링크·추가 식별 정보는 더 많은 비용을 요구한다.

출처: 책 627-630쪽 / 원문 PDF 564-567쪽

FAT(파일 할당 테이블)는 링크들을 파티션의 별도 연속 테이블로 모은 변형이다. 디렉터리의 시작 블록을 인덱스로 FAT를 찾으면 다음 블록 번호를 얻고 특수 값으로 끝을 표시한다. 원문 예의 사슬은 217→618→339이다. FAT가 메모리에 캐시되지 않으면 테이블과 데이터 위치 사이의 왕복 탐색이 늘 수 있다.

출처: 책 627-630쪽 / 원문 PDF 564-567쪽

그림 14.6 파일 할당 테이블(FAT)

그림·표 출처: 책 629쪽 / 원문 PDF 566쪽

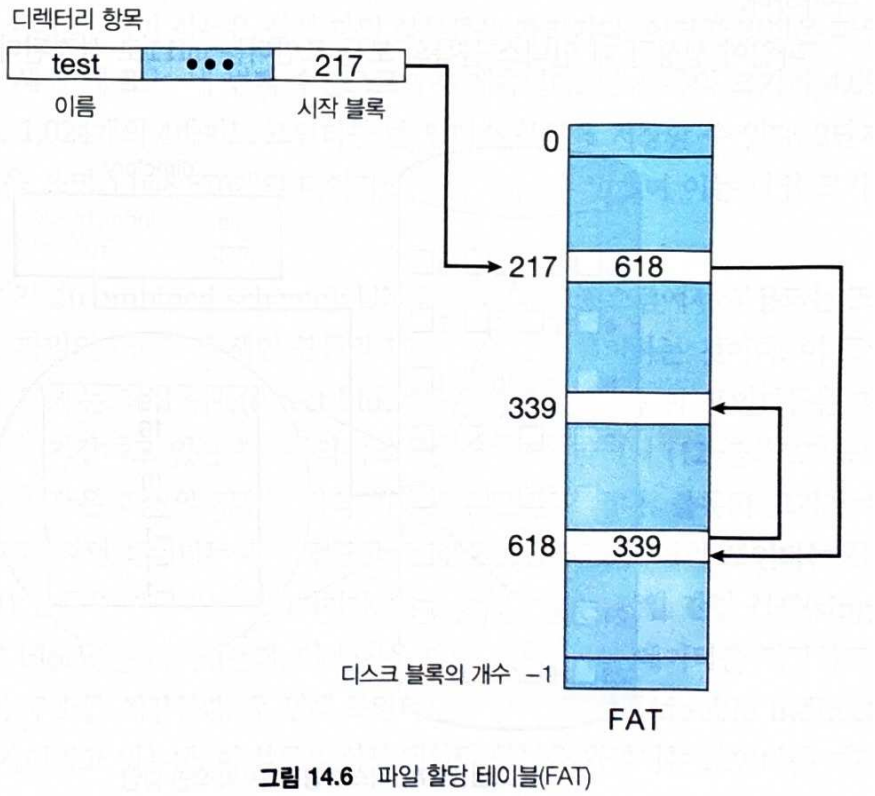


그림 14.6 파일 할당 테이블(FAT)

그림 14.6 파일 할당 테이블(FAT) - 원문 그림 부분 발췌

디렉터리의 시작 블록 217에서 FAT 항목을 따라 618·339로 연결되는 구조를 보존한다.

14.4.3 색인 할당

출처: 책 630-632쪽 / 원문 PDF 567-569쪽

출처: 책 630-632쪽 / 원문 PDF 567-569쪽

색인 할당은 파일마다 블록 주소 배열인 색인 블록을 두고 i번째 항목이 i번째 데이터 블록을 가리키게 한다. 디렉터리는 색인 블록의 주소를 갖는다. 새 파일은 포인터를 null로 시작하고 실제 블록을 쓸 때 주소를 넣는다. 외부 단편화 없이 직접 접근을 지원하지만 작은 파일에도 색인 블록 전체가 필요하여 포인터 오버헤드가 크다. 색인을 작게 하면 큰 파일을 표현할 별도 방법이 필요하다.

출처: 책 630-632쪽 / 원문 PDF 567-569쪽

확장 방법	구조와 조건
연결 기법	색인 블록의 마지막 포인터로 다음 색인 블록을 연결한다.
다중 수준 색인	상위 색인이 하위 색인 블록을 가리키고 최하위 색인이 데이터를 가리킨다. 필요하면 수준을 더한다.
결합 기법	inode 안의 직접 포인터와 단일·이중·삼중 간접 포인터를 함께 사용한다. 작은 파일은 추가 색인을 피하고 큰 파일은 간접 수준을 쓴다.

출처: 책 630-632쪽 / 원문 PDF 567-569쪽

원문 다중 색인 예는 4,096바이트 블록에 4바이트 포인터 1,024개를 넣고, 두 수준으로 1,048,576 데이터 블록, 4GB 파일을 표현한다. 결합 예의 15개 포인터 중 12개는 직접 블록이고 나머지 3개가 단일·이중·삼중 간접 블록이다. 4KB 블록이면 처음 48KB는 직접 포인터로 접근한다. 색인을 캐시해도 데이터 자체는 볼륨 전역에 흩어질 수 있다.

출처: 책 630-632쪽 / 원문 PDF 567-569쪽

그림 14.8 UNIX의 inode

그림·표 출처: 책 632쪽 / 원문 PDF 569쪽

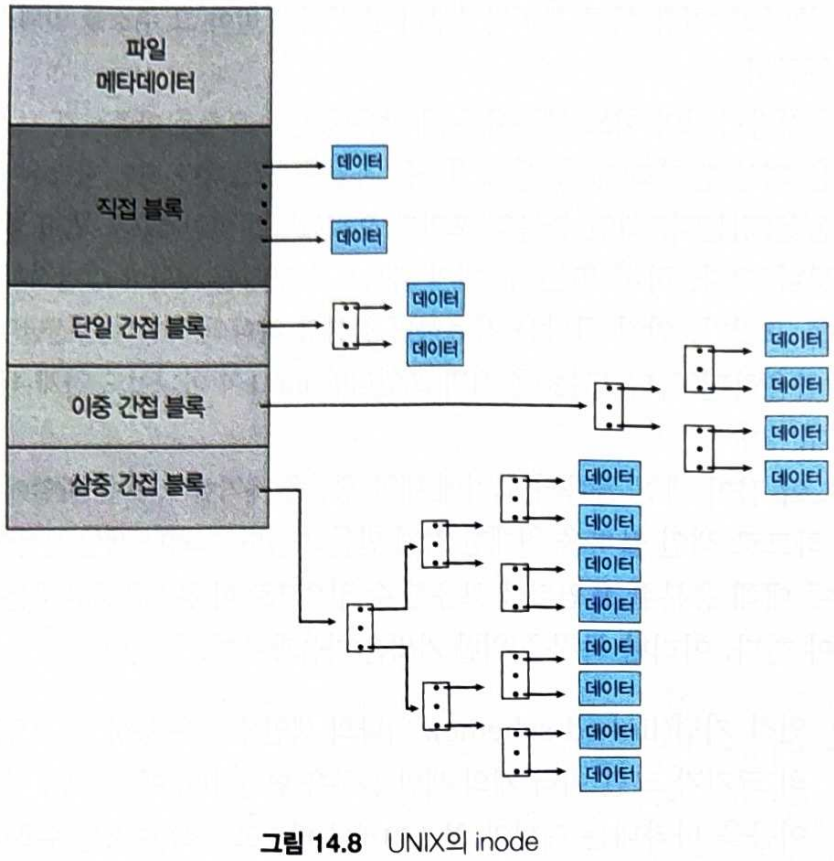


그림 14.8 UNIX의 inode

그림 14.8 UNIX의 inode - 원문 그림 부분 발췌

직접·단일 간접·이중 간접·삼중 간접 블록의 포인터 수준과 데이터 도달 관계를 보여준다.

14.4.4 할당 방법의 성능

출처: 책 632-633쪽 / 원문 PDF 569-570쪽

출처: 책 632-633쪽 / 원문 PDF 569-570쪽

방법	접근과 절충
연속	시작과 변위로 즉시 블록을 결정하지만 공간 확보·성장·외부 단편화가 문제이다.
연결	다음 블록의 순차 접근은 쉽지만 i번째 직접 접근에 여러 링크 읽기가 필요하다.
색인	색인의 메모리 상주 여부·구조·파일 크기·목표 위치에 따라 추가 I/O가 달라진다.

출처: 책 632-633쪽 / 원문 PDF 569-570쪽

시스템의 주 사용 패턴이 선택 기준이다. 순차·직접 파일을 다른 방식으로 저장하거나 작은 파일은 연속, 커지면 색인으로 전환하는 결합도 있다. HDD에서는 탐색을 줄이기 위한 추가 CPU 연산이 타당할 수 있으나, 헤드가 없는 NVM에 같은 최적화를 그대로 쓰면 불필요한 비용이 된다. NVM은 명령 수와 접근 경로를 줄이는 최적화가 필요하다는 것이 원문의 설명이다.

14.5 가용 공간의 관리

출처: 책 634-636쪽 / 원문 PDF 571-573쪽

14.5.1 비트 벡터

출처: 책 634-635쪽 / 원문 PDF 571-572쪽

출처: 책 634-635쪽 / 원문 PDF 571-572쪽

가용 공간 관리는 할당 시 빈 블록을 제거하고 삭제 시 되돌려 받는 역할이다. 목록이라는 이름이어도 반드시 연결 리스트일 필요는 없다. **비트맵**은 블록당 한 비트를 사용한다. 이 원문의 표기에서는 **1이 가용, 0이 할당**이다. 워드를 검사하여 처음 0이 아닌 워드와 그 안의 첫 1을 찾으면 첫 가용 블록을 구할 수 있다. 연속 가용 블록 탐색도 쉽다.

출처: 책 634-635쪽 / 원문 PDF 571-572쪽

원문 코드·기호

```
(워드의 비트 수) × (값이 0인 워드의 수) + 첫 번째 1비트의 변위
```

원문이 제시한 첫 가용 블록 번호의 계산식이다. 앞의 워드는 모두 할당 비트 0으로만 구성되어 있다는 검색 절차의 전제를 갖는다.

출처: 책 634-635쪽 / 원문 PDF 571-572쪽

하드웨어 비트 연산은 효율적이지만 큰 비트맵을 메모리에 두기 어렵다. 원문은 4KB 블록의 1TB 디스크에 32MB 비트맵이 필요하다고 설명한다. 블록·클러스터가 커지면 관리 비트 수는 줄지만 공간 할당의 단위도 커진다. 복구를 위해 장치상의 기록도 유지해야 한다.

14.5.2 연결 리스트

출처: 책 635쪽 / 원문 PDF 572쪽

출처: 책 635쪽 / 원문 PDF 572쪽

모든 가용 블록을 다음 가용 블록의 포인터로 연결하고 첫 포인터를 정해진 위치와 메모리에 둔다. 전체 순회는 디스크 접근이 많아 비싸지만 보통 할당에는 첫 가용 블록만 필요하다. FAT는 할당 사슬과 가용 블록의 관리를 같은 테이블에서 처리하여 별도 구조를 줄일 수 있다.

14.5.3 그룹핑

출처: 책 636쪽 / 원문 PDF 573쪽

출처: 책 636쪽 / 원문 PDF 573쪽

첫 가용 블록에 n개의 주소를 넣고 처음 n-1개는 실제 빈 블록, 마지막은 같은 형식의 다음 주소 묶음을 가진 블록을 가리킨다. 한 블록씩 따라가는 방식보다 여러 가용 주소를 쉽게 확보한다.

14.5.4 계수

출처: 책 636쪽 / 원문 PDF 573쪽

출처: 책 636쪽 / 원문 PDF 573쪽

연속 블록들이 함께 할당·반환되는 경향을 이용하여 첫 주소와 연속 길이만 기록한다. 개별 항목은 주소 하나보다 크지만 대개 길이가 1보다 커 전체 목록이 짧아진다. 효율적인 검색·삽입·삭제를 위해 균형 트리로 저장할 수 있다. 익스텐트 관리와 유사한 구조이다.

14.5.5 공간맵

출처: 책 636-637쪽 / 원문 PDF 573-574쪽

출처: 책 636-637쪽 / 원문 PDF 573-574쪽

큰 파일 시스템에서 산재한 블록을 반환할 때 비트맵 여러 블록을 갱신하는 비용이 커질 수 있다. ZFS는 장치를 **메타슬랩**으로 나누고 각 영역의 **공간맵**에 할당·반환 활동을 계수 형식으로 시간순 기록한다.

출처: 책 636-637쪽 / 원문 PDF 573-574쪽

공간맵 사용

1. 필요한 메타슬랩의 공간맵을 오프셋으로 색인된 메모리의 균형 트리로 적재한다.
2. 로그를 재실행하여 현재의 할당·가용 상태를 반영한다.
3. 인접 가용 구간을 결합하여 항목을 압축한다.
4. 트랜잭션 작업의 일부로 장치의 공간 정보를 갱신한다. 처리 중 새 요청은 로그를 통해 충족시킬 수 있다.

출처: 책 636-637쪽 / 원문 PDF 573-574쪽

로그와 균형 트리가 함께 가용 리스트 역할을 수행하여 구조 크기와 I/O를 제어한다.

14.5.6 사용하지 않는 트림

출처: 책 637쪽 / 원문 PDF 574쪽

출처: 책 637쪽 / 원문 PDF 574쪽

HDD는 해제된 블록을 다음 사용 때 덮어쓰면 되지만 NAND는 먼저 큰 블록을 지워야 한다. 파일 시스템이 어느 페이지가 더 이상 사용되지 않는지 장치에 알려야 미리 가비지 수집·삭제하여 쓰기 가능한 공간을 유지할 수 있다. 원문은 ATA의 **TRIM**, NVMe의 **unallocate** 명령을 제시한다. 이 기능이 없으면 거의 가득 찼을 때 쓰기 중 정리가 집중되어 성능이 급락하는 **쓰기 절벽(write cliff)**이 생길 수 있다.

14.6 효율과 성능

출처: 책 637-641쪽 / 원문 PDF 574-578쪽

14.6.1 효율

출처: 책 637-639쪽 / 원문 PDF 574-576쪽

출처: 책 637-639쪽 / 원문 PDF 574-576쪽

공간 효율과 성능은 함께 판단한다. inode 선할당은 공간을 쓰지만 데이터 근처에 분산하면 탐색을 줄인다. 큰 클러스터는 전송에 유리하지만 내부 단편화가 커, 작은 파일·마지막 부분에 작은 클러스터를 쓰는 변형이 있다. 마지막 접근 시각을 유지하려면 읽기 때도 메타데이터 기록이 필요하여 각 속성의 실제 가치와 비용을 고려해야 한다.

출처: 책 637-639쪽 / 원문 PDF 574-576쪽

큰 포인터는 표현 가능한 파일·공간을 늘리지만 색인·가용 공간 구조의 크기도 키운다. 고정 크기 테이블은 가득 차면 자원이 남아도 새 프로세스·파일을 만들지 못하고 크기 변경에 재부팅·재컴파일이 필요할 수 있다. 동적 할당은 이런 인위적 제한을 줄이지만 관리 알고리즘이 복잡해지고 할당·반환 비용이 생긴다. 원문의 과거 MS-DOS·Solaris 사례는 초기 구조 선택이 향후 확장에 미치는 영향을 보여준다.

14.6.2 성능

출처: 책 639-642쪽 / 원문 PDF 576-579쪽

출처: 책 639-642쪽 / 원문 PDF 576-579쪽

컨트롤러 캐시는 트랙·블록을 보관하고 운영체제는 메모리에 다시 캐시한다. **페이지 캐시**는 가상 메모리의 페이지로 파일 데이터를 관리하고 프로세스 페이지와 함께 이용할 수 있다. 메모리 사상 접근은 페이지 캐시, read/write는 별도 버퍼 캐시를 쓰면 동일 데이터가 두 번 복사되는 **이중 캐싱**이 생겨 공간·CPU·일관성 비용이 든다. **통합 버퍼 캐시**는 두 접근 경로가 같은 캐시를 사용하게 한다.

출처: 책 639-642쪽 / 원문 PDF 576-579쪽

그림 14.11 통합 버퍼 캐시를 사용하는 입출력

그림·표 출처: 책 641쪽 / 원문 PDF 578쪽

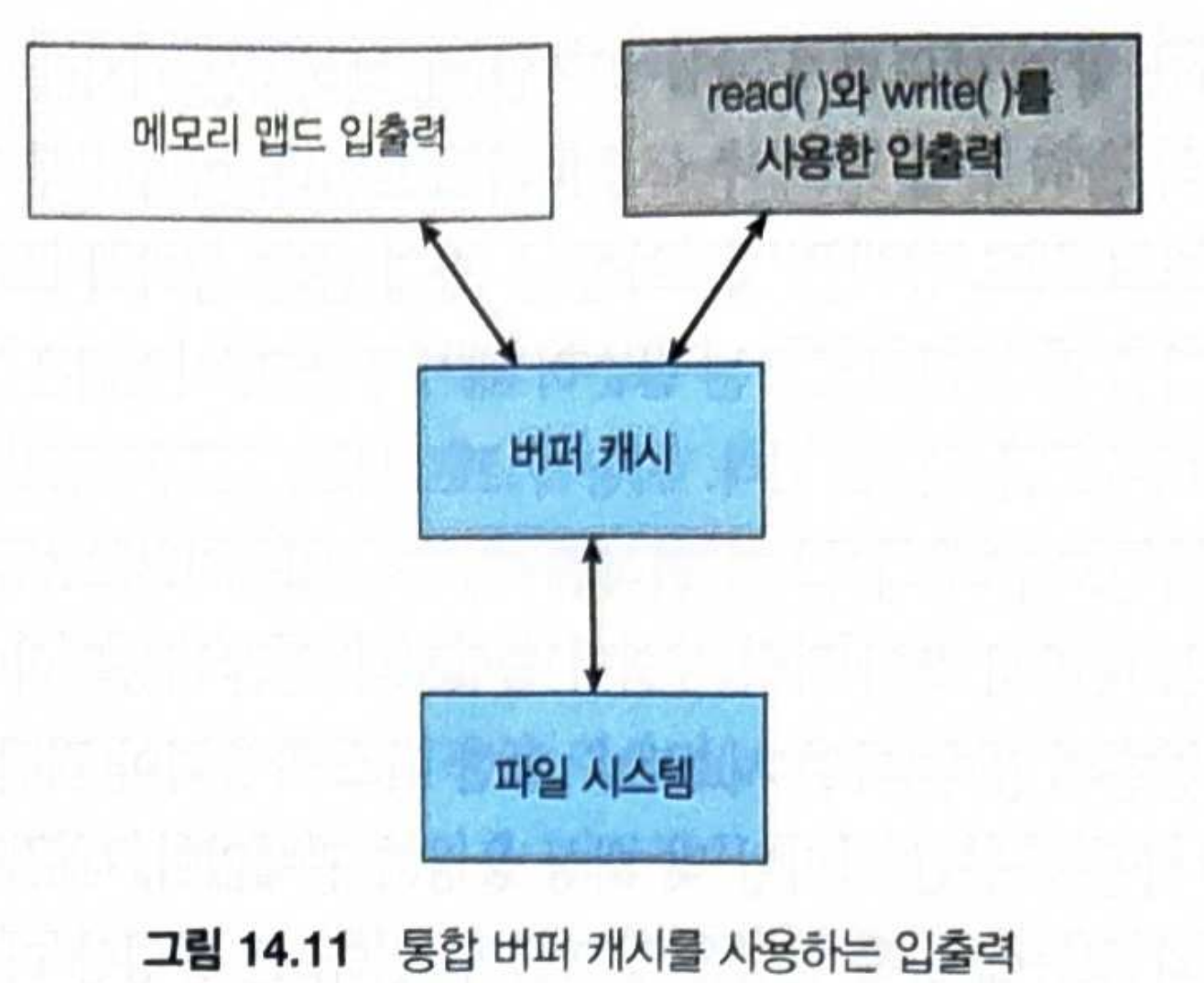


그림 14.11 통합 버퍼 캐시를 사용하는 입출력

그림 14.11 통합 버퍼 캐시를 사용하는 입출력 - 원문 그림 부분 발췌

메모리 맵드 I/O와 read/write의 두 경로가 하나의 캐시를 거쳐 파일 시스템으로 이어진다.

출처: 책 639-642쪽 / 원문 PDF 576-579쪽

프로세스 메모리와 파일 캐시가 가용 메모리를 공유하면 한쪽이 다른 쪽을 밀어낼 수 있어 교체 정책과 경계를 조정한다. 원문의 Solaris 발전은 단순한 범용 정책만으로 모든 부하를 만족시키기 어렵다는 사례이다.

출처: 책 639-642쪽 / 원문 PDF 576-579쪽

기법	원문의 조건·효과
동기식 쓰기	저장장치에 도달한 뒤 진행한다. 메타데이터·순서 보장이 필요한 작업에 사용한다.
비동기식 쓰기	캐시에 쓰고 먼저 복귀한다. 모아서 스케줄할 수 있지만 실제 매체 기록은 뒤에 이루어진다.
free-behind	순차 접근에서 다음 페이지를 요청하면 이전 페이지를 버퍼에서 제거한다.
read-ahead	요청 페이지와 이어질 페이지를 한 번에 읽어 미리 캐시한다. 순차 접근에서 이후 요청 가능성을 이용한다.

출처: 책 639-642쪽 / 원문 PDF 576-579쪽

순차 파일은 최근 페이지를 다시 쓸 가능성이 낮아 일반 LRU가 적합하지 않을 수 있다. 작은 비동기 출력은 캐싱 덕분에 입력보다 빠르게 보일 수 있지만 파일이 캐시 용량보다 크면 결국 장치 속도에 제약된다. 따라서 캐시가 대량 연속 I/O의 물리 병목을 없애는 것은 아니다.

14.7 복구

출처: 책 642-645쪽 / 원문 PDF 579-582쪽

14.7.1 일관성 검사

출처: 책 642-643쪽 / 원문 PDF 579-580쪽

출처: 책 642-643쪽 / 원문 PDF 579-580쪽

파일 생성 같은 작업은 디렉터리·FCB·데이터·가용 계수 등 여러 구조를 바꾼다. 일부만 기록되고 크래시가 나면 메모리와 장치 또는 자료구조 사이가 불일치한다. **일관성 검사기**는 디렉터리와 메타데이터를 실제 할당 상태와 비교하여 복구한다. 전체 검사는 오래 걸리므로 갱신 중 상태 비트를 설정하고 완료 후 소거하여 필요할 때 검사할 수 있다.

출처: 책 642-643쪽 / 원문 PDF 579-580쪽

복구 가능성은 할당 구조에 좌우된다. 원문은 데이터 블록의 링크가 있으면 재구성이 가능하지만 서로 관계를 모르는 색인 데이터 블록은 디렉터리 정보를 잃었을 때 어렵다고 설명한다. 메타데이터의 동기식 기록과 전원 보호 장치 버퍼는 일부 위험을 줄여도 크래시 도중 부분 갱신을 모두 없애지는 못한다.

14.7.2 로그 구조 파일 시스템

출처: 책 643-644쪽 / 원문 PDF 580-581쪽

출처: 책 643-644쪽 / 원문 PDF 580-581쪽

원문의 로그 기반 복구

1. 파일 시스템 메타데이터 변경을 로그에 순차 기록하고 한 작업의 연산 집합을 트랜잭션으로 묶는다.

2. 로그 기록이 확약되면 호출자에게 복귀할 수 있다. 이후 실제 자료구조에 변경을 재실행하고 진행·완료를 기록한다.
3. 크래시 뒤에는 확약되었지만 미완료인 트랜잭션을 끝까지 적용한다.
4. 확약 전에 중단된 트랜잭션의 변경은 원상 복구하여 일관성을 회복한다.

출처: 책 643-644쪽 / 원문 PDF 580-581쪽

로그는 원형 버퍼로 구현될 수 있으므로 아직 필요한 정보를 새 기록으로 덮지 않게 관리한다. 전체 메타데이터를 검사·수리하는 대신 필요한 로그를 처리하므로 복구의 지연·불확실성·사람의 개입을 줄인다. HDD에서는 동기식 임의 메타데이터 쓰기를 순차 로그 쓰기로 바꾸고 실제 구조 갱신을 비동기로 수행하여 성능도 높일 수 있다.

14.7.3 다른 해결 방안들

출처: 책 644-645쪽 / 원문 PDF 581-582쪽

출처: 책 644-645쪽 / 원문 PDF 581-582쪽

WAFL과 ZFS의 다른 접근은 기존 블록을 새 내용으로 덮지 않고 데이터·메타데이터 변경을 새 블록에 기록하는 것이다. 트랜잭션이 끝나면 포인터를 새 블록으로 바꾸고 옛 블록을 회수한다. 옛 포인터·블록을 유지하면 특정 시점의 **스냅숏**이 된다. 포인터 갱신의 정확성이 중요하며 WAFL도 일부 메타데이터 오염에 대응하는 검사기를 가진다고 원문은 설명한다. ZFS는 여기에 데이터·메타데이터 체크섬과 RAID를 결합하는 방식으로 소개된다.

14.7.4 백업과 복구

출처: 책 645-646쪽 / 원문 PDF 582-583쪽

출처: 책 645-646쪽 / 원문 PDF 582-583쪽

원문의 주기적 백업

1. 첫날 전체 파일을 전체 백업한다.
2. 다음 날부터 직전 백업 이후 바뀐 파일만 점증적으로 백업한다.
3. 일정 주기가 지나면 다시 전체 백업으로 돌아간다.
4. 복원은 전체 백업에서 시작하여 이후 필요한 점증 백업을 차례로 적용한다.

출처: 책 645-646쪽 / 원문 PDF 582-583쪽

주기가 길면 백업량을 줄일 수 있지만 복원에 읽을 매체가 많아진다. 중간 전체 백업은 복원 부담을 줄이는 대신 저장 공간을 더 쓴다. 손상 발견이 늦을 수 있어 재사용하지 않는 영구 백업과 다른 장소의 보관도 필요할 수 있다. 백업은 시스템 전체 고장뿐 아니라 실수로 삭제한 파일의 복구에도 이용된다.

14.8 예: WAFL 파일 시스템

출처: 책 646-650쪽 / 원문 PDF 583-587쪽

출처: 책 646-650쪽 / 원문 PDF 583-587쪽

WAFL(write-anywhere file layout)은 원문의 NetApp 파일 서버와 NVRAM 쓰기 캐시·랜덤 I/O 부하에 맞춰 설계되었다. 블록 기반이며 inode를 사용하고, 루트 inode 아래에 inode 파일·가용 블록 맵·가용 inode 맵 등을 모두 파일로 둔다. 메타데이터도 일반 파일이므로 고정 위치에 묶이지 않고 여러 디스크로 확장할 때 함께 커질 수 있다.

출처: 책 646-650쪽 / 원문 PDF 583-587쪽

WAFL 스냅숏

1. 루트 inode를 복사하여 특정 시점의 포인터를 남긴다.
2. 이후 변경은 기존 블록을 덮지 않고 새 공간에 기록한다. 현재 루트는 새 블록을 가리키고 스냅숏은 옛 블록을 계속 가리킨다.
3. 추가 맵 비트는 어느 스냅숏이 블록을 쓰는지 추적한다. 그 블록을 쓰는 모든 스냅숏이 없어지면 재사용할 수 있다.

출처: 책 646-650쪽 / 원문 PDF 583-587쪽

그림 14.13 WAFL의 스냅숏

그림·표 출처: 책 648쪽 / 원문 PDF 585쪽

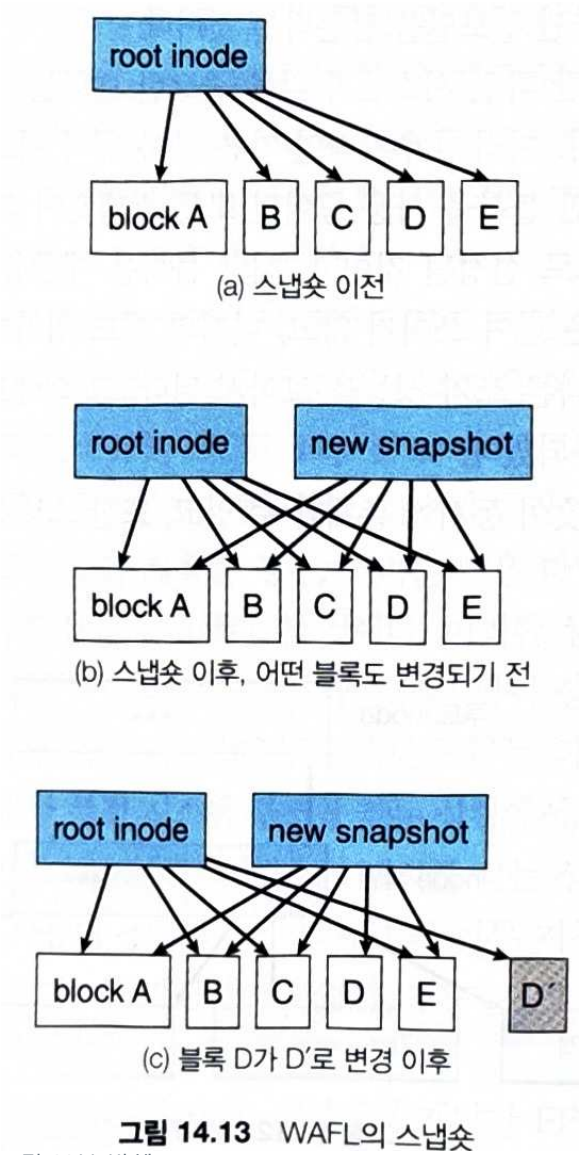


그림 14.13 WAFL의 스냅숏

그림 14.13 WAFL의 스냅숏 - 원문 그림 부분 발췌

스냅숏 전, 포인터만 복사한 직후, 블록 D를 D′로 변경한 뒤의 세 상태이다. 옛 블록을 남기고 새 블록에 기록하는 관계를 보존한다.

출처: 책 646-650쪽 / 원문 PDF 583-587쪽

스냅숏은 전체 파일을 복사하지 않아 적은 공간으로 여러 시점을 보관하고 백업·테스트·디버깅에 쓸 수 있다. **클론**은 읽기·쓰기 스냅숏으로 원본 스냅숏을 참조하다 변경을 새 블록에 기록한다. 원본을 유지하므로 테스트·업그레이드 실패 때 클론만 제거할 수 있고, 필요하면 원본을 대신하도록 승격할 수 있다. 원격 복제는 처음 스냅숏을 전송한 뒤 이후 스냅숏 사이에 바뀐 블록과 포인터를 적용하여 다른 시스템의 복사본을 유지한다.

출처: 책 646-650쪽 / 원문 PDF 583-587쪽

원문의 별도 APFS 설명은 공간 공유, 복제·스냅숏, 쓰기 시 복사, 암호화, 디렉터리 크기 관리, 원자적 안전 저장, I/O 병합을 소개한다. 컨테이너의 가용 공간을 여러 파일 시스템이 공유하고, 작은 쓰기를 큰 쓰기로 묶어 NVM 성능을 돕는 관계를 설명한다. 이 기능 목록의 상세 동작을 외부 지식으로 보충하지 않는다.

14.9 요약과 원문 연습문제의 점검 범위

출처: 책 650-653쪽 / 원문 PDF 587-590쪽

출처: 책 650-653쪽 / 원문 PDF 587-590쪽

원문 요약은 계층·할당·가용 공간·디렉터리와 안정성의 관계를 다시 정리한다. 연습문제는 블록 추가·제거에 따른 할당 방식별 I/O, 비트맵의 저장, 할당 선택 기준, 확장 영역, 캐시의 한계, 동적 테이블의 이익·비용을 다룬다. 원문에 없는 횟수 계산이나 풀이를 생성하지 않았다. 추가 자료와 참고문헌의 존재를 확인하고 외부 설명은 사용하지 않았다.

핵심 정리

- 파일 인터페이스와 실제 블록 배치는 계층별로 연결되며 FCB·디렉터리·열린 테이블의 역할이 다르다.
- 연속·연결·색인의 차이는 접근 속도뿐 아니라 확장·단편화·포인터·복구에 걸쳐 있다.
- 원문의 비트맵은 1=가용, 0=할당이다.
- TRIM은 파일 시스템의 해제 사실을 NAND 장치에 알려 미리 공간을 준비하게 한다.
- 캐시·비동기 쓰기는 빠른 반환과 실제 기록 완료를 구별해야 하며 로그·스냅숏·백업의 목적도 다르다.

혼동하기 쉬운 내용

세 가지 파일 할당

출처: 책 625-633쪽 / 원문 PDF 562-570쪽

방법	이점	중요 제약
연속	빠른 순차·직접 접근	외부 단편화·성장·선할당
연결	동적 확장·외부 단편화 회피	직접 접근·포인터 비용과 손상
색인	외부 단편화 없이 직접 접근	색인 공간·색인 접근 I/O

원문의 복구 방식

출처: 책 642-646쪽 / 원문 PDF 579-583쪽

방법	기준
일관성 검사	현재 구조를 검사해 불일치를 복구한다.
로그 기반	확약 로그의 재실행·미확약 변경의 복구를 수행한다.
새 블록 기록·스냅숏	옛 내용은 남기고 포인터를 새 내용으로 전환한다.
백업 복원	다른 매체의 전체·점증 사본에서 내용을 되살린다.

장 마무리 복습

- inode의 직접·간접 수준과 FAT의 사슬은 파일 내용의 위치를 찾는 서로 다른 구조이다.
- 가용 공간은 비트맵·연결·그룹·계수·로그/트리의 여러 방식으로 관리한다.
- HDD의 헤드 이동 최적화를 NVM에 그대로 적용하면 불필요한 CPU 비용이 될 수 있다.
- WAFL은 새 블록 기록과 루트 포인터의 보존으로 스냅숏·클론·복제를 연결한다.

제15장. 파일 시스템 내부구조

출처: 책 655-678쪽 / 원문 PDF 591-614쪽

학습 주제

- 여러 파일 시스템을 하나의 이름 공간에 연결하는 마운트와 VFS를 정리한다. 다중 사용자 공유·원격 고장·일관성 의미를 구분하고 원문의 NFS 버전 3 중심 설명으로 마운트·원격 연산·경로 변환·캐싱을 따라간다.

설명의 흐름

1. 파일 시스템의 다양성 → 마운트·부팅 → 다중 사용자 공유 → VFS → 원격 접근·인증·고장 → 일관성 의미 → NFS의 프로토콜과 캐시

15.1 파일 시스템

출처: 책 655-657쪽 / 원문 PDF 591-593쪽

출처: 책 655-657쪽 / 원문 PDF 591-593쪽

저장장치는 파티션을, 파티션·장치들의 볼륨은 파일 시스템을 보유한다. 볼륨 관리자는 여러 파티션에 걸친 볼륨을 제공할 수 있다. 한 컴퓨터에는 여러 종류의 파일 시스템이 함께 존재한다. 범용 저장뿐 아니라 메인 메모리의 임시 tmpfs, 커널 심볼을 노출하는 objfs, 프로세스 정보를 제공하는 procfs, 다른 파일 시스템을 다시 보이게 하는 lofs 등 특수 목적도 있다. 일부 가상 파일 시스템은 실제 디스크 파일이 아니라 커널 인터페이스를 파일처럼 보이게 한다.

15.2 파일 시스템 마운팅

출처: 책 657-659쪽 / 원문 PDF 593-595쪽

출처: 책 657-659쪽 / 원문 PDF 593-595쪽

원문의 마운트 절차

1. 장치 이름과 기존 이름 구조에 부착할 마운트 포인트를 지정한다. 시스템에 따라 파일 시스템 유형도 지정하거나 구조를 검사하여 알아낸다.
2. 장치 드라이버를 통해 디렉터리를 읽어 유효한 파일 시스템 형식인지 확인한다.
3. 지정 지점에 마운트되었다는 정보를 기록한다. 경로 순회는 그 지점에서 알맞은 파일 시스템으로 넘어간다.

출처: 책 657-659쪽 / 원문 PDF 593-595쪽

마운트 전에는 새 볼륨의 파일을 기존 이름 공간에서 사용할 수 없다. 원문의 /users 예는 기존 하부 구조 대신 새 볼륨의 하부 트리가 보이며 언마운트하면 원래 구조가 다시 보이는 것을 설명한다. 비어 있지 않은 디렉터리의 마운트를 금지할지, 원래 내용을 가릴지, 같은 파일 시스템을 여러 곳에 붙일지는 시스템의 의미와 정책에 따른다.

출처: 책 657-659쪽 / 원문 PDF 593-595쪽

그림 15.4 /users에 마운트 된 볼륨

그림·표 출처: 책 658쪽 / 원문 PDF 594쪽

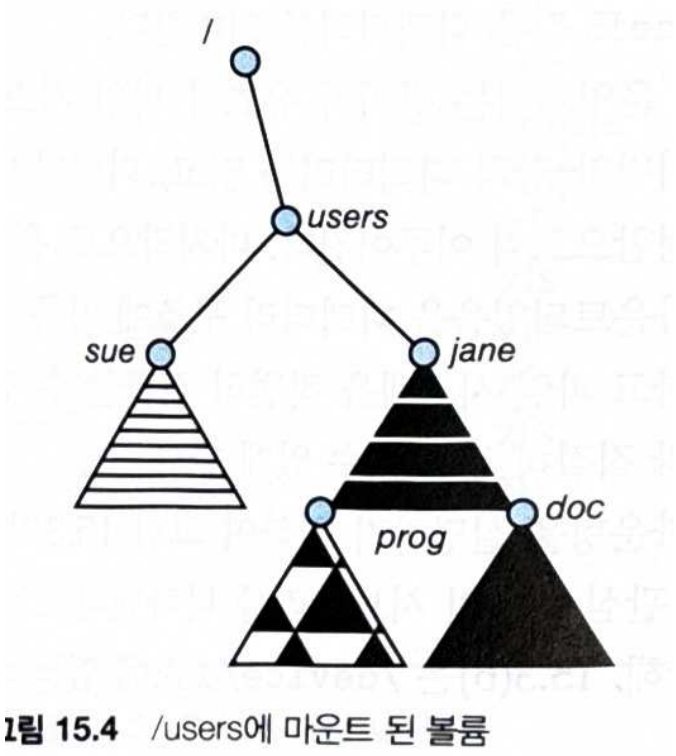


그림 15.4 /users에 마운트 된 볼륨 - 원문 그림 부분 발췌

기존 이름 공간의 /users가 새 볼륨의 하위 파일·디렉터리를 제공하는 마운트 결과이다.

출처: 책 657-659쪽 / 원문 PDF 593-595쪽

원문은 macOS의 자동 발견·마운트, Windows의 드라이브 문자와 디렉터리 마운트, UNIX의 구성 파일과 명시적 명령을 비교한다. 자동 여부와 이름 표시 방식은 운영체제에 따라 달라진다.

15.3 파티션과 마운팅

출처: 책 659-660쪽 / 원문 PDF 595-596쪽

출처: 책 659-660쪽 / 원문 PDF 595-596쪽

파티션은 파일 시스템을 가진 **cooked** 형태 또는 파일 시스템 없이 고유 형식을 사용하는 **raw** 형태일 수 있다. 스왑·일부 데이터베이스·RAID 관리 정보가 raw 사용의 예이다. 부팅할 때는 아직 일반 파일 시스템 코드가 적재되지 않았으므로 부트 정보가 별도 형식이어야 하며 부트스트랩 로더가 커널을 찾아 적재할 만큼 파일 시스템을 이해해야 한다.

출처: 책 659-660쪽 / 원문 PDF 595-596쪽

여러 파티션에 여러 운영체제를 둘 수 있어 부트 로더가 어느 것을 시작할지 선택한다. 로더가 이해하지 못하는 파일 시스템의 운영체제는 시작할 수 없어 루트 파일 시스템 선택에 제약이 된다. 루트 파티션은 부팅 때 마운트하고 다른 파티션은 자동 또는 나중에 수동으로 마운트한다.

출처: 책 659-660쪽 / 원문 PDF 595-596쪽

마운트 테이블은 유형과 장치를 연결한다. 원문의 Windows 예는 드라이브 문자에 파일 시스템 포인터를 연관시킨다. UNIX는 마운트 포인트 디렉터리의 메모리 inode에 플래그와 마운트 테이블 포인터를 두고 그 항목이 해당 파일 시스템의 슈퍼블록을 가리킨다. 경로 순회는 이 정보를 통해 파일 시스템을 전환한다.

15.4 파일 공유

출처: 책 660-661쪽 / 원문 PDF 596-597쪽

15.4.1 파일 공유와 다수의 사용자

출처: 책 660-661쪽 / 원문 PDF 596-597쪽

출처: 책 660-661쪽 / 원문 PDF 596-597쪽

공유는 공동 작업을 돕지만 이름·보호·상충되는 쓰기를 조정해야 한다. 파일·디렉터리의 소유자와 그룹 ID를 저장하고 요청한 사용자의 ID·그룹 ID와 비교하여 연산을 허용·거부한다. 소유자는 속성·허가를 제어하며 그룹은 공유 사용자 집합을 정한다.

출처: 책 660-661쪽 / 원문 PDF 596-597쪽

로컬에서 여러 파일 시스템을 마운트할 때도 ID와 허가를 검사한다. 이동식 장치를 다른 컴퓨터로 가져가면 ID가 다른 사용자를 가리킬 수 있으므로 ID가 일치하거나 소유권이 적절히 재설정되도록 해야 한다. 원격 시스템에서도 같은 신원 대응 문제가 확장된다.

15.5 가상 파일 시스템

출처: 책 662-664쪽 / 원문 PDF 598-600쪽

출처: 책 662-664쪽 / 원문 PDF 598-600쪽

VFS(virtual file system)는 기본 시스템 콜을 구체적 파일 시스템 구현에서 분리한다. 위에는 open·read·write·close와 디스크립터의 인터페이스, 가운데는 VFS, 아래에는 유형별 로컬 파일 시스템 또는 원격 프로토콜 구현을 둔다. 서로 다른 구현이 함께 존재하면서 사용자는 통합 이름 공간을 탐색할 수 있다.

출처: 책 662-664쪽 / 원문 PDF 598-600쪽

그림 15.5 가상 파일 시스템의 개략적 모습

그림·표 출처: 책 663쪽 / 원문 PDF 599쪽

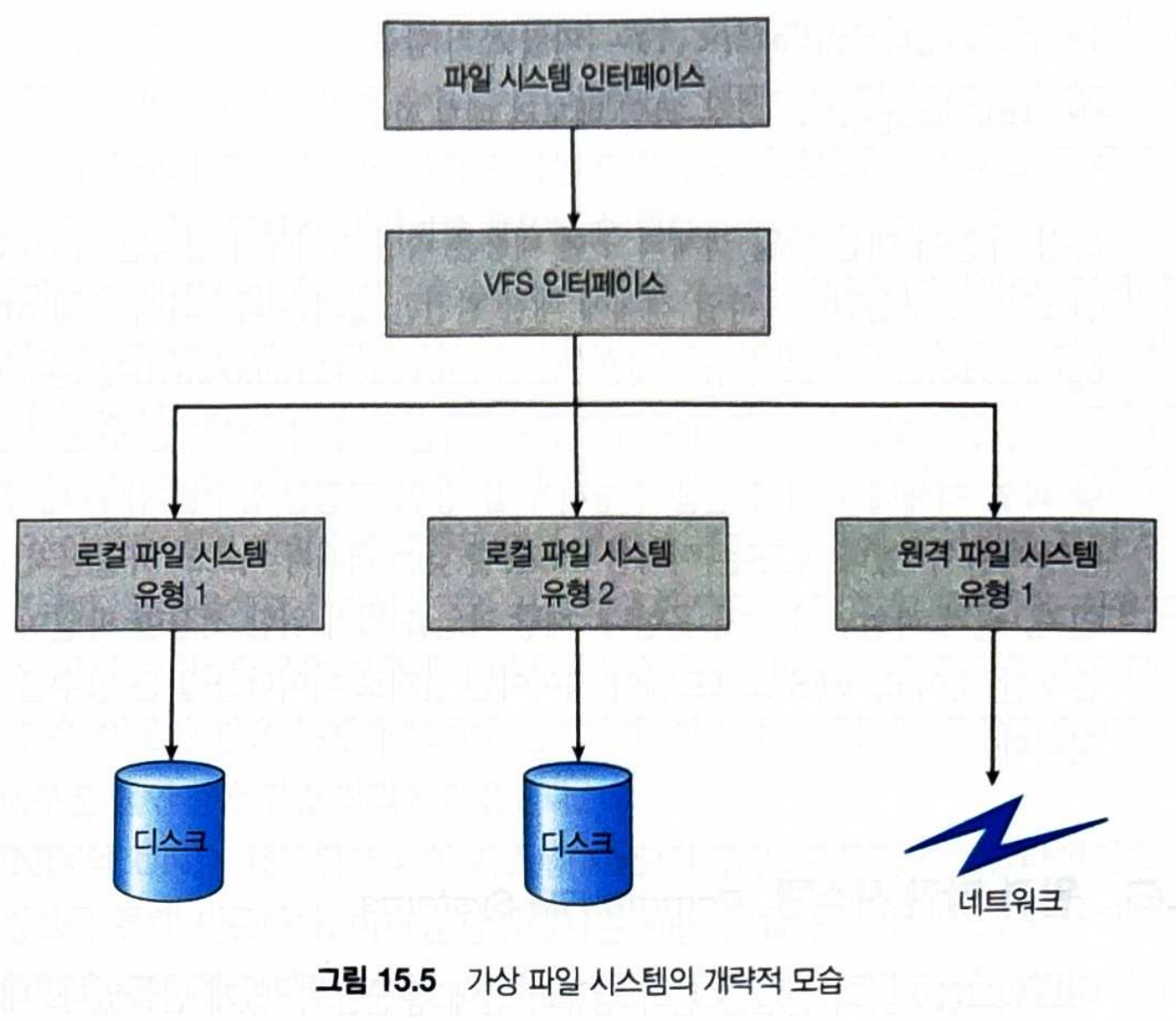


그림 15.5 가상 파일 시스템의 개략적 모습

그림 15.5 가상 파일 시스템의 개략적 모습 - 원문 그림 부분 발췌

동일한 파일 인터페이스가 VFS를 거쳐 여러 로컬 파일 시스템과 원격 파일 시스템에 연결된다.

출처: 책 662-664쪽 / 원문 PDF 598-600쪽

VFS의 **vnode**는 활성 파일·디렉터리를 나타내며 원문은 개별 파일 시스템 안에서 유일한 inode와 네트워크에서 파일을 구별해야 하는 vnode의 역할을 구분한다. 로컬 요청은 해당 파일 시스템 함수로, 원격 요청은 NFS 등의 프로토콜로 보낸다. 객체의 연산 함수 테이블이 실제 구현의 주소를 담으므로 상위 층은 적절한 함수를 선택한다.

출처: 책 662-664쪽 / 원문 PDF 598-600쪽

Linux VFS 객체	원문의 표현 대상
Inode object	각 파일
File object	열린 파일
Superblock object	전체 파일 시스템
Dentry object	각 디렉터리 항목

출처: 책 662-664쪽 / 원문 PDF 598-600쪽

파일 객체의 함수 집합에는 open·close·read·write·mmap 등이 있다. VFS는 파일의 구체적 종류마다 처리 코드를 직접 중복하기보다 해당 객체의 테이블에서 연산을 찾아 호출한다.

15.6 원격 파일 시스템

출처: 책 664쪽 / 원문 PDF 600쪽

출처: 책 664쪽 / 원문 PDF 600쪽

원격 공유의 방법으로 사용자가 ftp 등으로 직접 전송하는 방식, 원격 디렉터리를 통합하여 접근하는 **DFS(분산 파일 시스템)**, 웹·클라우드를 통한 접근이 소개된다. 익명 전송은 서버 계정 없이 가능할 수 있으나 DFS는 클라이언트와 서버의 더 긴밀한 통합을 요구하므로 복잡성이 커진다.

15.6.1 클라이언트 서버 모델

출처: 책 664-665쪽 / 원문 PDF 600-601쪽

출처: 책 664-665쪽 / 원문 PDF 600-601쪽

파일을 가진 컴퓨터는 서버, 접근하려는 컴퓨터는 클라이언트이다. 서버는 제공하는 볼륨·디렉터리와 허용 클라이언트를 명시하며 다수 서버·다수 클라이언트 관계가 가능하다. 한 컴퓨터가 동시에 양쪽 역할을 할 수 있다.

출처: 책 664-665쪽 / 원문 PDF 600-601쪽

네트워크 이름·IP만의 식별은 스푸핑에 취약하다. 암호화된 키를 쓰는 인증도 알고리즘 호환과 안전한 키 교환이 필요하다. 원문의 UNIX/NFS 사례에서는 클라이언트·서버의 사용자 ID가 일치해야 파일 접근 검사도 올바르다. 같은 사용자가 양쪽에서 서로 다른 ID를 가지면 서버가 다른 사람의 권한을 검사할 수 있다. 마운트 뒤 파일 연산은 사용자 신원과 함께 프로토콜로 전달되고 서버가 권한을 판단한다.

15.6.2 분산 정보 시스템

출처: 책 665-666쪽 / 원문 PDF 601-602쪽

출처: 책 665-666쪽 / 원문 PDF 601-602쪽

분산 정보·네이밍 서비스는 원격 컴퓨팅에 필요한 정보를 통일된 방식으로 제공한다. DNS는 호스트 이름을 네트워크 주소로 변환한다. 사용자 이름·암호·UID·GID·프린터 등의 정보도 중앙에서 관리할 수 있다. 원문의 NIS는 정보 집중을 제공하지만 인증의 안전성에 문제가 있고 NIS+는 더 안전하되 복잡한 사례이다.

출처: 책 665-666쪽 / 원문 PDF 601-602쪽

CIFS의 네트워크 로그인과 Active Directory, Kerberos 방식 인증, LDAP가 원문의 다른 사례이다. 통일된 사용자·자원 이름 공간은 시스템마다 흩어진 정보를 합쳐 관리 부담을 줄이고 한 번의 인증으로 조직 자원에 접근하는 단일 사인온을 가능하게 한다. 이 설명은 원문의 서비스 관계에 한정하며 새로운 구성 절차를 추가하지 않는다.

15.6.3 고장 모드

출처: 책 666-667쪽 / 원문 PDF 602-603쪽

출처: 책 666-667쪽 / 원문 PDF 602-603쪽

로컬 시스템도 매체·컨트롤러·메타데이터·케이블·관리 실수로 실패할 수 있다. 원격 시스템은 여기에 네트워크 고장·잘못된 구성·서버 중단·분할 등이 더해진다. 서버를 잃었을 때 요청을 종료할지 재개까지 미룰지는 프로토콜의 오류 의미이다. 즉시 종료하면 데이터 손실이 생길 수 있어 많은 DFS는 기다리거나 연기하는 방식을 제공한다.

출처: 책 666-667쪽 / 원문 PDF 602-603쪽

상태성 방식은 열린 파일·활동을 양쪽이 기억하여 복구에 이용한다. 원문의 **NFS 버전 3** 설명은 각 요청에 필요한 정보를 담는 무상태 방식을 채택하여 구현과 재시작을 단순하게 하지만 신원·거짓 요청의 보안 문제가 남는다고 설명한다. 버전 4는 보안·성능·기능 개선을 위해 상태를 도입한 것으로 소개된다. 뒤의 상세 NFS 절 역시 책이 명시한 버전 3 중심의 범위를 유지한다.

15.7 일관성의 의미

출처: 책 668쪽 / 원문 PDF 604쪽

출처: 책 668쪽 / 원문 PDF 604쪽

일관성 의미는 공유 파일의 한 사용자 변경이 다른 사용자에게 언제 보이는지 규정한다. 파일 시스템 코드로 구현되며 프로세스 동기화와 관련된다. 그러나 디스크·네트워크의 지연 때문에 복잡한 원자 연산을 모두 제공하면 성능 비용이 크다. 원문의 비교는 open과 close 사이의 접근을 하나의 **파일 세션**으로 가정한다.

15.7.1 UNIX의 의미

출처: 책 668쪽 / 원문 PDF 604쪽

출처: 책 668쪽 / 원문 PDF 604쪽

열린 파일에 대한 쓰기는 같은 파일을 연 다른 사용자에게 즉시 보일 수 있다. 현재 위치 포인터를 공유하는 모드에서는 한 사용자의 위치 전진이 모든 공유자에 영향을 준다. 파일은 접근이 서로 엇갈려 적용되는 단일 물리 이미지를 가지며 이 배타적 자원에 대한 경쟁은 대기를 낳을 수 있다.

15.7.2 세션 의미

출처: 책 668-669쪽 / 원문 PDF 604-605쪽

출처: 책 668-669쪽 / 원문 PDF 604-605쪽

원문의 Andrew/OpenAFS **세션 의미**에서는 한 사용자의 쓰기가 동시에 열어 둔 다른 사용자에게 즉시 보이지 않는다. 닫은 뒤에 시작되는 세션에서 변경이 보이고 이미 열린 인스턴스에는 반영되지 않는다. 한 파일에 일시적으로 서로 다른 이미지가 존재할 수 있어 여러 사용자가 각 이미지에서 병행하여 읽고 쓸 수 있다.

15.7.3 불변 공유 파일의 의미

출처: 책 669쪽 / 원문 PDF 605쪽

출처: 책 669쪽 / 원문 PDF 605쪽

공유된다고 선언한 파일은 더 이상 변경하지 못한다. 파일 이름도 재사용하지 않고 내용도 바꾸지 않아 이름이 고정된 내용을 가리킨다. 공유 규칙이 읽기 전용으로 명확하여 분산 환경에서 구현이 단순하다.

15.8 NFS

출처: 책 669-675쪽 / 원문 PDF 605-611쪽

15.8.1 NFS 개요

출처: 책 669-671쪽 / 원문 PDF 605-607쪽

출처: 책 669-671쪽 / 원문 PDF 605-607쪽

NFS는 원격 파일 접근의 소프트웨어 구현과 명세를 뜻한다. 원문은 Solaris 구현의 예와 일반 명세 설명을 함께 사용하며 **버전 3을 중심으로** 다룬다고 명시한다. 책의 "최신 버전"·배포 빈도 설명을 현재 상황으로 갱신하지 않는다. 독립된 시스템들이 클라이언트·서버 관계로 투명하게 파일을 공유하는 것이 목표이며 서버 고장은 관련 클라이언트에 영향을 준다.

출처: 책 669-671쪽 / 원문 PDF 605-607쪽

마운트 명령에는 원격 호스트·디렉터리를 명시해야 하지만 완료 뒤에는 로컬 하부 트리처럼 투명하게 접근한다. 클라이언트의 기존 마운트 포인트 내용은 가려진다. 일부 구현은 원격 마운트 안에 다른 시스템을 다시 마운트하는 연속 마운트를 허용한다. 그러나 서버가 이미 마운트한 다른 원격 파일 시스템까지 자동으로 보이는 **이행성은 없다**. 각 기계는 자신이 수행한 마운트에 의해서만 영향을 받는다.

출처: 책 669-671쪽 / 원문 PDF 605-607쪽

그림 15.7 NFS에서의 마운팅

그림·표 출처: 책 671쪽 / 원문 PDF 607쪽

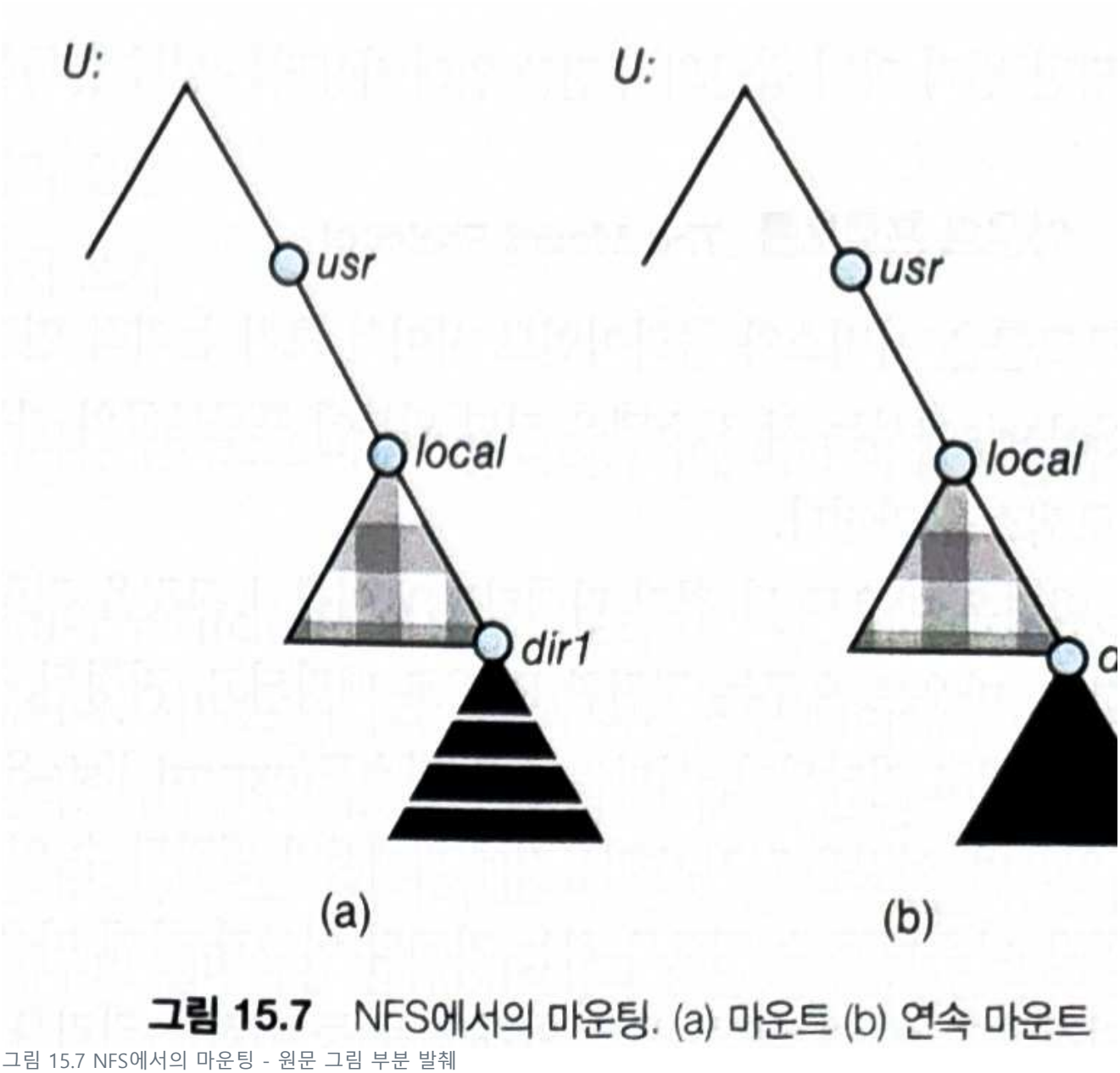


그림 15.7 NFS에서의 마운팅. (a) 마운트 (b) 연속 마운트

그림 15.7 NFS에서의 마운팅 - 원문 그림 부분 발췌

로컬 /usr/local에 원격 하위 트리를 마운트한 상태와 그 안에 다시 명시적으로 연속 마운트한 상태를 비교한다.

출처: 책 669-671쪽 / 원문 PDF 605-607쪽

서로 다른 하드웨어·운영체제·네트워크를 지원하기 위해 RPC와 **XDR(외부 자료 표현)**을 이용한다. 서비스는 초기 연결의 마운트 프로토콜과 실제 원격 연산의 NFS 프로토콜로 나뉜다. 공유 홈 디렉터리를 여러 기계에서 마운트하면 사용자 이동성도 제공할 수 있다.

15.8.2 마운트 프로토콜

출처: 책 672쪽 / 원문 PDF 608쪽

출처: 책 672쪽 / 원문 PDF 608쪽

NFS 마운트

1. 클라이언트가 서버 이름과 원격 디렉터리를 지정한 마운트 요구를 RPC로 보낸다.
2. 서버는 수출 리스트의 디렉터리·허가된 기계·접근 권리를 확인한다.
3. 허용하면 파일 시스템 식별자와 디렉터리 inode 번호 등 접근에 필요한 정보를 담은 파일 핸들을 반환한다.
4. 클라이언트는 이 핸들을 이후 원격 파일 접근에 사용한다.

출처: 책 672쪽 / 원문 PDF 608쪽

서버는 관리용 클라이언트·마운트 목록도 유지할 수 있으나 이는 실제 파일 연산의 정확성을 위한 상태와 다르다. 마운트 구성은 부팅 때 정할 수 있고 변경도 가능하며 언마운트·수출 리스트 조회 절차도 제공된다.

15.8.3 NFS 프로토콜

출처: 책 672-674쪽 / 원문 PDF 608-610쪽

출처: 책 672-674쪽 / 원문 PDF 608-610쪽

NFS 프로토콜은 디렉터리 검색·항목 읽기·링크/디렉터리 조작·속성 접근·파일 읽기/쓰기의 RPC를 제공한다. 원문이 설명하는 무상태 서비스에는 **open·close가 의도적으로 없다**. 각 요청이 고유 파일 식별자와 절대 오프셋 등 필요한 매개변수를 모두 가져야 한다. 재시도에서 같은 연산을 여러 번 적용해도 한 번 적용한 결과와 같도록 멱등성을 고려하고 일련번호로 중복·누락을 구별한다.

출처: 책 672-674쪽 / 원문 PDF 608-610쪽

마운트의 관리 목록은 크래시 뒤 복원하지 않아도 정확한 연산이 가능하므로 무상태 원칙과 모순되는 필수 상태로 보지 않는다. 원문은 클라이언트에 결과를 돌려주기 전에 서버의 수정 데이터·간접/상태 블록이 안전한 저장장치에 기록되어야 한다고 설명한다. 동기 기록의 비용을 줄이기 위해 배터리로 보호된 비휘발성 캐시가 완료를 먼저 보장하고 나중에 디스크에 기록할 수 있다.

출처: 책 672-674쪽 / 원문 PDF 608-610쪽

하나의 NFS 쓰기 RPC는 다른 쓰기와 섞이지 않지만 사용자 write 시스템 콜은 여러 RPC로 나뉠 수 있다. 따라서 전체 시스템 콜의 데이터가 다른 사용자의 쓰기와 섞일 가능성이 남는다. 원문은 별도의 락 서비스·협조적 접근이 필요하다고 설명한다. 원문의 전송 크기와 프로토콜 조건은 해당 설명 범위의 예이며 현재 네트워크의 일반 제한으로 확대하지 않는다.

출처: 책 672-674쪽 / 원문 PDF 608-610쪽

VFS를 통한 원격 연산

1. 클라이언트의 일반 시스템 콜을 vnode에 대한 VFS 연산으로 변환한다.
2. VFS가 원격 파일임을 확인해 NFS 프로시저를 호출한다.
3. RPC/XDR로 서버의 NFS 서비스에 전송한다.
4. 서버 VFS가 로컬 파일 시스템 연산으로 처리하고 같은 경로를 되돌아 결과를 보낸다.

출처: 책 672-674쪽 / 원문 PDF 608-610쪽

그림 15.8 NFS 구조의 개략도

그림·표 출처: 책 674쪽 / 원문 PDF 610쪽

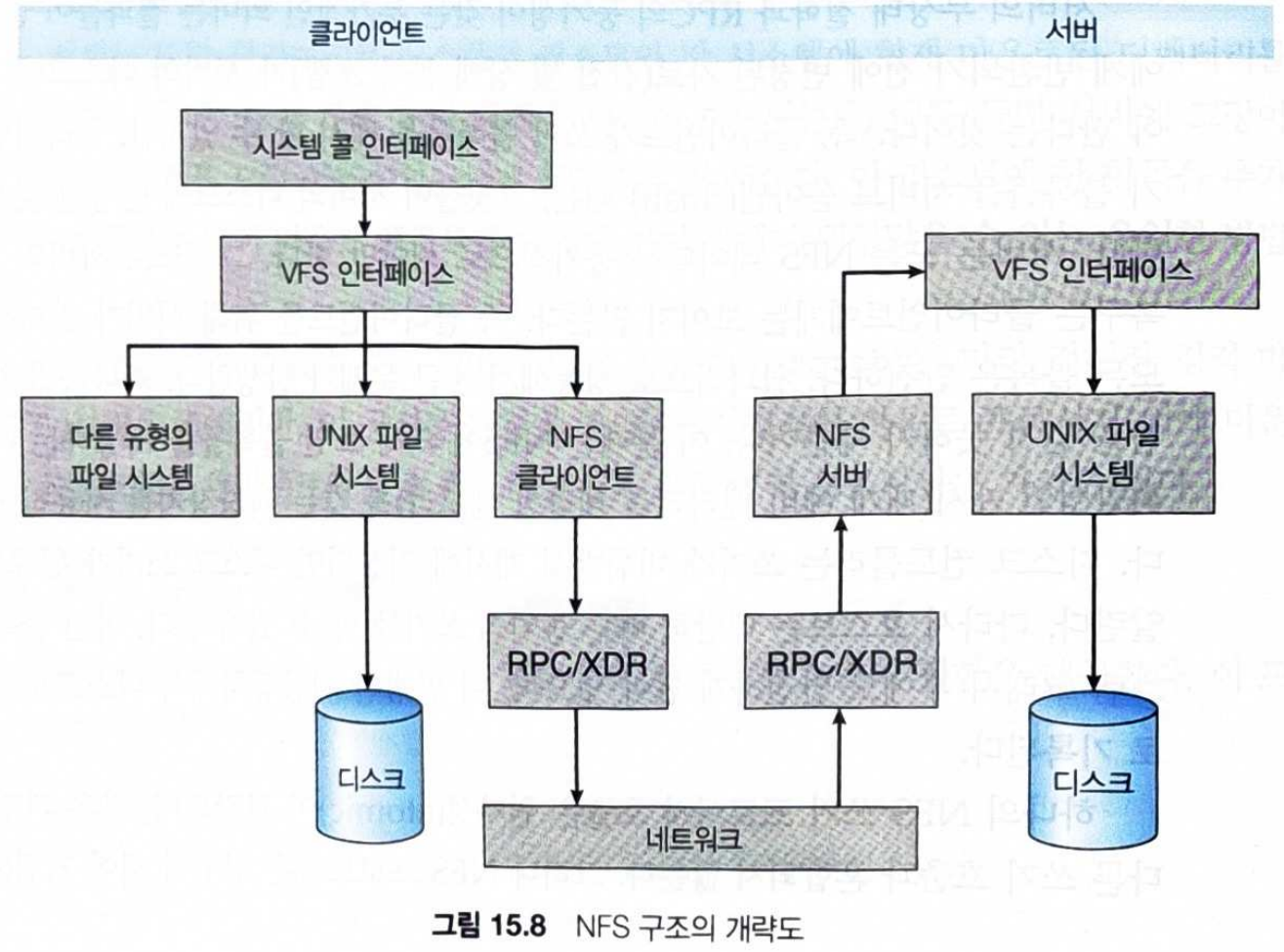


그림 15.8 NFS 구조의 개략도

그림 15.8 NFS 구조의 개략도 - 원문 그림 부분 발췌

클라이언트·서버 양쪽 VFS와 로컬 파일 시스템, NFS, RPC/XDR의 호출 연결을 보존한다.

15.8.4 경로 이름 변환

출처: 책 675쪽 / 원문 PDF 611쪽

출처: 책 675쪽 / 원문 PDF 611쪽

경로를 구성요소 이름으로 나누고 각 이름과 디렉터리 vnode 쌍에 대해 lookup한다. 마운트 포인트를 넘은 뒤의 각 구성요소는 별도 서버 RPC를 요구할 수 있다. 경로 전체를 한꺼번에 서버에 주면 빠를 수 있지만 클라이언트마다 다른 마운트 이름 공간과 중간의 추가 마운트를 무상태 서버가 알지 못한다는 제약이 있다.

출처: 책 675쪽 / 원문 PDF 611쪽

클라이언트의 디렉터리 이름 조회 캐시는 공통 접두어를 가진 경로의 반복 참조를 빠르게 한다. 반환된 속성이 캐시된 vnode와 맞지 않으면 캐시를 버린다. 클라이언트가 명시적으로 연속 마운트한 경로는 여러 서버를 지날 수 있지만 서버 자체의 다른 마운트를 자동으로 따라가는 것은 아니다.

15.8.5 원격 연산

출처: 책 675-676쪽 / 원문 PDF 611-612쪽

출처: 책 675-676쪽 / 원문 PDF 611-612쪽

개념상 파일 연산과 RPC가 대응하지만 실제로는 파일 블록·속성을 지역에 캐시하여 여러 연산이 RPC 없이 처리될 수 있다. 원문은 열린 시점에 속성의 최신 여부를 서버에 확인하고, 속성이 최신일 때만 관련 블록 캐시를 쓰며, 속성 캐시는 일정 시간 뒤 버린다고 설명한다. 미리 읽기와 지연 쓰기를 함께 사용하되 서버 기록 완료를 확인하기 전까지 지연 쓰기 블록을 해제하지 않는다.

출처: 책 675-676쪽 / 원문 PDF 611-612쪽

원문의 수치 사례에서 속성 캐시는 기본 60초 뒤 버리며, 한 기계에서 만든 새 파일은 다른 기계에서 30초 동안 보이지 않을 수 있다고 설명한다. 이 수치는 본문의 NFS 구현·성능 조정 사례를 그대로 기록한 것이며 모든 버전과 설정의 공통값이라는 의미로 사용하지 않는다.

출처: 책 675-676쪽 / 원문 PDF 611-612쪽

캐시와 지연 쓰기로 인해 다른 기계의 새 파일·수정이 곧바로 보이지 않을 수 있다. 이미 열린 파일에서 보이는 내용과 새 open에서 보이는 내용도 구별해야 한다. 따라서 원문의 NFS는 엄격한 UNIX 의미도 Andrew의 세션 의미도 그대로 제공하지 않는다고 결론짓는다. 성능을 위한 조정이 일관성 의미에 미치는 영향이 핵심이다.

15.9 요약과 원문 연습문제의 점검 범위

출처: 책 676-678쪽 / 원문 PDF 612-614쪽

출처: 책 676-678쪽 / 원문 PDF 612-614쪽

다양한 파일 시스템은 마운트와 VFS를 통해 통합되고, 파일 공유에는 신원·권한·일관성 의미가 필요하다. 원격 공유는 통신·인증·고장·경로 변환과 캐시 일관성까지 고려한다. 장말 문제는 VFS의 역할, 여러 파일 시스템 유형의 필요, procfs, 이름 공간 통합과 별도 표시, ftp와 NFS의 차이를 점검한다. 추가 자료·참고문헌은 확인했지만 원문에 없는 실습 명령이나 풀이를 작성하지 않았다.

핵심 정리

- 마운트는 파일 시스템을 이름 공간에 부착하고 VFS는 서로 다른 구현의 연산을 공통 인터페이스로 전달한다.
- 파일 소유자·그룹의 ID는 다른 장치·원격 기계에서도 올바르게 대응해야 한다.
- 파일 세션은 open부터 close까지이고 일관성 의미는 변경의 관찰 시점을 정한다.
- 원문의 NFS 상세 설명은 버전 3 중심이다. 무상태 요청과 관리용 마운트 목록의 성격을 구분한다.
- NFS의 단일 RPC 원자성과 응용 write 전체, 캐시 성능과 즉시 가시성을 혼동하지 않는다.

혼동하기 쉬운 내용

원문의 파일 공유 일관성 의미

출처: 책 668-669쪽 / 원문 PDF 604-605쪽

모델	변경이 보이는 시점·조건
UNIX	같이 열린 파일의 다른 사용자에게 즉시 보일 수 있다. 단일 이미지이다.
세션	닫은 이후 새로 시작한 세션에서 보이며 기존 열린 인스턴스에는 반영되지 않는다.
불변 공유	공유 선언 후 내용 변경·이름 재사용이 없다.

장 마무리 복습

- 부팅·파티션·볼륨·마운트는 커널을 찾고 여러 저장장치를 하나의 경로로 제공하는 연결이다.
- VFS는 함수 테이블로 로컬·원격 구현을 선택하고 NFS는 RPC/XDR를 통해 서버 VFS로 이어진다.
- 무상태성은 요청 자체의 완전한 정보를 요구하며 캐싱·지연 쓰기는 일관성 의미를 바꾼다.

제7부 도입. 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

학습 주제

- 제공된 마지막 도입 페이지에서 보안과 보호의 역할을 구별한다.

P7.1 도입에서 확인되는 보안과 보호의 역할

출처: 책 679쪽 / 원문 PDF 615쪽

보안은 저장된 자료와 코드의 무결성뿐 아니라 물리 자원의 보호를 위해 사용자 인증을 보장한다. 인가되지 않은 접근, 접근 후의 악의적인 파괴·자료 변경, 우연한 비일관성 도입을 막는 역할을 한다.

보호는 사용자의 파일 접근 유형을 제한해 제어된 접근을 제공한다. 운영체제에서 적절한 권한을 얻은 프로세스만 메모리 세그먼트·CPU 등의 자원에 작업할 수 있도록 한다. 프로그램·프로세스·사용자의 자원 접근을 제어하는 기법에는 부과할 제어를 지정하는 방법과 시행하는 방법이 필요하다.

이 페이지는 제7부의 도입이다. 뒤의 제16장 보안과 제17장 보호 본문은 제공되지 않아, 구체적인 위협·암호 기법·접근 행렬 등의 설명은 원문에서 확인되지 않음으로 남긴다.

핵심 정리

- 보안은 사용자 인증과 인가되지 않은 접근·자료 훼손의 방지를 다룬다.
- 보호는 권한에 따른 자원 접근 제어를 다룬다. 제어를 지정하고 시행하는 방법이 필요하다.

혼동하기 쉬운 내용

도입에서 구별한 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

개념	도입에 제시된 역할
보안	사용자 인증, 무결성과 물리 자원 보호, 인가되지 않은 접근과 파괴·변경 방지
보호	파일 접근 유형 제한과 권한에 따른 자원 접근 제어, 제어의 지정과 시행

장 마무리 복습

- 보안과 보호의 역할 구별은 제공된 도입의 범위에서만 복습한다.

원문 확인 메모

- 제16~21장 및 부록 본문은 미제공이다. 제7부 도입만으로 해당 장의 내용을 보충하지 않았다.

비교·공식·절차 모음

제1장. 서론

다중 프로그래밍과 다중 태스킹

출처: 책 24-26쪽 / 원문 PDF 28-30쪽

항목	다중 프로그래밍	다중 태스킹
목적	실행할 일이 있으면 CPU를 계속 사용	프로세스를 빈번하게 전환하여 빠른 응답
관계	여러 프로그램을 메모리에 유지	다중 프로그래밍의 논리적 확장

제3장. 프로세스

IPC의 기본 모델

출처: 책 137-139쪽 / 원문 PDF 84-86쪽

항목	공유 메모리	메시지 전달
교환 단위	공유 영역의 데이터	송수신 메시지
커널 관여	영역 설정 후 일반 메모리 접근	통상 각 통신에 시스템 콜 사용
책임	자료 형식과 동시 접근 조정	이름·봉쇄·버퍼링 규칙 선택

좀비와 고아

출처: 책 135-136쪽 / 원문 PDF 82-83쪽

항목	좀비	고아
핵심 조건	종료한 자식의 상태를 부모가 아직 수집하지 않음	부모가 자식보다 먼저 종료
처리	부모의 wait가 항목을 회수	다른 부모가 인계받아 종료 상태를 수집

제4장. 스레드와 병행성

데이터 병렬과 태스크 병렬

출처: 책 181-182쪽 / 원문 PDF 127-128쪽

기준	데이터 병렬	태스크 병렬
분할 대상	데이터 부분집합	서로 다른 작업
코어별 연산	동일	고유 연산
관계	두 방법을 혼합할 수 있다.	두 방법을 혼합할 수 있다.

비동기 취소와 지연 취소

출처: 책 208-210쪽 / 원문 PDF 154-156쪽

기준	비동기	지연
시점	즉시 종료	안전한 취소 점에서 요청을 확인
자원·공유 상태	갱신 중 중단·자원 미반환 위험	정리 기회를 제공

제5장. CPU 스케줄링

기본 CPU 스케줄링 선택 규칙

출처: 책 226-239쪽 / 원문 PDF 172-185쪽

알고리즘	선택 기준	주의
FCFS	도착·요청 순	비선점·호위 효과
SJF/SRTF	짧은 다음 버스트/남은 시간	예측이 필요함
RR	같은 q를 순환 배정	q와 문맥 교환의 절충
우선순위	가장 높은 순위	숫자 방향 확인·기아와 노화
다단계 큐	상위 큐 우선 또는 큐별 몫	소속이 일반적으로 고정
피드백 큐	실행 특성·대기에 따라 이동	승격·강등 등 매개변수 선택 필요

실시간 알고리즘의 순위

출처: 책 253-256쪽 / 원문 PDF 199-202쪽

기준	RM	EDF
우선순위	주기 짧을수록 높음·정적	기한 빠를수록 높음·동적
대상 조건	고정 수행 시간의 주기 태스크	주기·고정 수행 시간을 반드시 요구하지 않음
필요 정보	주기와 수행 시간	실행 가능 시 마감시간 통지

평가 방법의 범위

출처: 책 269-274쪽 / 원문 PDF 215-220쪽

방법	정보와 장점	한계
결정론적	정확한 고정 입력·빠르고 명확	해당 입력에 국한
큐잉	확률 분포로 평균 계산	분포·독립 가정의 근사
모의실험	모형과 추적 파일로 실제 순서 반영	시간·저장·모형 개발 비용
구현	실제 시스템 환경에서 실행	코드 변경·회귀 시험·환경 변화 비용

제12장. 입출력 시스템

봉쇄형·비봉쇄형·비동기식 I/O

출처: 책 554-556쪽 / 원문 PDF 493-495쪽

방식	반환과 완료
봉쇄형	완료를 기다려 결과를 받는다.
비봉쇄형	지금 가능한 전송량을 반환한다.
비동기식	먼저 돌아온 후 전체 요청 완료를 별도로 알린다.

버퍼·캐시·스풀

출처: 책 557-560쪽 / 원문 PDF 496-499쪽

개념	목적
버퍼	전송 중 임시 보관, 속도·크기 차이와 복제 시맨틱 지원
캐시	다른 곳에 있는 데이터의 빠른 사본으로 재사용
스풀	섞어 처리할 수 없는 장치에 작업별 출력을 모아 순차 전달

제13장. 파일 시스템 인터페이스

디렉터리 구조의 변화

출처: 책 594-603쪽 / 원문 PDF 532-541쪽

구조	해결·허용하는 것	새 제약
1단계	단순한 한 디렉터리	모든 파일명 고유
2단계	사용자별 같은 이름	공유와 경로 표현 필요
트리	사용자 하위 디렉터리·계층화	물리적 파일·하위 디렉터리 공유 제한
비순환 그래프	공유 허용	중복 순회·참조·삭제 관리
일반 그래프	순환까지 허용하는 유연성	탐색 종료·도달성·가비지 수집 문제

제14장. 파일 시스템 구현

세 가지 파일 할당

출처: 책 625-633쪽 / 원문 PDF 562-570쪽

방법	이점	중요 제약
연속	빠른 순차·직접 접근	외부 단편화·성장·선할당
연결	동적 확장·외부 단편화 회피	직접 접근·포인터 비용과 손상
색인	외부 단편화 없이 직접 접근	색인 공간·색인 접근 I/O

원문의 복구 방식

출처: 책 642-646쪽 / 원문 PDF 579-583쪽

방법	기준
일관성 검사	현재 구조를 검사해 불일치를 복구한다.
로그 기반	확약 로그의 재실행·미확약 변경의 복구를 수행한다.
새 블록 기록·스냅숏	옛 내용은 남기고 포인터를 새 내용으로 전환한다.
백업 복원	다른 매체의 전체·점증 사본에서 내용을 되살린다.

제15장. 파일 시스템 내부구조

원문의 파일 공유 일관성 의미

출처: 책 668-669쪽 / 원문 PDF 604-605쪽

모델	변경이 보이는 시점·조건
UNIX	같이 열린 파일의 다른 사용자에게 즉시 보일 수 있다. 단일 이미지이다.
세션	닫은 이후 새로 시작한 세션에서 보이며 기존 열린 인스턴스에는 반영되지 않는다.
불변 공유	공유 선언 후 내용 변경·이름 재사용이 없다.

제7부 도입. 보안과 보호

도입에서 구별한 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

개념	도입에 제시된 역할
보안	사용자 인증, 무결성과 물리 자원 보호, 인가되지 않은 접근과 파괴·변경 방지
보호	파일 접근 유형 제한과 권한에 따른 자원 접근 제어, 제어의 지정과 시행

주요 공식과 기호

제4장. 스레드와 병행성 / 4.2 다중 코어 프로그래밍

출처: 책 178-182쪽 / 원문 PDF 124-128쪽

원문 코드·기호

```
speedup ≤ 1 / (S + (1 − S) / N)
```

- S: 반드시 순차적으로 실행되어야 하는 부분
- N: 처리 코어 수

원문 예는 병렬 75%·순차 25%에서 2코어 약 1.6배, 4코어 2.28배를 제시한다. N이 무한히 커지면 1/S에 수렴하므로 순차 부분이 성능 향상을 제한한다.

제5장. CPU 스케줄링 / 5.3.2 최단 작업 우선 스케줄링

출처: 책 228-230쪽 / 원문 PDF 174-176쪽

원문 코드·기호

```
τ_{n+1} = α t_n + (1 − α)τ_n,  0 ≤ α ≤ 1
```

- t_n: n번째 실제 CPU 버스트 길이
- τ_n: 앞서 축적한 예측값
- τ_{n+1}: 다음 CPU 버스트 예측
- α: 최근 실제값과 과거 예측에 줄 상대 가중치

α=0이면 과거 예측만, α=1이면 가장 최근 버스트만 반영한다. α=1/2이면 둘에 같은 무게를 준다. 원문 그림은 α=1/2, 초기 예측 10을 사용한다.

제5장. CPU 스케줄링 / 5.3.3 라운드 로빈 스케줄링

출처: 책 230-233쪽 / 원문 PDF 176-179쪽

원문 코드·기호

```
다음 몫까지의 대기 상한: (n − 1)q
```

- n: 준비 큐의 프로세스 수
- q: 각 시간 할당량

각 프로세스가 최대 q 덩어리로 CPU의 1/n을 나누어 받는 원문 조건이다. 예를 들어 n=5, q=20ms이면 매 100ms마다 최대 20ms를 받는다.

제5장. CPU 스케줄링 / 5.6.2 우선순위 기반 스케줄링

출처: 책 252-253쪽 / 원문 PDF 198-199쪽

원문 코드·기호

```
0 ≤ t ≤ d ≤ p,  실행 빈도 = 1/p
```

- t: 주기마다 필요한 고정 수행 시간
- d: 서비스를 완료해야 하는 마감시간
- p: 반복 주기

원문 주기 태스크 모델의 관계이다. 승인 제어는 기한 내 완료 가능한 요청을 허용하고 그렇지 않으면 거절한다.

제5장. CPU 스케줄링 / 5.6.3 Rate-Monotonic 스케줄링

출처: 책 253-255쪽 / 원문 PDF 199-201쪽

원문 코드·기호

```
N(2^(1/N) − 1)
```

- N: 스케줄할 주기 프로세스 수

원문이 제시한 최악의 경우 보장 가능한 CPU 이용률 경계이다. 한 프로세스는 100%, 두 프로세스는 약 83%, 수가 커지면 약 69%로 내려간다. 경계 초과는 모든 경우 실패한다는 뜻이 아니라 보장할 수 없다는 뜻이다.

제5장. CPU 스케줄링 / 5.6.5 일정 비율의 몫 스케줄링

출처: 책 256-257쪽 / 원문 PDF 202-203쪽

원문 코드·기호

```
할당받는 CPU 시간 비율 = N/T
```

- T: 전체 시간 몫
- N: 응용에 할당한 몫

원문 예는 T=100에서 A=50·B=15·C=20을 주어 각각 50%·15%·20%를 보장한다. 승인 제어는 남은 몫 안의 요청만 허용한다.

제5장. CPU 스케줄링 / 5.8.2 큐잉 모델

출처: 책 271-272쪽 / 원문 PDF 217-218쪽

원문 코드·기호

$$n = \lambda \times W$$

- n: 서비스 중인 프로세스를 제외한 평균 큐 길이
- λ: 큐에 도착하는 평균률
- W: 큐의 평균대기 시간

Little의 공식은 원문의 안정 상태 조건에서 도착 수와 이탈 수가 같음을 이용한다. 두 변수를 알면 나머지를 구한다. 원문 예 λ=7개/초, n=14개에서 W=2초이다.

제9장. 메인 메모리 / 9.3.1 기본 방법

출처: 책 397-402쪽 / 원문 PDF 337-342쪽

원문 코드·기호

논리 주소 공간 2^m, 페이지 크기 2^n 바이트 → 상위 m−n비트: p / 하위 n비트: d

- m: 논리 주소의 비트 수
- n: 페이지 안 오프셋의 비트 수

페이지 크기는 하드웨어가 정하며 2의 거듭제곱이므로 비트로 나누어 변환할 수 있다.

제10장. 가상 메모리 / 10.2 요구 페이징

출처: 책 431-438쪽 / 원문 PDF 371-378쪽

원문 코드·기호

$$\text{실질 접근 시간} = (1-p) \times ma + p \times \text{페이지 폴트 시간}$$

- p: 페이지 폴트 확률, $0 \le p \le 1$
- ma: 메모리 접근 시간

폴트가 없을 때의 메모리 시간과 폴트가 있을 때의 처리 시간을 확률로 가중한다.

제10장. 가상 메모리 / 10.5.2 할당 알고리즘

출처: 책 455-456쪽 / 원문 PDF 395-396쪽

원문 코드·기호

$$S = \Sigma\, s_i, \quad a_i = (s_i / S) \times m$$

- s_i: 프로세스 p_i의 크기
- m: 가용 프레임 전체 수
- a_i: p_i에 할당하는 근사 프레임 수

원문의 비례 할당식. 10페이지·127페이지에 62프레임을 나누는 예는 각각 약4·57프레임으로 제시된다.

제10장. 가상 메모리 / 10.6.2 작업 집합 모델

출처: 책 464-465쪽 / 원문 PDF 404-405쪽

원문 코드·기호

$D = \Sigma WSS_i$

- WSS_i: 프로세스 i의 작업 집합 크기

- D: 전체 요구 프레임 수

- m: 시스템의 전체 메모리 프레임 수

D>m이면 지역성을 담을 프레임이 부족해 스래싱을 유발한다. 원문은 작업 집합에 맞게 할당하고 부족하면 한 프로세스를 연기하여 프레임을 나눈다.

제10장. 가상 메모리 / 10.9.3 TLB Reach

출처: 책 475-476쪽 / 원문 PDF 415-416쪽

원문 코드·기호

```
TLB reach = TLB 항목 수 × 페이지 크기
```

- TLB reach: TLB로 직접 접근 가능한 메모리 공간의 크기

작업 집합을 담을 수 있으면 좋다. 항목 수 증가는 연관 메모리의 가격·전력 비용이 있고 페이지 확대는 내부 단편화를 늘릴 수 있다.

제11장. 대용량 저장장치 구조 / 11.8.1 중복으로 신뢰성 향상

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

원문 코드·기호

```
100,000 / 100 = 1,000시간
```

원문의 단일 드라이브 MTBF가 100,000시간인 100개 드라이브 집합의 고장 간 평균 시간 예이다.

제11장. 대용량 저장장치 구조 / 11.8.1 중복으로 신뢰성 향상

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

원문 코드·기호

$100{,}000^2 / (2 \times 10) = 500 \times 10^6$시간

드라이브 고장이 독립적이고 단일 MTBF가 100,000시간, 평균 수리 시간이 10시간이라는 원문의 가정에서 얻는 평균 데이터 분실 시간이다. 원문은 약 57,000년으로 표현한다.

제11장. 대용량 저장장치 구조 / 11.8.2 병렬성을 이용한 성능 향상

출처: 책 519-520쪽 / 원문 PDF 458-459쪽

원문 코드·기호

```
(i mod n) + 1
```

원문은 n개 드라이브에서 파일의 i번째 블록을 배치하는 드라이브 번호를 이 형태로 제시한다. 원문의 인덱스 표현을 보존한다.

제13장. 파일 시스템 인터페이스 / 13.2.2 직접 접근

출처: 책 590-591쪽 / 원문 PDF 528-529쪽

원문 코드·기호

`L × N` 위치에서 `L`바이트

- L: 논리 레코드 길이

- N: 0부터 세는 레코드 번호

0번을 첫 레코드로 세고 논리 레코드의 길이가 L일 때 N번째 레코드의 요청 위치이다. 고정 길이라는 전제를 유지한다.

제14장. 파일 시스템 구현 / 14.5.1 비트 벡터

출처: 책 634-635쪽 / 원문 PDF 571-572쪽

원문 코드·기호

`(워드의 비트 수) × (값이 0인 워드의 수) + 첫 번째 1비트의 변위`

원문이 제시한 첫 가용 블록 번호의 계산식이다. 앞의 워드는 모두 할당 비트 0으로만 구성되어 있다는 검색 절차의 전제를 갖는다.

주요 처리 순서

제1장. 서론 / 1.2.1

출처: 책 7-12쪽 / 원문 PDF 11-16쪽

인터럽트 처리의 기본 순서

1. CPU는 명령어 실행을 마칠 때 인터럽트 요청 라인을 검사한다.

↓

2. 인터럽트 번호를 인터럽트 벡터의 인덱스로 사용해 해당 핸들러로 이동한다.

↓

3. 핸들러는 변경될 상태를 저장하고 원인을 확인하여 필요한 처리를 수행한다.

↓

4. 상태를 복원하고 인터럽트 복귀 명령으로 이전 실행을 재개한다.

제3장. 프로세스 / 3.2.1

출처: 책 123-126쪽 / 원문 PDF 70-73쪽

실행 중 프로세스의 이동

1. 입출력을 요청하면 해당 대기 큐에서 완료를 기다린다.

↓

2. 자식을 생성한 뒤 종료를 기다리면 자식 종료 대기 큐로 간다.

↓

3. 타임 슬라이스 만료나 인터럽트로 코어에서 제거되면 준비 큐로 돌아갈 수 있다.

↓

4. 이벤트 완료 후 대기에서 준비로 돌아와 다시 스케줄된다. 종료하면 큐에서 제거되고 PCB와 자원이 반환된다.

제4장. 스레드와 병행성 / 4.5.1

출처: 책 195-197쪽 / 원문 PDF 141-143쪽

스레드 풀 처리

1. 미리 만든 스레드들은 작업을 기다린다.

↓

2. 요청이 오면 새 스레드 대신 작업을 풀에 제출한다.

↓

3. 사용 가능한 스레드가 작업을 처리하고, 없으면 작업이 기다린다.

↓

4. 작업을 마친 스레드는 풀로 돌아가 다음 작업을 기다린다.

제5장. CPU 스케줄링 / 5.3.6

출처: 책 237-239쪽 / 원문 PDF 183-185쪽

원문의 3단계 예

1. 새 프로세스는 큐 0에서 8ms를 받는다.

↓

2. 끝나지 않으면 큐 1의 꼬리로 내려가고 큐 0이 비었을 때 16ms를 받는다.

↓

3. 또 끝나지 않으면 큐 2로 간다. 큐 0·1이 비었을 때 FCFS로 실행한다.

↓

4. 상위 큐에 새 프로세스가 오면 하위 큐의 실행을 선점한다.

제6장. 동기화 도구들 / 6.1

출처: 책 283-285쪽 / 원문 PDF 227-229쪽

원문에서 count가 4가 되는 교차 실행

1. 생산자가 count=5를 자신의 레지스터로 읽고 6으로 증가시킨다.

↓

2. 소비자가 아직 5인 count를 다른 레지스터로 읽고 4로 감소시킨다.

↓

3. 생산자가 6을 count에 저장한 뒤 소비자가 4를 저장한다. 실제 채워진 버퍼는 5개지만 count는 4이다.

제7장. 동기화 예제 / 7.4

출처: 책 334-342쪽 / 원문 PDF 277-285쪽

원문의 가득 찬 유한 버퍼 처리

1. 생산자가 insert()에서 버퍼가 가득 찼음을 확인하고 wait()하여 락을 놓는다.

↓

2. 소비자가 remove()에 진입해 항목을 제거하고 notify()한다. 이 순간에도 소비자가 락을 소유한다.

↓

3. 생산자는 대기 집합에서 진입 집합으로 이동한다.

↓

4. 소비자가 메소드를 종료하여 락을 반환한다. 생산자가 다시 획득하여 루프에서 빈 공간을 재확인하고 삽입한다.

제8장. 교착 상태 / 8.1

출처: 책 349-351쪽 / 원문 PDF 291-293쪽

정상 자원 사용 순서

1. 요청: 즉시 허용되지 않으면 자원을 얻을 때까지 기다린다.

↓

2. 사용: 할당받은 자원을 이용하여 작업한다.

↓

3. 방출: 사용을 마치면 반환한다.

제9장. 메인 메모리 / 9.3.1

출처: 책 397-402쪽 / 원문 PDF 337-342쪽

MMU의 주소 변환

1. 논리 주소에서 p를 추출한다.

↓

2. 페이지 테이블에서 p에 대응하는 f를 얻는다.

↓

3. p를 f로 바꾸고 d는 유지하여 물리 주소를 만든다.

제10장. 가상 메모리 / 10.2

출처: 책 431-438쪽 / 원문 PDF 371-378쪽

페이지 폴트 처리

1. 프로세스 내부 테이블을 검사하여 참조가 유효한지 확인한다.

↓

2. 무효 참조면 프로세스를 중단하고, 유효하지만 메모리에 없으면 페이지를 가져온다.

↓

3. 가용 프레임을 찾는다.

↓

4. 보조저장장치에 그 프레임으로 페이지 읽기를 요청한다.

↓

5. 읽기가 끝나면 페이지 테이블과 내부 테이블을 갱신한다.

↓

6. 중단된 명령어를 다시 수행한다.

제11장. 대용량 저장장치 구조 / 11.1.2

출처: 책 494-496쪽 / 원문 PDF 433-435쪽

FTL·가비지 수집·과잉 공급의 관계

1. 블록이 다시 기록되면 새 페이지가 현재 데이터를 보유하고 이전 페이지의 데이터는 무효가 된다. 플래시 변환 계층(FTL)은 유효한 논리 블록을 실제 페이지로 매핑하고 물리 블록 상태를 추적한다.

↓

2. 공간이 부족하면 유효하지 않은 페이지만 든 블록을 지운다. 유효·무효 페이지가 섞인 블록은 유효 데이터를 다른 위치로 복사한 뒤 지운다. 이것이 가비지 수집이다.

↓

3. 이 복사를 위한 여유 공간과 쓰기 성능을 확보하려고 장치는 과잉 공급 영역을 따로 둔다. 지운 블록은 장치 상태에 따라 과잉 공급 공간이나 가용 풀로 돌아간다.

↓

4. 삭제 횟수가 적은 블록에 데이터를 배치하여 후속 지우기를 분산한다. 이러한 마모 평준화는 일부 블록만 일찍 마모되어 전체 수명이 줄어드는 것을 막는다.

제12장. 입출력 시스템 / 12.2.2

출처: 책 539-540쪽 / 원문 PDF 478-479쪽

원문의 바이트 출력 핸드셰이킹

1. 호스트는 busy 비트가 소거될 때까지 검사한다.

↓

2. 명령 레지스터의 쓰기 비트를 설정하고 data-out에 출력 바이트를 쓴다.

↓

3. command-ready 비트를 설정한다.

↓

4. 컨트롤러는 command-ready를 알아차리고 busy를 설정한다.

↓

5. 컨트롤러는 쓰기 명령과 데이터를 읽어 장치로 출력한다.

↓

6. 완료하면 command-ready·오류·busy 비트를 소거한다. 이 과정은 바이트마다 반복된다.

제13장. 파일 시스템 인터페이스 / 13.3.5

출처: 책 601-603쪽 / 원문 PDF 539-541쪽

원문의 가비지 수집

1. 파일 시스템을 순회하면서 접근 가능한 모든 대상을 표시한다.

↓

2. 두 번째 탐색에서 표시되지 않은 대상을 수집하여 가용 공간 목록에 넣는다.

제14장. 파일 시스템 구현 / 14.2.2

출처: 책 622-623쪽 / 원문 PDF 559-560쪽

open부터 close까지

1. open이 먼저 범 시스템 테이블을 확인한다. 이미 열려 있으면 그 항목을 가리키는 프로세스별 항목을 만든다.

↓

2. 처음 열 때는 디렉터리에서 찾아 FCB를 범 시스템 테이블에 복사한다. 프로세스별 항목에는 현재 위치와 접근 모드 등을 둔다.

↓

3. 프로세스는 반환된 파일 디스크립터(UNIX) 또는 핸들(Windows)로 후속 연산을 한다. 매번 파일 이름을 다시 검색하지 않는다.

↓

4. close는 프로세스별 항목을 없애고 열린 계수를 줄인다. 마지막 사용자가 닫으면 갱신 정보를 디스크에 반영하고 범 시스템 항목을 회수한다.

제15장. 파일 시스템 내부구조 / 15.2

출처: 책 657-659쪽 / 원문 PDF 593-595쪽

원문의 마운트 절차

1. 장치 이름과 기존 이름 구조에 부착할 마운트 포인트를 지정한다. 시스템에 따라 파일 시스템 유형도 지정하거나 구조를 검사하여 알아낸다.

2. 장치 드라이버를 통해 디렉터리를 읽어 유효한 파일 시스템 형식인지 확인한다.
↓
3. 지정 지점에 마운트되었다는 정보를 기록한다. 경로 순회는 그 지점에서 알맞은 파일 시스템으로 넘어간다.
↓

알고리즘·원문 코드 찾아보기

제1장. 서론

코드·그림	원문 출처	교재 쪽
그림 1.6 저장장치 계층 구조	출처: 책 14쪽 / 원문 PDF 18쪽	10
그림 1.13 사용자 모드에서 커널 모드로의 전환	출처: 책 26쪽 / 원문 PDF 30쪽	13

제3장. 프로세스

코드·그림	원문 출처	교재 쪽
그림 3.8 fork()로 별도 프로세스 생성	출처: 책 132쪽 / 원문 PDF 79쪽	25
그림 3.12 공유 메모리 생산자	출처: 책 141쪽 / 원문 PDF 88쪽	27
그림 3.13 공유 메모리 소비자	출처: 책 141쪽 / 원문 PDF 88쪽	27
그림 3.2 프로세스 상태 다이어그램	출처: 책 121쪽 / 원문 PDF 68쪽	22
그림 3.16 POSIX 공유 메모리 생산자 코드	출처: 책 148쪽 / 원문 PDF 95쪽	30
그림 3.17 POSIX 공유 메모리 소비자 코드	출처: 책 149쪽 / 원문 PDF 96쪽	31
그림 3.29 RPC의 실행	출처: 책 168쪽 / 원문 PDF 115쪽	35

제4장. 스레드와 병행성

코드·그림	원문 출처	교재 쪽
그림 4.11 Pthreads 합산 프로그램	출처: 책 187쪽 / 원문 PDF 133쪽	41
OpenMP 병렬 for	출처: 책 202쪽 / 원문 PDF 148쪽	43
TBB parallel_for	출처: 책 205쪽 / 원문 PDF 151쪽	44
Amdahl의 법칙 그래프	출처: 책 180쪽 / 원문 PDF 126쪽	39
그림 4.21 Windows 스레드의 자료구조	출처: 책 214쪽 / 원문 PDF 160쪽	47

제5장. CPU 스케줄링

코드·그림	원문 출처	교재 쪽
그림 5.10의 경쟁 범위 설정 부분	출처: 책 241쪽 / 원문 PDF 187쪽	56
FCFS의 원문 간트차트	출처: 책 227쪽 / 원문 PDF 173쪽	51
지수 평균의 원문 전개식	출처: 책 230쪽 / 원문 PDF 176쪽	52
선점형 SJF의 원문 간트차트	출처: 책 230쪽 / 원문 PDF 176쪽	53
RR의 원문 간트차트(q=4ms)	출처: 책 231쪽 / 원문 PDF 177쪽	53
그림 5.9 다단계 피드백 큐	출처: 책 238쪽 / 원문 PDF 184쪽	55
그림 5.23 RM이 기한을 놓치는 예	출처: 책 255쪽 / 원문 PDF 201쪽	59
그림 5.24 EDF 스케줄링	출처: 책 256쪽 / 원문 PDF 202쪽	60

제6장. 동기화 도구들

코드·그림	원문 출처	교재 쪽
Peterson의 해결안: 프로세스 P_i의 구조	출처: 책 289쪽 / 원문 PDF 233쪽	67
원자적 compare_and_swap()의 정의	출처: 책 294쪽 / 원문 PDF 238쪽	69
대기 큐를 사용하는 세마포 wait()	출처: 책 301쪽 / 원문 PDF 245쪽	71
대기 큐를 사용하는 세마포 signal()	출처: 책 302쪽 / 원문 PDF 246쪽	71

제7장. 동기화 예제

코드·그림	원문 출처	교재 쪽
유한 버퍼 생산자 구조	출처: 책 320쪽 / 원문 PDF 263쪽	75
유한 버퍼 소비자 구조	출처: 책 321쪽 / 원문 PDF 264쪽	76
Pthread 조건 변수로 a == b를 기다림	출처: 책 333쪽 / 원문 PDF 276쪽	80
Java ReentrantLock의 획득·반환	출처: 책 339쪽 / 원문 PDF 282쪽	82
식사하는 철학자 문제의 모니터 해법	출처: 책 326쪽 / 원문 PDF 269쪽	78
Java 객체의 진입 집합과 대기 집합	출처: 책 337쪽 / 원문 PDF 280쪽	81

제8장. 교착 상태

코드·그림	원문 출처	교재 쪽
교착 상태인 자원 할당 그래프	출처: 책 357쪽 / 원문 PDF 299쪽	86
사이클이 있지만 교착 상태가 아닌 그래프	출처: 책 358쪽 / 원문 PDF 300쪽	87
은행원 안전성 알고리즘	출처: 책 369쪽 / 원문 PDF 311쪽	90
은행원 알고리즘의 원문 행렬 예	출처: 책 370쪽 / 원문 PDF 312쪽	91
여러 인스턴스의 교착 상태 탐지 절차	출처: 책 373쪽 / 원문 PDF 315쪽	92

제9장. 메인 메모리

코드·그림	원문 출처	교재 쪽
페이징 하드웨어의 페이지·프레임 변환	출처: 책 398쪽 / 원문 PDF 338쪽	97
TLB 적중률80%의 원문 계산	출처: 책 405쪽 / 원문 PDF 345쪽	98
TLB 적중률99%의 원문 계산	출처: 책 405쪽 / 원문 PDF 345쪽	98
x86-64 선형 주소의 원문 필드	출처: 책 421쪽 / 원문 PDF 361쪽	101

제10장. 가상 메모리

코드·그림	원문 출처	교재 쪽
원문의 배열 순회: 열을 먼저 진행	출처: 책 477쪽 / 원문 PDF 417쪽	114
원문의 배열 순회: 행을 먼저 진행	출처: 책 477쪽 / 원문 PDF 417쪽	114
요구 페이징의 원문 성능 계산	출처: 책 437쪽 / 원문 PDF 377쪽	104
FIFO의 원문 페이지 교체 추적	출처: 책 445쪽 / 원문 PDF 385쪽	105
최적 교체의 원문 추적	출처: 책 447쪽 / 원문 PDF 387쪽	106
LRU의 원문 추적	출처: 책 448쪽 / 원문 PDF 388쪽	106
작업 집합의 원문 참조 창	출처: 책 464쪽 / 원문 PDF 404쪽	110
버디 시스템의 원문 분할	출처: 책 470쪽 / 원문 PDF 410쪽	112

제11장. 대용량 저장장치 구조

코드·그림	원문 출처	교재 쪽
그림 11.6 FCFS 디스크 스케줄링	출처: 책 501쪽 / 원문 PDF 440쪽	120
그림 11.8 C-SCAN 디스크 스케줄링	출처: 책 503쪽 / 원문 PDF 442쪽	122
그림 11.15 RAID 레벨	출처: 책 521쪽 / 원문 PDF 460쪽	129
그림 11.16 RAID 0+1과 1+0의 단일 장애	출처: 책 524쪽 / 원문 PDF 463쪽	130
그림 11.17 ZFS의 메타데이터·데이터 체크섬	출처: 책 528쪽 / 원문 PDF 467쪽	132

제12장. 입출력 시스템

코드·그림	원문 출처	교재 쪽
그림 12.3 인터럽트 구동식 입출력 사이클	출처: 책 541쪽 / 원문 PDF 480쪽	136
그림 12.6 DMA 전송의 단계	출처: 책 546쪽 / 원문 PDF 485쪽	138
그림 12.14 입출력 요청의 생애 주기	출처: 책 567쪽 / 원문 PDF 506쪽	144
그림 12.15 STREAMS 구조	출처: 책 569쪽 / 원문 PDF 508쪽	145

제13장. 파일 시스템 인터페이스

코드·그림	원문 출처	교재 쪽
그림 13.2 Java의 파일 락킹 예제	출처: 책 586쪽 / 원문 PDF 524쪽	150
그림 13.10 비순환 그래프 디렉터리 구조	출처: 책 599쪽 / 원문 PDF 537쪽	155
그림 13.15 공유 메모리에 기록하는 Windows 생산자	출처: 책 611쪽 / 원문 PDF 549쪽	159
그림 13.16 공유 메모리를 읽는 Windows 소비자	출처: 책 612쪽 / 원문 PDF 550쪽	160

제14장. 파일 시스템 구현

코드·그림	원문 출처	교재 쪽
그림 14.6 파일 할당 테이블(FAT)	출처: 책 629쪽 / 원문 PDF 566쪽	165
그림 14.8 UNIX의 inode	출처: 책 632쪽 / 원문 PDF 569쪽	167
그림 14.11 통합 버퍼 캐시를 사용하는 입출력	출처: 책 641쪽 / 원문 PDF 578쪽	170
그림 14.13 WAFL의 스냅숏	출처: 책 648쪽 / 원문 PDF 585쪽	173

제15장. 파일 시스템 내부구조

코드·그림	원문 출처	교재 쪽
그림 15.4 /users에 마운트 된 볼륨	출처: 책 658쪽 / 원문 PDF 594쪽	176
그림 15.5 가상 파일 시스템의 개략적 모습	출처: 책 663쪽 / 원문 PDF 599쪽	178
그림 15.7 NFS에서의 마운팅	출처: 책 671쪽 / 원문 PDF 607쪽	182
그림 15.8 NFS 구조의 개략도	출처: 책 674쪽 / 원문 PDF 610쪽	184

주요 개념 복습

제1장. 서론

출처: 책 3-59쪽 / 원문 PDF 7-63쪽

- 하드웨어 자원을 추상화·할당·보호하는 것이 프로그램 실행 환경의 토대이다.
- CPU 제어의 이동은 인터럽트·트랩·시스템 콜 및 실행 모드 전환과 연결된다.
- 단일 처리기·다중 처리기·클러스터는 범용 계산 단위와 결합 방식의 차이로 구별한다.
- 저장 계층은 속도·용량·비용·휘발성의 절충이며 캐시에는 크기·교체·일관성 문제가 있다.

제3장. 프로세스

출처: 책 117-173쪽 / 원문 PDF 64-120쪽

- 프로세스 표현 → 큐 이동 → 문맥 교환의 연결을 따라 실행과 재개를 설명한다.
- 생성과 프로그램 교체는 다른 연산이며 자원 복사·공유와 부모 대기 조건을 구별한다.
- 공유 메모리의 접근 책임과 메시지 전달의 링크 설계 선택을 연결한다.
- 소켓의 바이트 스트림보다 높은 추상화를 RPC가 제공하며 스텁·정돈·바인딩이 이를 구현한다.

제4장. 스레드와 병행성

출처: 책 175-218쪽 / 원문 PDF 121-164쪽

- 응답성·자원 공유·경제성·규모 적응성을 원문의 사례와 연결한다.
- 독립 작업 분할만으로 충분하지 않으며 균형·데이터 분할·종속성·검증도 필요하다.
- 스레드 API의 생성·조인과 Executor·풀·fork-join의 작업 제출을 구별한다.
- 다중 스레드 fork·exec·신호·취소·TLS·LWP는 공유와 개별 실행 상태가 만나는 지점이다.

제5장. CPU 스케줄링

출처: 책 219-279쪽 / 원문 PDF 165-225쪽

- 버스트 특성 → 선택 기준 → 실제 디스패치 비용을 연결한다.
- 선점은 응답에 유리하지만 공유 자료 보호가 필요하다.
- 큐 구조·경쟁 범위·논리 CPU·물리 코어는 서로 다른 층의 결정이다.
- 부하·선호도·에너지·기한 등 목표가 달라지면 필요한 정책도 달라진다.
- 단일 수치 예제의 비교와 실제 시스템의 확률·시간 비용을 구별한다.

제6장. 동기화 도구들

출처: 책 283-317쪽 / 원문 PDF 227-261쪽

- 공유 데이터 → 경쟁 조건 → 임계구역 프로토콜의 관계를 연결한다.
- 하드웨어 지원 → 원자적 변수·락 → 세마포·모니터로 추상화 수준이 높아진다.
- 바쁜 대기와 봉쇄 대기의 비용은 문맥 교환과 경합 정도에 따라 다르다.

제7장. 동기화 예제

출처: 책 319-347쪽 / 원문 PDF 262-290쪽

- 고전 문제의 요구 조건 → 세마포·모니터의 상태와 연산 순서를 연결한다.
- 커널의 짧은 보호에는 스핀·선점 제어, 오래 기다리는 상황에는 봉쇄 도구라는 원문 맥락을 복습한다.
- 트랜잭션·OpenMP·함수형 방식은 각각 원자성 책임·지시 기반 보호·상태 모델을 바꾸는 접근으로 읽는다.

제8장. 교착 상태

출처: 책 349-381쪽 / 원문 PDF 291-323쪽

- 요청·사용·방출 → 대기 고리 → 네 조건의 제거 또는 상태 검사로 연결한다.
- 예방은 요청 규칙, 회피는 미래 최대 요구와 안전성, 탐지는 현재 요청·할당을 사용한다.
- 교착 복구의 선택은 잃는 계산·자료 무결성·반복 희생의 비용과 연결된다.

제9장. 메인 메모리

출처: 책 385-426쪽 / 원문 PDF 325-366쪽

- 프로세스별 주소 공간 → 하드웨어 보호 → 바인딩·MMU 변환의 관계를 연결한다.
- 연속 공간의 단편화 → 비연속 프레임 할당 → 페이지 테이블 → TLB의 흐름으로 복습한다.
- 전체 주소 비트 수·실제 사용하는 비트·페이지 크기·단계 수를 서로 다른 기준으로 구분한다.

제10장. 가상 메모리

출처: 책 427-488쪽 / 원문 PDF 367-428쪽

- 가상 주소 추상화 → 필요한 페이지 적재 → 빈 프레임이 없으면 교체라는 흐름을 복습한다.
- 폴트율과 서비스 시간 → 실질 접근 시간 → 지역성·할당·작업 집합으로 연결한다.
- 스래싱 때 CPU 이용률 하락을 단순한 프로세스 부족으로 해석하면 악순환이 생긴다.
- 압축과 고정·커널 할당은 일반 사용자 페이지 교체와 다른 조건을 가진다.

제11장. 대용량 저장장치 구조

출처: 책 491-534쪽 / 원문 PDF 430-473쪽

- 물리 구조 → 논리 블록 → 요청 큐 → 매핑·복구라는 연결을 먼저 떠올린다.
- 스왑 파일과 raw 파티션은 관리 편의와 성능·크기 변경의 조건이 다르다.
- NAS의 파일, iSCSI의 논리 블록, SAN의 저장장치 연결, 클라우드의 API 접근을 구별한다.
- ZFS는 포인터에 둔 체크섬과 공유 풀을 통해 무결성·크기 관리 문제를 다룬다.

제12장. 입출력 시스템

출처: 책 535-576쪽 / 원문 PDF 474-515쪽

- 응용은 장치를 직접 제어하기보다 드라이버와 커널 서비스의 표준 인터페이스를 사용한다.
- 일관성·보호·전력 관리 역시 I/O 서브시스템의 역할이다.
- STREAMS는 양방향 큐와 메시지·흐름 제어로 모듈을 결합한다.
- 성능은 장치 속도뿐 아니라 문맥 교환·인터럽트·복사·계층 간 이동 비용에 좌우된다.

제13장. 파일 시스템 인터페이스

출처: 책 579-615쪽 / 원문 PDF 517-553쪽

- 파일 열기는 반복 이름 검색을 줄이고 열린 테이블의 사용 상태를 유지한다.
- 파일 구조를 운영체제가 많이 지원할수록 편의와 구현·확장 제약의 절충이 생긴다.
- 메모리 사상은 파일 접근을 가상 메모리 참조로 바꾸며 공유 물리 페이지와 쓰기 시 복사는 서로 다른 공유 의미를 가진다.

제14장. 파일 시스템 구현

출처: 책 617-653쪽 / 원문 PDF 554-590쪽

- inode의 직접·간접 수준과 FAT의 사슬은 파일 내용의 위치를 찾는 서로 다른 구조이다.
- 가용 공간은 비트맵·연결·그룹·계수·로그/트리의 여러 방식으로 관리한다.
- HDD의 헤드 이동 최적화를 NVM에 그대로 적용하면 불필요한 CPU 비용이 될 수 있다.
- WAFL은 새 블록 기록과 루트 포인터의 보존으로 스냅숏·클론·복제를 연결한다.

제15장. 파일 시스템 내부구조

출처: 책 655-678쪽 / 원문 PDF 591-614쪽

- 부팅·파티션·볼륨·마운트는 커널을 찾고 여러 저장장치를 하나의 경로로 제공하는 연결이다.
- VFS는 함수 테이블로 로컬·원격 구현을 선택하고 NFS는 RPC/XDR를 통해 서버 VFS로 이어진다.
- 무상태성은 요청 자체의 완전한 정보를 요구하며 캐싱·지연 쓰기는 일관성 의미를 바꾼다.

제7부 도입. 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

- 보안과 보호의 역할 구별은 제공된 도입의 범위에서만 복습한다.

개념 색인

개념	교재 쪽	개념	교재 쪽
2차 기회	107	ACL	157
Amdahl의 법칙	38	ARMv8	101
ASID	97	Belady의 모순	105
C-SCAN	121	CAS	68
CFS	61	clone	47
Dispatcher 객체	79	DMA	11, 137
ECC	123	EDF	60
Executor	42	FAT	165
FCB	162	FCFS	51, 120
FIFO 페이지 교체	105	fork	24
fork-join	43	FTL	117
GCD	44	inode	166
IPC	26	ISAM	152
Java 모니터	80	Little의 공식	63
LRU	106	LWP	46
MMU	94	NFS	181
NUMA	11, 109	OpenMP	43
OpenMP 임계구역	82	OPT	105
PAE	100	PCB	22
PCS	55	Peterson의 해결안	67
PFF	111	POSIX 동기화	79
Pthreads	41	RAID	128
raw 파티션	177	Readers-Writers	76
ReentrantLock	80	RM	59
RPC	33	RR	53
SCAN	121	SCS	55
SJF	51	SMP	11
SRTF	51	STREAMS	145
TLB	97	TLB reach	113
TLS	46	TRIM	169

개념	교재 쪽	개념	교재 쪽
VFS	177	vnode	177
vruntime	61	WAFL	172
wakelock	143	XDR	181
ZFS	131	가상 메모리	103
가상화	15	객체 저장소	133
경로 이름 변환	184	경쟁 상황	66
고아	26	공간맵	169
공유 메모리	26	교착 상태	84
교착 상태 회복	92	권고적 락	149
기명 세마포	79	내부 단편화	95, 151
노화	54	다중 태스킹	12
다중 프로그래밍	12	대기 그래프	91
동적 연결	95	동적 적재	95
디스패처	50	라이브니스	73
라이브락	84	랑데부	29
로그 기반 복구	171	마운트	176
메모리 맵드 I/O	134	메모리 사상 파일	157
메모리 압축	111	메모리 장벽	68
모니터	72	무상태	183
문맥 교환	24	바인더	36
버디 시스템	111	병렬성	38
병행성	38	보안 / 보호의 도입	186
복제 시맨틱	141	부하 균등화	57
분산 정보 시스템	179	비동기 I/O	140
비순환 그래프	154	비트맵	16
색인 할당	166	선점	50
세마포	70	세션 의미	180
소켓	33	순환 대기	88
스냅숏	172	스래싱	109
스레드	37	스레드 풀	42
스와핑	100	스왑 맵	125
스풀링	142	스핀락	70

개념	교재 쪽	개념	교재 쪽
슬랩 할당	111	식사하는 철학자	77
실시간 시스템	18	쓰기 시 복사	104
쓰기 증폭	123	안전 상태	88
역 페이지 테이블	99	열린 파일 테이블	149
외부 단편화	95	요구 페이징	103
우선순위 역전	73	원자적 변수	69
유한 버퍼	74	은행원 알고리즘	89
익스텐트	164	인터럽트	9
인터럽트 벡터	135	임계구역	66
자원 할당 그래프	85	작업 집합	110
전역·지역 교체	109	조건 변수	72
좀비	26	주소 바인딩	94
지수 평균	51	직접 접근	152
참조 계수	154	처리기 선호도	57
총처리 시간	50	캐시 일관성	14
통합 버퍼 캐시	170	트랜잭션 메모리	82
특권 명령	12	파이프	32
페이지 잠금	115	페이지 폴트	103
페이징	96	폴링	135
프로세스	21	피드백 큐	54
호위 효과	51		

최종 학습 점검표

제1장. 서론

[ ] 인터럽트 발생부터 상태 복원까지의 순서를 설명할 수 있는가?

출처: 책 7-12쪽 / 원문 PDF 11-16쪽

[ ] 다중 프로그래밍과 다중 태스킹의 목적을 구별할 수 있는가?

출처: 책 24-26쪽 / 원문 PDF 28-30쪽

[ ] 사용자·커널 모드와 타이머의 보호 역할을 설명할 수 있는가?

출처: 책 26-29쪽 / 원문 PDF 30-33쪽

[ ] 보호와 보안, 가상화와 에뮬레이션을 구별할 수 있는가?

출처: 책 35-38쪽 / 원문 PDF 39-42쪽

[ ] 리스트·스택·큐·트리·맵·비트맵의 원문 용도를 설명할 수 있는가?

출처: 책 39-43쪽 / 원문 PDF 43-47쪽

제3장. 프로세스

[ ] 프로세스의 네 메모리 섹션과 다섯 상태를 설명할 수 있는가?

출처: 책 118-122쪽 / 원문 PDF 65-69쪽

[ ] 준비·대기 큐와 문맥 교환의 저장·복원 흐름을 설명할 수 있는가?

출처: 책 123-128쪽 / 원문 PDF 70-75쪽

[ ] fork·exec·wait와 좀비·고아의 차이를 설명할 수 있는가?

출처: 책 129-136쪽 / 원문 PDF 76-83쪽

[ ] 메시지 명명·봉쇄·버퍼 용량을 서로 구별할 수 있는가?

출처: 책 142-146쪽 / 원문 PDF 89-93쪽

[ ] 파이프의 방향·수명·프로세스 관계·통신 범위를 설명할 수 있는가?

출처: 책 155-161쪽 / 원문 PDF 102-108쪽

[ ] RPC 정돈과 바인딩 및 원문 전달 의미를 설명할 수 있는가?

출처: 책 164-169쪽 / 원문 PDF 111-116쪽

제4장. 스레드와 병행성

[ ] 스레드의 개별 상태와 공유 자원을 구분할 수 있는가?

출처: 책 176-178쪽 / 원문 PDF 122-124쪽

[ ] 병행·병렬과 Amdahl 식의 변수·조건을 설명할 수 있는가?

출처: 책 179-182쪽 / 원문 PDF 125-128쪽

[ ] 다대일·일대일·다대다·두 수준 모델의 차이를 설명할 수 있는가?

출처: 책 182-185쪽 / 원문 PDF 128-131쪽

[ ] 풀·fork-join·OpenMP·GCD·TBB의 작업 배정 원리를 설명할 수 있는가?

출처: 책 194-206쪽 / 원문 PDF 140-152쪽

[ ] 비동기·지연 취소와 TLS의 범위를 구별할 수 있는가?

출처: 책 208-211쪽 / 원문 PDF 154-157쪽

[ ] Windows 자료구조와 Linux clone의 공유 방식을 설명할 수 있는가?

출처: 책 213-215쪽 / 원문 PDF 159-161쪽

제5장. CPU 스케줄링

[ ] 다섯 평가 기준과 선점·디스패치의 의미를 구분할 수 있는가?

출처: 책 221-226쪽 / 원문 PDF 167-172쪽

[ ] FCFS·SJF·SRTF·RR의 원문 간트차트를 읽을 수 있는가?

출처: 책 226-233쪽 / 원문 PDF 172-179쪽

[ ] 지수 평균의 기호와 α 변화의 의미를 설명할 수 있는가?

출처: 책 229-230쪽 / 원문 PDF 175-176쪽

[ ] 기아·노화·다단계 큐·피드백 이동을 설명할 수 있는가?

출처: 책 233-239쪽 / 원문 PDF 179-185쪽

[ ] PCS·SCS와 부하 균등화·처리기 선호도를 구별할 수 있는가?

출처: 책 239-249쪽 / 원문 PDF 185-195쪽

[ ] 주기 t·d·p 및 RM·EDF의 순위 차이를 설명할 수 있는가?

출처: 책 253-256쪽 / 원문 PDF 199-202쪽

[ ] CFS의 nice·vruntime·도메인 관계를 설명할 수 있는가?

출처: 책 259-262쪽 / 원문 PDF 205-208쪽

[ ] Little 공식과 각 평가 방법의 가정을 확인할 수 있는가?

출처: 책 269-274쪽 / 원문 PDF 215-220쪽

제6장. 동기화 도구들

[ ] 임계구역의 세 요구 조건과 차이를 설명할 수 있는가?

출처: 책 286-287쪽 / 원문 PDF 230-231쪽

[ ] Peterson 해결안의 두 프로세스 전제와 현대 구조에서의 한계를 설명할 수 있는가?

출처: 책 288-292쪽 / 원문 PDF 232-236쪽

[ ] 세마포 대기 큐 구현에서 음수 값과 wakeup의 의미를 설명할 수 있는가?

출처: 책 301-303쪽 / 원문 PDF 245-247쪽

[ ] 모니터 조건 변수의 signal과 세마포 signal의 차이를 설명할 수 있는가?

출처: 책 305-307쪽 / 원문 PDF 249-251쪽

[ ] 우선순위 역전과 우선순위 상속의 관계를 설명할 수 있는가?

출처: 책 312-313쪽 / 원문 PDF 256-257쪽

제7장. 동기화 예제

[ ] 유한 버퍼 생산자·소비자의 wait와 signal 순서를 설명할 수 있는가?

출처: 책 320-321쪽 / 원문 PDF 263-264쪽

[ ] Reader 우선과 writer 우선의 기아 가능성을 구분할 수 있는가?

출처: 책 321-323쪽 / 원문 PDF 264-266쪽

[ ] 철학자 문제에서 상호 배제·교착·기아의 차이를 설명할 수 있는가?

출처: 책 324-327쪽 / 원문 PDF 267-270쪽

[ ] POSIX 조건 변수의 wait와 signal이 mutex에 미치는 영향을 설명할 수 있는가?

출처: 책 333-334쪽 / 원문 PDF 276-277쪽

[ ] Java notify 이후 락 재획득까지의 단계를 설명할 수 있는가?

출처: 책 335-339쪽 / 원문 PDF 278-282쪽

[ ] STM과 HTM 및 OpenMP의 조건과 한계를 설명할 수 있는가?

출처: 책 342-345쪽 / 원문 PDF 285-288쪽

제8장. 교착 상태

[ ] 교착과 라이브락의 차이를 설명할 수 있는가?

출처: 책 351-354쪽 / 원문 PDF 293-296쪽

[ ] 사이클과 교착의 관계를 인스턴스 수에 따라 구분할 수 있는가?

출처: 책 355-358쪽 / 원문 PDF 297-300쪽

[ ] 안전 순서·불안전 상태·교착 상태의 관계를 설명할 수 있는가?

출처: 책 365-366쪽 / 원문 PDF 307-308쪽

[ ] 은행원 자료구조와 안전성·요청 검사 순서를 설명할 수 있는가?

출처: 책 368-371쪽 / 원문 PDF 310-313쪽

[ ] 현재 Request를 쓰는 탐지와 Need를 쓰는 회피를 구별할 수 있는가?

출처: 책 368-374쪽 / 원문 PDF 310-316쪽

[ ] 선점 복구의 희생자·후퇴·기아 문제를 설명할 수 있는가?

출처: 책 376-377쪽 / 원문 PDF 318-319쪽

제9장. 메인 메모리

[ ] 컴파일·적재·실행 시간 바인딩과 재배치 조건을 설명할 수 있는가?

출처: 책 388-391쪽 / 원문 PDF 328-331쪽

[ ] 동적 적재와 동적 연결의 차이를 설명할 수 있는가?

출처: 책 391-393쪽 / 원문 PDF 331-333쪽

[ ] 최초·최적·최악 적합과 내부·외부 단편화를 구분할 수 있는가?

출처: 책 394-397쪽 / 원문 PDF 334-337쪽

[ ] 페이지 번호·프레임 번호·오프셋을 원문 예에 연결할 수 있는가?

출처: 책 397-400쪽 / 원문 PDF 337-340쪽

[ ] TLB 미스와 ASID·flush·보호 비트의 역할을 설명할 수 있는가?

출처: 책 403-407쪽 / 원문 PDF 343-347쪽

[ ] 계층적·해시·역 테이블의 구조 차이를 설명할 수 있는가?

출처: 책 409-414쪽 / 원문 PDF 349-354쪽

[ ] IA-32·x86-64·ARMv8의 원문 변환 흐름을 구분할 수 있는가?

출처: 책 417-423쪽 / 원문 PDF 357-363쪽

제10장. 가상 메모리

[ ] 유효 참조의 미적재와 무효 참조를 구별하여 폴트 처리 순서를 설명할 수 있는가?

출처: 책 431-435쪽 / 원문 PDF 371-375쪽

[ ] 폴트 확률·메모리 시간·서비스 시간이 실질 접근 시간에 미치는 영향을 설명할 수 있는가?

출처: 책 436-438쪽 / 원문 PDF 376-378쪽

[ ] COW와 vfork의 공유·수정 차이를 설명할 수 있는가?

출처: 책 438-440쪽 / 원문 PDF 378-380쪽

[ ] 동일 참조열의 FIFO15·OPT9·LRU12 결과와 각 선택 기준을 설명할 수 있는가?

출처: 책 444-449쪽 / 원문 PDF 384-389쪽

[ ] 참조·변경 비트와 2차 기회 등급을 설명할 수 있는가?

출처: 책 449-452쪽 / 원문 PDF 389-392쪽

[ ] 전역·지역 할당과 주요·사소한 폴트를 구별할 수 있는가?

출처: 책 456-459쪽 / 원문 PDF 396-399쪽

[ ] 작업 집합 Δ·WSS·D와 PFF의 제어 기준을 설명할 수 있는가?

출처: 책 461-467쪽 / 원문 PDF 401-407쪽

[ ] 버디·슬랩 및 I/O 잠금의 목적·제약을 설명할 수 있는가?

출처: 책 469-479쪽 / 원문 PDF 409-419쪽

제11장. 대용량 저장장치 구조

[ ] 대용량 저장장치 구조의 개관의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 491-499쪽 / 원문 PDF 430-438쪽

[ ] 하드 디스크 드라이브의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 492-493쪽 / 원문 PDF 431-432쪽

[ ] 비휘발성 메모리 장치와 NAND 컨트롤러의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 494-496쪽 / 원문 PDF 433-435쪽

[ ] 휘발성 메모리와 자기 테이프의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 496-498쪽 / 원문 PDF 435-437쪽

[ ] 보조저장장치 연결 방법의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 498쪽 / 원문 PDF 437쪽

[ ] 주소 매핑의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 498-499쪽 / 원문 PDF 437-438쪽

[ ] 디스크 스케줄링의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 500쪽 / 원문 PDF 439쪽

[ ] 선입 선처리 스케줄링의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 501쪽 / 원문 PDF 440쪽

[ ] SCAN 스케줄링의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 501-502쪽 / 원문 PDF 440-441쪽

[ ] C-SCAN 스케줄링의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 502-503쪽 / 원문 PDF 441-442쪽

[ ] 디스크 스케줄링 알고리즘의 선택의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 503-504쪽 / 원문 PDF 442-443쪽

[ ] NVM 스케줄링의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 504-505쪽 / 원문 PDF 443-444쪽

[ ] 오류 감지 및 수정의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 505-506쪽 / 원문 PDF 444-445쪽

[ ] 드라이브 포매팅, 파티션, 볼륨의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 506-508쪽 / 원문 PDF 445-447쪽

[ ] 부트 블록의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 508-509쪽 / 원문 PDF 447-448쪽

[ ] 손상된 블록의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 509-511쪽 / 원문 PDF 448-450쪽

[ ] 스왑 공간 관리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 511-513쪽 / 원문 PDF 450-452쪽

[ ] 저장장치 연결의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 513-517쪽 / 원문 PDF 452-456쪽

[ ] 중복으로 신뢰성 향상의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 517-519쪽 / 원문 PDF 456-458쪽

[ ] 병렬성을 이용한 성능 향상의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 519-520쪽 / 원문 PDF 458-459쪽

[ ] RAID 레벨과 구현의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 520-525쪽 / 원문 PDF 459-464쪽

[ ] RAID 레벨 선택의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 525-527쪽 / 원문 PDF 464-466쪽

[ ] 확장의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 527쪽 / 원문 PDF 466쪽

[ ] RAID의 문제점들의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 527-529쪽 / 원문 PDF 466-468쪽

[ ] 객체 저장소의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 529-530쪽 / 원문 PDF 468-469쪽

[ ] 요약과 원문 연습문제의 점검 범위의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 530-534쪽 / 원문 PDF 469-473쪽

제12장. 입출력 시스템

[ ] 개관의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 535-536쪽 / 원문 PDF 474-475쪽

[ ] 입출력 하드웨어의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 536-538쪽 / 원문 PDF 475-477쪽

[ ] 메모리 맵드 입출력의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 538-539쪽 / 원문 PDF 477-478쪽

[ ] 폴링의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 539-540쪽 / 원문 PDF 478-479쪽

[ ] 인터럽트의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 540-545쪽 / 원문 PDF 479-484쪽

[ ] 직접 메모리 접근(DMA)의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 545-547쪽 / 원문 PDF 484-486쪽

[ ] 입출력 하드웨어 요약의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 547-548쪽 / 원문 PDF 486-487쪽

[ ] 응용 입출력 인터페이스의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 548-551쪽 / 원문 PDF 487-490쪽

[ ] 블록 장치와 문자 장치의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 551-552쪽 / 원문 PDF 490-491쪽

[ ] 네트워크 장치의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 552쪽 / 원문 PDF 491쪽

[ ] 클록과 타이머의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 553-554쪽 / 원문 PDF 492-493쪽

[ ] 봉쇄형과 비봉쇄형 입출력의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 554-556쪽 / 원문 PDF 493-495쪽

[ ] 벡터형 입출력의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 556쪽 / 원문 PDF 495쪽

[ ] I/O 스케줄링의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 556-557쪽 / 원문 PDF 495-496쪽

[ ] 버퍼링의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 557-559쪽 / 원문 PDF 496-498쪽

[ ] 캐싱의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 559쪽 / 원문 PDF 498쪽

[ ] 스풀링 및 장치 예약의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 559-560쪽 / 원문 PDF 498-499쪽

[ ] 오류 처리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 560-561쪽 / 원문 PDF 499-500쪽

[ ] 입출력 보호의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 561-562쪽 / 원문 PDF 500-501쪽

[ ] 커널 자료구조의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 562-563쪽 / 원문 PDF 501-502쪽

[ ] 전원 관리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 563-565쪽 / 원문 PDF 502-504쪽

[ ] 커널 입출력 서브시스템 요약의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 565쪽 / 원문 PDF 504쪽

[ ] 입출력 요구를 하드웨어 연산으로 변환의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 566-568쪽 / 원문 PDF 505-507쪽

[ ] STREAMS의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 568-570쪽 / 원문 PDF 507-509쪽

[ ] 성능의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 570-574쪽 / 원문 PDF 509-513쪽

[ ] 요약과 원문 연습문제의 점검 범위의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 574-576쪽 / 원문 PDF 513-515쪽

제13장. 파일 시스템 인터페이스

[ ] 파일 개념의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 579-580쪽 / 원문 PDF 517-518쪽

[ ] 파일 속성의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 580-581쪽 / 원문 PDF 518-519쪽

[ ] 파일 연산과 열린 파일 관리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 582-586쪽 / 원문 PDF 520-524쪽

[ ] 파일 유형의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 587-588쪽 / 원문 PDF 525-526쪽

[ ] 파일 구조의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 588-589쪽 / 원문 PDF 526-527쪽

[ ] 파일의 내부 구조의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 589쪽 / 원문 PDF 527쪽

[ ] 순차 접근의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 590쪽 / 원문 PDF 528쪽

[ ] 직접 접근의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 590-591쪽 / 원문 PDF 528-529쪽

[ ] 기타 접근 방법의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 591-593쪽 / 원문 PDF 529-531쪽

[ ] 디렉터리 구조의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 593쪽 / 원문 PDF 531쪽

[ ] 1단계 디렉터리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 594쪽 / 원문 PDF 532쪽

[ ] 2단계 디렉터리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 594-596쪽 / 원문 PDF 532-534쪽

[ ] 트리 구조 디렉터리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 596-598쪽 / 원문 PDF 534-536쪽

[ ] 비순환 그래프 디렉터리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 599-601쪽 / 원문 PDF 537-539쪽

[ ] 일반 그래프 디렉터리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 601-603쪽 / 원문 PDF 539-541쪽

[ ] 보호와 접근의 유형의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 603-604쪽 / 원문 PDF 541-542쪽

[ ] 접근 제어의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 604-607쪽 / 원문 PDF 542-545쪽

[ ] 다른 보호 방법의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 607-608쪽 / 원문 PDF 545-546쪽

[ ] 메모리 사상 파일의 기본 기법의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 608-610쪽 / 원문 PDF 546-548쪽

[ ] Windows API에서 공유 메모리의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 610-613쪽 / 원문 PDF 548-551쪽

[ ] 요약과 원문 연습문제의 점검 범위의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 613-615쪽 / 원문 PDF 551-553쪽

제14장. 파일 시스템 구현

[ ] 파일 시스템 구조의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 618-620쪽 / 원문 PDF 555-557쪽

[ ] 파일 시스템 구현 개요의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 620-622쪽 / 원문 PDF 557-559쪽

[ ] 사용법의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 622-623쪽 / 원문 PDF 559-560쪽

[ ] 선형 리스트의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 624쪽 / 원문 PDF 561쪽

[ ] 해시 테이블의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 624-625쪽 / 원문 PDF 561-562쪽

[ ] 연속 할당의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 625-627쪽 / 원문 PDF 562-564쪽

[ ] 연결 할당과 FAT의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 627-630쪽 / 원문 PDF 564-567쪽

[ ] 색인 할당의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 630-632쪽 / 원문 PDF 567-569쪽

[ ] 할당 방법의 성능의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 632-633쪽 / 원문 PDF 569-570쪽

[ ] 비트 벡터의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 634-635쪽 / 원문 PDF 571-572쪽

[ ] 연결 리스트의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 635쪽 / 원문 PDF 572쪽

[ ] 그룹핑의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 636쪽 / 원문 PDF 573쪽

[ ] 계수의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 636쪽 / 원문 PDF 573쪽

[ ] 공간맵의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 636-637쪽 / 원문 PDF 573-574쪽

[ ] 사용하지 않는 트림의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 637쪽 / 원문 PDF 574쪽

[ ] 효율의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 637-639쪽 / 원문 PDF 574-576쪽

[ ] 성능의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 639-642쪽 / 원문 PDF 576-579쪽

[ ] 일관성 검사의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 642-643쪽 / 원문 PDF 579-580쪽

[ ] 로그 구조 파일 시스템의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 643-644쪽 / 원문 PDF 580-581쪽

[ ] 다른 해결 방안들의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 644-645쪽 / 원문 PDF 581-582쪽

[ ] 백업과 복구의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 645-646쪽 / 원문 PDF 582-583쪽

[ ] 예: WAFL 파일 시스템의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 646-650쪽 / 원문 PDF 583-587쪽

[ ] 요약과 원문 연습문제의 점검 범위의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 650-653쪽 / 원문 PDF 587-590쪽

제15장. 파일 시스템 내부구조

[ ] 파일 시스템의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 655-657쪽 / 원문 PDF 591-593쪽

[ ] 파일 시스템 마운팅의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 657-659쪽 / 원문 PDF 593-595쪽

[ ] 파티션과 마운팅의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 659-660쪽 / 원문 PDF 595-596쪽

[ ] 파일 공유와 다수의 사용자의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 660-661쪽 / 원문 PDF 596-597쪽

[ ] 가상 파일 시스템의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 662-664쪽 / 원문 PDF 598-600쪽

[ ] 원격 파일 시스템의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 664쪽 / 원문 PDF 600쪽

[ ] 클라이언트 서버 모델의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 664-665쪽 / 원문 PDF 600-601쪽

[ ] 분산 정보 시스템의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 665-666쪽 / 원문 PDF 601-602쪽

[ ] 고장 모드의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 666-667쪽 / 원문 PDF 602-603쪽

[ ] 일관성의 의미의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 668쪽 / 원문 PDF 604쪽

[ ] UNIX의 의미의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 668쪽 / 원문 PDF 604쪽

[ ] 세션 의미의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 668-669쪽 / 원문 PDF 604-605쪽

[ ] 불변 공유 파일의 의미의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 669쪽 / 원문 PDF 605쪽

[ ] NFS 개요의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 669-671쪽 / 원문 PDF 605-607쪽

[ ] 마운트 프로토콜의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 672쪽 / 원문 PDF 608쪽

[ ] NFS 프로토콜의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 672-674쪽 / 원문 PDF 608-610쪽

[ ] 경로 이름 변환의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 675쪽 / 원문 PDF 611쪽

[ ] 원격 연산의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 675-676쪽 / 원문 PDF 611-612쪽

[ ] 요약과 원문 연습문제의 점검 범위의 핵심 개념·조건·처리 관계를 설명할 수 있는가?

출처: 책 676-678쪽 / 원문 PDF 612-614쪽

제7부 도입. 보안과 보호

[ ] 제공된 도입에 따라 보안과 보호의 역할을 구별할 수 있는가?

출처: 책 679쪽 / 원문 PDF 615쪽